Persistent and Fault-Tolerant UAV-IoT Coverage

Research Bundle

Mawn Mid-Semester Report — supporting material

The one thing you must know

The report's parameter table currently makes the stated objective *physically unachievable*. With $|\Omega| = 100 \times 100$ m^2, $M = 20$ sensors, $N = 5$ UAVs, $K = 2$ and $R_{\text{cov}} = 15$ m, the fleet supplies

$$N\pi R_{\text{cov}}^2 = 3\,534 \text{ m}^2$$

of coverage against a requirement of

$$KA = 2 \times 10\,000 = 20\,000 \text{ m}^2,$$

only 17.7% of what is needed. The necessary condition is

$$R_{\text{cov}} \geq \sqrt{\frac{KA}{\pi N}} = 35.68 \text{ m},$$

and the table says 15 m.
Fix: $\Omega = 80 \times 80$ m, $R_{\text{cov}} = 35$ m, $R_{\text{comm}} = 45$ m, $B^{\max} = 60$ kJ, charge 0.001/step. Surplus ratio $\sigma = 1.50$, dropping to 1.20 after a failure.
Two further must-fixes: the articulation indicator a_i collides with the action a_i (rename to κ_i), and reference `chen2025` is attributed to the wrong author (Zhuang Chen, not Chen).

29 September 2026

Contents

0.1 PROJECT 1 — RESEARCH FOLDER . . . 4
0.1.1 READ IN THIS ORDER . . . 4
0.1.2 THE ONE THING YOU MUST KNOW . . . 4
0.1.3 THE OTHER TWO MUST-FIXES . . . 5
0.1.4 WHAT IS ALREADY GOOD . . . 5
0.1.5 SCRIPTS . . . 5
0.1.6 COMPILED VERSION . . . 5
0.2 10 — REPORT BACKBONE: WHAT TO WRITE, IN WHAT ORDER . . . 6
0.2.1 1. HONEST ASSESSMENT OF WHAT YOU HAVE . . . 6
0.2.2 2. THE THREE EDITS . . . 6
0.2.3 3. THE FOUR ADDITIONS . . . 7
0.2.4 4. TARGET STRUCTURE . . . 8
0.2.5 5. WHAT TO SAY WHEN THEY ASK . . . 8
0.2.6 6. FINAL CHECKLIST . . . 9
0.2.7 7. WHERE EVERYTHING IS . . . 9
0.3 00 — MATHEMATICAL & SCIENTIFIC AUDIT . . . 10
0.3.1 PART 1 — THE FATAL FINDING . . . 10
0.3.2 PART 2 — NOTATION COLLISIONS (fix these; an examiner will spot them) . . . 11
0.3.3 PART 3 — VERIFIED CORRECT (do not touch) . . . 11
0.3.4 PART 4 — MODELLING WEAKNESSES (worth fixing, not fatal) . . . 12
0.3.5 PART 5 — CORRECTED PARAMETER SET . . . 14
0.3.6 PART 6 — CHECKLIST BEFORE SUBMISSION . . . 14
0.4 01 — THE PROBLEM STATEMENT, WRITTEN AS ONE MATHEMATICAL OBJECT . . . 16
0.4.1 THE ONE-SLIDE VERSION . . . 16
0.4.2 1. SYSTEM ENTITIES . . . 16
0.4.3 2. STATE . . . 16
0.4.4 3. OBSERVATION (why this is a Dec-POMDP, not an MDP) . . . 17
0.4.5 4. ACTION SPACE AND DYNAMICS . . . 17
0.4.6 5. THE FOUR HARD REQUIREMENTS . . . 17
0.4.7 6. FEASIBILITY — the part the current paper omits and MUST add . . . 18
0.4.8 7. CONNECTIVITY FEASIBILITY . . . 18
0.4.9 8. OBJECTIVE . . . 19
0.4.10 9. RECOVERY . . . 19
0.4.11 10. THE LEARNER . . . 20
0.4.12 11. HOW TO SAY IT OUT LOUD (30 seconds) . . . 20
0.4.13 12. WHAT YOUR PROFESSOR PROBABLY MEANT . . . 20
0.5 02 — FEASIBILITY ANALYSIS AND CORRECTED PARAMETERS . . . 21
0.5.1 1. THE ERROR . . . 21
0.5.2 2. THE COUNTING BOUND . . . 21
0.5.3 3. EVALUATION ON THE PAPER'S PARAMETERS . . . 21
0.5.4 4. WHY THIS MATTERS FOR THE REPORT . . . 22
0.5.5 5. WHICH KNOB TO TURN . . . 22
0.5.6 6. CONNECTIVITY FEASIBILITY . . . 23
0.5.7 7. SCENARIO COMPARISON . . . 24
0.5.8 8. RECOMMENDED PARAMETER TABLE . . . 24
0.5.9 9. THE SENTENCE TO ADD TO THE PAPER . . . 25
0.5.10 10. REPRODUCING . . . 25
0.6 09 — REFERENCE VERIFICATION . . . 26
0.6.1 PART 1 — VERIFICATION TABLE . . . 26
0.6.2 PART 2 — THE ONE ERROR TO FIX . . . 26

0.6.3 PART 3 — ADDITIONAL VERIFIED PAPERS TO STRENGTHEN RELATED WORK 27
0.6.4 PART 4 — REVISED SURVEY STRUCTURE . . . 29
0.7 Mathematical Reference: MARL Theory, Coverage Theory, Graph/Connectivity Theory, and Energy Modelling . . . 30
0.8 PART A — MARL THEORY . . . 30
0.8.1 A.1 MDP and Dec-POMDP . . . 30
0.8.2 A.2 POMDP belief update equations . . . 31
0.8.3 A.3 CTDE . . . 32
0.8.4 A.4 Policy gradients: REINFORCE and the log-derivative trick . . . 34
0.8.5 A.5 Value, Q, advantage $A = Q - V$ . . . 35
0.8.6 A.6 GAE — full derivation, bias–variance, TD(λ) equivalence . . . 35
0.8.7 A.7 TRPO — full math . . . 38
0.8.8 A.8 PPO . . . 40
0.8.9 A.9 MAPPO (Yu et al. 2022) . . . 42
0.8.10 A.10 Variance of the multi-agent gradient estimator; non-stationarity; recent theory . . . 44
0.8.11 A.11 Complexity: shared actor, centralised critic . . . 47
0.8.12 A.12 Reward shaping: potential-based theorem (Ng, Harada, Russell 1999) . . . 49
0.9 Theory Background: Coverage, Graph Connectivity, and Energy . . . 53
0.10 PART A — COVERAGE THEORY . . . 53
0.10.1 1. The Art Gallery Problem: O'Rourke's formulation, Chvtal's theorem, Fisk's proof . . . 53
0.10.2 2. k-Fold Coverage: Definition, Reduction, Complexity . . . 55
0.10.3 3. Connected Multi-Robot Coverage . . . 56
0.10.4 4. Persistent / Continuous Coverage: the Steady-State Problem . . . 57
0.10.5 5. Persistent Coverage Complexity (cell decomposition and the approximation ratio) . . . 58
0.10.6 6. Disk-Covering Asymptotics: **VERIFICATION OF THE CLAIMED CONSTANT** 59
0.10.7 7. **THE KEY FEASIBILITY BOUND** — Proof from First Principles . . . 60
0.10.8 8. Worked Numerical Example — $A = 100 \times 100$, $M = 20$, $N = 5$, $K = 2$, $R_{\text{cov}} = 15$ m . . . 62
0.11 PART B — GRAPH AND CONNECTIVITY THEORY . . . 64
0.11.1 9. Connectivity, k-Connectivity, and Menger's Theorems . . . 64
0.11.2 10. Articulation Points . . . 66
0.11.3 11. Bridges (Cut Edges) . . . 67
0.11.4 12. Graph Laplacian, Algebraic Connectivity, and Fiedler's Theorem . . . 68
0.11.5 13. Unit Disk Graphs . . . 69
0.11.6 14. Random Geometric Graphs . . . 71
0.11.7 15. The Connectivity Threshold . . . 72
0.11.8 16. Local Articulation Point Detection: the ACTUAL Conditions . . . 73
0.11.9 17. 2-Vertex-Connectivity, and Why It Implies Tolerance to a Single Node Failure . . . 76
0.11.10 18. Why Preserving 2-Connectivity Implies 1-Fault Tolerance — The Graph-Theoretic Reason, Spelled Out . . . 77
0.12 PART C — ENERGY . . . 79
0.12.1 19. The Peukert Model . . . 79
0.12.2 20. The Kinetic Battery Model (KiBaM) . . . 80
0.12.3 21. Li-Ion Discharge: Why SoC Is *Not* Linear in Remaining Energy . . . 82
0.12.4 22. Charging: the CC–CV Profile, Time Model, and Power Limits . . . 84
0.12.5 23. Partial Charging: When to Fully Charge and When Not To . . . 86
0.12.6 24. The Assignment Problem: Hungarian Algorithm, LAP vs MILP, and When Queueing Wins . . . 88
0.13 PART D — BIBLIOGRAPHY . . . 91
0.13.1 D.1 Sources verified to exist in this session (29 September 2026) . . . 91
0.13.2 D.2 Sources cited in the text that certainly exist but whose exact bibliographic details I did **not** re-verify in this session . . . 94
0.13.3 D.3 The three accuracy flags, restated as action items for the report . . . 94
0.14 Wireless Channel Models for Simulating a UAV–IoT Network . . . 96
0.14.1 Primary sources (exact identifiers) . . . 96
0.14.2 1. 3GPP TR 38.901 path-loss models . . . 97
0.14.3 2. Shadowing and fading . . . 103
0.14.4 3. Link budget, SINR and rate . . . 106
0.14.5 4. How the simulation stack turns this into packet outcomes . . . 110
0.14.6 5. Aerial-to-ground and aerial-to-aerial specifics . . . 115
0.14.7 6. Simu5G: what it models and what it does not . . . 118
0.14.8 7. Latency, PDR and throughput metrics (ITU-T definitions) . . . 120

0.14.9 8. Worked numerical example — every arithmetic step . . . 121
0.14.10 9. Corrections and caveats — a consolidated list . . . 126
0.14.11 10. Recommended configuration for a UAV-IoT persistent-coverage study . . . 128
0.14.12 11. Formula index (quick reference) . . . 128
0.14.13 12. References . . . 129
0.15 Simulation Toolchain & Open-Source Simulators . . . 131
0.15.1 0. Executive summary — corrections before you start . . . 131
0.15.2 1. OMNeT++ 6.4.0 . . . 132
0.15.3 2. INET 4.7.0 (and 4.6.0 / 4.5.4) . . . 138
0.15.4 3. Simu5G . . . 144
0.15.5 4. The Python OMNeT++ bridge . . . 146
0.15.6 5. Open-source UAV RL simulators — comparison . . . 151
0.15.7 6. RL algorithm implementation reality . . . 155
0.15.8 7. Reproducibility requirements . . . 157
0.15.9 8. License and citation requirements . . . 159
0.15.10 9. Traps — a consolidated list . . . 161

Part I — What to do tonight

0.1. PROJECT 1 — RESEARCH FOLDER

Topic: Persistent and Fault-Tolerant UAV-IoT Coverage: a Multi-Agent Reinforcement Learning Framework for Joint Coverage, Connectivity, and Energy Management **Compiled:** 29 September 2026 **For:** report due 30 September 2026

0.1.1. READ IN THIS ORDER

#	File	Read it if you want to. . .
1	10_REPORT_BACKBONE.md	**know exactly what to change tonight.** Start here.
2	00_MATH_AUDIT.md	see the equation-by-equation audit and every fix
3	01_PROBLEM_FORMULATION.md	see the problem stated as **one formal object** (the professor's ask)
4	02_FEASIBILITY_AND_PARAMETERS.md	understand the **fatal R_{cov} error** and the corrected table
5	09_REFERENCES_VERIFIED.md	fix the one bad reference, add 14 verified papers
6	04_MARL_THEORY.md	Dec-POMDP, PG, GAE, TRPO, PPO, MAPPO with full derivations
7	05_07_THEORY_COVERAGE_GRAPH_ENERGY.md	art gallery, k-coverage, Menger, Tarjan, Laplacian, KiBaM, Hungarian
8	03_WIRELESS_MODELS.md	3GPP TR 38.901, fading, link budget, INET/Simu5G radio models
9	08_SIMULATION_STACK.md	OMNeT++ 6.4, INET 4.7, Simu5G 1.7, RL bridges, version traps

---

0.1.2. THE ONE THING YOU MUST KNOW

The report's parameter table makes the stated objective physically impossible.

With $|\Omega| = 100 \times 100$ m^2, $M = 20$ sensors, $N = 5$ UAVs, $K = 2$ and $R_{\text{cov}} = 15$ m, the fleet can supply at most

$$N\pi R_{\text{cov}}^2 = 3\,534 \text{ m}^2$$

of coverage against a requirement of

$$KA = 2 \times 10\,000 = 20\,000 \text{ m}^2.$$

That is **17.7 %** of what is needed. The condition $n_d(t) \geq K$ is **unsatisfiable for every controller** — MAPPO, heuristic, or omniscient optimal. The dominant reward term $\lambda_O U_t$ is then pinned near its maximum, all policies score identically, and the entire coverage experiment produces no signal.

The necessary condition is

$$R_{\text{cov}} \geq \sqrt{\frac{KA}{\pi N}} = 35.68 \text{ m}$$

and 30 **m** after packing correction. The table says 15 m.

Fix: use $\Omega = 80 \times 80$ m, $R_{\text{cov}} = 35$ m, $R_{\text{comm}} = 45$ m, $B^{\max} = 60$ kJ, charge 0.001/step. That gives a surplus ratio $\sigma = 1.50$, dropping to 1.20 after a failure — so the reserve UAV is a genuine safety factor rather than the only route to feasibility.

Proof, arithmetic, six-scenario comparison and the corrected table are in `02_FEASIBILITY_AND_PARAMETERS.md`. Run `python feasibility_check.py` to reproduce it.

0.1.3. THE OTHER TWO MUST-FIXES

1. **Notation collision.** `a_i` is used both for the articulation-point indicator (Eq. 12–13) and for the action (Eq. 2, Eq. 16). Rename the indicator to κ_i.
2. **Wrong reference.** `chen2025` — the first author is **Zhuang** Chen (), not Chen. Attributed to the wrong person. Correct BibTeX in `09_REFERENCES_VERIFIED.md`. The other 13 references are all verified correct.

0.1.4. WHAT IS ALREADY GOOD

Do not rewrite these parts:

- 10-paper survey where each entry states its *limitation relative to this work*
- The gap hedge: *"a survey-derived gap statement, not a claim that no related component has ever appeared in the broader literature"*
- The refusal to guarantee connectivity from local observations
- The refusal to assign artificial recovery times to unrecovered failures
- A 12-metric evaluation table instead of one aggregate score
- The 3-phase implementation plan (Python → OMNeT++/INET → Simu5G)

0.1.5. SCRIPTS

Script	Run	Shows
`feasibility_check.py`	`python feasibility_check.py`	the current table is infeasible; endurance, charge-rate and connectivity audits
`parameter_search.py`	`python parameter_search.py`	tests each knob, derives $R_{\text{cov}} \geq 35.68$ m, Monte-Carlo connectivity
`scenario_compare.py`	`python scenario_compare.py`	compares 6 scenarios, picks the recommended set

All CPU-only, under a minute total. Their output is reproduced in the markdown files, so every number in the report is reproducible. **Include them as an appendix.**

0.1.6. COMPILED VERSION

`RESEARCH_BUNDLE.pdf` — all documents in one file, if you prefer reading a PDF rather than markdown.

0.2. 10 — REPORT BACKBONE: WHAT TO WRITE, IN WHAT ORDER

Read this last, after the other documents. It tells you what the final report should contain and exactly which edits to make to `Mlawn-midsem-tex.tex`.

Current state: 805 lines, 9 pages, IEEEtran, 14 references, 8 sections. The structure is already good. **You are not rewriting — you are fixing three things and adding four.**

0.2.1. 1. HONEST ASSESSMENT OF WHAT YOU HAVE

Already strong — do not touch: - Literature survey with 10 papers, each with a stated *limitation relative to your work* (not just a summary) - The research-gap hedge: *"a survey-derived gap statement, not a claim that no related component has ever appeared in the broader literature"* — this is exactly right epistemically. Examiners reward it. - The refusal to guarantee connectivity from local observations - The refusal to assign artificial recovery times to unrecovered failures - The staged 3-phase implementation plan - A metric table that reports 12 separate quantities instead of one aggregate

Broken — must fix: 1. $R_{\text{cov}} = 15$ m makes the objective impossible (Section 2 below) 2. `a_i` means two different things (articulation indicator vs action) 3. Reference `chen2025` attributes the paper to the wrong person

Missing — high value: 4. A feasibility argument (proves you thought about whether the instance works) 5. Any numerical results at all 6. Complexity analysis for the scalability claim

---

0.2.2. 2. THE THREE EDITS

Edit 1 — Table IV parameter fix

Replace these values:

Parameter	Current	**New**
Operating region Ω	100×100 m	80×80 **m**
Coverage radius R_{cov}	15 m	**35 m**
Communication radius R_{comm}	40 m	**45 m**
Battery capacity $B^{\max}$	30 kJ	**60 kJ**
Charge increment	0.01/step	**0.001/step**

and add two rows: `Episode length T = 2000 steps`, `Charging pads S = 2`.

Why each: - R_{cov}: 15 → 35 m makes $\sigma = 1.50$ instead of 0.18. **Mandatory.** - $|\Omega|$: 100→80 m keeps the coverage disk at a physically sensible 60 % of the region instead of 7 %. Without this, $R_{\text{cov}} = 35$ m would need a 100 m region where the disk is 38 % — also fine actually, but 80 m gives better connectivity behaviour. - R_{comm}: 40 → 45 m puts $P(\text{connected}) = 0.63$, a genuinely marginal regime worth studying. 40 m gives 0.18 — usually disconnected, so connectivity would dominate everything and swamp the other four objectives. - $B^{\max}$: 30 → 60 kJ gives 11.8 min endurance instead of 5.9 min, so agents actually reach the energy-critical regime the reward targets. - charge rate: 0.01 → 0.001/step gives a 16.7 min charge against 11.8 min flight (ratio 1.42) instead of 1.7 min against 5.9 min (ratio 0.28). Real Li-ion is ~60 min charge on a ~10 min quadrotor.

Edit 2 — rename the articulation indicator

Global search-and-replace, because `a_i` is overloaded:

Eq.	From	To
12	$a_i(t) = \mathbb{1}[G_t - i \text{ is disconnected}]$	$\kappa_i(t) = \mathbb{1}[G_t^{-i} \text{ is disconnected}]$
13	$\sum_i a_i(t)$	$\sum_i \kappa_i(t)$
16	$\mathbb{1}[\mathbf{a_i} = \text{Hover}]$	$\mathbb{1}[\mathbf{a}_i = \text{Hover}]$

Also add $G_t^{-i} := G_t$ with vertex i removed, and use G_t^{-i} in Eq. 12 instead of $G_t - i$ for typesetting consistency.

And define the active set formally, right after Eq. 3:

$$\mathcal{U}(t) = \{\, i \in \{1, \ldots, N\} : F_i(t) = 0 \,\}$$

Edit 3 — fix the reference

`chen2025` → `zhuang2025`, first author **Zhuang Chen**, pages 856–861. Full BibTeX in `09_REFERENCES_VERIFIED.md`. Verify the DOI resolves in a browser; if not, drop the reference — you have 13 other verified ones.

0.2.3. 3. THE FOUR ADDITIONS

Addition 1 — Feasibility argument (Sec. III-F) — HIGHEST VALUE

Insert after the service condition $n_d(t) \geq K$. The exact paragraph is in `02_FEASIBILITY_AND_PARAMETERS.md` 9. It is four lines of prose and one equation, and it converts the report's biggest weakness into a demonstration of rigour.

This is the best value-for-effort edit in the report.

Addition 2 — Why Dec-POMDP (Sec. III-B)

The paper states the Dec-POMDP tuple but never argues the problem is not an MDP. Add two sentences:

> The problem is a Dec-POMDP for two independent reasons. First, observability is partial: $\mathbf{o}_i \subsetneq \mathbf{s}_t$, since a UAV cannot observe peers beyond radio range, sensors it does not cover, or global topology. Second, execution is decentralised: even with global observability at training time, the deployed controller must satisfy $\pi_i : \mathcal{O}_i \to \mathcal{A}_i$ individually, so the joint action $\mathcal{A} = \mathcal{A}_1 \times \cdots \times \mathcal{A}_N$ with $|\mathcal{A}| = 6^N$ is never available at deployment.

Addition 3 — Complexity (Sec. III-M or IV)

The scalability claim in Table VIII is currently asserted. Make it rigorous in five lines:

Quantity	Cost
Actor inference, per agent per step	$O(1)$ — o_i is fixed-size via top-K_n + mask
Actor inference, whole swarm	$O(N)$
Critic (training only)	$O(N + M)$
Charging arbitration (Hungarian)	$O(r^3)$, r = concurrent requests $\leq N$
Graph recomputation	$O(N^2)$ brute force, $O(N \log N)$ with spatial hash
Articulation points (Tarjan)	$O(\|V\| + \|E\|)$

> Because the observation is fixed-size, the deployed policy costs $O(1)$ per agent regardless of swarm size. Execution cost is therefore independent of N, which is the formal basis for the scalability metric in Table VIII.

Addition 4 — At least some numbers

Zero numbers is the biggest remaining risk. The three heuristic baselines need **no learning** and are cheap:

Baseline	Implementation	Runtime
Fixed formation + threshold recharge	~30 lines	seconds
Greedy lowest-battery recharge	~40 lines	seconds
Reactive reconfiguration	~80 lines	~1 h

Run all three on 5 seeds under the corrected parameters, fill in 4–5 rows of Table VIII with mean ± std. This alone converts the report from a proposal into a study. MAPPO results can remain future work — that is a legitimate position *if* baselines are present.

Suggested appendix: the three scripts in this folder, with their output. It makes every number in the paper reproducible, which is worth marks by itself.

0.2.4. 4. TARGET STRUCTURE

Keep the current section order. It is already correct for an IEEE conference paper.

```
I.   Introduction
       goal, gap, research question, 4 contributions
II.  Literature Survey
       Table I: 10 papers, each with "limitation relative to this work"
       Sec. II-C: research gap  <- RESTRUCTURE as a capability matrix
III. System Model
       A. Architecture (Fig. 1)
       B. Dec-POMDP formulation         <- ADD why-Dec-POMDP paragraph
       C. Global state
       D. Local observation
       E. Action space and motion
       F. Persistent K-coverage         <- ADD feasibility paragraph
       G. Connectivity model
       H. Battery model
       I. Charging coordination
       J. Safety constraints
       K. Failure and recovery
       L. Objective and reward          <- ADD sign-consistency sentence
       M. Parameters and scope          <- REPLACE Table IV
       N. Complexity                    <- NEW
IV.  Proposed ML Algorithm
       A. CTDE MAPPO, B. Feasibility mask, C. Failure-aware execution,
       D. Algorithm 1, E. Training protocol, F. Hyperparameters
V.   Comparison Models                <- ADD A1, A5, A9 from ref verification
VI.  Performance Metrics
VII. Experimental Design Matrix
VIII. Results                          <- NEW (heuristic baselines only)
```

0.2.5. 5. WHAT TO SAY WHEN THEY ASK

"Is the problem actually solvable?" > Yes, and I show the condition. K-coverage of area A by N sensing disks > needs $N\pi R_{\text{cov}}^2 \geq KA$, so $R_{\text{cov}} \geq> \sqrt{KA/(\pi N)}$. For my parameters that is 28.5 m counting, 30.0 m > packing-corrected, and I use 35 m — a surplus ratio of 1.50, dropping to 1.20 > after a failure. My original value of 15 m gave a ratio of 0.18, which means > the objective was unachievable; I found that by checking feasibility.

(Saying this unprompted is worth more than any single section of the report.)

"Why MAPPO and not QMIX?" > MAPPO keeps a centralised critic during training and a shared actor at > deployment, so the action space stays linear in N per agent. QMIX's > monotonic factorisation assumes the team Q decomposes additively over agents; > coverage, connectivity and charging contention are coupled non-additively > through shared sensors and shared pads, so that assumption does not hold. > Also, the action mask for the return-energy constraint makes the feasible set > state-dependent, which is awkward for a monotonic decomposition.

"How do you guarantee connectivity?" > I don't, and I don't claim to. A UAV provably cannot detect a global > articulation point from one-hop observations. So the mask enforces the locally > checkable part — the move doesn't isolate the mover — the reward penalises > articulation count softly through λ_C, and connectivity is *reported as > a metric*, not asserted as an invariant.

"Why is the reserve a resource and not an agent?" > Deterministic activation and assignment. Making it an agent would add a > seventh actor to a problem where the joint action space is already 6^N, for a > role that is a single binary decision. The reserve uses the same shared actor > once it is inserted, so it does not need its own policy.

"Your results show no improvement." > I report heuristic baselines only. The contribution at this stage is the > formulation, the feasibility analysis, and the evaluation protocol. The MAPPO > experiments are specified in Section VII but not yet run, and I would rather > defer the claim than state an unsupported one.

0.2.6. 6. FINAL CHECKLIST

Do these three, then submit: - [ ] `R_{\mathrm{cov}}` 15 → 35 m, `$\Omega$` 100 → 80 m, `R_{\mathrm{comm}}` 40 → 45 m, `B^{max}` 30 → 60 kJ, charge 0.01 → 0.001 - [ ] `a_i` (articulation) → `$\kappa_i$` everywhere; define `$\mathcal{U}(t)$` - [ ] `chen2025` → `zhuang2025`, author Zhuang Chen, pp. 856–861

Then, in priority order: - [ ] Add the feasibility paragraph (Sec. III-F) — 15 minutes, biggest payoff - [ ] Add the why-Dec-POMDP paragraph (Sec. III-B) — 5 minutes - [ ] Add the sign-consistency sentence (Sec. III-L) — 2 minutes - [ ] Add the complexity table (Sec. III-N) — 15 minutes - [ ] Run the three heuristic baselines, fill in 4–5 rows of Table VIII - [ ] Add A1 (Shi et al. TMC 2025) and A9 (Scherer & Rinner 2016) to related work - [ ] Restructure the gap statement as a capability matrix

Leave alone — these are already right: - The two honesty disclaimers (connectivity guarantee, unrecovered failures) - The "survey-derived gap statement" hedge - The 12-metric evaluation table - The 3-phase implementation plan - The "numerical claims deferred" position *if* you add baseline numbers

---

0.2.7. 7. WHERE EVERYTHING IS

File	What it is
`00_MATH_AUDIT.md`	Equation-by-equation audit + all fixes
`01_PROBLEM_FORMULATION.md`	The problem as one formal object (professor's ask)
`02_FEASIBILITY_AND_PARAMETERS.md`	The R_{cov} proof + corrected table
`03_WIRELESS_MODELS.md`	3GPP TR 38.901, fading, link budget, INET/Simu5G radio models
`04_MARL_THEORY.md`	Dec-POMDP, PG, GAE, TRPO, PPO, MAPPO derivations
`05_07_THEORY_COVERAGE_GRAPH_ENERGY.md`	Art gallery, k-coverage, Menger, Tarjan, Laplacian, KiBaM, Hungarian
`08_SIMULATION_STACK.md`	OMNeT++ 6.4, INET 4.7, Simu5G 1.7, RL bridges, version traps
`09_REFERENCES_VERIFIED.md`	14 refs checked + 14 new verified papers
`10_REPORT_BACKBONE.md`	This file
`feasibility_check.py`	Proves the current table is infeasible
`parameter_search.py`	Tests each knob, derives the minimum R_{cov}
`scenario_compare.py`	Compares 6 scenarios with Monte-Carlo connectivity

Part II — The audit and the fix

0.3. 00 — MATHEMATICAL & SCIENTIFIC AUDIT

File audited: `Project1/Mlawn-midsem-tex.tex` (805 lines, IEEEtran, 9-page PDF) **Date:** 29 September 2026 **Verdict:** The paper is **structurally strong and mathematically honest**, but it contains **one fatal parameter error** that makes the stated objective physically unachievable, plus **several notation collisions** and **3 weaker modelling problems**.

Fix the CRITICAL item before submission. The rest are improvements.

0.3.1. PART 1 — THE FATAL FINDING

1.1 `R_cov = 15 m` makes `K`-coverage impossible for *every* controller

The claim in the paper (Eq. 12–16, Sec. III-F):

> "The primary service condition is $n_d(t) \geq K$ for all d whenever the geometry and available fleet make it feasible."

and Table IV sets $R_{\text{cov}} = 15$ m, $M = 20$, $N = 5$, $K = 2$, $|\Omega| = 100 \times 100 = 10\,000$ m^2.

The counting bound. For K-coverage of a region of area A by N sensing disks of radius r, coverage of all M sensors at level K requires $K \cdot M$ distinct (sensor, UAV) incidence pairs. Each UAV supplies at most πr^2 of covered area, hence at most $M\pi r^2/A$ sensors. Summing over N UAVs:

$$\underbrace{N \cdot \pi r^2}_{\text{area available}} \geq \underbrace{K \cdot A}_{\text{area required}} \qquad \Longleftrightarrow \qquad r \geq \sqrt{\frac{KA}{\pi N}}.$$

Substituting the paper's values:

$$r \geq \sqrt{\frac{2 \times 10\,000}{\pi \times 5}} = \sqrt{1273.24} = \mathbf{35.68}\ \text{m}$$

The paper uses **15 m** — short by a factor of **2.38**×.

Direct consequence (computed, not estimated):

Quantity	Value
Coverage area required, KA	20 000 m
Coverage area supplied, $N\pi R_{\text{cov}}^2$	3 534 m
Ratio	**0.177**
Expected coverage slots, $N \cdot (M/A)\pi R_{\text{cov}}^2$	7.07
Coverage slots required, KM	40
Fraction of requirement achievable	**17.7 %**

What this means for the report. With these parameters, $n_d(t) \geq K$ is **never** satisfiable. Not by MAPPO, not by a heuristic, not by an omniscient optimal controller. The reward's dominant term $\lambda_O U_t$ is therefore permanently pinned near its maximum, the agent receives essentially a constant gradient, and the entire coverage-learning experiment produces no signal. Every baseline will score the same. This is the single most likely reason an examiner would reject the report after reading Table IV.

Packing-corrected bound. The counting bound ignores that coverage disks overlap. Densest packing of congruent disks has density $\delta = \pi/(2\sqrt{3}) \approx 0.9069$, so:

$$r \geq \sqrt{\frac{KA}{\delta\,\pi N}} = \sqrt{\frac{20\,000}{0.9069\,\pi \times 5}} = \mathbf{37.47}\ \text{m}$$

Best case does not save it. Even with a perfectly uniform sensor grid (5×4, spacing 20 × 25 m), one UAV with $r = 15$ m covers at most **1** sensor, so the swarm supplies 5 slots against 40 required. The problem is not a placement failure — it is geometrically impossible.

Fix: set $R_{\text{cov}} \geq 40$ m (see Part 4 for the recommended set).

0.3.2. PART 2 — NOTATION COLLISIONS (fix these; an examiner will spot them)

2.1 `$a_i$` means two different things — CRITICAL

Location	Usage	Meaning
Eq. 2 (policy)	$a_i(t) \sim \pi_\theta(a_i \mid \mathbf{o}_i(t))$	the **action**
Eq. 16	$P_i(t) = P_{\text{hover}}\mathbb{1}[\mathbf{a_i} = \text{Hover}] + \dots$	the **action**
Eq. 12	$a_i(t) = \mathbb{1}[G_t - i \text{ is disconnected}]$	**articulation-point indicator**
Eq. 13	$\text{Crit}_t = \frac{1}{\lvert V_t^{act}\rvert} \sum_i a_i(t)$	uses the indicator

Eq. 13 is built on the indicator meaning while Eq. 2 and Eq. 16 use the action meaning. This is a genuine ambiguity, not a style issue.

Fix: rename the articulation indicator. Suggested:

$$\kappa_i(t) \;=\; \mathbb{1}\big[G_t^{-i} \text{ is disconnected}\big]\,, \qquad \text{Crit}_t = \frac{1}{|V_t^{\text{act}}|} \sum_{i \in V_t^{\text{act}}} \kappa_i(t)$$

Note also that κ then does not collide with the future use of $\kappa(G)$ for vertex connectivity, which appears in earlier drafts — pick one. The report already uses ρ_i for the *local* proxy, which is the right distinction, so keeping κ_i for the *global* one is consistent.

2.2 `$\gamma$` is safe, but check `$\lambda$`

The report uses γ for the discount factor (Eq. 3) and $\lambda_E, \lambda_D, \lambda_W, \lambda_R, \lambda_C$ for objective weights, and λ for the GAE parameter (Table VI). **This is internally consistent** — no collision — but note that λ (GAE) and $\lambda_\bullet$ (rewards) look alike. Add one sentence distinguishing them, or rename GAE's to λ_{GAE}.

(This was a real collision in the earlier Beamer version of the deck, where `\gamma` doubled as a charging-cost coefficient. That is fixed.)

2.3 `$\mathcal{V}_t^{\mathrm{act}}$` is used but never defined

Eq. 11 and Eq. 13 sum over $\mathcal{V}_t^{\text{act}}$ (the set of *active* UAVs). The text says "A failed UAV is removed from the active set" but never writes the set formally. Add:

$$\mathcal{V}_t^{\text{act}} \;=\; \big\{\, i \in \{1, \dots, N\} \;:\; F_i(t) = 0 \,\big\}, \qquad \mathcal{V}_t^{\text{act}} \subset \mathcal{V}_t^{\text{act}} \cup \{N+1\}$$

2.4 `$S$` is overloaded with the objective sum index

`$S$` is the number of charging pads (Sec. III-A, $\mathbf{Q}_t \in \{0,1\}^S$). In Eq. 30 the sums $\sum_s \sum_s$ use lowercase s as a *pad index*, which is fine, but make sure the text never writes S when it means the pad count in the same sentence as the objective.

2.5 Two different coverage quantities, both named confusingly

Symbol	Eq.	Meaning
C_t	10	instantaneous *soft* score, $\min(n_d/K, 1)$ averaged
C_K	Table VIII	*hard* fraction of sensor-slots with $n_d \geq K$

These are not the same and the names are close. Suggest C_t^{soft} and C_K^{hard}, or state the relationship explicitly. Note $C_t = 1 \iff n_d \geq K \;\forall d$, so $C_t \leq C_K$ always — worth one sentence.

0.3.3. PART 3 — VERIFIED CORRECT (do not touch)

I checked these carefully. They are right.

Eq.	Statement	Verdict
1	Dec-POMDP tuple $\langle N, \mathcal{S}, \{\mathcal{A}_i\}, \{\mathcal{O}_i\}, \mathcal{T}, R, \gamma\rangle$	[OK] correct
2	Objective $\max_\pi \mathbb{E}_\pi[\sum_{t=0}^{T-1} \gamma^t R_t]$	[OK] correct
4	$o_i(t) = [\mathbf{p}_i, \mathbf{v}_i, b_i, \mathcal{N}_i, C_i^{local}, \rho_i, \tau_i^{return}, E_i^{pad}]$	[OK] correct
5	$x_{id}(t) = \mathbb{1}[\lVert\mathbf{p}_i - \mathbf{q}_d\rVert_2 \le R_{cov}]$	[OK] correct
6	$n_d(t) = \sum_{i \in V^{act}} x_{id}(t)$	[OK] correct
7	$C_t = \frac{1}{M}\sum_d \min(n_d/K, 1)$	[OK] correct
8	$U_t = \frac{1}{M}\sum_d \mathbb{1}[n_d < K]$	[OK] correct
9	$G_t = (V_t, E_t),\ E_t = \{(i,j) : \lVert\mathbf{p}_i - \mathbf{p}_j\rVert_2 \le R_{comm}\}$	[OK] correct
11	$a_i(t) = \mathbb{1}[G_t - i$ disconnected]	[OK] correct *math* (bad symbol, 2.1)
14	$E_i(t+1) = \text{clip}(E_i(t) - P_i(t)\Delta t + P_{ch,i}\Delta t,\ 0, B^{max})$	[OK] correct
15	$E_i^{req} = E_i^{pad} + E_i^{rejoin} + E^{reserve}$	[OK] correct
17	$c_{is} = \alpha\tilde{d}_{is} + \beta\tilde{q}_s + \mu\tilde{u}_i$	[OK] correct; **normalised tildes are good practice**
18	Hungarian assignment with $\sum_s y_{is} \le 1,\ \sum_i y_{is} \le 1$	[OK] correct LAP
21	Safety $\lVert\mathbf{v}_i\rVert \le v_{max},\ \mathbf{p}_i \in \Omega,\ \lVert\mathbf{p}_i - \mathbf{p}_j\rVert \ge d_{safe}$	[OK] correct
23	$\tau_f = \inf\{\tau \ge 0 : n_d(t_f + \tau) \ge K\ \forall d\}$	[OK] correct
25	$J_{out}(\pi) = \mathbb{E}_\pi[\sum \gamma^t U_t]$	[OK] correct
28	Team reward $r_t = -(\lambda_O U_t + \lambda_E E_t + \dots)$	[OK] correct, and **sign-consistent** with the max-return objective
31	GAE $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}$	[OK] correct
33	Clipped PPO actor loss	[OK] correct form
34	Critic loss with $\hat{R}_t = \hat{A}_t + V_\phi(\mathbf{s}_t)$	[OK] correct

Sign consistency — worth stating explicitly in the report. The paper minimises J_{out} (Eq. 25) but the RL *maximises* return, and Eq. 28 negates the reward. This is internally consistent, but an examiner may pause on it. Add one line: *"Since the learner maximises expected return while the control objective is a minimisation, the team reward is the negated weighted cost (Eq. 28); the multi-objective problem of Eq. 26 and the RL objective are therefore equivalent up to sign."* That sentence removes the ambiguity entirely.

0.3.4. PART 4 — MODELLING WEAKNESSES (worth fixing, not fatal)

4.1 Endurance is only ~6 minutes — the energy regime is never exercised

$$t_{\text{move}} = \frac{B^{max}}{P_{\text{move}} + P_{\text{comm}}} = \frac{30\,000}{75 + 10} = 352.8 \text{ s} = 5.9 \text{ min}$$

The return threshold lands at

$$E^{req} \approx \underbrace{2404}_{\text{corner}\to\text{corner}} + \underbrace{1700}_{\text{rejoin}} + \underbrace{3000}_{10\%\text{ reserve}} = 7104 \text{ J} = 24\%\ B^{max}, \qquad \text{so } b_i < 0.76.$$

So the agent only ever operates in the top 76 % of the battery window, and the "energy-critical" behaviour the reward is designed to shape is rarely visited. With $T = 2000$ steps at $\Delta t = 1$ s, a UAV gets roughly **5–6 recharge cycles per episode** — enough to matter, but the dynamics are heavily dominated by transit.

Recommended: $B^{max} = 60$ kJ, $P_{\text{move}} = 75$ W, $P_{\text{comm}} = 10$ W → endurance 11.8 min, return threshold 17 %, giving ~3 cycles per 2000-step episode.

4.2 Charging is ~6× too fast

`0.01 battery fraction / step` at $\Delta t = 1$ s means a full 0→1 charge takes **100 s**, versus 353 s of flight. Real Li-ion at 1C takes ~60 min. Charging contention is therefore almost free, which undercuts one of the paper's five claimed contributions.

Recommended: `0.001 battery fraction / step` at $\Delta t = 1\,\text{s} \to 1000\,\text{s}$ (16.7 min) full charge; charge/flight ratio 1.42, close to a real 1C pack. (Matches `mei2026`'s 0.547 coverage / 38-slot outage scale, which is a useful comparability point.)

4.3 $\Delta t = 1$ s is too coarse relative to R_{cov}

At $v_{max} = 5$ m/s and $\Delta t = 1$ s a UAV moves 5 m per step, so with $R_{\text{cov}} = 15$ m the coverage disk is only **6 control steps** across its diameter. The 6-way discrete action space is then dominated by moves that change which sensors are covered very little — a weak credit-assignment signal.

Rule of thumb: choose Δt so that

$$\frac{2R_{\text{cov}}}{v_{max}\Delta t} \;\geq\; 10 \qquad \Longleftrightarrow \qquad \Delta t \;\leq\; \frac{2R_{\text{cov}}}{10\,v_{max}}.$$

With the corrected $R_{\text{cov}} = 35$ m and $v_{max} = 5$ m/s this gives $\Delta t \leq 1.4$ s, so the existing 1 s is **acceptable once R_{cov} is fixed**. No change needed. (The problem was R_{cov}, not Δt.)

4.4 The Dec-POMDP is claimed but not argued

Sec. III-B states the Dec-POMDP tuple but never says *why* it is not an MDP. The report should state explicitly that partial observability is inherent: each UAV cannot observe s_t, so the controller is a Dec-POMDP by construction, and decentralised execution additionally forbids access to the joint action. One paragraph, and it strengthens the contribution claim.

4.5 No complexity analysis

A one-line complexity result strengthens the scalability contribution and is trivial to prove:

- **Actor inference:** $O(1)$ per agent per step, since o_i is fixed-size (top-K_n neighbours with masking). Total $O(N)$.
- **Critic (training only):** $O(N + M)$ per step, since s_t is linear in the number of agents and sensors.
- **Hungarian charging step:** $O(r^3)$ in the number of concurrent charge requests r (typically $r \leq N$), executed once per slot.
- **Graph recomputation:** $O(N^2)$ brute force, or $O(N \log N)$ with a spatial hash; articulation-point detection is $O(|V| + |E|)$ via Tarjan.

Because the actor is $O(1)$ per agent, the *deployed* policy cost is independent of N. That is the formal argument for the scalability claim in Table VIII, and it is currently asserted but not shown.

4.6 Connectivity is enforced by masking, but the paper is careful — keep that

Sec. III-H correctly notes that connectivity cannot be *guaranteed* from one-hop observations, and the paper says so. This is the right call and a strength. Make sure the reward's λ_C term and the local proxy ρ_i are presented as a *soft* mechanism, with connectivity as an evaluation metric rather than a claimed invariant. The report already does this — preserve it.

4.7 "Numerical performance claims are intentionally deferred"

This is honest and defensible, but a mid-semester report with **zero** numbers is a risk. Minimum viable addition: run the **heuristic baselines only** (no MARL needed) and report Table VIII metrics for 3–5 seeds. Baselines are cheap:

Baseline	Effort	Expected runtime
Fixed formation + threshold recharge	trivial	minutes
Greedy lowest-battery recharge	trivial	minutes
Reactive reconfiguration	small	~1 h

Even three heuristic rows with mean ± std makes the report concrete and directly tests the feasibility fix in Part 5.

0.3.5. PART 5 — CORRECTED PARAMETER SET

Computed by Monte-Carlo and counting bounds (`parameter_search.py`, `scenario_compare.py` in this folder). Define the surplus ratio

$$\sigma \;=\; \frac{\text{expected coverage slots}}{\text{required slots}} = \frac{N\,(M/A)\,\pi R_{\text{cov}}^2}{KM}$$

$\sigma \geq 1$ is necessary; $\sigma \gtrsim 1.5$ leaves room for movement and a failure. P(connected) is the Monte-Carlo probability that a random geometric graph on N uniform points is connected.

Recommendation — Scenario S-E (primary)

Parameter	Paper	**Recommended**	Why
Ω	100×100 m	80×80 **m**	keeps radii physically plausible
M	20	**20**	unchanged
N / N_R	5 / 1	**5 / 1**	unchanged
K	2	**2**	unchanged
R_{cov}	15 m	**35 m**	$\sigma = 1.50$ — feasible with slack
R_{comm}	40 m	**45 m**	$P(\text{conn}) = 0.63$, $P(\text{conn} \mid \text{failure}) = 0.56$
v_{max}	5 m/s	**5 m/s**	unchanged
d_{safe}	5 m	**5 m**	unchanged
Δt	1 s	**1 s**	≥ 14 steps across $2R_{\text{cov}}$ — fine
$B^{\max}$	30 kJ	**60 kJ**	11.8 min endurance
$P_{\text{hover}}/P_{\text{move}}/P_{\text{comm}}$	50/75/10 W	**50/75/10 W**	unchanged
charge increment	0.01/step	**0.001/step**	16.7 min full charge, ratio 1.42
S (pads)	2	**2**	unchanged
T	—	**2000 steps**	~3 recharge cycles per episode

With $R_{\text{cov}} = 35$ m the coverage disk is 60 % of the region area, so a UAV sees ~10 of 20 sensors on average — genuinely partial coverage, not trivial.

Failure-tolerance check. After losing one UAV, σ falls to $4 \cdot (M/A)\pi R_{\text{cov}}^2/(KM) = 1.20$ — still above 1. The reserve is therefore a *safety factor* rather than the only route to feasibility, which is exactly the claim the paper wants to make. P(connected) drops from 0.63 to 0.56, so the reserve demonstrably helps connectivity too.

Alternative — Scenario S-C (if you want connectivity to be the hard constraint)

$|\Omega| = 60 \times 60$ m, $R_{\text{cov}} = 30$ m, $R_{\text{comm}} = 40$ m: $\sigma = 1.96$ (coverage is comfortable) but $P(\text{conn}) = 0.86$ falling to 0.78 after a failure. Use this if the story you want is "coverage is solvable, connectivity is what breaks."

What NOT to do

Increasing N to fix $R_{\text{cov}} = 15$ m requires $N \geq 29$ (counting) or $N \geq 32$ (packing-corrected). That destroys the scalability argument and makes a 30-agent swarm intractable to train. Reducing M below 4 makes the problem trivial. Increasing $|\Omega|$ makes it strictly worse.

0.3.6. PART 6 — CHECKLIST BEFORE SUBMISSION

Must fix: - [ ] Set $R_{\text{cov}} \geq 35$ m and state the counting bound in the paper. Add a one-sentence feasibility argument to Sec. III-F: *"K-coverage is reachable under the stated fleet only if* $R_{\text{cov}} \geq \sqrt{KA/(\pi N)}$*; Table IV satisfies this."* - [ ] Rename the articulation indicator to avoid clashing with the action (2.1) and propagate through Eqs. 12, 13, 16. - [ ] Fix reference `chen2025` → author is **Zhuang** Chen, not Z. Chen (see `09_REFERENCES_VERIFIED.md`).

Strongly recommended: - [ ] Define $\mathcal{V}_t^{\text{act}}$ formally. - [ ] Add the sign-consistency sentence between Eq. 26 and Eq. 28. - [ ] Fix `$B^{\mathrm{max}}$` to 60 kJ and the charge rate to 0.001/step. - [ ] Run the three heuristic baselines and fill in at least a few rows of Table VIII. Zero numbers is the biggest remaining risk. - [ ] Add the one-paragraph justification of *why Dec-POMDP* (4.4). - [ ] Add the complexity paragraph (4.5).

Optional polish: - [ ] Distinguish C_t (soft) from C_K (hard) by name. - [ ] Rename GAE's λ to λ_{GAE}. - [ ] Change `qiu2019` to the peer-reviewed ICC 2020 version (doi `10.1109/ICC40277.2020.9149258`). - [ ] Change `mathew2015` title to "**Multirobot** Rendezvous Planning..." (IEEE-exact form). - [ ] Mark `sun2025` explicitly as an arXiv preprint.

Do not change (already correct and honest): - The refusal to claim connectivity guarantees from local observations (4.6). - The refusal to assign artificial recovery times to unrecovered failures (III-I). - The deferral of numerical claims *if* you add baseline numbers. - The "survey-derived gap statement, not a claim that no related component has ever appeared" hedge in Sec. II-C. This is exactly the right epistemic framing.

0.4. 01 — THE PROBLEM STATEMENT, WRITTEN AS ONE MATHEMATICAL OBJECT

Purpose. Your professor asked you to *put the problem statement into math so it is easy to explain and verify.* The current paper defines the model across Sections III-A to III-M, which is normal for a paper but awkward when you have to state "what is the problem?" in one breath. This document collapses the whole problem into a **single numbered statement** that you can put on one slide and defend line by line.

Everything here is consistent with `Mlawn-midsem-tex.tex` after the fixes in `00_MATH_AUDIT.md`.

0.4.1. THE ONE-SLIDE VERSION

> **Persistent, Fault-Tolerant UAV-IoT Coverage (PFT-UAV-IoT).** Given N cooperative UAVs, M fixed ground sensors in a region Ω, a finite battery per UAV, S shared charging pads, and stochastic single-UAV failures, find decentralised policies $\{\pi_i\}$ such that, at every time step, (C1) every sensor is covered by $\geq K$ UAVs, (C2) the UAV communication graph stays connected, (C3) no UAV commits to a path it cannot survive on its remaining energy, and (C4) no hard physical constraint is violated — while minimising expected long-horizon service loss.

Now the formal object.

0.4.2. 1. SYSTEM ENTITIES

Symbol	Type	Meaning
$\mathcal{U}$	$\subseteq \{1,\dots,N\}$	active UAV index set
$\mathcal{U}_R$	$\{N+1\}$	reserve UAV index set
$\mathcal{D}$	$= \{1,\dots,M\}$	sensor index set
$\mathcal{S}$	$= \{1,\dots,S\}$	charging-pad index set
$\Omega \subset \mathbb{R}^2$	region	operating area, $\|\Omega\| = A$
z_0	$\mathbb{R}_+$	nominal UAV altitude (fixed)

Fix from the audit: the active set is written formally, not implied:

$$\mathcal{U}(t) \;=\; \big\{\, i \in \{1,\dots,N\} \;:\; F_i(t) = 0 \,\big\}$$

A UAV with $F_i(t) = 1$ is removed from all coverage, connectivity and energy sums.

0.4.3. 2. STATE

$$\boxed{\mathbf{s}_t = \Big\{ \big(\mathbf{p}_i, \mathbf{v}_i, b_i, m_i, F_i\big)_{i\in\{1..N\}},\; \big(\mathbf{q}_d\big)_{d\in\mathcal{D}},\; \mathbf{Q}_t \Big\}}$$

$$\mathbf{s}_t = \Big\{ \big(\mathbf{p}_i, \mathbf{v}_i, b_i, m_i, F_i\big)_{i=1}^{N},\; \big(\mathbf{q}_d\big)_{d=1}^{M},\; \mathbf{Q}_t \Big\} \quad \text{with}$$

Symbol	Domain	Meaning
$\mathbf{p}_i(t)$	Ω	horizontal position
$\mathbf{v}_i(t)$	$\{\mathbf{v} : \|\mathbf{v}\| \leq v_{\max}\}$	velocity
$b_i(t)$	$[0,1]$	normalised battery, $E_i = b_i B^{\max}$
$m_i(t)$	$\{\mathsf{serve}, \mathsf{transit}, \mathsf{charge}\}$	mode
$F_i(t)$	$\{0,1\}$	failure flag
$\mathbf{q}_d$	Ω	fixed sensor position
$\mathbf{Q}_t$	$\{0,1\}^S$	pad occupancy

0.4.4. 3. OBSERVATION (why this is a Dec-POMDP, not an MDP)

$$\mathbf{o}_i(t) = \Big[\mathbf{p}_i,\ \mathbf{v}_i,\ b_i,\ \mathcal{N}_i(t),\ C_i^{\text{local}}(t),\ \rho_i(t),\ \tau_i^{\text{return}}(t),\ E_i^{\text{pad}}(t)\Big]$$

The formal argument the current paper is missing. The problem is a Dec-POMDP for two independent reasons:

1. **Partial observability.** $\mathbf{o}_i \subsetneq \mathbf{s}_t$. A UAV cannot observe other UAVs' exact positions beyond radio range, cannot see sensors it is not covering, and cannot see the global graph topology.
2. **Decentralised execution.** Even if $\mathbf{s}_t$ were observable at training time, the deployed controller must satisfy $\pi_i : \mathcal{O}_i \to \mathcal{A}_i$ **individually** — the joint action is never available at deployment.

The paper's own line — *"this distinction avoids assuming that an individual UAV can magically obtain the complete global topology"* (Sec. III-D) — is the right insight; it just needs to be stated as a structural claim, not a caveat.

Global vs local criticality — keep the distinction, rename the symbol. The *global* articulation indicator is an evaluation quantity:

$$\kappa_i(t) = \mathbb{1}\big[G_t^{-i} \text{ is disconnected}\big], \qquad G_t^{-i} := G_t \text{ with vertex } i \text{ and its edges removed}$$

The *local* proxy ρ_i is what the actor actually receives, computed from 1-hop (and optionally 2-hop) neighbour exchange. **Never** feed κ_i to the actor; feed ρ_i and evaluate κ_i. (This is the §§2.1 collision from the audit.)

0.4.5. 4. ACTION SPACE AND DYNAMICS

$$\mathcal{A}_i = \{\mathsf{Hover}, +x, -x, +y, -y, \mathsf{Charge}\}, \qquad |\mathcal{A}_i| = 6$$

$$\mathbf{p}_i(t+1) = \Pi_\Omega(\mathbf{p}_i(t) + \mathbf{v}_i(t)\,\Delta t)\,, \qquad \|\mathbf{v}_i(t)\| \le v_{\max}$$

Π_Ω projects into the permitted airspace — this is how the workspace constraint is *structural* rather than penalised.

Why 6 and not 5. The joint action space is $|\mathcal{A}| = 6^N$: $6^4 = 1296$, $6^6 = 46\,656$, $6^{10} = 60\,466\,176$, $6^{20} \approx 3.7 \times 10^{15}$. Any tabular or joint-Q approach is dead beyond $N \approx 8$.

(Note: the earlier Beamer deck used 5 actions and 5^N. The current paper's 6 actions are the better choice — the extra action is what lets the agent request charging at all. Keep 6 and make the deck match the paper.)

0.4.6. 5. THE FOUR HARD REQUIREMENTS

This is the part the professor is asking for. Each requirement is a predicate on $(\mathbf{s}_t, \mathbf{a}_t)$ that must hold at every step. They are **hard** — enforced by the feasibility mask, not by reward.

C1 — Persistent K-coverage

$$x_{id}(t) = \mathbb{1}\big[\|\mathbf{p}_i(t) - \mathbf{q}_d\|_2 \le R_{\text{cov}}\big], \qquad n_d(t) = \sum_{i \in \mathcal{U}(t)} x_{id}(t), \qquad \boxed{n_d(t) \ge K \quad \forall d \in \mathcal{D}}$$

C2 — Swarm connectivity

$$G_t = (V_t, E_t), \qquad E_t = \big\{(i,j) : \|\mathbf{p}_i - \mathbf{p}_j\|_2 \le R_{\text{comm}}\big\}$$

$$\boxed{G_t \text{ connected}, \quad \forall t}$$

Stronger form for single-failure tolerance: require $\kappa(G_t) \ge 2$ (2-vertex-connectivity), which by Menger's theorem guarantees the graph stays connected after **any** single vertex removal.

> **Important honesty note — keep the paper's current stance.** Connectivity **cannot** be guaranteed from one-hop local observations. A UAV provably cannot detect a global articulation point without global topology. So: - **Feasibility mask** enforces the *locally checkable* part of C2 (a move that would isolate the mover or exceed radio range). - The **reward** penalises Crit_t softly. - **Connectivity is reported as a metric**, not claimed as an invariant.
>
> This is exactly what Sec. III-H already says. Do not weaken it.

C3 — Energy reachability

$$E_i^{\text{req}}(t) = E_i^{\text{pad}}(t) + E_i^{\text{rejoin}}(t) + E^{\text{reserve}}$$

$$\boxed{E_i(t) \;\geq\; E_i^{\text{req}}(t) \quad \text{before committing to any move}}$$

If $E_i(t) < E_i^{\text{req}}(t)$, the only feasible action is Charge. This is the *predictive* part of the contribution — it replaces a fixed battery threshold with a **return-feasibility** test.

C4 — Physical safety

$$\|\mathbf{v}_i(t)\| \leq v_{\max}, \qquad \mathbf{p}_i(t) \in \Omega, \qquad \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \geq d_{\text{safe}} \quad \forall i \neq j$$

0.4.7. 6. FEASIBILITY — the part the current paper omits and MUST add

K-coverage is achievable only if the fleet is large enough. This is a necessary condition on the parameters, and it is the check that caught the fatal error in the current Table IV.

Proposition (area-counting bound). *If N UAVs, each covering a disk of radius R_{cov}, achieve K-coverage of every one of M sensors uniformly distributed in a region of area A, then necessarily*

$$N\,\pi R_{\text{cov}}^2 \;\geq\; K\,A \qquad \Longleftrightarrow \qquad R_{\text{cov}} \;\geq\; \sqrt{\frac{KA}{\pi N}}.$$

Proof. K-coverage of all M sensors requires KM incidence pairs (i,d) with $x_{id}=1$, so $\sum_d n_d \geq KM$. For a uniformly distributed sensor field of density $\sigma_d = M/A$, the expected number of sensors in one coverage disk is $\sigma_d \pi R_{\text{cov}}^2$. Summing over N UAVs, $\mathbb{E}[\sum_d n_d] = N(M/A)\pi R_{\text{cov}}^2$. Requiring the expectation to reach KM gives the bound. ■

Packing-corrected version. Disks cannot tile without overlap. Densest packing of congruent disks has density $\delta = \pi/(2\sqrt{3}) \approx 0.9069$, so

$$R_{\text{cov}} \;\geq\; \sqrt{\frac{KA}{\delta\,\pi N}}$$

Slack factor. In practice you want a surplus

$$\sigma := \frac{N(M/A)\pi R_{\text{cov}}^2}{KM} \;\gtrsim\; 1.5$$

so that movement error and a single failure do not immediately break C1. After losing one UAV the requirement is $\sigma' \geq 1$, which $N=5,\ R_{\text{cov}}=35$ m, $A=6400$ satisfies ($\sigma = 1.50 \to \sigma' = 1.20$).

Applied to the current paper's Table IV ($A = 10^4$ m, $M=20$, $N=5$, $K=2$):

$$R_{\text{cov}} \geq \sqrt{\frac{2\cdot 10^4}{5\pi}} = 35.7 \text{ m} \quad \text{but the table says 15 m}$$

This is the single change that makes the whole study work. Full numbers and a corrected table are in `02_FEASIBILITY_AND_PARAMETERS.md`.

0.4.8. 7. CONNECTIVITY FEASIBILITY

Proposition (random geometric graph). *For N points placed uniformly and independently in a region of area A, with edges between points within distance R, the expected edge count is*

$$\mathbb{E}[|E|] = \binom{N}{2}\frac{\pi R^2}{A}$$

and a connected graph on N vertices requires $|E| \geq N-1$. Hence $P(\text{connected})$ is appreciable only when

$$\binom{N}{2}\frac{\pi R^2}{A} \;\gtrsim\; N-1.$$

Refined threshold (Penrose 1997). The connectivity threshold radius r_n for n points satisfies $\mathbb{E}[|E|] \sim \frac{1}{2}n\ln n$, giving

$$r_n \sim \sqrt{\frac{n \ln n}{\pi n}} \approx \sqrt{\frac{\ln n}{\pi}}.$$

Monte-Carlo for $A = 6400$ m, $N = 5$:

R_{comm}	P(connected)	P(connected | 1 failure)
30 m	0.036	—
40 m	0.089	—
45 m	**0.632**	**0.562**
50 m	0.732	—
60 m	0.90+	—

$R_{\text{comm}} = 45$ m puts the study in a genuinely **marginal connectivity regime** — the swarm is connected most of the time but disconnects occasionally. That is the interesting regime for a fault-tolerance paper. State it deliberately.

0.4.9. 8. OBJECTIVE

Primary quantity — persistent service loss:

$$J_{\text{out}}(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t U_t\right], \qquad U_t = \frac{1}{M}\sum_{d=1}^{M} \mathbb{1}\big[n_d(t) < K\big]$$

Constrained multi-objective programme:

$$\min_\pi \; J_{\text{out}}(\pi) + \lambda_E J_E(\pi) + \lambda_D J_D(\pi) + \lambda_W J_W(\pi) + \lambda_R J_R(\pi) + \lambda_C J_C(\pi)$$

$$\text{s.t.} \quad \text{C1, C2 (soft), C3, C4}$$

Team reward handed to the learner (negated, because RL maximises return):

$$r_t = -\Big(\lambda_O U_t + \lambda_E E_t + \lambda_D D_t + \lambda_W W_t + \lambda_R R_t + \lambda_C \,\text{Crit}_t\Big)$$

Sign-consistency sentence to add (this trips examiners):

> *The learner maximises expected return while the control objective of (26) is a minimisation. The team reward (28) is therefore the negated weighted cost, and the two are equivalent up to sign. λ_O dominates the secondary efficiency terms so that service is never traded away for energy savings.*

0.4.10. 9. RECOVERY

$$\tau_f = \inf\Big\{\tau \geq 0 : \; n_d(t_f + \tau) \geq K \;\; \forall d \in \mathcal{D}\Big\}$$

Resilience requirement: $\tau_f \leq \tau_{\max}$ when recovery is achievable.

If the set is empty, $\tau_f = \infty$ and the event is reported as an unrecovered failure. Do not assign an artificial recovery time. The current paper gets this exactly right — keep it.

Reserve activation. The reserve $N + 1$ is a *resource*, not an RL agent. It enters $\mathcal{U}(t)$ when a failure is detected and $b_{N+1} \geq b_{\text{thr}}$. It uses the same shared actor π_θ after insertion.

0.4.11. 10. THE LEARNER

$$a_i(t) \sim \pi_\theta(a_i \mid \mathbf{o}_i(t)) \quad \text{(deployed, per UAV)}$$
$$V_\phi(\mathbf{s}_t) \quad \text{(training only, centralised)}$$

CTDE statement. The critic minimises $\mathcal{L}^{\text{critic}}$ against $\mathbf{s}_t$; the actor maximises $\mathcal{L}^{\text{actor}}$ against $\mathbf{o}_i$. At deployment the critic is deleted and the shared actor runs independently on each UAV.

Parameter sharing is valid here because all active UAVs are interchangeable: same dynamics, same action semantics, same reward. A shared θ therefore generalises across UAVs and across N, which is what makes the policy $O(1)$ per agent.

Complexity (add this to the paper — it is the formal scalability claim):

Quantity	Cost
Actor inference, per agent per step	$O(1)$ (fixed-size o_i via top-K_n + mask)
Actor inference, whole swarm	$O(N)$
Critic, per step (training only)	$O(N+M)$
Charging arbitration (Hungarian)	$O(r^3)$, r = concurrent requests $\leq N$
Graph recomputation, brute force	$O(N^2)$
Graph recomputation, spatial hash	$O(N \log N)$
Articulation points (Tarjan)	$O(\lvert V\rvert + \lvert E\rvert) = O(N^2)$ worst case

Because the deployed policy is $O(1)$ per agent, execution cost is independent of swarm size. That is the rigorous version of the scalability claim in Table VIII.

0.4.12. 11. HOW TO SAY IT OUT LOUD (30 seconds)

> "Our problem is a Dec-POMDP. There are N UAVs over M fixed sensors. Each UAV sees only its neighbours, not the global state, and must choose one of six actions independently — so it is partial observability plus decentralised execution. We require four things to hold at every step: K-coverage of every sensor, a connected communication graph, enough energy to reach a charger before committing to any move, and no physical-safety violation. We minimise expected long-horizon service loss, penalising energy, movement, charging wait, recovery delay and topology fragility. We learn it with MAPPO under centralised training with a global-state critic and decentralised execution with a shared actor. Coverage, connectivity, charging arbitration and a reserve UAV are all handled outside the policy as hard feasibility layers, because the actor cannot observe enough to guarantee them."

That paragraph contains the entire paper. If you can say it, you understand it.

0.4.13. 12. WHAT YOUR PROFESSOR PROBABLY MEANT

When a professor says *"put the problem statement in math"*, they are usually complaining about one of these five things. Check which applies to you:

Symptom	What they want	Where you are now
Problem stated in prose only	One formal definition object they can point at	**Now provided** — 1–5
Requirements not quantified	Every "should maintain coverage" turned into a predicate or an inequality	**Now provided** — C1–C4
No objective function	An explicit $\min_\pi J(\pi)$ with named terms	**Now provided** — 8
No feasibility check	Proof the instance is solvable at all	**Now provided** — 6, and this is what caught the R_{cov} error
Variables undefined	A complete symbol table with domains	**Now provided** — 1–4

The last two are the ones that separate a C-grade report from an A-grade one, and the feasibility argument in 6 is the single highest-value addition you can make tomorrow.

0.5. 02 — FEASIBILITY ANALYSIS AND CORRECTED PARAMETERS

This document is the highest-value thing in the folder. It contains a proof that the parameter table in `Mlawn-midsem-tex.tex` makes the stated objective physically unachievable, and a corrected table that fixes it.

Scripts: `feasibility_check.py`, `parameter_search.py`, `scenario_compare.py` (all in this folder, all runnable, all outputs reproduced below).

0.5.1. 1. THE ERROR

`Mlawn-midsem-tex.tex`, Table IV:

Parameter	Value
Operating region Ω	100×100 m^2
IoT sensors M	20
Active UAVs N	5
Coverage radius R_{cov}	**15 m**
Coverage target K	2

The paper's Sec. III-F states the primary service condition as $n_d(t) \geq K$ for all d, and Sec. III-M states the objective as minimising expected under-coverage J_{out}.

These are incompatible. With $R_{\text{cov}} = 15$ m the swarm *cannot* satisfy $K = 2$ for any controller.

0.5.2. 2. THE COUNTING BOUND

Theorem. If N UAVs each covering a disk of radius R_{cov} achieve K-coverage of all M sensors uniformly distributed in a region of area A, then necessarily

$$N\,\pi R_{\text{cov}}^2 \;\geq\; K\,A \qquad \Longleftrightarrow \qquad \boxed{R_{\text{cov}} \;\geq\; \sqrt{\frac{KA}{\pi N}}}$$

Proof. K-coverage of all M sensors requires at least KM incidence pairs with $x_{id} = 1$, hence $\sum_{d=1}^{M} n_d \geq KM$. The sensor field has density $\sigma_d = M/A$. A UAV at position $\mathbf{p}$ covers a disk of area πR_{cov}^2, containing on average $\sigma_d \pi R_{\text{cov}}^2$ sensors. Summing over N UAVs, $\mathbb{E}[\sum_d n_d] = N(M/A)\pi R_{\text{cov}}^2$. Requiring this to reach KM:

$$\frac{N(M/A)\pi R_{\text{cov}}^2}{KM} \;\geq\; 1 \;\Longleftrightarrow\; \frac{N\pi R_{\text{cov}}^2}{A} \;\geq\; K \qquad \blacksquare$$

Packing-corrected form. Disks cannot tile the plane. The densest packing of congruent disks has density

$$\delta = \frac{\pi}{2\sqrt{3}} \approx 0.9069$$

so a stricter necessary condition is

$$R_{\text{cov}} \;\geq\; \sqrt{\frac{KA}{\delta\,\pi N}}$$

0.5.3. 3. EVALUATION ON THE PAPER'S PARAMETERS

$$R_{\text{cov}} \;\geq\; \sqrt{\frac{2 \times 10\,000}{\pi \times 5}} = \sqrt{1273.24} = \mathbf{35.68}\ \text{m} \qquad \text{(counting)}$$

$$R_{\text{cov}} \;\geq\; \sqrt{\frac{20\,000}{0.9069\,\pi \times 5}} = \mathbf{37.47}\ \text{m} \qquad \text{(packing-corrected)}$$

The paper uses **15 m** — **2.38× too small**.

Quantity	Value
Area required, KA	20 000 m
Area supplied, $N\pi R_{\text{cov}}^2$	3 534 m
Ratio	**0.177**
Expected coverage slots	7.07
Required slots, KM	40
Fraction achievable	**17.7 %**

3.1 Even single-coverage fails

The theory document (`05_07_...md` 8) sharpens this: the *maximum* coverage level attainable is

$$K_{\max} = \left\lfloor \frac{N\pi R_{\text{cov}}^2}{KA} \right\rfloor = \left\lfloor \frac{3534.29}{20000} \right\rfloor = \mathbf{0}$$

So the configuration cannot even **single-cover** all 20 sensors. Best-case coverage of the *entire region* would need

$$R_{\text{cov}} \geq \sqrt{\frac{KA}{N\pi}}\bigg|_{K=1} = 35.68 \text{ m},$$

and the binding requirement at $K = 2$ (accounting for discrete sensor placement) is **39.24 m**.

3.2 Best case does not rescue it

Uniform sensor grids, $R_{\text{cov}} = 15$ m:

Grid	M	Spacing	Max sensors per UAV	Max slots	Need	Verdict
5×4	20	20×25 m	1	5	40	INFEASIBLE
4×5	20	25×20 m	1	5	40	INFEASIBLE
4×4	16	25×25 m	1	5	32	INFEASIBLE

One UAV covers at most **one** sensor. This is not a placement failure — it is geometrically impossible.

0.5.4. 4. WHY THIS MATTERS FOR THE REPORT

With the objective permanently unachievable:

1. U_t is pinned near 1 for **every** policy.
2. The dominant reward term $\lambda_O U_t$ supplies a near-constant gradient.
3. MAPPO, MADDPG, greedy and fixed-formation baselines all score identically.
4. The "learning" part of the study becomes untestable.
5. Every table in Section VI becomes a table of noise.

An examiner who computes $N\pi R^2 \geq KA$ will find this in about thirty seconds. It is the single highest-risk item in the report.

0.5.5. 5. WHICH KNOB TO TURN

Four options, with the arithmetic:

R_{cov}	expected slots	required	surplus σ	verdict

Option A — increase R_{cov} [OK] RECOMMENDED

R_{cov}	expected slots	required	surplus σ	verdict
15 m	7.07	40	0.18	infeasible (current)
25 m	19.63	40	0.49	infeasible
35 m	38.48	40	0.96	infeasible (just barely)
36 m	40.72	40	**1.02**	feasible at the bound
40 m	50.27	40	1.26	feasible
45 m	63.62	40	1.59	feasible
50 m	78.54	40	1.96	feasible

The coverage radius is the *natural* knob: it is the physical sensing range of the UAV, and increasing it does not fight the rest of the design.

Option B — increase N [X] REJECT

$$N \geq \frac{KM}{(M/A)\pi R^2} = \frac{40}{1.414} = 28.3 \qquad \text{(packing-corrected: 31.2)}$$

A 29–32 UAV swarm. This (a) destroys the scalability contribution, (b) makes centralised-critic training intractable, and (c) negates the whole point of decentralised execution.

Option C — reduce M [X] REJECT

At $N = 5$, $K = 2$, $R_{\text{cov}} = 15$ m: $M_{\max} = 3$. Too trivial.

Option D — increase $|\Omega|$ [X] REJECT

Makes it strictly worse: $\sigma \propto 1/A$.

0.5.6. 6. CONNECTIVITY FEASIBILITY

Expected edges in a random geometric graph on N uniform points:

$$\mathbb{E}\big[|E|\big] = \binom{N}{2} \frac{\pi R_{\text{comm}}^2}{A}$$

For $N = 5$, $R_{\text{comm}} = 40$ m, $A = 10^4$ m: $\mathbb{E}[|E|] = 5.03$, versus $N - 1 = 4$ needed for connectivity. Marginal. Monte-Carlo says $P(\text{connected})$ is only **0.18** — the paper's own configuration is *usually disconnected.*

Connectivity threshold radius (Penrose 1997, $\mathbb{E}[|E|] \sim \frac{1}{2} n \ln n$):

N	r_n	paper's R_{comm}
5	45.6 m	40 m — **below threshold**
6	42.2 m	40 m
10	34.2 m	40 m — above
20	25.9 m	40 m — well above

So both parameters are wrong in the same direction — coverage too small, connectivity too small. The configuration is simultaneously infeasible on C1 and usually failing on C2.

0.5.7. 7. SCENARIO COMPARISON

σ = expected coverage slots / required slots (want $\gtrsim 1.5$). $P(\text{conn})$ = Monte-Carlo probability the graph is connected, 8 000 trials.

scenario	$\|\Omega\|$	M	R_{cov}	R_{comm}	σ	$P(\text{conn})$	$P(\text{conn}\,\|\text{fail})$	disk/Ω
S-A as-written	10 000	20	15	40	**0.18**	0.18	0.17	7 %
S-B bigger radius	10 000	20	40	50	1.26	0.46	0.40	50 %
S-C smaller region	3 600	20	30	40	**1.96**	**0.86**	**0.78**	79 %
S-D smaller region 2	3 600	20	35	45	2.67	0.95	0.91	107 %
S-E mid	6 400	20	35	45	**1.50**	**0.63**	**0.56**	60 %
S-F mid alt	6 400	16	35	45	1.50	0.63	0.56	60 %

Note S-D has disk/$\Omega = 107\%$ — one UAV's coverage disk is larger than the region, which makes coverage trivial and is not a good scenario.

S-E is the best choice. Both constraints are non-trivial: - $\sigma = 1.50$: coverage needs work but is achievable - $P(\text{conn}) = 0.63$: connectivity is genuinely marginal - one UAV's disk is 60 % of the region: partial coverage, not trivial - after a failure $\sigma' = 1.20$ and $P(\text{conn}) = 0.56$: the reserve helps both constraints measurably

---

0.5.8. 8. RECOMMENDED PARAMETER TABLE

Replace Table IV in the paper with this:

Parameter	Symbol	Value
Operating region	Ω	80×80 m^2
Active UAVs	N	5
Reserve UAVs	N_R	1
IoT sensors	M	20
Coverage target	K	2
Coverage radius	R_{cov}	**35 m**
Communication radius	R_{comm}	**45 m**
Control interval	Δt	1 s
Maximum speed	$v_{\max}$	5 m/s
Safety distance	d_{safe}	5 m
Battery capacity	$B^{\max}$	**60 kJ**
Hover power	P_{hover}	50 W
Move power	P_{move}	75 W
Communication power	P_{comm}	10 W
Charging pads	S	2
Charge increment	—	**0.001 per step**
Episode length	T	2000 steps

Bold entries are the three changes from the current table plus one addition.

Feasibility check for the new table

$$N\pi R_{\text{cov}}^2 = 5\pi(35)^2 = 19\,242 \text{ m}^2 \;\geq\; KA = 2 \times 6\,400 = 12\,800 \text{ m}^2 \qquad \checkmark\ (\sigma = 1.50)$$

$$R_{\text{cov}} = 35 \text{ m} \;\geq\; \sqrt{\frac{2 \cdot 6\,400}{0.9069\,\pi \cdot 5}} = 29.98 \text{ m} \quad \checkmark$$

Endurance and charging

Config	Flight (85 W)	Charge 0→1	ratio	Return at
current 30 kJ, 0.01/step	353 s = 5.9 min	100 s = 1.7 min	**0.28**	24 %
recommended 60 kJ, 0.001/step	706 s = 11.8 min	1000 s = 16.7 min	**1.42**	17 %
reference 120 kJ, 0.0005/step	1412 s = 23.5 min	2000 s = 33.3 min	1.42	13 %

Real Li-ion at 1C charges in ~60 min against a ~10 min quadrotor endurance → ratio ≈ 6. The current 0.28 is ~20× too fast; 1.42 is the same order and conservative. Use the recommended row.

Time-step resolution

Rule of thumb: at least 10 control steps across the coverage diameter,

$$\frac{2R_{\text{cov}}}{v_{max}\Delta t} \;\geq\; 10$$

With $R_{\text{cov}} = 35$ m, $v_{max} = 5$ m/s, $\Delta t = 1$ s: **14 steps** ✓. So $\Delta t = 1$ s is fine — it was $R_{\text{cov}} = 15$ m that broke it (6 steps). No change needed to Δt.

0.5.9. 9. THE SENTENCE TO ADD TO THE PAPER

Put this in Sec. III-F, immediately after the service condition:

> **Feasibility.** Persistent K-coverage is achievable only when the sensing radius is large enough for the fleet. A necessary condition is obtained by comparing the area available for coverage with the area required: N disks of radius R_{cov} supply $N\pi R_{\text{cov}}^2$, while K-coverage of a region of area A requires KA. Hence $R_{\text{cov}} \geq \sqrt{KA/(\pi N)}$; accounting for the $\pi/(2\sqrt{3}) \approx 0.907$ packing density of congruent disks gives $R_{\text{cov}} \geq \sqrt{KA/(\delta\pi N)}$. Substituting the parameters of Table IV ($A = 6400$ m^2, $M = 20$, $N = 5$, $K = 2$) yields $R_{\text{cov}} \geq 28.5$ m (counting) and 30.0 m (packing-corrected), both satisfied by the chosen $R_{\text{cov}} = 35$ m. The resulting surplus ratio $\sigma = N\pi R_{\text{cov}}^2/(KA) = 1.50$ leaves margin for movement error and for a single-UAV failure, after which $\sigma = 1.20$ still exceeds 1.

This is four lines of prose, one equation, and it converts the report's biggest weakness into a demonstration of rigour. **It is the single best value-for-effort edit in the entire report.**

0.5.10. 10. REPRODUCING

cd Project1\research python feasibility_check.py # proves the current table is infeasible python parameter_search.py # tests each knob, derives R_cov >= 35.68 m python scenario_compare.py # compares 6 scenarios, Monte-Carlo connectivity

All three run in under a minute on CPU. Include the outputs as an appendix so the numbers in the paper are reproducible — that is worth marks on its own.

0.6. 09 — REFERENCE VERIFICATION

Checked: 29 September 2026, by web search + Crossref + arXiv + publisher records. **Result: 13 of 14 references are real and correctly cited. One has a wrong author name. No fabricated references found.**

A fabricated or misattributed reference is the fastest way to lose marks. Fix `chen2025` before you submit.

0.6.1. PART 1 — VERIFICATION TABLE

#	Key	Verdict	Action
1	`liu2018`	[OK] VERIFIED	none
2	`qiu2019`	[OK] VERIFIED	optional: use ICC 2020 version
3	`yu2022`	[OK] VERIFIED	none
4	`omoniwa2022`	[OK] VERIFIED	none
5	`zareh2016`	[OK] VERIFIED	optional: full surname is Eshghdoust
6	`basu2004`	[OK] VERIFIED	none
7	`mathew2015`	[OK] VERIFIED	title: use "**Multirobot**"
8	`arribas2023`	[OK] VERIFIED	none
9	`jiang2024`	[OK] VERIFIED	none
10	`khan2025`	[OK] VERIFIED	none
11	`sun2025`	[OK] VERIFIED — genuine	mark as arXiv preprint
12	`chen2025`	[X] **WRONG AUTHOR**	`Z. Chen` → `Z. Zhuang`
13	`carvalho2026`	[OK] VERIFIED — genuine	none
14	`mei2026`	[OK] VERIFIED — genuine	none

0.6.2. PART 2 — THE ONE ERROR TO FIX

`chen2025` — **the first author is Zhuang, not Chen**

The paper is real, but the name is being misattributed. ICCCS 2025 lists the authors in Given-Name-Surname order, so "Chen Zhuang" () has surname **Zhuang**. Writing "Z. Chen" attributes the paper to a different person.

Corrected BibTeX:

@inproceedings{zhuang2025marl, author = {Zhuang Chen and Yuxiang Huang and Chaoyi Zhu and Wuyang Zhou}, title = {Multi-Agent Reinforcement Learning Based Trajectory Design for UAV-Assisted Emergency Wireless Communication}, booktitle = {2025 10th International Conference on Computer and Communication Systems (ICCCS)}, address = {Chengdu, China}, publisher = {IEEE}, year = {2025}, pages = {856--861}, doi = {10.1109/ICCCS65393.2025.11069505} }

> **Caveat.** Existence, authorship and page range (856–861) were confirmed from the ICCCS 2025 programme and proceedings table of contents, but the DOI string could not be independently resolved (IEEE Xplore blocks automated queries). **Open the DOI in a browser before submitting.** If it does not resolve, drop the reference rather than risk it — you have 13 other verified refs.

Cosmetic fixes

- N. Mathew, S. L. Smith, and S. L. Waslander, ``Multi-robot rendezvous planning - for recharging in persistent tasks,'' IEEE Transactions on Robotics, ... + N. Mathew, S. L. Smith, and S. L. Waslander, ``Multirobot rendezvous planning + for recharging in persistent tasks,'' IEEE Transactions on Robotics, ...

IEEE's published title is one word. "Multi-robot" is the preprint form.

Two optional upgrades

% qiu2019 — replace the preprint with the peer-reviewed version @inproceedings{qiu2020, author = {Jin Qiu and Jiangbin Lyu and Liqun Fu}, title = {Placement Optimization of Aerial Base Stations With Deep Reinforcement Learning}, booktitle = {IEEE International Conference on Communications (ICC)}, pages = {1--6}, year = {2020}, doi = {10.1109/ICC40277.2020.9149258} }

% zareh2016 — formal first-author surname author = {Zareh Eshghdoust and Lorenzo Sabattini and Claudio Secchi}

Note on sun2025

This is a **genuine arXiv preprint** (arXiv:2509.23905, 28 Sep 2025) and the **MAHPPO-AM** acronym is correctly attributed — the paper explicitly defines "multi-agent hybrid proximal policy optimization with action masking (MAHPPO-AM)" and uses it throughout. But it has **no peer-reviewed version**.

If your course requires peer-reviewed sources, either mark it `[preprint]` in the text or pair it with one of A2/A3 below.

0.6.3. PART 3 — ADDITIONAL VERIFIED PAPERS TO STRENGTHEN RELATED WORK

All confirmed to exist. **A1 and A9 are the two you most need.**

A1 — The closest published competitor. Add this.

> R. Shi, X. Yu, Y. Wang, Y. Tian, Z. Liu, W. Wu, X.-P. Zhang, and M. M. Veloso, "Symmetry-Informed MARL: A Decentralized and Cooperative UAV Swarm Control Approach for Communication Coverage," *IEEE Transactions on Mobile Computing*, vol. 24, no. 9, pp. 8039–8056, Sep. 2025. doi: **10.1109/TMC.2025.3553285**

Why it matters. This is the peer-reviewed work closest to your problem: permutation/equivariant-network MARL for decentralised UAV swarm communication coverage. It explicitly benchmarks against *both* `liu2018` and `yu2022` — the same two papers you build on. **Omitting it is the most conspicuous gap in your related-work section.** Cite it, and position your work against it: they do coverage only; you add persistent K-redundancy, connectivity criticality, charging contention and failure recovery.

A9 — Foundational for your novelty claim. Add this.

> J. Scherer and B. Rinner, "Persistent multi-UAV surveillance with energy and communication constraints," *IEEE Int. Conf. Automation Science and Engineering (CASE)*, pp. 1225–1230, 2016. doi: **10.1109/COASE.2016.7743546**

Why it matters. This is the canonical formulation of **persistent coverage subject to both energy/recharge feasibility AND a maintained multi-hop connection to a base station** — i.e. literally your "connected persistent coverage" problem statement. If your contribution claim is "persistent coverage + connectivity + energy", **this is the paper you must cite and differentiate from.** Not citing it invites a "you did not acknowledge the prior art" comment.

A2 — CTDE-MAPPO for large cellular UAV swarms

> N. D. M. Quang, R. Chong, Z. Wei, C. Liu, and D. W. K. Ng, "Game-Theoretic Multi-Agent Reinforcement Learning for Swarm Trajectory Planning in Low-Altitude Wireless Networks," arXiv: **2606.16386**, 2026.

Useful because it explicitly argues why value-decomposition methods (QMIX) are unsuited to non-decomposable congestion coupling. That argument is relevant to your choice of a *policy-gradient* MAPPO over a value-based baseline.

A3 — Beyond vanilla MAPPO

> P. Zhang and G. Li, "A cooperative dynamic target search approach for multi-UAV systems utilizing the MAPPO algorithm," *Discover Artificial Intelligence*, vol. 5, art. 153, 2025. doi: **10.1007/s44163-025-00411-9**

Attention-augmented MAPPO (AS-MAPPO) head-to-head vs plain MAPPO and MADDPG. Good for showing you know what the MAPPO variant literature looks like.

A4 — Joint trajectory + communication + power

C. Wang, C. Wei, Z. Yin, K. Jin, and X. Li, "Collaborative planning of multi-UAV trajectories and communication strategies considering channel resource constraints," *Chinese Journal of Aeronautics*, 2025. doi: **10.7527/S1000-6893.2025.31837**

End-to-end MAPPO for trajectories + association + power. Recent and communication-facing — matches your Phase 2/3 direction.

A5 — Energy-constrained coverage planning (strongest recent baseline)

D. Datsko, F. Nekov, R. Pnika, and M. Saska, "Energy-Aware Multi-UAV Coverage Mission Planning With Optimal Speed of Flight," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2893–2900, Mar. 2024. doi: **10.1109/LRA.2024.3358581** — code: `github.com/ctu-mrs/EnergyAwareMCPP`

The first multi-UAV coverage method to enforce **battery-capacity constraints during planning**, validated in a real two-UAV flight. This is the baseline an examiner is most likely to name. Cite it.

A6 — UAV+UGV charging rendezvous

M. S. Mondal, S. Ramasamy, J. D. Humann, J. M. Dotterweich, and P. A. Bhounsule, "Deep Reinforcement Learning Enabled Persistent Surveillance with Energy-Aware UAV-UGV Systems," *IEEE Transactions on Intelligent Vehicles*, vol. 11, no. 8, pp. 914–929, 2026. doi: **10.1109/TIV.2026.3696943**

Jointly plans UAV surveillance routes and UGV recharging rendezvous with a transformer DRL policy. Directly relevant to your charging subproblem.

A7 — Scalability of MARL coverage

X. Chen *et al.*, "Transformer-based reinforcement learning for scalable multi-UAV area coverage," *IEEE Trans. Intelligent Transportation Systems*, vol. 25, p. 10062, 2024. doi: **10.1109/TITS.2024.3358010**

Answers the obvious examiner question about your swarm size.

A8 — The canonical continuous-coverage UAV paper

H. Shakhatreh, A. Khreishah, J. Chakareski, H. B. Salameh, and I. Khalil, "On the Continuous Coverage Problem for a Swarm of UAVs," *2016 IEEE 37th Sarnoff Symposium*, pp. 130–135, 2016. doi: **10.1109/SARNOF.2016.7846742**

The theoretical ancestor of the continuous/persistent coverage problem. Older than your 2024–26 window but indispensable for the framing.

A10 — k-connectivity restoration for drone swarms

O. Ceylan, Z. A. Dagdeviren, M. Challenger, and O. Dagdeviren, "Design and Analysis of k-Connectivity Restoration Algorithms for Fault-Tolerant Drone Swarms in Harsh Civil Environments," *Drones*, vol. 10, no. 1, art. 16, 2026. doi: **10.3390/drones10010016** — code: `github.com/netosresearch-eg`

The natural successor to your `basu2004` and `zareh2016` citations, and it explicitly critiques existing k=1 RL-based restoration work. Strong for your fault-tolerance section.

A11 — k-connectivity via IP model

M. A, Z. A. Dagdeviren, V. K. Akram, H. U. Yldz, O. Dadeviren, and B. Tavl, "Enhancing Drone Network Resilience: Investigating Strategies for k-Connectivity Restoration," *Applied Sciences*, 2025.

Caveat: the paper and author list were confirmed but **no DOI could be retrieved**. Check MDPI before citing.

A12 — Persistent emergency comms, fleet-size minimisation

C. Liu, X. Xin, Y. Dai, and D. Xu, "Cost Optimization of UAV Swarm Network for Persistent Emergency Communication," *IEEE Trans. Green Communications and Networking*, vol. 10, pp. 1734–1748, 2026.

Minimises UAV count while guaranteeing continuous multi-hop backhaul under energy constraints. A strong **non-RL optimality reference** — good for arguing your learned policy is competitive with combinatorial methods.

A13 — Spare-drone fleet sizing (foundational)

E. Hartuv, N. Agmon, and S. Kraus, "Scheduling spare drones for persistent task performance under energy constraints," *AAMAS 2018*, pp. 532–540. doi: **10.65109/YXEM5998**

The combinatorial core of "how many UAVs + how many chargers". Underpins `arribas2023` and much of the 2026 literature.

A14 — Mobile charging stations (standard reference)

X. Lin, Y. Yazcoglu, and D. Aksaray, "Robust Planning for Persistent Surveillance With Energy-Constrained UAVs and Mobile Charging Stations," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4157–4164, 2022. doi: **10.1109/LRA.2022.3146938**

The standard reference for mobile charging stations in persistent missions. Full partial-charging analysis is in `05_07_THEORY_COVERAGE_GRAPH_ENERGY.md` Part C.23.

0.6.4. PART 4 — REVISED SURVEY STRUCTURE

Your current Table I groups 10 papers by *component*. An examiner looking for a research gap wants to see the *intersection* clearly. Restructure as a **capability matrix**:

Paper	Cov	Conn	Energy	Charge	Fault	Persist	RL
liu2018	✓	✓	✓				✓
qiu2019	✓						✓
omoniwa2022	✓	✓	✓				✓
zareh2016		✓			✓		
basu2004		✓			✓		
mathew2015			✓	✓		✓	
arribas2023			✓	✓		✓	
jiang2024			✓		✓		✓
khan2025	✓						✓
mei2026	✓		✓	✓		✓	✓
Scherer & Rinner (A9)	✓	✓	✓	✓		✓	
Shi et al. (A1)	✓						✓
Ceylan et al. (A10)		✓			✓		
This work	✓	✓	✓	✓	✓	✓	✓

The gap sentence that falls out of the matrix — and it is defensible because every cell was verified:

> *Scherer & Rinner (2016) come closest by coupling persistent coverage with both energy and connectivity, but they treat service locations as planned rather than reconfigured after failure. Shi et al. (2025) give decentralised communication coverage with symmetry-informed MARL, but without persistent redundancy, charging contention or failure recovery. Ceylan et al. (2026) solve k-connectivity restoration combinatorially, without learned coverage or energy management. No surveyed work simultaneously models (i) persistent K-coverage, (ii) communication-graph criticality, (iii) contested charging pads, (iv) failure-triggered reserve recovery, and (v) decentralised motion execution. That intersection is the target of this work.*

This is exactly the hedging your current Sec. II-C already does well (*"a survey-derived gap statement, not a claim that no related component has ever appeared"*). Keep that framing — it is the mark of honest scholarship.

Part III — Theoretical background

0.7. Mathematical Reference: MARL Theory, Coverage Theory, Graph/Connec Theory, and Energy Modelling

Compiled: 29 September 2026 **Purpose:** Rigorous reference for a report on multi-agent reinforcement learning for persistent, fault-tolerant UAV-IoT coverage.

Notation: $\mathcal{S}$ state space, $\mathcal{A}$ action space, N agents, M observation dimension, P polygon complexity, r sensor range, K coverage multiplicity, n vertex count, V number of vertices, E edges, λ GAE/decay parameter, δ trust-region radius, ϵ PPO clip, γ discount.

Verification status. Every theorem, constant and equation was checked against primary sources (arXiv full text, publisher PDFs, journal TOC pages) during compilation. **Three items could not be verified and are flagged in place with [!]:** 15 (Orden & Lpez-Polo complexity constants), 17 (Gordon–Orfao–Rote 1999 — see warning), 31 (Lin et al. / Liu et al. partial-charging specifics). Treat those three as *report-author assertions requiring primary-source confirmation.*

0.8. PART A — MARL THEORY

0.8.1. A.1 MDP and Dec-POMDP

A.1.1 Markov Decision Process

Definition A.1 (MDP). $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ with $\mathcal{S}$ finite state space, $\mathcal{A}$ finite action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0,1]$ transition kernel with $\sum_{s'} P(s'|s,a) = 1$, $R(s,a,s')$ expected reward, $\gamma \in [0,1)$ discount.

Definition A.2 (policy). $\pi : \mathcal{S} \to \Delta(\mathcal{A})$; $\pi^\star \in \arg\max_\pi J(\pi)$.

Definition A.3 (value functions).

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t\geq 0} \gamma^t r_{t+1} \,\middle|\, s_0 = s\right], \quad Q^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{t\geq 0} \gamma^t r_{t+1} \,\middle|\, s_0 = s, a_0 = a\right].$$

Derivation (Bellman expectation). Condition on $(s_0, a_0) = (s, a)$ and split the return at $t = 1$:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)\big[r(s,a) + \gamma V^\pi(s')\big],$$

$$\boxed{Q^\pi(s,a) = \sum_{s'} P(s'|s,a)\left[r(s,a) + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')\right].}$$

A.1.2 Dec-POMDP

Definition A.4 (Dec-POMDP; Bernstein, Zilberstein 1996; Oliehoek & Amato 2016).

$$\mathcal{D} = \Big\langle \mathcal{I}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, P, \{R_i\}, O, \gamma \Big\rangle,$$

with $\mathcal{I} = \{1, \ldots, N\}$; **global** transition kernel $P : \mathcal{S} \times \prod_i \mathcal{A}_i \times \mathcal{S} \to [0,1]$; rewards $R_i : \mathcal{S} \times \prod_j \mathcal{A}_j \times \mathcal{S} \to \mathbb{R}$ (fully cooperative: $R_i \equiv R$, one team reward); observation function $O : \mathcal{S} \times \prod_i \mathcal{A}_i \times \prod_i \mathcal{O}_i \to \Omega$ with samples $o_i \sim O(\cdot|s, \mathbf{a})$; γ.

Joint spaces. $\mathcal{O} = \prod_{i=1}^N \mathcal{O}_i$, $\mathcal{A} = \prod_{i=1}^N \mathcal{A}_i$. For continuous actions $a_i \in \mathbb{R}^M$ the joint action is $\mathbf{a} \in \mathbb{R}^{NM}$; for discrete actions $|\mathcal{A}| = |\mathcal{A}_i|^N$ — **exponential in** N.

Definition A.5 (decentralised policy class).

$$\pi^{\text{dec}} = \Big\{ \boldsymbol{\pi} : \ \exists \pi_i \text{ Markov with } \boldsymbol{\pi}(\mathbf{a}|s) = \prod_{i=1}^N \pi_i(a_i|o_i) \Big\}.$$

Definition A.6 (history / observation). $h_i^t := (o_i^0, a_i^0, o_i^1, a_i^1, \ldots, a_i^{t-1}, o_i^{t-1})$.

Why decentralized execution changes the problem — formal

Proposition A.7 (effective kernel seen by agent i). With the other policies π_j ($j \neq i$) frozen,

$$\bar{P}_i(s'|s, o_i) = \sum_{\mathbf{a}^{-i} \in \prod_{j\neq i} \mathcal{A}_j} \Big[\prod_{j\neq i} \pi_j(a_j|o_j)\Big] P(s'|s, a_i, \mathbf{a}^{-i}).$$

Proposition A.8 (non-stationarity). If agent j updates $\pi_j^{(k)} \to \pi_j^{(k+1)}$, then in general $\bar{P}_i^{(k)} \neq \bar{P}_i^{(k+1)}$, so $V^{(k)}$ no longer equals $V^{\pi_i^{(k)}, \boldsymbol{\pi}^{-i,(k)}}$. The Bellman operator

$$(\mathcal{T}^{\pi_i,(k)} V)(s) = \sum_a \pi_i(a|s) \sum_{s'} \bar{P}_i^{(k)}(s'|s, a)\big[r + \gamma V(s')\big]$$

is **not the same operator across iterations** — the γ-contraction argument of A.3 fails.

Exact dynamic-programming cost. $\bar{P}_i$ requires a sum over $\prod_{j\neq i} |\mathcal{A}_j|$ terms, so exact DP is $O(\prod_i |\mathcal{A}_i|)$ per state. The remedy is a **factored, centralised critic**, not a joint Q.

Example A.9 (2×2 common-reward matrix game — minimal non-stationarity demo). $R = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. With $\pi_1(1) = \pi_2(1) = \varepsilon$, $J(\varepsilon, \varepsilon) = 2\varepsilon^2$. Each agent's greedy best response is to play row/column 1, so simultaneous greedy updates cycle $(1,1) \to (1,2) \to (2,1) \to (2,2) \to (1,1)$ — **non-convergence**. A centralised V breaks the symmetry and has a unique joint optimum at $(1,1)$ where $J = 1$.

A.1.3 Joint belief states

Definition A.10 (belief). $b^t : \mathcal{S} \to [0,1]$, $\sum_s b^t(s) = 1$, the posterior $\Pr(s_t \mid \mathbf{h}^t)$.

Notation convention (used throughout Parts A–D). $\mathcal{S}$ denotes the **global** state space, and s is a global state. Where this document writes $\mathbf{s} = (s^1, \ldots, s^N) \in \mathcal{S}^N$ (A.1.3, A.3), it means either (i) $\mathcal{S} \equiv \mathcal{S}^N$ with **factorised** dynamics $P(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = \prod_i P^i(s'^i|s, \mathbf{a})$ and **localised** observations $O(\mathbf{o}|\mathbf{s}, \mathbf{a}) = \prod_i O_i(o_i|s, a_i)$ — the standard Dec-POMDP assumption, needed for the belief factorisation to be exact — or (ii) $\mathbf{s}$ is a *sufficient statistic / encoding* of $s \in \mathcal{S}$ (e.g. a permutation-invariant aggregation such as a DeepSets/attention encoding). Statements requiring the genuine global state (e.g. the PDL of A.3, which is defined on $(\mathcal{S}, \mathcal{A})$) are read with the true $\mathcal{S}$; statements requiring a *product* structure (belief factorisation, the vectorised critic input of A.11.2) are read with (i). **The two are not interchangeable** — writing $\sum_s b(s)$ while simultaneously assuming $\mathcal{S} = \mathcal{S}^N$ silently changes the dimension of the sum from $|\mathcal{S}|$ to $|\mathcal{S}|^N$.

Definition A.11 (local/decentralised belief). $b_i^t(s) = \Pr(s_t = s \mid h_i^t)$ — the quantity actually available to agent i. Generally $b_i^t \neq b^t$ and b_i^t is **not computable from h_i^t alone** when other agents' observations carry information about s_t.

Definition A.12 (Nash equilibrium in belief space). $\boldsymbol{\pi}^\star = (\pi_1^\star, \ldots, \pi_N^\star)$ satisfies $J(\pi_i', \pi_{-i}^\star) \leq J(\pi_i^\star, \pi_{-i}^\star)$ for all i and all decentralised π_i'.

Complexity (the formal justification for all of MARL). Dec-POMDP is **PEXP-complete** (Bernstein, Givan, Zilberstein 1994) — between NP and EXP. Hardness comes from (a) partial observability forcing belief maintenance over $\mathcal{S}$ and (b) decentralisation forcing the policy onto local histories whose count grows as $\sum_i (|\mathcal{O}_i||\mathcal{A}_i|)^t$.

0.8.2. A.2 POMDP belief update equations

Definition A.13 (POMDP). $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma, \mathcal{O}, O \rangle$, with policy $\pi(a|b)$ on beliefs.

Prediction step (belief update given known action a_t):

$$\boxed{\bar{b}_{t+1}(s') = \sum_{s \in \mathcal{S}} P(s'|s, a_t)\, b_t(s)}$$

Correction step (sensor update given observation o_{t+1}):

$$\boxed{b_{t+1}(s') = \frac{O(o_{t+1}|s', a_t)\, \bar{b}_{t+1}(s')}{\sum_{\tilde{s} \in \mathcal{S}} O(o_{t+1}|\tilde{s}, a_t)\, \bar{b}_{t+1}(\tilde{s})}}$$

Joint / factorised form. With $\mathbf{s}' \in \mathcal{S}^N$, $\mathbf{o}_{t+1} \in \prod_i \mathcal{O}_i$, and *decentralised* $O(\mathbf{o}|s,\mathbf{a}) = \prod_i O_i(o_i|s,a_i)$:

$$\bar{b}_{t+1}(\mathbf{s}') = \sum_{\mathbf{s}} P(\mathbf{s}'|\mathbf{s},\mathbf{a}_t) b_t(\mathbf{s}), \qquad b_{t+1}(\mathbf{s}') = \frac{\prod_i O_i(o_i^{t+1}|s',a_i)\,\bar{b}_{t+1}(\mathbf{s}')}{\sum_{\tilde{\mathbf{s}}} \prod_i O_i(o_i^{t+1}|\tilde{s},a_i)\bar{b}_{t+1}(\tilde{\mathbf{s}})}.$$

Proposition A.14 (sufficiency of the belief). $\Pr(s_{t+1} \mid h^{t+1}) = \Pr(s_{t+1} \mid b_{t+1}, a_{t+1})$, so value functions live on beliefs:

$$V^\pi(b) = \sum_a \pi(a|b) \sum_o \sum_{s'} O(o|s',a)\, T(s'|b,a)\, Q^\pi(s',a).$$

What MAPPO actually does instead (important honesty point). Modern MARL does **not** maintain b^t. Two substitutes: 1. **Observation-history concatenation** (SMAC/GRF practice): input is o_i^t plus previous k observations (approximating $h_i^{t-k:t}$) plus agent-ID one-hot $\mathbf{e}_i \in \{0,1\}^N$; the network *implicitly learns* the belief filter. 2. **RNN actor/critic:** recurrent state $c_i^t = f(o_i^{t-1}, a_i^{t-1}, c_i^{t-1})$ trained so that $c_i^t \approx b_i^t$.

Link to GAE (precise). The λ-return below is a *biased, truncated* estimate of the true advantage. This is what makes the GAE derivation (A.6) the correct estimator for the belief-free setting.

0.8.3. A.3 CTDE

Definition A.15 (CTDE; Lowe et al. 2017; Foerster et al. 2018). A scheme is **Centralized Training with Decentralized Execution** if there exist (i) during **training**, a centralised critic $V_\phi(s^1,\dots,s^N,a^1,\dots,a^{N-1})$ or $V_\phi(s^1,\dots,s^N)$ using information unavailable at execution (true states, other agents' observations/actions); and (ii) during **execution**, a decentralised actor $\pi_\theta(a_i|o_i)$ depending only on o_i. The critic is **discarded at deployment**, so the executed policy class is exactly π^{dec}.

Objective. $\max_\theta \mathbb{E}\big[\sum_t \gamma^t \min(r_t(\theta)\hat{A}_t^{\text{clip}}(\theta),\, r_t(\theta)\hat{A}_t)\big]$ subject to θ being executable with only o_i.

Why CTDE works — the precise argument

Write $\mathcal{S}$ for the **joint/global** state space, $\mathcal{A} = \mathcal{A}^N$, and define

$$\rho_t^{\boldsymbol{\pi}}(s) = (1-\gamma)\sum_{k\ge t}\gamma^{k-t}\Pr_{\boldsymbol{\pi}}(s_k = s), \qquad J(\boldsymbol{\pi}) = \mathbb{E}_{\boldsymbol{\pi}}\Big[\sum_{t\ge 0}\gamma^t r_{t+1}\Big].$$

Lemma A.15a (Performance Difference Lemma, joint form). For any two Markov policies $\boldsymbol{\pi}, \tilde{\boldsymbol{\pi}}$ on $(\mathcal{S},\mathcal{A})$ with $Q^{\boldsymbol{\pi}}, V^{\boldsymbol{\pi}}$ of $\boldsymbol{\pi}$,

$$\boxed{J(\tilde{\boldsymbol{\pi}}) - J(\boldsymbol{\pi}) \;=\; \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim\rho^{\boldsymbol{\pi}}}\,\mathbb{E}_{\mathbf{a}\sim\tilde{\boldsymbol{\pi}}(\cdot|s)}\big[\,A^{\boldsymbol{\pi}}(s,\mathbf{a})\,\big]}$$

with the **joint** advantage $A^{\boldsymbol{\pi}}(s,\mathbf{a}) = Q^{\boldsymbol{\pi}}(s,\mathbf{a}) - V^{\boldsymbol{\pi}}(s)$, $Q^{\boldsymbol{\pi}}(s,\mathbf{a}) = r(s,\mathbf{a}) + \gamma\mathbb{E}_{s'}[V^{\boldsymbol{\pi}}(s')]$. **There is no sum over agents here** — the joint advantage appears exactly once.

Proof. Start from $Q^{\boldsymbol{\pi}} - \frac{1}{1-\gamma}V^{\boldsymbol{\pi}}$ and note it satisfies the Bellman equation for $\boldsymbol{\pi}$: $T^\pi r(s,\mathbf{a}) = Q^\pi(s,\mathbf{a}) - \frac{1}{1-\gamma}V^\pi(s)$. Hence, for $v(s,\mathbf{a}) := Q^{\boldsymbol{\pi}}(s,\mathbf{a}) - \frac{1}{1-\gamma}V^{\boldsymbol{\pi}}(s)$,

$$v(s,\mathbf{a}) = r(s,\mathbf{a}) + \gamma\,\mathbb{E}_{s'}\big[v(s',\mathbf{a}')\big]_{\mathbf{a}'\sim\tilde{\boldsymbol{\pi}}} \qquad \text{(Bellman equation of } \tilde{\boldsymbol{\pi}} \text{ applied to } v).$$

Iterate the $\tilde{\boldsymbol{\pi}}$-expectation k times; by induction the remaining tail is $\gamma^k\mathbb{E}_{\tilde{\pi}}[v(s_k,\mathbf{a}_k)]$ with the $k=0$ base case just stated. Since $\|\mathbb{E}[v]\|_\infty < \infty$ for bounded v and $0 \le \gamma < 1$, letting $k \to \infty$ gives

$$v(s,\mathbf{a}) = \mathbb{E}_{\tilde{\boldsymbol{\pi}}}\Big[\sum_{t=0}^{k-1}\gamma^t r_{t+1}\Big] + \gamma^k\mathbb{E}_{\tilde{\pi}}[v(s_k,\mathbf{a}_k)].$$

Taking $\mathbf{a} \sim \tilde{\boldsymbol{\pi}}(\cdot|s_0)$, dividing by $1-\gamma$ and letting $k \to \infty$:

$$\frac{Q^{\boldsymbol{\pi}}(s_0,\mathbf{a}_0)}{1-\gamma} - \frac{V^{\boldsymbol{\pi}}(s_0)}{1-\gamma} = \frac{1}{1-\gamma}\mathbb{E}_{\tilde{\pi}}\Big[\sum_{t\ge 0}\gamma^t r_{t+1}\Big] = J(\tilde{\boldsymbol{\pi}}).$$

Left-hand side is $J(\boldsymbol{\pi}) + \frac{1}{1-\gamma}\big(Q^{\boldsymbol{\pi}}(s_0,\mathbf{a}_0) - V^{\boldsymbol{\pi}}(s_0)\big) = J(\boldsymbol{\pi}) + \frac{1}{1-\gamma}A^{\boldsymbol{\pi}}(s_0,\mathbf{a}_0)$, which rearranges to the boxed identity. ■

Credit-assigned form (the version that actually needs a sum over i). One may replace the joint $A^{\boldsymbol{\pi}}$ by **any** collection of terms $c_i(s, \mathbf{a})$ satisfying

$$\sum_{i=1}^{N} c_i(s, \mathbf{a}) \equiv A^{\boldsymbol{\pi}}(s, \mathbf{a}) \qquad \text{for all } (s, \mathbf{a}),$$

and the lemma still holds with $\sum_i \mathbb{E}_{\tilde{\pi}}[c_i]$. Two honest instantiations: - $c_i \equiv A^{\boldsymbol{\pi}}$ for a single index i_0 and $c_i \equiv 0$ otherwise (degenerate but exact); - an **orthogonal-projector decomposition**: let L be the vector of per-agent score functions $\nabla_\theta \log \pi_\theta(a^i|s)$, and let $\boldsymbol{\pi}^{-i}$ denote the product of all other agents' policies. Since $\tilde{\pi}_i(a^i|s) = \pi_i(a^i|s)\frac{\tilde{\pi}_i(a^i|s)}{\pi_i(a^i|s)}$, we have exactly

$$\mathbb{E}_{\tilde{\pi}}\Big[\sum_{i=1}^{N} \nabla_{\theta^i} \log \pi_i(a^i|s) \cdot \nabla_{\theta^i} \log \frac{\tilde{\pi}_i(a^i|s)}{\pi_i(a^i|s)} \cdot c_i\Big] = \mathbb{E}_{\tilde{\pi}}\Big[\Big(\sum_i \nabla_{\theta^i} \log \pi_i(a^i|s)\Big) \cdot \sum_i c_i\Big] = \mathbb{E}_{\tilde{\pi}}\Big[\nabla_\theta \log \tilde{\pi}(\mathbf{a}|s) \cdot A^{\boldsymbol{\pi}}(s, \mathbf{a})\Big],$$

where the last step uses the PDL. So the c_i must be *constructed* to sum to the joint advantage (e.g. the HATRPO/HAPPO decomposition of Kuba et al. 2022, Lemma 1, which recursively peels off one agent's marginal advantage); one may not simply write N copies of the same advantage.

[!] Common error. Writing $J(\tilde{\boldsymbol{\pi}}) - J(\boldsymbol{\pi}) = \frac{1}{1-\gamma}\sum_i \mathbb{E}[A_i]$ with A_i all equal to the *same* team advantage inflates the RHS by a factor N and is not a consequence of any theorem.

The three genuine mechanisms. 1. **Variance reduction (provable, unbiased).** For any baseline $b(s, \mathbf{a}_{-i})$ that does not depend on a_i, $\mathbb{E}_\pi[b \nabla \log \pi_i(a_i|s)] = 0$, so subtracting it leaves the gradient unbiased (Proposition A.19 below), while

$$\mathrm{Var}_\pi(\hat{A}_b) = \mathbb{E}[A^2] - \mathbb{E}\big[\mathbb{E}(A \mid \text{baseline-info})^2\big].$$

A richer baseline-info (global state) increases the subtracted term and strictly decreases variance. This is the classical baseline-minimisation argument and it is the *real* reason CTDE helps. 2. **Partial mitigation of non-stationarity (heuristic).** The centralised critic absorbs other agents' policy movement into the function approximator, so the actor sees a less rapidly moving target. This is **not** a convergence guarantee. 3. **Zero benefit at execution.** The actor stays in π^{dec}. Centralised information buys training-time variance reduction only.

Assumptions of CTDE (each can fail in a UAV deployment)

#	Assumption	Failure in UAV-IoT coverage
A1	Simulator can supply global state + all observations/actions in training	No simulator in the real world; global state requires a latency-bounded central fabric, contradicting "decentralised execution"
A2	Centralised critic is a fixed-dim function of global state	Global state has dim $O(NM)$; a fixed-size MLP critic degrades in N (A.11.3)
A3	Decentralised policy is Markov in o_i	Low-altitude occlusion $\Rightarrow$ o_i is not a sufficient statistic; need history/RNN
A4	Reward shared and **stationary**	Persistent coverage with SoC is a *non-stationary-in-time* MDP: R depends on slowly varying battery state, so really $R(s, a, \tau)$. **Genuine gap**
A5	No execution-time communication (or it is folded into o_i)	If agents talk at execution, the problem is a Dec-POMDP *with communication* (Oliehoek et al. 2016) with a richer policy class

#	Assumption	Failure in UAV-IoT coverage
A6	Homogeneous agents share parameters	Heterogeneous fleets (different battery mass/payload) violate it; parameter sharing can be exponentially worse in the worst case (Kuba et al. 2022, Prop. 1)

Explicitly not a guarantee. CTDE gives no monotonic improvement or convergence guarantee by itself. Guarantees come from combining CTDE with a trust region, and for a rigorous statement one must use either a **joint ratio** (TRPO) or the **centralised trust region over the product of independent ratios** (Sun et al. 2023). This distinction is the crux of A.8 and A.10.

0.8.4. A.4 Policy gradients: REINFORCE and the log-derivative trick

A.4.1 Setup

$g^\gamma(\theta) := \nabla_\theta J(\theta)$, $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_t \gamma^t r_{t+1}] = \int p_\theta(\tau) J(\tau) d\tau$ with $J(\tau) = \sum_{t\geq 0} \gamma^t r_{t+1}$.

A.4.2 The log-derivative identity

Lemma A.16 (log-derivative trick / score function). For differentiable density p_θ and integrable f:

$$\nabla_\theta \mathbb{E}_{X\sim p_\theta}[f(X)] = \int f(x) \nabla_\theta p_\theta(x) dx = \int f(x) p_\theta(x) \nabla_\theta \log p_\theta(x) dx = \mathbb{E}_{X\sim p_\theta}\big[f(X) \nabla_\theta \log p_\theta(X)\big].$$

Proof. Chain: $\nabla_\theta \int f p_\theta = \int f \nabla_\theta p_\theta$; then $\nabla_\theta p_\theta = p_\theta \cdot \nabla_\theta p_\theta / p_\theta = p_\theta \nabla_\theta \log p_\theta$ using $\nabla_\theta \log p_\theta = \nabla_\theta p_\theta / p_\theta$. □

Corollary A.17 (score of categorical policy). For $\pi_\theta(a|s) = \mathrm{softmax}_\theta(f_\theta(s,\cdot))_a$:

$$\boxed{\nabla_\theta \log \pi_\theta(a|s) = \nabla_\theta f_\theta(s,a) - \sum_{\tilde{a}} \pi_\theta(\tilde{a}|s) \nabla_\theta f_\theta(s,\tilde{a})}$$

For linear logits $f_\theta = \theta^\top \phi(s,a)$: $\nabla_\theta \log \pi_\theta(a|s) = \phi(s,a) - \sum_{\tilde{a}} \pi_\theta(\tilde{a}|s) \phi(s,\tilde{a})$.

Corollary A.18 (score of a diagonal Gaussian; used for UAV continuous control). Let $\pi_\theta(a|s) = \prod_{j=1}^M \mathcal{N}\big(a_j;\, \mu_{\theta,j}(s),\, \sigma^2_{\theta,j}(s)\big)$ with $\mu_{\theta,j}, \sigma_{\theta,j} > 0$ depending on s. Then, componentwise,

$$\frac{\partial}{\partial \theta} \log \pi_\theta(a|s) = \sum_{j=1}^M \left[\underbrace{\frac{a_j - \mu_{\theta,j}(s)}{\sigma^2_{\theta,j}(s)} \nabla_\theta \mu_{\theta,j}(s)}_{\text{mean term}} + \underbrace{\left(\frac{(a_j - \mu_{\theta,j}(s))^2}{\sigma^2_{\theta,j}(s)} - 1 \right) \nabla_\theta \log \sigma_{\theta,j}(s)}_{\text{scale term}} \right].$$

Derivation. The j-th factor contributes $-\frac{(a_j-\mu_j)^2}{2\sigma_j^2} - \log \sigma_j$. Differentiate the first term with the chain rule: $\nabla_\theta \sigma_j^{-2} = -2\sigma_j^{-3} \nabla_\theta \sigma_j$, so $\nabla_\theta\big[-\frac{1}{2}(a_j-\mu_j)^2 \sigma_j^{-2}\big] = \frac{(a_j-\mu_j)^2}{\sigma_j^2} \nabla_\theta \log \sigma_j + \frac{a_j-\mu_j}{\sigma_j^2} \nabla_\theta \mu_j$; and $\nabla_\theta[-\log \sigma_j] = -\nabla_\theta \log \sigma_j$. The two $\nabla_\theta \log \sigma_j$ terms combine to the coefficient $\big((a_j-\mu_j)^2/\sigma_j^2 - 1\big)$. □

Two corollaries used later. 1. **With state-independent σ (MAPPO's UAV velocity commands).** The scale term vanishes and the score reduces to the mean term, so $\nabla_\theta \log \pi_\theta(a|s) = \frac{a-\mu_\theta(s)}{\sigma^2} \odot \nabla_\theta \mu_\theta(s)$. 2. **Zero conditional mean (needed by Prop. A.21 and the TRPO bound).** $\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a|s)] = 0$, since for each j the mean term has mean 0 by symmetry of the Gaussian and the scale term has expectation $\frac{\mathbb{E}[(a_j-\mu_j)^2]}{\sigma_j^2} - 1 = \frac{\sigma_j^2}{\sigma_j^2} - 1 = 0$.

[!] Common error. The scale coefficient is $\frac{(a_j-\mu_j)^2}{\sigma_j^2} - 1$ — it contains **no** log of any residual. Expressions such as $\log \frac{a_j-\mu_j}{\sigma_j}$ arise from integrating a Gaussian score and are not scores.

A.4.3 REINFORCE — full derivation

Using Lemma A.16 with $X = \tau$: $g^\gamma(\theta) = \mathbb{E}_\theta\big[J(\tau)\nabla_\theta \log p_\theta(\tau)\big]$. Factor $p_\theta(\tau) = \prod_t \pi_\theta(a_t|s_t)P(s_{t+1}|s_t,a_t)$; the P terms are θ-independent and log of a product is a sum, so $\nabla_\theta \log p_\theta(\tau) = \sum_{t\geq 0}\nabla_\theta \log \pi_\theta(a_t|s_t)$. Therefore

$$g^\gamma(\theta) = \mathbb{E}_\theta\Big[\Big(\sum_{t\geq 0}\gamma^t r_{t+1}\Big)\Big(\sum_{t\geq 0}\nabla_\theta \log \pi_\theta(a_t|s_t)\Big)\Big].$$

The expected score of a non-selected action vanishes ($\sum_a \pi(a|s)\nabla \log \pi(a|s) = \nabla \sum_a \pi(a|s) = 0$), so

$$\boxed{g^\gamma(\theta) = \sum_{t\geq 0}\gamma^t\, \mathbb{E}_\theta\big[r_{t+1}\nabla_\theta \log \pi_\theta(a_t|s_t)\big]}$$

— **REINFORCE** (Williams 1992). Sample estimate with horizon T: $g^\gamma \approx \sum_{t<T}\gamma^t r_{t+1}\nabla_\theta \log \pi_\theta(a_t|s_t)$.

Variance. Grows as $O(T^2)$ in the horizon (each of T score terms times a partial return of order T; $O(T^2)$ cross-terms). This motivates $V/Q/A$ entirely.

A.4.4 Baseline property (unbiasedness)

Proposition A.19 (baseline invariance). For any $b(s)$ depending on state only,

$$\mathbb{E}_{\pi_\theta}\Big[\sum_t \gamma^t b(s_t)\nabla_\theta \log \pi_\theta(a_t|s_t)\Big] = \sum_t \gamma^t \mathbb{E}\big[b(s_t)\underbrace{\mathbb{E}_{a_t\sim\pi}\nabla_\theta \log \pi(a_t|s_t)}_{=\nabla_\theta \sum_a \pi(a|s)=0}\big] = 0.$$

Hence the estimator with $(r_{t+1} - b(s_t))$ is **unbiased**. The same holds for a baseline $b(s, \mathbf{a}_{-i})$ independent of a_i — the multi-agent CTDE version. This is the entire justification for actor–critic: V_ϕ is a *learned* state-only baseline.

0.8.5. A.5 Value, Q, advantage $A = Q - V$

Definition A.20. $A^\pi(s,a) := Q^\pi(s,a) - V^\pi(s)$.

Decomposition intuition. $Q^\pi(s,a) = \underbrace{V^\pi(s)}_{\text{state quality}} + \underbrace{A^\pi(s,a)}_{\text{action quality}}$; the first term is a-independent and carries no information about *which action to take*.

Proposition A.21 (formal justification). $\mathbb{E}_{a\sim\pi(\cdot|s)}\big[V^\pi(s)\nabla_\theta \log \pi_\theta(a|s)\big] = V^\pi(s)\nabla_\theta \sum_a \pi_\theta(a|s) = 0$. Hence Q^π and A^π give the **same expected gradient** (both equal a baseline plus an a-independent constant), but A^π has strictly lower variance. □

Exact relationship to the TD residual. $A^\pi(s_t,a_t) = r_{t+1} - V^\pi(s_t) + \gamma V^\pi(s_{t+1}) - V^\pi(s_t) = \delta_t^\pi$ with $\delta_t^\pi := r_{t+1} + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$. When $V = V^{\pi,\gamma}$ is exact, δ_t^π is an **unbiased** one-step estimator of $A^{\pi,\gamma}$:

$$\mathbb{E}_{s_{t+1}}[\delta_t] = \mathbb{E}\big[r_{t+1} + \gamma V^{\pi,\gamma}(s_{t+1})\big] - V^{\pi,\gamma}(s_t) = Q^{\pi,\gamma}(s_t,a_t) - V^{\pi,\gamma}(s_t) = A^{\pi,\gamma}(s_t,a_t).$$

(Verified verbatim: Schulman et al. 2016, Eq. 10.)

Multi-agent. For a shared team reward, $A^{\boldsymbol{\pi}}(s,\mathbf{a}) = Q^{\boldsymbol{\pi}}(s,\mathbf{a}) - V^{\boldsymbol{\pi}}(s)$; MAPPO computes **one** $A^{\boldsymbol{\pi}}$ and reuses it for every agent's ratio — a deliberate choice, replaced by the joint decomposition of Kuba et al. (2022) in HATRPO/HAPPO (A.9.5).

0.8.6. A.6 GAE — full derivation, bias–variance, TD(λ) equivalence

Reference: Schulman, Moritz, Levine, Jordan, Abbeel, *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, ICLR 2016, arXiv:1506.02438 (earlier: NIPS 2015, "Generalized Advantage Estimation for Policy Gradients").

A.6.1 Construction

Definition A.22 (k-step estimator). With the reward-indexing convention fixed throughout A.1–A.12 as r_{t+1} = reward received on the transition $s_t \to s_{t+1}$,

$$\hat{A}_t^{V,k} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V, \qquad \boxed{\delta_t^V := r_{t+1} + \gamma V(s_{t+1}) - V(s_t)}$$

(Schulman et al. 2016 write $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$ with the *other* convention r_t = reward on $s_t \to s_{t+1}$; the two are the same formula with the index shifted by one, and mixing them is the usual source of off-by-one GAE implementations.)

Derivation (telescoping). Expanding: $\hat{A}_t^{V,k} = \sum_{l<k} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}) - V(s_t)$, because the $\gamma^l V(s_{t+l+1})$ terms cancel against the $\gamma^{l+1} V(s_{t+l+1})$ of the next summand. □

Definition A.23 (GAE). For $\lambda \in [0,1]$: $\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} := (1-\lambda) \sum_{k \geq 1} \lambda^{k-1} \hat{A}_t^{V,k}$.

Full derivation of the closed form. Swap the order of summation and collect by δ_{t+l}^V. For fixed l, the contributing values of k are $k = l+1, l+2, \ldots$, so

$$\sum_{k=l+1}^{\infty} \lambda^{k-1} \gamma^l = \gamma^l \sum_{j \geq 0} \lambda^{l+j} = \gamma^l \frac{\lambda^l}{1-\lambda} = \frac{(\gamma\lambda)^l}{1-\lambda}.$$

Hence, cancelling $(1-\lambda)$:

$$\boxed{\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}^V}$$

(Schulman et al. 2016, Eq. 16 — verified.)

Boundary cases. $\lambda = 1$: $\hat{A}_t = \sum_{l \geq 0} \gamma^l \delta_{t+l}^V = \sum_{l \geq 0} \gamma^l r_{t+l} - V(s_t)$ — MC return minus baseline, **unbiased for any** V, high variance. $\lambda = 0$: $\hat{A}_t = \delta_t^V$ — one-step TD residual, **biased unless** $V = V^{\pi,\gamma}$, low variance.

Resulting estimator. $\hat{g}^{\mathrm{GAE}} = \mathbb{E}\big[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}^V\big]$; unbiased iff $\lambda = 1$.

A.6.2 Bias–variance as a function of λ — two *distinct* effects

Schulman et al. 2016 3 stress that γ and λ play different roles; conflating them is the usual error.

1. $\gamma < 1$ **biases the gradient estimate regardless of V accuracy.** Reason: $g^\gamma \neq g$ — the discounted objective is a *different* objective. Also γ sets the scale of $V^{\pi,\gamma}$: for per-step reward of order R, $V \approx R/(1-\gamma)$, which inflates advantage magnitudes and gradient scale.
2. $\lambda < 1$ **biases only when V is inaccurate.** Writing $\varepsilon_t := V^{\pi,\gamma}(s_t) - V(s_t)$ and $\delta_t^\pi := r_{t+1} + \gamma V^{\pi,\gamma}(s_{t+1}) - V^{\pi,\gamma}(s_t) = A^{\pi,\gamma}(s_t, a_t)$:
$$\hat{A}_t^{\mathrm{GAE}} - A^{\pi,\gamma}(s_t, a_t) = \sum_{l \geq 0} (\gamma\lambda)^l \big(\delta_{t+l}^V - \delta_{t+l}^\pi\big),$$
so the bias is an exponentially-weighted average of the value errors:
$$\mathrm{Bias} \asymp \sum_{l \geq 0} (\gamma\lambda)^l \varepsilon_{t+l}.$$

Monotone statements (rigorous). $\lambda \uparrow 1 \Rightarrow$ variance $\uparrow$, bias $\to 0$. $\lambda \downarrow 0 \Rightarrow$ variance $\downarrow$, bias $\uparrow$. Effective bootstrap horizon: weights $(\gamma\lambda)^l$ decay with time constant $\approx (1-\gamma\lambda)^{-1}$, so $\lambda \approx 1 - \frac{1-\gamma}{\gamma} k$ gives an effective k-step backup. Empirically (MuJoCo): best $\lambda \in [0.9, 0.99]$, best $\gamma \approx 0.99$; $\lambda = 0$ **is disastrous in high dimensions**, $\lambda = 1$ **is very noisy**. MAPPO defaults: $\gamma = 0.99$, $\lambda = 0.95$.

A.6.3 TD(λ) equivalence theorem

Theorem A.24 (λ-return minus V = GAE). Define the n-step bootstrapped return and the λ-return by

$$G_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}), \qquad G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}.$$

Then, **identically as random variables**,

$$\boxed{G_t^\lambda - V(s_t) = \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V}$$

Proof. (i) **Telescoping identity.** By the derivation above, $\hat{A}_t^{V,n} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}^V$, and separately

$$\sum_{l=0}^{n-1} \gamma^l r_{t+l+1} + \gamma^n V(s_{t+n}) - V(s_t) = \sum_{l=0}^{n-1} \gamma^l \big(r_{t+l+1} + \gamma V(s_{t+l+1}) - V(s_{t+l})\big) = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}^V.$$

Hence

$$G_t^{(n)} - V(s_t) = \Big[\sum_{l=0}^{n-1} \gamma^l r_{t+l+1} + \gamma^n V(s_{t+n})\Big] - V(s_t) = \hat{A}_t^{V,n}. \tag{A.24.1}$$

(ii) Substitute (A.24.1) into the λ-return definition:

$$G_t^\lambda - V(s_t) = (1-\lambda)\sum_{n\geq 1} \lambda^{n-1} \hat{A}_t^{V,n} \overset{\text{Def. A.23}}{=} \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}. \qquad \blacksquare$$

Note that the identity holds **pointwise on trajectories**, not merely in expectation — this is an algebraic rewriting, which is why GAE is exactly the λ-return advantage and not merely similar to it.

Independent check via the λ-return recursion. With $G_t^{(0)} = V(s_t)$ and $G_t^{(n)} = r_{t+1} + \gamma G_{t+1}^{(n-1)}$,

$$\begin{aligned}
G_t^\lambda &= (1-\lambda)\sum_{n\geq 1} \lambda^{n-1} G_t^{(n)} \\
&= (1-\lambda)\sum_{n\geq 1} \lambda^{n-1}\Big(r_{t+1} + \gamma G_{t+1}^{(n-1)}\Big) = r_{t+1} + \gamma(1-\lambda)\sum_{m\geq 0} \lambda^m G_{t+1}^{(m)} \qquad (m := n-1) \\
&= r_{t+1} + \gamma(1-\lambda)\Big[\sum_{m\geq 1} \lambda^{m-1} G_{t+1}^{(m)} + \sum_{m\geq 0} \lambda^m G_{t+1}^{(m)} - \sum_{m\geq 1} \lambda^{m-1} G_{t+1}^{(m)}\Big] \\
&= r_{t+1} + \gamma(1-\lambda)\Big[\underbrace{\frac{G_{t+1}^\lambda}{1-\lambda}}_{m\geq 1 \text{ part}} + \underbrace{\lambda\frac{G_{t+1}^\lambda}{1-\lambda}}_{m\geq 1 \text{ part}} - \sum_{m\geq 1} \lambda^{m-1} G_{t+1}^{(m)}\Big] \\
&= r_{t+1} + \gamma\big[G_{t+1}^\lambda - (1-\lambda)V(s_{t+1})\big].
\end{aligned}$$

Now set $A_t^\lambda := G_t^\lambda - V(s_t)$. The recursion gives

$$A_t^\lambda = r_{t+1} + \gamma\big[A_{t+1}^\lambda + V(s_{t+1})\big] - \gamma(1-\lambda)V(s_{t+1}) - V(s_t) = r_{t+1} + \gamma A_{t+1}^\lambda + \lambda\gamma V(s_{t+1}) - V(s_t) = \delta_t^V + \gamma\lambda A_{t+1}^\lambda.$$

This is **exactly the GAE recursion** (A.6.4), so unrolling from $A_N^\lambda \to 0$ gives $A_t^\lambda = \sum_{l\geq 0} (\gamma\lambda)^l \delta_{t+l}^V$. $\square$

Why the reindexing step is legitimate (the one place people go wrong): $\sum_{m\geq 0} \lambda^m G_{t+1}^{(m)} = \lambda \sum_{m\geq 0} \lambda^m G_{t+1}^{(m)}$ would be wrong; the correct statement is that the $m \geq 1$ part of $\sum_{m\geq 0} \lambda^m G^{(m)}$ equals $\lambda \cdot G^\lambda/(1-\lambda)$ while the $m = 0$ term contributes $\lambda^0 V(s_{t+1}) = V(s_{t+1})$. Hence $\sum_{m\geq 0} \lambda^m G_{t+1}^{(m)} = \frac{\lambda}{1-\lambda} G_{t+1}^\lambda + V(s_{t+1})$, and multiplying by $(1-\lambda)$ gives $\lambda G_{t+1}^\lambda + (1-\lambda)V(s_{t+1})$, which is the third line above.

The precise statement, and the widespread misstatement. TD(λ) *estimates the value function* $G^\lambda \to V^\pi$ (biased for $\lambda < 1$, variance $\to 0$ as $\lambda \to 0$); GAE *estimates the advantage* $G^\lambda - V \to A^\pi$ (unbiased at $\lambda = 1$, variance $\to 0$ as $\lambda \to 0$). Schulman et al. 2016 4: *"The construction we used above is closely analogous to the one used to define TD(λ) [...] however TD(λ) is an estimator of the value function whereas here we are estimating the advantage function."* Note the parameter roles differ: TD(λ) bootstraps with discount γ and trace decay λ; GAE uses the **single** effective factor $\gamma\lambda$, so λ_{TD} and λ_{GAE} are not interchangeable numerically.

Interpretation (the point to make in a report). TD(λ) estimates V; GAE estimates A **with the same weighting**. So "use GAE with λ" = "use a TD(λ) backup, applied to the advantage instead of the value." Verbatim (Schulman et al. 2016): *"The construction we used above is closely analogous to the one used to define TD(λ) [...] however TD(λ) is an estimator of the value function whereas here we are estimating the advantage function."*

[!] Do not conflate GAE with potential-based reward shaping (Ng–Harada–Russell). They *look* alike and it is tempting to write "GAE is potential-based shaping with $\Phi = V$, discount $\gamma\lambda$," but that is **false**, and the difference is instructive: - PBRS with potential Φ and discount $\tilde{\gamma}$ produces the *reward* $\tilde{r}_{t+1} = r_{t+1} + \tilde{\gamma}\Phi(s_{t+1}) - \Phi(s_t)$. For $\tilde{\gamma} = \gamma$ and $\Phi = V$ this is $r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ — a reward whose structure matches δ^V but whose *discounted return is unchanged in value*: by the Ng–Harada–Russell result, $J_{\tilde{r}}(\pi) = J_r(\pi) - \frac{1}{1-\gamma}\mathbb{E}[\Phi(s_0)]$, so the shaping **does not bias the objective for any** λ. That is the whole point of PBRS. - GAE instead forms a $\gamma\lambda$-weighted sum of δ^V's. Setting $\tilde{\gamma} = \gamma\lambda \neq \gamma$ would break PBRS's policy-invariance requirement, and no potential makes the GAE *return* policy-invariant; GAE is a **variance-reduction estimator of the gradient**, not a redefinition of the objective.

The correct one-line relationship: the GAE recursion $\hat{A}_t = \delta_t^V + \gamma\lambda\hat{A}_{t+1}$ and the λ-return recursion $G_t^\lambda = r_{t+1} + \gamma G_{t+1}^\lambda - \gamma(1-\lambda)V(s_{t+1})$ differ only in where $\hat{A}$ sits relative to the baseline V; **subtracting V from a TD(λ) quantity yields GAE** (proved in A.6.3). No reward-shaping claim is needed, and none is correct.

A.6.4 Finite-episode form (what is implemented)

$$V(H_N) = 0, \qquad \delta_t^V = R_{t+1} + \gamma V(H_{t+1}) - V(H_t), \qquad \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{N-t-1} (\gamma\lambda)^l \delta_{t+l}^V.$$

Computed by the **reverse-time recurrence**:

for t = T-1, ..., 0: adv[t] = delta[t] + gamma*lam*(1-done[t+1])*adv[t+1] ret[t] = adv[t] + value[t] # value-normalized variant, A.9.4

Cost $O(T)$ per environment step — **no extra asymptotic cost** over the forward pass. This is the computational reason GAE is used universally.

0.8.7. A.7 TRPO — full math

Reference: Schulman, Wolski, Dhariwal, Radford, Klimov, *Trust Region Policy Optimization*, ICML 2015, arXiv:1502.05477 (original 2013 version arXiv:1305.5991, using natural gradients / CG with Fisher-vector products).

A.7.1 The theory

PDL. $J(\tilde{\pi}) - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim\rho^{\pi}}\big[\sum_a \tilde{\pi}(a|s) A^{\pi}(s,a)\big]$. *Proof sketch.* Substitute $A^{\pi} = Q^{\pi} - V^{\pi}$ and use $\mathbb{E}_{\tilde{\pi}}[A^{\pi}(s,a)] = Q^{\tilde{\pi}}(s,a) - V^{\pi}(s)$; the recursion $V^{\pi} = \mathbb{E}_{\pi}[Q^{\pi}]$ telescopes against the visit distribution. □

Definition A.25a (Surrogate value function). The Fisher information matrix is

$$F_{\theta} := \mathbb{E}_{s\sim\rho^{\theta_{\mathrm{old}}},\mathbf{a}\sim\pi_{\theta_{\mathrm{old}}}}\Big[\nabla_{\theta}\log\pi_{\theta_{\mathrm{old}}}(\mathbf{a}|s)\,\nabla_{\theta}\log\pi_{\theta_{\mathrm{old}}}(\mathbf{a}|s)^{\top}\Big].$$

Note $F \succeq 0$ automatically (it is a covariance of scores) and, since $\mathbb{E}[\nabla_{\theta}\log\pi] = 0$ (Prop. A.21), $\mathbb{E}[F]$ has a **zero eigenvalue along the constant direction**: for softmax over linear logits with feature dim $m < |\mathcal{A}|$, $F = \mathbb{E}_s\big[\Phi^{\top}\mathrm{Cov}_{a\sim\pi}(\phi(s,a))\Phi\big]$ is singular, so TRPO works in the subspace orthogonal to the null space. Let $x := \nabla_{\theta}\log\pi_{\theta_{\mathrm{old}}}(\mathbf{a}|s)$ (a *random* column vector, not a matrix) and write

$$g := \mathbb{E}_s[x], \qquad s^{\star} := \sqrt{\frac{\delta}{g^{\top}F^{-1}g}}\,F^{-1}g.$$

(On the subspace where F is invertible; otherwise project g onto $\mathrm{null}(F)^{\perp}$ first. This is why implementations add a small damping term $\epsilon\mathbb{I}$.)

> **Theorem A.25b (TRPO lower bound; Schulman et al. 2015, Thm 4 / Lemma 6).** There exist constants C depending only on γ and the reward bound (with $|r| \le R_{\max}$) such that for every policy $\tilde{\pi}$,
>
> $$J(\tilde{\pi}) \;\ge\; \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim\rho^{\theta_{\mathrm{old}}}}\mathbb{E}_{\mathbf{a}\sim\pi_{\theta_{\mathrm{old}}}}\big[\hat{A}^{\theta_{\mathrm{old}}}(s,\mathbf{a})\big] \;+\; \frac{1}{1-\gamma}\,\mathbb{E}_s\big[x^{\top}F^{-1}x\big] \;-\; \frac{C}{1-\gamma}\,\mathbb{E}_s\big[x^{\top}Fx\big].$$

Derivation. (i) By the PDL with $\tilde{\pi}$,

$$J(\tilde{\pi}) - J(\pi_{\theta_{\mathrm{old}}}) = \frac{1}{1-\gamma}\mathbb{E}_s\mathbb{E}_{\mathbf{a}\sim\pi_{\theta_{\mathrm{old}}}}\big[A^{\pi_{\mathrm{old}}}(s,\mathbf{a})\big],$$

and by definition $\mathbb{E}_{\pi_{\mathrm{old}}}[A^{\pi_{\mathrm{old}}}] = 0$, so the first term vanishes identically — a lower bound on the improvement must come from the *second-order* part. (ii) The key Taylor expansion (Schulman et al. 2015, Lemma 3) is for the **surrogate** objective $L(\theta) = \mathbb{E}_s[x^{\top}(\theta - \theta_{\mathrm{old}})]$, using the "score" analogue of f-divergence results:

$$L(\theta) - L(\theta_{\mathrm{old}}) = \mathbb{E}\left[\frac{1}{2}\Delta\theta^{\top}F\Delta\theta\right] + \mathbb{E}\left[\sum_{i\ge 3}\frac{\Delta\theta^{\top}\nabla^{i-1}\pi_{\theta_{\mathrm{old}}}}{\|x\|}\,\frac{\Delta\theta^{i-1}}{i!}x\right], \qquad \Delta\theta := \theta - \theta_{\mathrm{old}}.$$

The remainder is bounded by $\frac{C}{6}\mathbb{E}[(\Delta\theta^{\top}F\Delta\theta)^{3/2}]$-type terms, and is made $O(\Delta\theta^2)$ after multiplying by the advantage; the standard treatment (Schulman et al. 2015 Eq. 9 + the surrounding discussion) absorbs it into $\frac{C}{2}\mathbb{E}_s[x^{\top}Fx]$ by the inequality $\mathbb{E}[\|F^{1/2}\Delta\theta\|^2 \cdot \|F^{1/2}\Delta\theta\|] \le C\|F^{1/2}\Delta\theta\|^2$ valid inside a trust region. (iii) For the maximiser $\Delta\theta^{\star} = \sqrt{\delta}\,F^{-1}g/\|F^{-1}g\|$, substitute directly: the gain term is

$$\mathbb{E}_s\big[(\Delta\theta^{\star})^{\top}x\big] = \sqrt{\delta}\,\frac{g^{\top}F^{-1}g}{\sqrt{\delta/(g^{\top}F^{-1}g)}\cdot\|F^{-1}g\|}\Big|_{\mathrm{exact}} = \frac{\delta\,(g^{\top}F^{-1}g)}{\|F^{-1}g\|^2}\cdot\frac{\|F^{-1}g\|}{\sqrt{\delta}}\cdot\frac{1}{\sqrt{\delta}} \;\Rightarrow\; \sqrt{\delta}\,\|F^{-1/2}g\|,$$

and since $g = F(F^{-1}g)$, the quadratic penalty is $\frac{C}{2}\mathbb{E}_s[x^\top Fx]\cdot\frac{\delta}{g^\top F^{-1}g} = \frac{C\delta}{2}$. Net gain $= \sqrt{\delta}\,\|F^{-1/2}g\| - \frac{C\delta}{2} > 0$ for $\delta < \frac{2}{C}\|F^{-1/2}g\|^2 = \frac{2g^\top F^{-1}g}{C}$. Hence

$$\boxed{J\big(\pi_{\theta_{\text{old}}+\Delta\theta^\star}\big) \;\geq\; J(\pi_{\theta_{\text{old}}}) \;+\; \frac{1}{1-\gamma}\Big(\sqrt{\delta}\,\big\|F^{-1/2}g\big\| - \tfrac{C\delta}{2}\Big) \;>\; J(\pi_{\theta_{\text{old}}})}$$

[!] Corrections to the common malformed version of this theorem. (i) x is a *vector* $\nabla_\theta \log\pi(\mathbf{a}|s)$, and F^{-1} is $|\theta|\times|\theta|$; the bound is $\mathbb{E}_s[x^\top F^{-1}x] - \frac{C}{2}\mathbb{E}_s[x^\top Fx]$, **not** $\mathbb{E}[x^\top F^{-1}x] - \frac{C}{2}\mathbb{E}[x^\top F^{-1}x]$ — the two F occurrences must differ, and the dimensions only match with F in the penalty. (ii) The constant multiplies $\mathbb{E}_s[x^\top Fx]$, i.e. the *Fisher quadratic form in the mean-field/Fisher metric*, not the KL. (iii) Substituting $\Delta\theta = CF^{-1}x$ pointwise (with x random) does **not** give "gain C, penalty $C/2$"; the substitution must be made with the *deterministic* vector g (or the mean of the score-weighted quantities), as done above. (iv) C depends on γ and the reward bound, not on θ.

This is the mathematical content of "monotonic improvement", and it is a guarantee over the *true* objective J — not over the surrogate. It holds only in the region where the third-order remainder is controlled.

A.7.2 The TRPO objective

$$\max_\theta\; L_{\text{sur}}(\theta,\theta_{\text{old}}) = \mathbb{E}_{s\sim\rho^{\theta_{\text{old}}},\mathbf{a}\sim\pi_{\theta_{\text{old}}}}\big[r(\mathbf{a}|s)A^{\theta_{\text{old}}}(s,\mathbf{a})\big]$$
$$\text{s.t.}\quad \mathbb{E}_{s\sim\rho^{\theta_{\text{old}}}}\big[D_{\text{KL}}\big(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s)\big)\big] \leq \delta.$$

Remark (exact vs average KL — a real subtlety). The bound A.25 is stated in terms of the *second-order Taylor quantity* $\mathbb{E}[x^\top Fx]$, not the average KL itself. The Bernstein–Fisher expansion $D_{\text{KL}}(p_\theta\|p_{\theta_{\text{old}}}) = \frac{1}{2}(\theta-\theta_{\text{old}})^\top F(\theta-\theta_{\text{old}}) + O(\|\Delta\theta\|^3)$ (valid when the score has zero conditional mean) makes the KL constraint **equivalent to a trust region in the Fisher metric**; TRPO is therefore a natural-gradient method with step $\Delta\theta \propto F^{-1}\nabla L$. In practice Schulman et al. 2015 present both: constrain the **average** $\text{KL}(\bar\pi\|\pi)$ plus a **cumulative** per-state penalty $\beta\sum_s \text{KL}(\pi_{\text{old}}(s)\|\pi_\theta(s))$ with backtracking line search, or constrain the **maximum** per-state KL.

A.7.3 Solving it: conjugate gradient + Fisher-vector products

$\max_\Delta g^\top\Delta$ s.t. $\frac{1}{2}\Delta^\top F\Delta \leq \delta$ has solution $\Delta^\star = \sqrt{\frac{\delta}{g^\top F^{-1}g}}F^{-1}g$. Inverting $F\in\mathbb{R}^{|\theta|\times|\theta|}$ is infeasible. Lagrangian $\mathcal{L} = g^\top\Delta - \frac{\alpha}{2}(\Delta^\top F\Delta - \delta)$, $\nabla_\Delta\mathcal{L} = g - \alpha F\Delta = 0 \Rightarrow \Delta = \alpha^{-1}F^{-1}g$, with $\alpha = \sqrt{\delta/(g^\top F^{-1}g)}$.

Algorithm (Schulman et al. 2015, Alg. 1–2). 1. Batch of N samples; $\hat g = \frac{1}{N}\sum_i \nabla\log\pi(\mathbf{a}_i|s_i)\hat A_i$; accumulator $\hat X = \frac{1}{N}\sum_i \nabla\log\pi_i\hat A_i\hat A_i^\top$. 2. **Fisher-vector product** $\hat Fv \approx \frac{1}{N}\sum_i \nabla\log\pi(\mathbf{a}_i|s_i)\big(\nabla\log\pi(\mathbf{a}_i|s_i)^\top v\big)$ — **one backward pass** suffices, because a Hessian–vector product with weights removed equals a gradient of a weighted log-likelihood. 3. Solve $Fx = g$ approximately by **conjugate gradient**, K iterations (typically 10), using the FVP as operator. 4. $\hat s = \sqrt{\frac{\delta}{g^\top x}}$; step $\Delta x = \hat s x$. 5. Shrink $\hat s$ by $\alpha := 0.5$ until $\mathbb{E}_s[D_{\text{KL}}] \leq \delta$; then **line search** on the actual objective; accept. 6. Gradient step on V_ϕ with Huber loss on $(V_\phi(s_t) - \hat R_t)^2$.

Cost: $O(K)$ gradient-equivalent passes with $K \ll |\theta|$ — a small constant multiple of PPO's cost. This is exactly why PPO (first-order) is the practical default.

A.7.4 Continuous control: closed-form KL

For $\pi_\theta(\mathbf{a}|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta^2\mathbb{I})$ with **state-independent** σ:

$$D_{\text{KL}}(\pi_{\text{old}}\|\pi_\theta) = \frac{1}{2}\sum_{j=1}^{M}\left[\log\frac{\sigma_{\text{old},j}^2}{\sigma_{\theta,j}^2} + \frac{\sigma_{\theta,j}^2}{\sigma_{\text{old},j}^2} - 1\right].$$

With σ held fixed during the update this reduces to the closed form

$$D_{\text{KL}} = \frac{1}{2}\sum_{j=1}^{M}\frac{(\mu_{\theta,j} - \mu_{\text{old},j})^2}{\sigma_j^2},$$

— **no sampling, no FVP**. This is a large part of why continuous-control TRPO/PPO is cheap, and it is the formula to use for UAV velocity commands with fixed exploration noise. If σ is state-dependent and updated, the full log-difference form above is required.

0.8.8. A.8 PPO

Reference: Schulman, Wolski, Dhariwal, Radford, Klimov, *Proximal Policy Optimization Algorithms*, arXiv:1707.06347, 2017.

A.8.1 From TRPO to first order

Step 1 (importance weighting). $r_t(\theta) = \pi_\theta(\mathbf{a}_t|s_t)/\pi_{\theta_{\text{old}}}(\mathbf{a}_t|s_t)$.

Step 2 (conservative objective). TRPO's penalty relaxation is $\max_\theta L_{\text{sur}}(\theta) - \beta D_{\text{KL}}(\pi_{\text{old}}\|\pi_\theta)$. Using $x \le \max(x, 1+\epsilon)$ ("cap the improvement") and $\min(x, 1+\epsilon)$ ("pessimistic"), PPO defines

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big]}$$

Step 3 (why clipping is conservative — the full case analysis). Put $u = r_t(\theta)$.

Case $\hat{A}_t > 0$ (good action; we want to raise its probability). $\min(u\hat{A}, \text{clip}(u)\hat{A}) = \hat{A}\min(u, 1+\epsilon)$. For $u \le 1+\epsilon$ the objective equals $u\hat{A}$ (ordinary surrogate, gradient flows, and the ratio may move *down* to $1-\epsilon$ without clipping). For $u > 1+\epsilon$ it is **constant** — gradient **zero**. So for a good action, once the ratio exceeds $1+\epsilon$, the update is **cut off**.

Case $\hat{A}_t < 0$ (bad action; we want to lower its probability). The min picks the more negative, $\hat{A}\max(u, 1-\epsilon)$. For $u \ge 1-\epsilon$ the objective equals $u\hat{A}$; for $u < 1-\epsilon$ it is constant — **zero gradient**.

> **Proposition A.26 (pessimistic lower bound).** $L^{\text{CLIP}}(\theta) \le L_{\text{sur}}(\theta)$ for every θ. *Proof.* Four cases: ($\hat{A} > 0$, $u > 1+\epsilon$): CLIP $= \hat{A}(1+\epsilon) < \hat{A}u =$ SUR. ($\hat{A} < 0$, $u < 1-\epsilon$): $\hat{A} < 0$ and $1-\epsilon > u$ so $\hat{A}(1-\epsilon) < \hat{A}u =$ SUR. The other two cases give CLIP = SUR. ■

> **Corollary A.27 (conservative policy update).** Maximising L^{CLIP} cannot raise the probability of a very bad action ($\hat{A} < 0$) beyond ratio $1-\epsilon$, nor that of a very good action ($\hat{A} \ge 0$) beyond $1+\epsilon$. Therefore a large L_{sur} can never be achieved by exploiting the surrogate's optimism: L^{CLIP} is PPO's surrogate for TRPO's KL constraint.

Honest caveat that belongs in a rigorous report. Clipping does **not** actually bound the ratio, because a minibatch's ratios can be shifted; the realised mean ratio can exceed $1+\epsilon$ (measured by Schulman et al. 2017, Fig. 1). Andrychowicz et al., *What Matters In On-Policy Reinforcement Learning*, ICLR 2021, shows the *implementation details* (minibatch size, epochs, gradient clipping, advantage normalisation, value loss, initialisation, LR schedule) matter as much as the clipping itself. PPO's guarantee should therefore be stated as **empirical stabilisation**, not a proven KL bound.

A.8.2 KL early-stopping heuristic

Definition A.28. Schulman et al. 2017 2: after each minibatch update, estimate the KL and terminate the epoch when $\mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}\|\pi_\theta)] \ge \text{KL}_{\text{target}}$ (typically ≈ 0.01–0.05). **Rationale:** the number of gradient steps is what actually drives drift from π_{old}; clipping caps the *per-sample* penalty but the *aggregate* KL still grows with epochs. Early stopping directly controls the constraint TRPO enforces by construction.

Estimating the KL from samples (the k_1, k_2, k_3 family; Schulman et al., arXiv:1506.02438, 2). With $r_i = \pi_\theta(a_i|s_i)/\pi_{\theta_{\text{old}}}(a_i|s_i)$ and $D := D_{\text{KL}}(\pi_{\theta_{\text{old}}}\|\pi_\theta) = \mathbb{E}_{a\sim\pi_{\text{old}}}[\log\pi_{\text{old}}(a) - \log\pi_\theta(a)]$, the identity

$$D = \mathbb{E}_{a\sim\pi_{\text{old}}}\big[-\log r(a)\big]$$

gives the **unbiased but high-variance** estimator $\hat{D}_1 = \frac{1}{N}\sum_i[-\log r_i]$. The three standard low-variance alternatives are

$$\hat{D}_2 = \frac{1}{N}\sum_i \big[r_i - 1 - \log r_i\big] \quad \text{(non-negative, second-order accurate)},$$

$$\hat{D}_3 = \frac{1}{N}\sum_i \Big[\frac{1}{2}(r_i - 1)^2\Big] \quad \text{(non-negative, unbiased to second order, lower variance still)},$$

$$\hat{D}_4 = \frac{1}{N}\sum_i \Big[(r_i - 1) - \log r_i + \tfrac{1}{2}(r_i - 1)^2\Big] = \frac{1}{N}\sum_i \Big[\tfrac{3}{2}r_i - 2 - \log r_i\Big].$$

Non-negativity, checked. The three corrections are all pointwise ≥ 0: - $r - 1 - \log r \ge 0$ for $r > 0$, with equality only at $r = 1$: let $f(r) = r - 1 - \log r$; $f(1) = 0$, $f'(r) = 1 - \frac{1}{r}$, so $f' < 0$ on $(0,1)$ and $f' > 0$ on $(1,\infty)$, and $\lim_{r\to 0^+} f(r) = +\infty$. □ - $\frac{1}{2}(r-1)^2 \ge 0$. □ - For $\hat{D}_4$: $f_4(r) = \frac{3}{2}r - 2 - \log r$, $f_4(1) = \frac{3}{2} - 2 = -\frac{1}{2} \ne 0$! So

$\hat{D}_4$ as written is **not** a KL estimator at $r = 1$ and is **not** non-negative — it is negative for $1/3 \le r < 1$. The correctly normalised fourth-order estimator is

$$\hat{D}_4 = \frac{1}{N}\sum_i \Big[(r_i{-}1){-}\log r_i{+}\tfrac{1}{2}(r_i{-}1)^2\Big] \text{ **with the } \tfrac{1}{2}(r{-}1)^2 \text{ term halved once more**, i.e. } \hat{D}_4 = \tfrac{1}{2}\big(\hat{D}_2{+}\hat{D}_3\big) = \frac{1}{N}\sum_i \Big[r_i{-}1{+}$$

which satisfies $\hat{D}_4(1) = 0$ and $f_4'(1) = 1 - 1 + 0 = 0$, $f_4''(1) = 1 + 1 + 1/2 > 0$, so $f_4 \ge 0$ locally and globally by the same argument.

Taylor check of accuracy order. With $r = e^{-\varepsilon}$ ($\varepsilon > 0$ small, r small $\Rightarrow \pi_\theta \ll \pi_{\text{old}}$):

$$-\log r = \varepsilon, \quad r - 1 - \log r = e^{-\varepsilon} - 1 + \varepsilon = \tfrac{1}{2}\varepsilon^2 - \tfrac{1}{6}\varepsilon^3 + O(\varepsilon^4), \quad \tfrac{1}{2}(r-1)^2 = \tfrac{1}{2}(1 - e^{-\varepsilon})^2 = \tfrac{1}{2}\varepsilon^2 - \tfrac{1}{2}\varepsilon^3 + O(\varepsilon^4).$$

So $\hat{D}_1$ has error $O(\varepsilon)$, while $\hat{D}_2, \hat{D}_3$ are second-order accurate; averaging them ($\hat{D}_4$) is fourth-order accurate. **This is the content of "second-order accurate, always non-negative"** — accuracy in ε, not in the ratio.

[!] Correction to the draft version. The estimator written as $-r + \log r + \frac{1}{2}(r-1)^2 = \frac{3}{2}r - 2 - \log r$ is **not** a KL estimator: at $r = 1$ it equals $-\frac{1}{2}$ (should be 0), and it is negative for $\frac{1}{3} \le r < 1$, so the early-stopping test $\hat{D} \ge \text{KL}_{\text{target}}$ can trigger spuriously. The correct form is the average of the two non-negative second-order estimators.

A.8.3 Full PPO algorithm (MAPPO variant)

for iteration k: for actor i in parallel: execute pi_theta_old for T steps; collect (obs, act, logp, rew, val, done) compute GAE: adv[t] = delta[t] + gamma*lam*(1-done[t+1])*adv[t+1] optionally value-normalise: ret[t] = adv[t] + value[t] for epoch e = 1..E: for minibatch B: for i = 1..N: ratio_i = exp(logp_i(theta) - logp_i(theta_old)) L_pi = mean_B(min(ratio_i*A_i, clip(ratio_i,1-eps,1+eps)*A_i)) L_v = mean_B(max((V_phi-ret)^2, (clip(V_phi,ret-eps_v,ret+eps_v)-ret)^2)) # value clipping maximise L_pi - c_v * L_v compute approx KL; if KL > target: break

The **value clipping** term regresses V_ϕ against a *range* rather than a point, removing the incentive to move V far in the wrong direction — critical in MARL where a badly scaled centralised critic poisons every agent's advantage (Mnih et al. 2016; Poplar; adopted in MAPPO).

A.8.4 The multi-agent subtlety: independent ratios

The problem. MAPPO uses per-agent ratios $r_t^{(i)} = \pi_{\theta^i}(a_t^i|o_t^i)/\pi_{\theta^i_{\text{old}}}(a_t^i|o_t^i)$, not the joint ratio $\prod_i r_t^{(i)}$. Standard TRPO analysis (needing joint KL $\sum_i D_{\text{KL}}(\pi^i_{\text{old}}\|\pi^i)$ and the joint ratio) **does not apply**.

> **Theorem A.29 (Sun, Harb, White, Abbasi-Yadkori, Krause, AAMAS 2023, arXiv:2202.00082 — "Trust Region Bounds for Decentralized PPO Under Non-stationarity", Thms 4.3 & 5.3).** Assuming the advantage function has converged to a fixed point (their Assumption 5.2), a **monotonic improvement** guarantee holds for the independent-ratio objective, provided the **centralised** trust region is enforced: if every independent ratio satisfies
>
> $$\lambda_j^k = \frac{\tilde{\pi}^j(a_k|s_k)}{\pi^j(a_k|s_k)} \in [1 - \epsilon_j,\, 1 + \epsilon_j] \quad \forall k, j,$$
>
> then
>
> $$D_{\text{TV}}^{\max}\big(\pi^k(\cdot|s), \tilde{\pi}^k(\cdot|s)\big) < \sum_{j=1}^{N} \epsilon_j,$$
>
> and this centralised trust region is what delivers the improvement. **In practice N independent clip ranges of size ϵ induce a joint allowance $N\epsilon$, so the clip range must be scaled $\epsilon \asymp 1/N$.** They also prove the IPPO and MAPPO surrogate objectives **coincide** at critic fixed points (via Lyu et al. 2021, Lemmas 1–2), so the centralised-vs-decentralised critic choice is *not* what makes MAPPO work.

Independent confirmation (different angle). CoPPO (Xiao et al., *Coordinated Proximal Policy Optimization*, arXiv:2305.12080) derives

$$J(\tilde{\boldsymbol{\pi}}) \;\ge\; \tilde{L}_\pi(\tilde{\boldsymbol{\pi}}) - \frac{4}{1-\gamma}\sqrt{\sum_{i=1}^{N}(1 - \alpha_i)},$$

with per-agent gap parameters α_i. The cross terms in $\mathbb{E}[\prod_i r_i]$ produce a contribution of order $\prod_i(1 - \alpha_i)$, so when any α_i is bounded away from 1 the penalty scales **exponentially in** N. This is the quantitative version of "naive independent learners blow up."

0.8.9. A.9 MAPPO (Yu et al. 2022)

Reference: Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, Yi Wu, *The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games*, **NeurIPS 2022** (arXiv:2103.01955v4). Code: `github.com/zoeyuchao/mappo`.

> **Definition A.30 (MAPPO).** PPO in the CTDE paradigm with a **centralised value function**: the critic V_ϕ takes the global state; the actor π_θ takes only the local observation. Design choices: (i) parameter sharing, (ii) centralised critic, (iii) value normalisation, (iv) carefully chosen global-state representation. Verbatim: *"$V_\phi(s)$ is used for variance reduction and is only utilized during training; hence, it can take as input extra global information not present in the agent's local observation, allowing PPO in multi-agent domains to follow the CTDE structure. [...] we refer to PPO with centralized value function inputs as MAPPO."*

A.9.1 Decentralised vs centralised — the exact split

Component	Input	At execution?	Role
Actor $\pi_\theta(a_i\|o_i)$	local o_i (+ agent-ID one-hot)	**Yes**	must be decentralised
Critic V_ϕ	**global** state: AS (agent-specific $[o_i, o_{-i}]$), EP (episode/return + agent ID + local obs), or CL (full concatenation)	**No — discarded**	baseline / variance reduction only
Actor optimiser	$\nabla_\theta L^{\text{CLIP}}$ with $\hat{A}_t$ from GAE over centralised V_ϕ	No	training
Critic optimiser	$\nabla_\phi L_v$ on $\hat{R}_t$	No	training

The GAE residual is $\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ with V_ϕ on **global** states, and the **same** $\hat{A}_t$ is reused for all N agents' ratios.

A.9.2 Parameter sharing

Definition A.31. One shared actor and one shared critic:

$$L_\pi(\theta) = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{nB}\sum_{b}\min\left(r_b^{(i)}\hat{A}_b,\ \text{clip}(r_b^{(i)}, 1-\epsilon, 1+\epsilon)\hat{A}_b\right),$$

with the single shared θ inside every $r_b^{(i)}$. Agent identity injected as $\mathbf{e}_i \in \mathbb{R}^N$.

Why it helps (three reasons). 1. **Statistical strength:** effective dataset is $nB \cdot N$ samples, not nB. For N UAVs with few samples each this is the difference between learning and not learning. 2. **Deployment efficiency:** a *single* on-board model — a $k\times$ reduction in memory/compute, decisive for small UAV payloads (A.11). 3. **Implicit coordination regularisation:** gradients from all agents accumulate in the *same* weight vector, so a direction that helps one agent is shared by all.

Reported finding (verbatim, Appendix C.2): *"MAPPO with parameter sharing outperforms MAPPO without parameter sharing by a clear margin"* (median evaluation win rate on selected SMAC maps, 6 random seeds, MAPPO-Ind vs MAPPO).

[!] Precise ranking of the ablation effects — do not call parameter sharing "the biggest win." Yu et al. 2022 5 and App. C.2 report parameter sharing as a clear win **relative to MAPPO-Ind**, and it is the paper's *default* configuration. But the paper's own **most emphatic recommendation is value normalisation** ("never hurts training and often improves the final performance significantly", critical in MPE `spread`), and the report's own claim that *parameter sharing* is the single largest effect is an over-reading: the ablations are made relative to MAPPO's default, and the paper does not rank parameter sharing above value normalisation. The defensible statements are: - parameter sharing > no parameter sharing, by a clear margin, on the tested SMAC maps, 6 seeds (App. C.2); - value normalisation is the single most robust stabiliser across *all* benchmarks (Suggestion 1, 5); - critic-state design (EP < CL < AS ≈ FP) is the second-largest effect (Suggestion 2); - these three are **interacting**; the paper does not report a factorial decomposition, so no additive attribution of the total gain is available.

Theoretical caution (must be stated). Kuba et al. 2022 (Prop. 1) prove parameter sharing can yield an **exponentially worse** suboptimal outcome in a worst-case constructed game, because the shared gradient

can be dominated by agents whose local optima conflict. It is an **inductive bias**, valid under homogeneity — true for a homogeneous UAV swarm, **false** for a heterogeneous fleet.

A.9.3 The five factors that matter — exact reported findings (Yu et al. 2022 5)

Suggestion 1 — Value normalisation (strongest recommendation). *Verbatim suggestion: "Utilize value normalization to stabilize value learning."* - **Method:** the value network regresses to **normalised** targets using a **running mean and standard deviation of the value targets**; when computing the GAE, the running average **de-normalises** the network output so advantages are on the correct scale. - **Why (verbatim):** *"Through the training process of MAPPO, value targets can drastically change due to differences in the realized returns, leading to instability in value learning."* - **Evidence:** in MPE `spread` (episode returns ranging below -200 to 0) value normalisation is **critical**; it also positively affects several SMAC maps, either improving final performance or reducing training variance. Stated: value normalisation *"never hurts training and often improves the final performance significantly."* - **Direct transfer to UAV work:** persistent coverage has wildly different reward scales across mission phases (well-charged vs low-SoC), so target standardisation is essential.

Suggestion 2 — Critic input composition. *Verbatim: "When available, include both local, agent-specific features and global features in the value function input. Also check that these features do not unnecessarily increase the input dimension."* - Ordering: **EP** (episode-return + agent ID + local obs) < **CL** (all local observations concatenated) < **AS** (agent-specific $[o_i, o_{-i}]$) $\approx$ **FP** (featured-pruned AS). - Key result: **CL is ineffective, particularly in maps with many agents** — it inflates dimensionality without adding information. AS and FP are both strong; FP beats AS on some maps and wins where AS is subpar. - Verbatim: *"the state dimensionality, agent-specific features, and global information are all important in forming an effective global state."*

Suggestion 3 — Training data usage (epochs / minibatch / rollout). Very sensitive. Reported: episode length 200 (SMAC), 25 PPO epochs, 5 minibatches, buffer 400 (MPE) / 5000 (SMAC) episodes, batch 32 (MPE) / 1 (SMAC), $\gamma = 0.99$, $\epsilon = 0.2$ (value & policy), lr 5×10^{-4}, value-loss coefficient $c_v = 1$. The *number of gradient steps* per rollout controls trust-region drift, consistent with A.8.2.

Suggestion 4 — Clipping strength. Small ϵ (0.05) slows learning on hard maps (MMM2, 3s5z vs 3s6z) but gives **consistently high final performance and smaller standard deviation** in training curves — lower ϵ trades speed for stability. Large ϵ (0.2, 0.3, 0.5) still works well empirically; $\epsilon = 0.2$ is the reported default. **Consistency note with theory:** Sun et al. 2023 (A.8.4) would predict $\epsilon \asymp 1/N$ for *guaranteed* improvement; Yu et al. separately report that good ϵ values are *"highly sensitive to the number of agents."* The two are consistent once one reads Yu et al.'s large-ϵ results as indicating the **guarantee** is lost while **performance** is retained.

Suggestion 5 — Value/policy clipping. Using the same ϵ for both; keeping both is better than removing either.

Other quotable findings. - **PPO is surprisingly strong in MARL:** PPO-based methods match or beat off-policy competitors (MADDPG, QMIX family) on SMAC, GRF, Hanabi, MPE, *"with minimal hyperparameter tuning and without any domain-specific algorithmic modifications or architectures"* (abstract). - **Value clipping** stabilises the centralised critic. - **Recurrent policy needed for partial observability** (MPE `comm`) — the one benchmark where they use agent-specific (non-shared) parameters.

A.9.4 Sequential update order (HATRPO/HAPPO — *not* MAPPO)

This is a common confusion in reports: **MAPPO does not use a sequential update.** That is the design of the HATRPO/HAPPO family. Precise statement:

> **Theorem A.32 (Kuba, Chen, Wen, Wen, Sun, Wang, Yang, ICLR 2022, arXiv:2109.11251, Lemma 1 — Multi-Agent Advantage Decomposition).** In **any** cooperative Markov game, for any state s, any joint policy $\boldsymbol{\pi}$, and any permutation $i_1, \dots, i_N$,
>
> $$A_{\boldsymbol{\pi}}^{i_{1:N}}\left(s, \mathbf{a}^{i_{1:N}}\right) = \sum_{j=1}^{N} A_{\boldsymbol{\pi}}^{i_j}\left(s, \mathbf{a}^{i_{1:j-1}}, a^{i_j}\right),$$
>
> i.e. the joint advantage decomposes **exactly** into a sum of per-agent advantages, each conditioning only on the actions of agents earlier in the order. **No decomposability assumption on V or Q is required** (unlike VDN, QMIX, Q-DPP). *Proof idea.* Induct on N; use $Q_{\boldsymbol{\pi}}(s, \mathbf{a}) = \mathbb{E}_{s'}[r + \gamma Q_{\boldsymbol{\pi}}(s', \mathbf{a}')]$ and $V_{\boldsymbol{\pi}}(s) = \mathbb{E}_{s'}[\mathbb{E}_{\mathbf{a}' \sim \boldsymbol{\pi}}[r + \gamma V(s')]]$; the inner term recurses. □
>
> **Theorem A.33 (monotonic improvement, Kuba et al. 2022 Thm 2).** With $L_{\boldsymbol{\pi}}^{i_{1:m}}\left(\tilde{\pi}^{i_{1:m-1}}, \tilde{\pi}^{i_m}\right) = \mathbb{E}\left[\frac{\tilde{\pi}^{i_m}}{\pi^{i_m}} A_{\boldsymbol{\pi}}^{i_m}\right]$, the sequential scheme
>
> $$\tilde{\pi}_{k+1}^{i_m} = \arg\max_{\pi^{i_m}} \left\{ L_{\boldsymbol{\pi}_k}^{i_{1:m}}\left(\tilde{\pi}_{k+1}^{i_{1:m-1}}, \pi^{i_m}\right) - C\, D_{\mathrm{KL}}^{\max}\left(\pi_k^{i_m}, \pi^{i_m}\right) \right\}$$

satisfies $J(\boldsymbol{\pi}_{k+1}) \geq J(\boldsymbol{\pi}_k)$ for all k. Each agent can always make its summand ≥ 0 by not updating ($\tilde{\pi}^{i_m} = \pi^{i_m}$), so positive updates accumulate. Also: the update order can be **randomised** each iteration without hurting convergence (and often helping it).

Empirical findings of Kuba et al. 2022: HATRPO/HAPPO *significantly outperform* IPPO, MAPPO and MADDPG on Multi-Agent MuJoCo and SMAC. They need **no parameter sharing** and **no decomposability** of the joint Q, and they show MAPPO/IPPO's reliance on parameter sharing can be **exponentially worse** in the worst case.

MAPPO vs HATRPO/HAPPO.

	MAPPO	HATRPO / HAPPO
Ratios	**independent** $r^{(i)}$	**sequential**, conditioned on prior agents' new policies
Critic	one centralised V_ϕ on global state	one **joint** advantage estimator for all agents (saves N critics)
Parameter sharing	required in practice	**not** required (heterogeneous supported)
Monotonic improvement	only with independent-ratio theory (Sun et al. 2023) + small ϵ	**guaranteed** (Kuba et al. 2022)
Extra training info	global state	sequential action prefix $\mathbf{a}^{i_{1:m-1}}$
Empirically (Kuba 2022)	strong on SMAC	stronger on MuJoCo, strong on SMAC

Recommendation for the report: homogeneous UAVs + robustness goal ⇒ MAPPO, whose ablated design rules (A.9.3) are directly actionable. Heterogeneous fleet ⇒ HAPPO, which needs no parameter sharing, has a monotonic-improvement guarantee, and uses a single critic.

A.9.5 Other 2023–2026 results

- **Li & He (2025)**, *Bridging Theory and Practice of MAPPO for Cooperative Multi-Agent Continuous Control* — theoretical analysis of MAPPO in continuous control; TRPO-augmented variant **removes the parameter-sharing requirement, supports heterogeneous agents, and uses a squashed Gaussian** for bounded actions. Directly applicable to UAV continuous action spaces with hard speed/acceleration envelopes.
- **Li & He (2023)**, *Multi-Agent Trust Region Policy Optimization*, IEEE TNNLS 35(9):12873–12887.
- **PRD-MAPPO (2024)**, *Assigning Credit with Partial Reward Decoupling in MAPPO* — soft attention restricting agent i's advantage to a relevant set $\mathcal{R}_i(s_t)$:

$$\delta_{i,t} = \sum_{j \in \mathcal{R}_i(s_t)} r_t^{(j)} + \gamma V_\psi^i(s_{t+1}, \mathbf{a}^i_{-t+1}) - V_\psi^i(s_t, \mathbf{a}^i_{-t}).$$

  Reports that PRD-MAPPO **avoids the gradient-variance spikes present in MAPPO** and reduces advantage computation from $O(M^2)$ to $O(M)$ critic calls. **Directly useful for UAV coverage:** the natural relevant set is the k nearest neighbours in the coverage graph.
- **Constrained/safe MARL PPO (NeurIPS 2024)** — Lagrangian-constrained MARL PPO, for hard geo-fence/minimum-altitude constraints.

0.8.10. A.10 Variance of the multi-agent gradient estimator; non-stationarity; recent theory

A.10.1 Why naive independent learners diverge

Definition A.34 (independent learner). Agent i at iteration k updates with $\hat{g}_i^{(k)} = \frac{1}{B}\sum_b \nabla_{\theta^i} \log \pi_{\theta^i}(a_b^i|o_b^i)\hat{A}_b^i$, where $\hat{A}_b^i$ is estimated by a critic trained **only on agent i's own data**, with no access to other agents' state/actions.

(1) Non-stationary target (fundamental). $Q_i^{\boldsymbol{\pi}_k}$ changes as others update. The iteration $\theta_{k+1}^i = \theta_k^i + \alpha_k g^i(\theta_k^i; \boldsymbol{\pi}_k)$ is stochastic approximation toward a *moving* sequence of fixed points $\arg\max_\theta J(\theta^i; \boldsymbol{\pi}_k)$ that are themselves moving. There is no contraction. Classical two-timescale actor–critic theory (Borkar 1997) requires the critic to converge to $Q^{\boldsymbol{\pi}_k}$ — but $\boldsymbol{\pi}_k$ changes every step, so the tracking error never vanishes.

(2) Multiplicative variance growth. From CoPPO (A.8.4): the joint improvement bound's error term scales as $\sqrt{\sum_{i=1}^{N}(1-\alpha_i)}$, arising from cross terms of order $\prod_i(1-\alpha_i)$ in $\mathbb{E}[\prod_i r_i]$. **Exponential in** N when any $\alpha_i < 1$ is bounded away from 1.

(3) The deadly triad (Sutton & Barto 2018 3). The conjunction of: **- function approximation** (neural value function, limited capacity); **- bootstrapping** (GAE/TD targets depend on the current estimate); **- off-policy / non-stationary data** (targets generated by a joint policy that is being updated — in MARL the data distribution is non-stationary *by construction* because all agents update simultaneously).

Divergence mechanism: the Bellman operator is a γ-contraction in ℓ_∞ **for the true operator**; function approximation destroys this (empirically, spectral radius of the approximate operator > 1 implies divergence), and off-policy/non-stationary data removes the sampling assumptions that make the update a descent method.

Minimal demonstration: Example A.9 — the 2×2 game oscillates forever with independent greedy updates; a centralised V has a unique optimum at $(1, 1)$ where $J = 1$. **CTDE's only contribution is a correct, low-variance, low-bias signal about the joint effect of actions; it does not change the policy class.**

A.10.2 The MAPPO variance argument — stated correctly

MAPPO computes **one** team advantage $\hat{A}_t$ per time step and reuses it for all N agents. Decompose $\hat{A}_b = A_b^{\text{true}} + \varepsilon_b$ with ε_b the critic's estimation error, and write the shared-signal gradient estimator

$$\hat{g} = \frac{1}{BN}\sum_{b=1}^{B}\sum_{i=1}^{N} \nabla_{\theta^i} \log \pi_{\theta^i}(a_b^i|o_b^i)\big(A_b^{\text{true}} + \varepsilon_b\big) = \hat{g}_{\text{signal}} + \hat{g}_{\text{noise}}.$$

The correct comparison is against the IPPO-style estimator with per-agent advantages $\hat{A}_b^i = A_b^i + \varepsilon_b^i$:

$$\hat{g}_{\text{noise}}^{\text{IPPO}} = \frac{1}{BN}\sum_{b}\sum_{i} \nabla_{\theta^i} \log \pi_i(a_b^i|o_b^i)\varepsilon_b^i.$$

Case 1 — the critic errors are independent across agents ($\mathbb{E}[\varepsilon_b^i \varepsilon_b^j] = 0$ for $i \neq j$) and have equal variance σ_ε^2. The two noise terms are then both sums of BN zero-mean terms with comparable per-term scale, so

$$\|\hat{g}_{\text{noise}}\|_{\text{shared}}^2 \asymp \|\hat{g}_{\text{noise}}^{\text{IPPO}}\|^2 \asymp \frac{NB\,\sigma_\varepsilon^2\,\mathbb{E}\|\nabla \log \pi\|^2}{(BN)^2},$$

i.e. the **relative** noise level is the same, and the only difference is the quality of the advantage itself: one critic with the *global* state has a **lower bias and lower variance** for the team advantage than N independent local critics, because $A^{\boldsymbol{\pi}}(s, \mathbf{a}) = Q^{\boldsymbol{\pi}} - V^{\boldsymbol{\pi}}$ is a functional of the **joint** state-action pair, and N local critics each see only o_i and must average away the effect of the other $N-1$ agents' actions. The honest statement is therefore:

> MAPPO's shared advantage is a **bias/variance improvement in the advantage estimate**, obtained by conditioning V on the full joint state. The *averaging over agents* contributes a factor $1/\sqrt{N}$ (the usual independent-average rate) **only if** the per-agent score terms were being averaged with independent noise — which they are not, because they all carry the *same* ε_b.

Case 2 — the honest "shared signal" effect, and its correct order. Because the same ε_b is multiplied by N score terms, the noise is coherent across i:

$$\hat{g}_{\text{noise}} = \frac{1}{BN}\sum_{b} \varepsilon_b \underbrace{\Big(\sum_{i=1}^{N} \nabla_{\theta^i} \log \pi_i(a_b^i|o_b^i)\Big)}_{=:\, G_b,\ \text{the joint score}}, \qquad \mathbb{E}\,\|\hat{g}_{\text{noise}}\|^2 = \frac{\sigma_\varepsilon^2}{BN^2}\mathbb{E}\|G_b\|^2 = \frac{\sigma_\varepsilon^2}{N\,B}\,\mathbb{E}\|G_b\|^2.$$

Compare with IPPO: there the corresponding quantity is $\frac{1}{BN^2}\sum_{b,i}\mathbb{E}[\varepsilon^{i\,2}\|\nabla_i\|^2]$. So if $\mathbb{E}\|G_b\|^2 \asymp N\,\mathbb{E}\|\nabla_i\|^2$ (scores roughly orthogonal across agents) the two coincide in order, while if the agents' scores are **correlated** (the genuinely cooperative regime, where they move together) $\mathbb{E}\|G_b\|^2$ grows like N^2 and the shared-signal estimator is *worse*, not better.

[!] The "1/N instead of $1/\sqrt{N}$" claim is false. Two errors: (i) averaging N independent zero-mean terms reduces the standard deviation by $1/\sqrt{N}$, never $1/N$ — $1/N$ is the reduction in *variance*, not in the noise level; (ii) MAPPO's noise terms are *not* independent (the same ε_b appears in all N of them), so the standard $1/N$ variance-of-the-average argument does not apply at all. What MAPPO actually gains is a better *target* (global-state critic), not a magic $1/N$ averaging factor.

The defensible, citable justification for a shared team advantage is therefore: 1. **Bias reduction** (the real one): $V(s^1, \ldots, s^N)$ conditions on the full joint state, so $A^{\boldsymbol{\pi}}$ is estimated without the partialling-out error that N local critics incur. Formally, $\mathbb{E}[V^{\text{global}}] \leq \mathbb{E}[V^{\text{local}}]$-style comparisons require a nested-information assumption, but the qualitative statement — the A that is correct for the credit assignment is a joint functional, so estimate it jointly — is standard. 2. **Non-stationarity absorption** (A.10.1(1)): a single global critic is updated on the same batch, so all N agents consume a *simultaneously refreshed* target rather than N independently drifting ones. 3. **Practical**: 1 critic forward pass per environment step instead of N, which is why MAPPO is affordable (see A.11.2).

A.10.3 Recent theoretical results (2023–2026)

Paper	Result	Relevance
Kuba, Chen, Wen, Wen, Sun, Wang, Yang (ICLR 2022), arXiv:2109.11251	Multi-agent advantage decomposition (exact, no decomposability assumption) + sequential trust region ⇒ **monotonic improvement**; randomised agent order is fine	First rigorous multi-agent trust-region theory. HATRPO/HAPPO
Sun, Harb, White, Abbasi-Yadkori, Krause (AAMAS 2023), arXiv:2202.00082	**Monotonic improvement for IPPO/MAPPO with independent ratios** if each ratio in $[1-\epsilon_j, 1+\epsilon_j]$ and advantages converged. Joint TV trust region $= \sum_j \epsilon_j$. IPPO ≡ MAPPO objectives at critic fixed points	Direct justification for MAPPO's design and for the empirical finding that ϵ must scale with N
Xiao et al. (2023), CoPPO, arXiv:2305.12080	Approximate monotonic improvement for a theoretically-grounded joint objective; variance penalty $\sqrt{\sum_i (1-\alpha_i)}$ — multiplicative in N; interprets objective as **dynamic credit assignment**	Quantitative explanation of independent-learner blow-up
Zhao, Yang, Wang, Lee (ICML 2023), PMLR v202, *Local Optimization Achieves Global Optimality in MARL*	**First provably convergent multi-agent PPO** in cooperative Markov games with function approximation. Multi-agent PDL + **localized action value** as ideal descent direction; **sequential** per-agent updates. Rate $\frac{N}{1-\gamma}\log\frac{\lvert\mathcal{A}\rvert K}{1-\gamma}$ up to statistical errors from critic evaluation with n samples, K = iterations. Sample complexity **polynomial in** N — "breaking the curse of scalability". Requires only **single-policy concentrability** (weaker than prior bounds)	The strongest convergence guarantee for PPO-style multi-agent algorithms. The concrete error bound to cite
Li & He (2025)	Theoretical analysis of MAPPO in continuous control; TRPO-augmented variant removes parameter sharing, supports heterogeneity, **squashed Gaussian** for bounded actions	Directly applicable to UAV continuous control with hard speed limits
PRD-MAPPO (2024)	Attention-based relevant-set restriction; removes gradient-variance spikes; $O(M)$ critic calls	Actionable for UAV coverage with spatial neighbourhoods
Constrained MARL PPO (NeurIPS 2024)	Lagrangian-constrained MARL PPO	Hard coverage / geo-fence constraints

Honest closing statement for the report. As of September 2026 there is **no** end-to-end convergence proof for *MAPPO as actually implemented* (parameter sharing + advantage normalisation + GAE + value clipping + multiple epochs + independent ratios, all interacting). What exists is: (i) **monotonic improvement** under an idealised trust-region assumption (Kuba et al.; Sun et al. with $\epsilon \asymp 1/N$ and a converged advantage); (ii) **global convergence to the optimal decentralised policy at rate** $\frac{N}{1-\gamma}\log\frac{|\mathcal{A}|K}{1-\gamma}$ for a *specific* multi-agent PPO with sequential updates and no parameter sharing (Zhao et al. 2023); (iii) everything else is empirical.

0.8.11. A.11 Complexity: shared actor, centralised critic

Setup: N agents; per-agent $o_i \in \mathbb{R}^M$; $a_i \in \mathbb{R}^{d_a}$; actor MLP L_a layers width H (input M or $M+N$ with agent-ID one-hot); critic MLP L_c layers width H_c, input d_{crit}.

FLOP convention. A fully-connected layer (in d_{in}, out H) costs $\approx 2d_{\text{in}}H$ MACs forward, the same backward; a forward+backward pass $\approx 6d_{\text{in}}H$ FLOPs.

A.11.1 Actor: $O(1)$ per agent, $O(N)$ total — proof

Claim. With parameter sharing, per-agent actor inference cost is

$$C_{\text{actor}}^{\text{agent}} = \Theta\big(L_a\,(d_{\text{in}} + L_a H)\big) = \Theta\big(L_a H(d_{\text{in}} + H)\big),$$

independent of N; total over the team is $\Theta(N)$.

Proof. A parameter-shared actor performs **one** forward pass of the *same* network on the agent's own input. Weights are shared, so the parameter count $\Theta(L_a d_{\text{in}}H + L_a H^2 L_a + H d_a)$ is independent of N (except the N-dim agent-ID one-hot, contributing $\Theta(NH)$ parameters and $\Theta(NH)$ FLOPs to layer 1). Forward FLOPs:

$$\text{FLOPs}_{\text{fwd}} = 2Hd_{\text{in}} + 2(L_a-1)H^2 + 2Hd_a = \Theta\big(L_a H(d_{\text{in}}+H)\big),$$

with no N term other than the agent-ID input. □

The "3 orders of magnitude" argument. Per-agent networks: total parameters $\Theta(NL_aH^2)$, storage $\Theta(NL_aH^2)$. Shared: parameters and storage $\Theta(L_aH^2)$, total FLOPs still $\Theta(NL_aH^2)$. For $N = 100$, $L_a = 3$, $H = 256$: $100\times$ **storage reduction** — a shared actor fits an embedded autopilot budget where per-agent networks do not.

Communication-free scaling. The actor runs on-board with local o_i only, so per-step execution communication is $\Theta(1)$. For very large N replace the one-hot with a random/hash embedding $\mathbf{e}_i \in \mathbb{R}^{d_e}$, $d_e \ll N$, restoring $O(1)$ input dim.

A.11.2 Critic: the $O(N+M)$ claim, stated exactly

[!] The unqualified phrase "centralised critic has $O(N+M)$ input" is false for MAPPO's actual state. The three critic-state designs in the MAPPO paper are (Yu et al. 2022, 5 and App. C):

Critic state	Concatenation	Input dim	Asymptotics
AS (*agent state*)	$[o_i^{1\ldots k}, o^{i\,1\ldots N-1}]$ — agent i's own recent observations + **all other agents' current observations**	$kM + (N-1)M = \Theta(NM)$	$\Theta(NM)$
EP (*episode return*)	$[\mathbf{R}_t^i,\ i_t,\ o_t^i]$ — past episode returns, one-hot agent ID, own observation	$R_{\text{hist}} + N + M$	$\Theta(N+M)$
FP (*future plan*)	$[R^{i}_{-t},\ i_{-t},\ o i_t,\ o\hat{}i_\{t+1\}]$ + $ future plan	$R_{\text{hist}} + N + 2M$	$\Theta(N+M)$
Local-only	o_t^i	M	$\Theta(M)$

So $O(N+M)$ is the EP/FP row, not the AS/CL row. MAPPO's recommended and most-used configuration is **AS with a discrete mask** (per-agent index selecting which others are within comms range) or **AS + discretised input**, which is where the $\Theta(NM)$ comes from.

Per-call cost. One forward pass of an L_c-layer, width-H_c MLP on a d-dimensional input costs $\Theta(L_c H_c(d + H_c)) = \Theta(L_c H_c\, d)$ for $d \gg H_c$. Hence $C_{\text{crit}} = \Theta(L_c H_c NM)$ for AS, and $\Theta(L_c H_c(N + M))$ for EP/FP.

Per iteration. MAPPO performs **one** critic forward per collected timestep (the *same* global/agent state is reused for all N agents, because all agents share one advantage). With T steps and K epochs of minibatch updates, the count is T forward passes and TK (mini)batches:

$$\boxed{\text{Critic} = \Theta\big(T\, L_c H_c\, NM\big) \quad \text{(AS)}, \qquad \Theta\big(T\, L_c H_c (N + M)\big) \quad \text{(EP/FP)}}$$

$$\boxed{\text{Actor} = \Theta\big(T\, N\, L_a H (M + N)\big)}$$

[!] The "$O(N^2M)$ total" figure in the earlier draft was wrong in its counting. It assumed N *distinct* critic inputs (one per agent) per timestep. MAPPO does **not** do that: it computes **one** V from the (concatenated or agent-indexed) state and broadcasts it. The N^2M figure corresponds to running N *separate* N-input critics, which is the IPPO/"N individual centralised critics" variant, not MAPPO. Both numbers should be reported, with the design they belong to named.

Ratio (corrected). $\frac{\text{Critic}}{\text{Actor}} = \Theta\Big(\frac{L_c H_c NM}{N\, L_a H(M+N)}\Big) = \Theta\Big(\frac{L_c H_c M}{L_a H(M+N)}\Big)$, which is $O(1)$ **in** N for the AS design. With $L_c = L_a = 3$, $H_c = H = 256$, $M = 32$, $N \gg 1$: $\approx \frac{32}{32+N} \approx 0.28$ at $N = 100$ — the critic is *not* asymptotically dominant. The genuine constraint is **input dimensionality and memory**, not FLOPs: a $32 \times 100 = 3200$-dim MLP input with $H_c = 256$ allocates $3200 \times 256 \approx 8.2 \times 10^5$ weights in the first layer alone, and the network must be retrained whenever N changes. This motivates Yu et al.'s **feature pruning / discretisation** of the AS state, and is the honest reason to state the complexity as a *memory and re-trainability* issue rather than a compute-dominance issue.

Scaling limit and the natural fix for coverage. With $d_{\text{crit}} = \Theta(NM)$ the AS critic cannot be reused across swarm sizes. The domain-standard fix is a **permutation-invariant / factored critic**: a DeepSets or attention encoder $\rho(\sum_i \phi(o_i))$ with $\phi : \mathbb{R}^M \to \mathbb{R}^{d_e}$ makes $d_{\text{crit}} = \Theta(d_e) = \Theta(1)$ **for all** N and is exactly what makes the critic permutation-equivariant. For persistent coverage the natural factored critic is per-Voronoi-cell, $V(s) = \sum_c \psi_c(s|_{V_c})$ with ψ_c acting on the agents in cell c, so cost is $\Theta(L_c H_c M)$ per step and the structure mirrors the cell decomposition of B.4.

Scaling limit. $d_{\text{crit}} = \Theta(NM)$ is feasible up to $N \sim 10^2$; beyond that use **mean-field / per-region / factored critics**. For coverage, a per-Voronoi-cell value function $V_{\phi_{\text{cell}}}(\text{cell}_i)$ is natural (each UAV's value depends mainly on its own cell and immediate neighbours), reducing d_{crit} to $\Theta(M)$.

A.11.3 Summary table

Let T = environment steps per iteration, K = epochs, B = batch size ($TK \geq T$ minibatch updates). $C_{\text{actor}} = \Theta(L_a H(M + N))$ is the cost of **one** shared forward pass.

Operation	Input dim	Cost of one call	Per iteration (T steps)	Grows in N?
Actor inference (on-board, shared)	$M + N$ (one-hot) or $M + d_e$	$\Theta(L_a H(M + N))$	$\Theta(TNL_a H(M + N))$ at execution; $\Theta(TKL_a H(M + N))$ in training (batched)	**no** (if d_e fixed) / yes via one-hot
Critic, **AS** state (MAPPO default)	NM	$\Theta(L_c H_c NM)$	$\Theta(TL_c H_c NM)$	**yes**, $\Theta(NM)$
Critic, **EP**/FP state	$N + M$	$\Theta(L_c H_c(N + M))$	$\Theta(TL_c H_c(N + M))$	linearly in N
Critic, N separate AS critics (IPPO-style)	NM each	$\Theta(L_c H_c NM)$	$\Theta(TL_c H_c N^2 M)$	**yes**, $\Theta(N^2M)$
Critic, factored/DeepSets/per-cell	d_e or M_{cell}	$\Theta(L_c H_c M)$	$\Theta(TL_c H_c M)$	**no**

The three sentences to put in the report, each matching one row: 1. **Actor is $O(1)$ per agent in N** provided agent identity enters as a fixed-size embedding (d_e) rather than an N-dimensional one-hot; with a one-hot it is $O(N)$ per agent and $O(N^2)$ total. State the assumption; do not claim $O(1)$ without it. 2. **A single centralised critic with the AS state is $O(NM)$ per step, i.e. $O(1)$ in N relative to the**

N**-critic variant's** $O(N^2M)$ — this is the honest complexity win of CTDE: *one* global evaluation instead of N per-agent ones. Its real cost is the NM-dimensional input, which is a memory/generalisability problem, not a FLOP problem. 3. **The** $O(N+M)$ **bound is the EP/FP critic state**, and it is the configuration that stays dimension-light; it is not what AS does.

Communication feasibility check. Decentralised execution: $\Theta(1)$ per step. Centralised training requires shipping all N observations each control period: $\Theta(NM)$ values/frequency. At $N = 100$, $M = 32$, 4-byte floats, 1 Hz: $100 \times 32 \times 4 \times 1 = 12.8$ kB/s — trivially feasible over cellular/mesh backhaul. A useful sanity number for the report. **Caveat:** CTDE's centralisation here is a *training-time* abstraction; a real system needs this $\Theta(NM)$/step fabric to exist, which is exactly assumption A1 in the table of A.3 and the reason many deployments substitute a range-limited or hierarchical critic.

0.8.12. A.12 Reward shaping: potential-based theorem (Ng, Harada, Russell 1999)

Reference: Andrew Y. Ng, Daishi Harada, Stuart Russell, *Policy invariance under reward transformations: Theory and application to reward shaping*, **ICML 1999**, pp. 278–287.

A.12.1 The exact statement

Let $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, \gamma, R \rangle$ and $\mathcal{M}_0 = \langle \mathcal{S}, \mathcal{A}, T, \gamma, R_0 \rangle$ with $R_0 = R + F$.

Definition A.35 (potential-based shaping function). $F : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is **potential-based** if there exists $\Phi : \mathcal{S} \to \mathbb{R}$ such that for all $s \neq s'$, a, s':

$$\boxed{F(s, a, s') = \gamma\, \Phi(s') - \Phi(s)}$$

Theorem A.36 (Ng–Harada–Russell 1999, Theorem 1 — exact statement). Let any shaping reward function $F : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ be given. We say F is a potential-based shaping function if there exists $\Phi : \mathcal{S} \to \mathbb{R}$ such that for all $s \neq s'$, a, s': $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$. Then: **(Sufficiency)** If F is potential-based, then **every optimal policy in** $\mathcal{M}_0 = \langle \mathcal{S}, \mathcal{A}, T, \gamma, R_0 \rangle$ **will also be an optimal policy in** $\mathcal{M}$ (and vice versa). **(Necessity)** If F is **not** potential-based (i.e. no such Φ exists satisfying the equation), then there exist (proper) transition functions T and a reward function $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ such that **no optimal policy in** $\mathcal{M}_0$ **is optimal in** $\mathcal{M}$. Verbatim scope note: *"this is the strongest possible theorem of this form"*; with no prior knowledge of T and R, potential-based F is **necessary and sufficient** for consistency.

Corollary A.37 (Ng et al., Corollary 2). If $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ (and, if $\gamma = 1$, that $\Phi(s') = 0$), then for all s, a:

$$Q^*_{M_0}(s, a) = Q^*_M(s, a) - \Phi(s), \qquad V^*_{M_0}(s) = V^*_M(s) - \Phi(s).$$

Remark A.38 (Ng et al., Remark 1). These identities hold for **any** policy π, not only the optimal one: $Q^\pi_{M_0}(s, a) = Q^\pi_M(s, a) - \Phi(s)$, $V^\pi_{M_0}(s) = V^\pi_M(s) - \Phi(s)$. Hence potential-based shaping **preserves near-optimal policies**. (Some presentations additionally require $\|\Phi\|_\infty < \infty$, guaranteeing V is well defined for infinite discounted problems.)

A.12.2 Proof of sufficiency — step by step

Step 0 — normalisation. Replacing $\Phi(s)$ by $\Phi(s) - \Phi(s_0)$ for a constant c changes F by $\gamma c - c = -c(1-\gamma)$, a **constant** added to every reward, which does not change the optimal policy. Hence for $\gamma = 1$ we may assume $\Phi(s_0) = 0$ (Ng et al. fix this normalisation). (In the discounted case $\gamma < 1$ this is unnecessary.)

Step 1 — candidate. Show $\tilde{Q}(s, a) := Q^*_{M_0}(s, a) - \Phi(s)$ is the optimal Q of $\mathcal{M}$.

Step 2 — verify the Bellman optimality equation of $\mathcal{M}$**.**

$$\begin{aligned}
\mathbb{E}_{s'}\big[R_0(s, a, s') + \gamma\tilde{Q}(s', a')\big] &= \mathbb{E}_{s'}\big[R(s, a, s') + F(s, a, s') + \gamma Q^*_{M_0}(s', a') - \gamma\Phi(s')\big] \\
&= \mathbb{E}_{s'}\big[R(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_{F(s,a,s')} + \gamma Q^*_{M_0}(s', a') - \gamma\Phi(s')\big] \\
&= \mathbb{E}_{s'}\big[R(s, a, s') - \Phi(s) + \gamma Q^*_{M_0}(s', a')\big] \\
&= \mathbb{E}_{s'}\big[R(s, a, s') - \Phi(s) + \gamma\big(Q^*_M(s', a') - \Phi(s')\big)\big] \\
&= \mathbb{E}_{s'}\big[R(s, a, s') + \gamma \max_{a'} \tilde{Q}(s', a')\big],
\end{aligned}$$

using the inductive hypothesis $Q^*_{M_0} = \tilde{Q} + \Phi$. This is precisely the Bellman optimality equation of $\mathcal{M}$ for $\tilde{Q}$. (The undiscounted case needs $s = s'$ treated separately — as Ng et al. note.)

Step 3 — conclude by contraction. T^* is a γ-contraction on bounded functions, so it has a unique fixed point. $\tilde{Q}$ is a fixed point, hence $\tilde{Q} = Q^*_M$, i.e. $Q^*_{M_0}(s,a) = Q^*_M(s,a) - \Phi(s)$.

Step 4 — policy invariance. π is optimal in $\mathcal{M}_0$ iff it maximises $Q^*_{M_0}$:

$$\pi \in \arg\max_\pi \mathbb{E}\big[Q^*_{M_0}(s_0, \pi(s_0))\big] \iff \pi \in \arg\max_\pi \mathbb{E}\big[Q^*_M(s_0, \pi(s_0)) - \Phi(s_0)\big] \iff \pi \in \arg\max_\pi \mathbb{E}\big[Q^*_M(s_0, \pi(s_0))\big],$$

because $\Phi(s_0)$ is π-independent. So $\arg\max_\pi Q^*_{M_0} = \arg\max_\pi Q^*_M$ **as sets**. ■

A.12.3 Proof of necessity (sketch, contrapositive)

Ng et al., Appendix A. If F is not potential-based, there exist s, a, s' with $F(s,a,s') \neq \gamma\Phi(s') - \Phi(s)$ for *every* candidate Φ. Construct an MDP with a state x having two actions: "stay" (reward 0 forever) and "go" (enters a trap y with a one-time reward). R is chosen so the *shaped* problem prefers "go" (because $F(x, \text{go}, y)$ is large) while the *unshaped* problem prefers "stay". Key mechanism: with n intermediate states, the telescoped sum of a potential-based F over the path is bounded by $\Phi(\text{start}) - \Phi(\text{end})$ — **independent of** n — whereas a non-potential F can be arranged to accumulate $\Theta(n)$. So the shaped problem's total return flips sign relative to the unshaped one as n varies, and no invariant guarantee is possible. □

A.12.4 Telescoping and no positive cycles

Corollary A.39 (telescoping). For any trajectory $\tau = (s_0, a_0, s_1, \ldots, s_T)$:

$$\sum_{t=0}^{T-1} \gamma^t F(s_t, a_t, s_{t+1}) = \gamma^T \Phi(s_T) - \Phi(s_0).$$

Proof: $\sum_t \gamma^{t+1}\Phi(s_{t+1}) - \sum_t \gamma^t \Phi(s_t)$ telescopes. □

Corollary A.40 (no positive cycles). For a closed cycle $s_0 \to \cdots \to s_T = s_0$, the shaped contribution is $(\gamma^T - 1)\Phi(s_0)$; for $\gamma = 1$ it is **exactly 0**. Potential-based shaping **cannot** create a positive-reward cycle — exactly the pathology that naive shaping creates: Naive & Barto (1990) reward-for-progress produced agents that "rode a bicycle in tiny circles near the start state" (Randal, strm & Barto 1998) or a soccer robot that "vibrated" next to the ball.

A.12.5 Application: justifying a coverage-based shaping term

Construction. Unshaped mission reward:

$$R(s_t, \mathbf{a}_t) = \alpha \mathbb{1}[\text{battery-constrained coverage requirement met at } t] - \alpha_{\text{fail}} \mathbb{1}[\text{SoC hits reserve floor}] - \alpha_{\text{coll}} \mathbb{1}[\text{collision}].$$

Sparse and delayed. Add the shaping

$$\boxed{F(s_t, \mathbf{a}_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t), \qquad \Phi(s) = -\kappa \mathcal{U}(s)}$$

where $\mathcal{U}(s)$ is a **state-only** potential: - **Coverage-deficit potential.** With multi-coverage count field $\mathcal{C}(s,x)$ and requirement K:

$$\mathcal{U}(s) = \int_{\mathcal{R}} \max\big(0,\ K - \mathcal{C}(s,x)\big)\, dx \ \geq 0.$$

Since $\mathcal{U}$ is bounded on bounded $\mathcal{R}$, Φ is bounded and Theorem A.36 applies. As the swarm closes gaps $\mathcal{U}$ falls, so Φ rises and $F > 0$ on improvement, $F < 0$ on degradation. **The requirement that Φ depend on s only, not on (s,a,s'), is the entire content of the theorem. - Energy potential.** $\Phi_{\text{energy}}(s) = -\lambda\,\text{SoC}(s)$, so $F > 0$ when SoC rises — the KiBaM-aware reward (Part D) made policy-invariant. It cannot make "sit on the pad forever" optimal, because the net effect is bounded and telescopes. - **Connectivity potential.** $\Phi_{\text{conn}}(s) = -\mu\lambda_2(s)$ (or $-\mu\times$#components of $G_R(s)$) from Part C, so F rewards increases in algebraic connectivity and penalises fragmentation. This is the correct way to *bias* toward biconnectivity (C.27, C.28) without changing the optimal policy — the **hard** requirement must be enforced separately by a constrained-MARL Lagrangian (A.12.7).

The claim to make in the report, stated precisely.

By Theorem A.36 (Ng, Harada & Russell 1999), adding $F(s,a,s') = \gamma\Phi(s') - \Phi(s)$ to the mission reward leaves the set of optimal policies invariant; by Corollary A.37 all Q- and V-functions are invariant up to the additive potential $\Phi(s)$. By Remark A.38 this holds for **all** policies, and since Φ depends only on s, the shaped objective is $J_{M_0}(\pi) = J_M(\pi) - \Phi(s_0)$, a **constant shift for every policy** — so $\nabla_\theta J_{M_0} = \nabla_\theta J_M$ identically. The shaping therefore cannot change what the optimal policy is; it can only change the finite-sample learning signal, by increasing the density of informative reward along near-optimal trajectories. Conversely, Theorem A.36 (necessity) says a non-potential term — e.g. a per-step bonus $b \cdot \mathbb{1}$[moved closer to the nearest uncovered point], which is a *state-action* function, not a difference of potentials — **can** and generically **will** produce a policy optimal for the shaped problem and suboptimal for the real one. Any coverage bonus in the report not of the form $\gamma\Phi(s') - \Phi(s)$ must therefore be justified as a change to the **task** (i.e. it *is* the reward), not as an auxiliary signal.

Note on a checkable convention error. Some papers use $F = \Phi(s') - \Phi(s)$ (undiscounted form) and call it "potential-inspired". Ng et al. explicitly warn this is **not** covered by Theorem 1 unless $\gamma = 1$ (where the theorem carries the parenthetical "and, if $\gamma = 1$, that $\Phi(s') = 0$"). A report writing $F = \Phi(s') - \Phi(s)$ with $\gamma < 1$ uses the potential-*inspired* form and **loses the guarantee**.

A.12.6 GAE and potential-based shaping — the relationship, stated correctly

These are two different objects and **must not be identified**. (This corrects a common error; the accurate statement is given here in full.)

	Potential-based shaping (A.12)	GAE (A.6)
What is constructed	a new **reward** $r_{t+1} + F(s_t, a_t, s_{t+1})$	an **estimator** $\hat{A}_t$ of the advantage of the *unchanged* reward
Discount used	γ (fixed by the MDP)	$\gamma\lambda$ (a tunable trace decay)
Effect on J	$J_{M_0}(\pi) = J_M(\pi) - \Phi(s_0)$, invariant for **all** π	none — J is untouched; only $\hat{g}$'s bias/variance changes
Policy optimality	provably preserved (Thm A.36)	irrelevant; the objective is untouched
λ or Φ role	Φ is a state-only bias	λ interpolates bias ($\lambda \to 1$) $\to$ variance ($\lambda \to 0$)

The one true structural link. The GAE recursion and the λ-return recursion are the *same recursion shifted by the baseline*:

$$\hat{A}_t^{\mathrm{GAE}} = \delta_t^V + \gamma\lambda \hat{A}_{t+1}^{\mathrm{GAE}} \quad \Longleftrightarrow \quad G_t^\lambda = r_{t+1} + \gamma G_{t+1}^\lambda - \gamma(1-\lambda)V(s_{t+1}),$$

and $G_t^\lambda - V(s_t) = \hat{A}_t^{\mathrm{GAE}}$ (Theorem A.24). That is: **GAE is a TD(λ) backup applied to the advantage rather than the value.** No claim about potential-based reward shaping is involved, and none is correct.

Why the tempting identification fails. If one tries $\Phi = V$ and $\tilde{\gamma} = \gamma\lambda$ in $F(s,a,s') = \tilde{\gamma}\Phi(s') - \Phi(s)$, then the shaped reward is

$$\tilde{r}_{t+1} = r_{t+1} + \gamma\lambda V(s_{t+1}) - V(s_t) = \delta_{t+1}^V + (\gamma\lambda - \gamma)V(s_{t+2}),$$

which is **not** δ_{t+1}^V and whose $\gamma\lambda$-discounted sum telescopes to $-\Phi(s_0)/(1-\gamma\lambda)$ plus the *unchanged* J — not to $\hat{A}^{\mathrm{GAE}}$. Setting $\tilde{\gamma} \neq \gamma$ additionally destroys the policy-invariance guarantee of Theorem A.36 (which is stated for a fixed γ), so the identification is wrong twice over.

A.12.7 When shaping is *not* the right tool: constrained MARL

A hard requirement such as "maintain K-fold coverage **at all times**" or "keep the communication graph 2-connected **at all times**" is a **constraint**, not a preference. Potential-based shaping provably cannot enforce it (it only re-ranks policies that already satisfy the underlying objective). The correct machinery is **CMDP with a Lagrangian**:

$$\max_\pi J_{\mathrm{mission}}(\pi) \quad \text{s.t. } J_{\mathcal{C}}(\pi) \le d, \qquad \mathcal{L}(\pi,\lambda) = J_{\mathrm{mission}}(\pi) - \lambda\big(J_{\mathcal{C}}(\pi) - d\big),\ \lambda \ge 0,$$

with dual ascent $\lambda \leftarrow [\lambda + \eta(J_{\mathcal{C}} - d)]_+$. This is the framework of the NeurIPS 2024 constrained-MARL PPO line and is the mathematically correct way to impose the Part B/C feasibility conditions ($N\pi r^2 \ge KA$, $\lambda_2 > 0$, $\lambda_3 > 0$) on the learned policy. **A report that uses only reward shaping to enforce a hard fault-tolerance constraint is making an error.** For a *hard* persistent-coverage constraint, prefer a safety layer (e.g. a CBF-CQL shield maintaining $\lambda_2 \ge \lambda_2^{\min}$, C.24) over reward.

Part IV — Coverage, graph and energy theory

0.9. Theory Background: Coverage, Graph Connectivity, and Energy

Companion notes for the MAPPO report on persistent, fault-tolerant UAV–IoT coverage.

Problem setting assumed throughout. A region $\Omega \subseteq \mathbb{R}^2$ of area $A = |\Omega|$. There are N UAVs, each with an omnidirectional sensing footprint that is a **disk** $D_i(t) = B(p_i(t), R_{\text{cov}})$ of radius R_{cov}, and M fixed ground sensors located at $s_1, \dots, s_M \in \Omega$. The requirement is K**-fold coverage**:

$$\forall\, q \in \Omega : \qquad N_q := \Big|\big\{ i : \|p_i(t) - q\| \le R_{\text{cov}} \big\}\Big| \;\ge\; K,$$

and in the discrete (sensor-centric) form, which is what the simulation actually constrains,

$$\forall\, m \in \{1, \dots, M\} : \qquad \#\{\, i : \|p_i(t) - s_m\| \le R_{\text{cov}} \,\} \;\ge\; K.$$

Additionally the UAVs must induce a connected communication graph with communication range R_{comm}, must be fault-tolerant to stochastic single-UAV failure, and have finite batteries replenished at a limited number of charging pads.

Date of compilation: 29 September 2026 (web search performed on this date to verify citations).

Scope and honesty note. Every formula below is in LaTeX and has been checked algebraically or by direct numerical recomputation. Where I could **not** verify a bibliographic detail in a live search, I say so explicitly in the text and in Part D rather than fabricating it. Such items are flagged: (i) the "Gordon–Orfao–Rote" persistent-coverage citation (5), (ii) the exact closed-form algebraic constants of the KiBaM constant-current solution and the exact numeric parameter set of Marquis & DeCicca (20), (iii) the DOI `10.1109/CDC.2016.7798253` printed in the report for the Zareh–Sabattini–Secchi biconnectivity paper (16, D), and (iv) a small number of classical pagination details flagged inline.

0.10. PART A — COVERAGE THEORY

0.10.1. 1. The Art Gallery Problem: O'Rourke's formulation, Chvtal's theorem, Fisk's proof

1.1 Definitions (O'Rourke's formulation)

Let $P \subseteq \mathbb{R}^2$ be a **simple polygon**: a closed region bounded by a closed polygonal chain with n vertices whose boundary is non-self-intersecting, and with no holes. A **vertex guard** is a guard placed at one of the n vertices of P, with omnidirectional 360° field of view. A guard g **sees** a point $q \in P$ iff the closed segment $[g, q]$ lies entirely in P.

A set $G \subseteq V(P)$ of vertex guards **guards** P iff

$$\forall\, q \in P : \qquad \exists\, g \in G \text{ with } [g, q] \subseteq P.$$

Write $g(P) = \min\{|G| : G \text{ guards } P\}$ for the **guard number**. Two standard variants: **point guards** (guards anywhere in P, not only at vertices) and **bounded guards** (guards restricted to a point of prescribed radius at each vertex — the variant closest to fixed-radius UAV footprints).

The canonical reference is O'Rourke's monograph, *Art Gallery Theorems and Algorithms*, Cambridge University Press, 1987, which formalises the above and catalogues the $O(n \log n)$–$O(n^2)$ family of exact guard-placement algorithms. The question was posed to Chvtal by Victor Klee in 1973 (O'Rourke, 1987, p. 1).

1.2 Chvtal's theorem, stated with all hypotheses

> **Theorem (Chvtal, 1975).** *Let P be a simple polygon with n vertices and no holes. Then every such polygon can be guarded by at most $\lfloor n/3 \rfloor$ point guards, and the bound is tight: for infinitely many values of n there exist simple polygons with n vertices that require exactly $\lfloor n/3 \rfloor$ vertex guards.*

Hypotheses, itemised (all are load-bearing):

1. P is a **simple** (non-self-intersecting, single-boundary) polygon.
2. P has **no holes**.
3. $n \geq 3$.
4. Guards have **unlimited range** and 360° field of view (no R-range restriction).
5. The bound is *worst-case over all polygons with n vertices*; for a specific convex n-gon, $g(P) = 1$.

Chvtal's original proof is combinatorial (it inductively cuts off a small piece at an ear of the triangulation). Citation: V. Chvtal, "A combinatorial theorem in plane geometry," *Journal of Combinatorial Theory, Series B*, vol. 18, no. 1, pp. 39–41, 1975, doi: 10.1016/0095-8956(75)90061-1.

Extension beyond vertex guards (important for us). Chvtal's upper bound remains valid if the vertex restriction is dropped: $\lfloor n/3 \rfloor$ guards placed at *arbitrary* points of P suffice. Consequently a disk of radius r is a *bounded* guard, and the $\lfloor n/3 \rfloor$ theorem does **not** transfer to k-fold or r-bounded guarding.

Holes. With h holes, Gyri's partition theorem gives a guard set of size $\lfloor (n+2h)/3 \rfloor$: P can be partitioned into $\lfloor (n+2h)/3 \rfloor$ convex polygons, each guarded by one vertex guard of the original polygon. *(Classical; exact volume/page details of Gyri (1986) not re-verified in this session — see Part D.)*

Complexity. Deciding whether P is guardable by at most k guards is **NP-complete** (Lee & Lin, 1975/76, classical; see O'Rourke 1987 and Kahn–Klawe–Kleitman 1986, IEEE Trans. Inf. Theory 32(2), 276–282, doi: 10.1109/TIT.1986.1057165). This is what makes $\lfloor n/3 \rfloor$ a *bound* and not an *algorithm*: computing $g(P)$ is hard.

1.3 Fisk's proof of $\lfloor n/3 \rfloor$ (complete, six steps)

The proof rests on exactly two ingredients: (i) every simple polygon admits a triangulation, and the graph of that triangulation is 3-colourable; (ii) every triangle of the triangulation is guarded by a guard placed at any one of its vertices.

Step 1 — Triangulation exists. Every simple n-gon P admits a straight-line triangulation T into exactly $n-2$ triangles whose vertex set is $V(P)$ and whose union is P with pairwise-disjoint triangle interiors. (Standard, O'Rourke 1987, Ch. 1.)

Step 2 — The triangulation graph is 3-colourable. Let $G_T = (V(T), E(T))$. Claim: G_T has a proper 3-colouring. *Proof by induction on n:* $n = 3$ is trivial. For $n > 3$, the weak dual of T (a vertex per triangle, edges for shared diagonals) is a tree, hence has a leaf, so T has an **ear**: a triangle σ with two boundary edges, whose tip vertex v has degree 2 in G_T. Colour $G_T - v$ by induction; the two neighbours of v use at most two of the three colours; assign to v a colour that is missing among its neighbours. ■ (Equivalently: Grtzsch's theorem — every *triangle-free* planar graph is 3-colourable — combined with the 4-Colour Theorem. The ear induction is the elementary proof and is what Fisk used.)

Step 3 — Colour classes partition the vertices. Write $V(T) = C_1 \dot\cup C_2 \dot\cup C_3$, with each C_j an independent set in G_T. Since $|C_1| + |C_2| + |C_3| = n$, the pigeonhole principle gives

$$\min_j |C_j| \;\leq\; \left\lfloor \frac{n}{3} \right\rfloor .$$

Step 4 — Every triangle sees all three colours. Every face of G_T is a triangle whose three vertices are mutually adjacent, hence pairwise differently coloured; so each triangle contains exactly one vertex of each C_j.

Step 5 — Place guards on the smallest class. Let $C = C_j$ be a class of minimum size and place a guard at each vertex of C. Then $|C| \leq \lfloor n/3 \rfloor$ guards are deployed, and **every** triangle of T contains exactly one vertex of C (by Step 4).

Step 6 — The guards see everything. Fix any $q \in P$. Since the triangles of T cover P, q lies in some triangle $\sigma \in T$. That triangle has a vertex $g \in \sigma \cap C$. A triangle is **convex**, so $[g, q] \subseteq \sigma \subseteq P$; the guard at g sees q. Since q was arbitrary, C guards P. ■

Tightness. Chvtal's "comb" family (a spine with k mutually hidden teeth) has $3k$ vertices, and a single guard can see at most one tooth, so at least $k = \lfloor 3k/3 \rfloor$ guards are necessary. More generally, the standard statement is: for every k there is a polygon P_k with $g(P_k) = \lfloor |V(P_k)|/3 \rfloor$.

Where Fisk appears. S. Fisk, "A short proof of Chvtal's watchman theorem," *Journal of Combinatorial Theory, Series B*, vol. 24, no. 3, p. 374, 1978, doi: 10.1016/0095-8956(78)90059-X. (The result also appeared at FICS 1978, pp. 126–134.)

Reading for the UAV report. Fisk's proof is a *colouring* argument, and the guarantee it delivers is $\lfloor n/3 \rfloor$ guards for **1-fold** coverage of a polygon by **unbounded** guards. Our problem has neither property: it is K-fold, and each guard is a disk of bounded radius R_{cov}. The art-gallery theorem is therefore a **sanity check on the shape of the answer**, **not** a feasibility certificate. The relevant certificate is the area-counting bound of 7.

0.10.2. 2. k-Fold Coverage: Definition, Reduction, Complexity

2.1 Definition

Let P be a polygon and G a set of guards. G is a **k-fold guard set** for P if

$$\forall\, q \in P: \qquad \big|\{\, g \in G : [g,q] \subseteq P \,\}\big| \;\geq\; k.$$

The k-fold guard number is $g_k(P) = \min\{|G| : G \text{ is a } k\text{-fold guard set}\}$. Note $g_1(P) = g(P)$ and, by monotonicity,

$$g_1(P) \;\leq\; g_2(P) \;\leq\; \cdots \;\leq\; g_k(P).$$

In the UAV setting the guard is a disk rather than a point, so the operative notion is **k-fold disk coverage of Ω by N disks of radius R_{cov}**, with the counting function

$$\nu(x) \;=\; \sum_{i=1}^{N} \mathbb{1}\big\{\, x \in B(p_i, R_{\mathrm{cov}}) \,\big\}, \qquad \nu(x) \geq K \;\; \forall x \in \Omega.$$

2.2 Reduction to the standard (1-fold) problem

(a) Decisional reduction, k-fold $\Rightarrow$ 1-fold. If P is k-fold guardable by N guards, it is 1-fold guardable by N guards; the converse fails. Hence the k-fold minimum is a strictly harder optimisation and every 1-fold upper bound transfers downward:

$$g_k(P) \;\geq\; g_1(P) \;\geq\; \left\lfloor \frac{n}{3} \right\rfloor.$$

(b) Constructive reduction, 1-fold $\Rightarrow$ k-fold (the one we actually use). Fisk's Step 5 shows that **every** colour class of a 3-colouring of the triangulation graph is a valid 1-fold guard set, because every triangle contains exactly one vertex of each class. Therefore

$$C_1,\; C_2,\; C_3 \quad \text{are three pairwise disjoint 1-fold guard sets with } |C_1| + |C_2| + |C_3| = n.$$

For $k \leq 3$ this gives a k-fold guard set of size equal to the sum of the k smallest colour classes, hence

$$g_k(P) \;\leq\; \sum_{j=1}^{k} \big(j \text{ smallest } |C_j|\big) \;\leq\; \left\lfloor \frac{kn}{3} \right\rfloor + O(1),$$

and in particular $g_2(P) \leq n - \lceil n/3 \rceil = \lfloor 2n/3 \rfloor$. *(Flag: the sharpest published k-fold bounds in the polygon setting have been improved over the decades; I have not re-verified the current state of the art and am deliberately giving the* derivable* bound rather than the best known one.)*

(c) The reduction that is actually correct for UAVs. k-fold disk coverage of a **continuous** region is *not* an art-gallery problem: art-gallery guards are unbounded points on a polygon. The correct reformulation is a **set multi-cover**: universe Ω (or the M sensor locations), sets $\mathcal{D}_i = B(p_i, R_{\mathrm{cov}}) \cap \Omega$, and the task is to choose N sets such that every universe element is covered at least K times, minimising N (equivalently minimising R_{cov}, or maximising K — the three coincide at the optimum). This is the (N, K) **multi-cover / k-fold covering** problem. Generic polynomial approximation is via weighted set cover, for which the $H_N = \Theta(\ln N)$ ratio is optimal under standard complexity hypotheses (Johnson 1974; Chvtal 1979; for the *connected* variant see Raz & Hu, "Connected set cover problem and its applications," Proc. AAIM 2006, LNCS vol. 4041, pp. 243–254).

2.3 Complexity summary

Problem	Status	Reference
Guard a simple n-gon with $\leq k$ guards	NP-complete	Lee & Lin (1975/76), classical; via Kahn–Klawe–Kleitman 1986, IEEE Trans. Inf. Theory 32(2), 276–282
Cover a polygon with m disks of radius r (decision)	NP-hard / NP-complete	Hershberger & Suri, "Disk covering: a greedy approximation algorithm," SODA 1999 (classical; details not re-verified)

Problem	Status	Reference
k-fold version of either	NP-hard for every $k \geq 1$; inherits the H_N-factor hardness of set multi-cover	—
Connected coverage (coverage + connected backbone)	NP-hard	MRCTC NP-hard — Fraigniaud, Larsen & Misra, 2006

Bottom line for the report. The K-coverage feasibility check inside the MAPPO reward is a *feasibility test on a given configuration*, not a combinatorial optimisation, so NP-hardness does not hurt the training loop. What we need from theory is a **necessary condition** — a cheap, computable certificate of infeasibility — and that is exactly 7.

0.10.3. 3. Connected Multi-Robot Coverage

3.1 Definition

Let $R = \{r_1, \ldots, r_N\}$ be mobile robots in an environment Q (here Ω), each sensing a disk $B(r_i, r_s)$ of radius r_s and communicating within range r_c. Two problems:

- **Discretised / static connected coverage.** A set $S \subseteq R$ of robots is *simultaneously covering* if $\bigcup_{i \in S} B(p_i, r_s) \supseteq \Omega$. A set S is *simultaneously connected* if the graph $\mathcal{G}(S) = (S, E_S)$ with $E_S = \{\{i, j\} : \|p_i - p_j\| \leq r_c\}$ is connected. **Connected coverage** is the *existential* combination.
- **Sweep / continuous connected coverage.** There must exist a schedule of the robots such that at every instant the active set is simultaneously covering *and* connected. This is strictly stronger and is the model relevant to a persistent mission.

3.2 Discretisation

The continuous problem is attacked by a **cellular decomposition**: partition Ω into a finite set of cells $\{\Sigma_1, \ldots, \Sigma_c\}$ (grid cells, Morse cells, or a Voronoi/Delaunay tessellation), compute a **spanning tree** of the cell-adjacency graph, and convert it into a coverage tour. The exact-realisation step assigns each cell a single representative (sweep) point and proves that if the representative lies inside an ε-thickened cell, the whole cell is covered. This is the basis of the Bouaziza–Debled **DARP** algorithm ("DARP: divide areas algorithm for optimal multi-robot coverage path planning," *J. Intell. Robot. Syst.*, Springer, doi: 10.1007/s10846-016-0461-x) and of the sweep-coverage literature generally.

3.3 NP-hardness and approximation ratios

- **Connected coverage on a graph is NP-hard** in essentially every formulation. For **trees** it is NP-hard: the *Multi-Robot Connected Tree Coverage* (MRCTC) problem is NP-hard (Fraigniaud, Larsen & Misra, 2006). For **general topological graphs with separate movement and communication edge sets**, multi-robot coverage and multi-agent path finding are **PSPACE-complete** (Charrier et al., 2020).
- **Exact generalisation.** Coverage of graphs of bounded treewidth is solvable exactly by dynamic programming. The *Heterogeneous Multi-Robot Graph Coverage with Proximity and Connectivity Constraints* problem (AAAI 2025; document verified at `u.cs.biu.ac.il/~sarit/data/articles/AAAI_25__k_Tree_Coverage_with_C` gives a PTAS with approximation error $1 + O(\epsilon)$ independent of d for the $d = 2$, $k = 3$, tree case. *(I verified the document exists and the claim as stated in it; the author list should be read off the PDF before citing.)*
- **Approximation ratios — an honesty point.** In the tree case the literature reports **speedup factors** γ (the ratio of multi-robot to single-robot traversal time) rather than a ratio to the true optimum. This is frequently glossed over in secondary summaries. For the general graph case, the standard technique is *connection fusion* (Czyzowicz, Kowalski, Dryniecki & Stojmenovi): take a ρ-approximate set cover and connect its components by adding at most a constant number of edges per component, yielding a $O(\rho)$-approximation for the connected variant. For Euclidean static connected cover, geometric set cover gives $O(1)$ ratios for disks. *(Full pagination for the connection-fusion paper not re-verified — see Part D.)*

Consequence for our design. The K-coverage requirement and the connectivity requirement are coupled but *separable*: coverage is controlled by R_{cov} and N (area counting, 7); connectivity is controlled by R_{comm} and the relay structure (13–15). The two radii are independent knobs.

0.10.4. 4. Persistent / Continuous Coverage: the Steady-State Problem

4.1 Formulation

The **steady-state coverage problem** (a.k.a. minimum number of sensors for continuous coverage) is:

> Given a region Ω and a sensing radius r, find the minimum N such that there exist positions $p_1, \dots, p_N \in \Omega$ with
>
> $$\Omega \subseteq \bigcup_{i=1}^{N} B(p_i, r).$$

The *persistent* form replaces "there exist positions" by "there exist trajectories $p_i(t)$ such that the condition holds $\forall t$". For a static Ω and fixed N this is *equivalent* to the static form (place the robots at the static solution and never move). The genuine difference appears when robots have **kinematic constraints** (speed limits, collision avoidance) or when the optimised quantity is the **time between consecutive visits** (latency / sweep period). Our problem is the latter.

4.2 Minimum sensors for continuous coverage of a disk region — actual formulas

Let $\Omega = B(0, R)$ and let N disks of radius $r < R$ cover it. There are **two independent necessary conditions**, and their maximum is far tighter than the area bound alone.

(D1) Area condition.

$$\pi R^2 \;\le\; \sum_{i=1}^{N} \pi r^2 = N\pi r^2 \quad\Longrightarrow\quad N \;\ge\; \frac{R^2}{r^2}, \qquad \text{i.e.} \qquad r \;\ge\; \frac{R}{\sqrt{N}}.$$

(D2) Boundary-arc condition (the binding one). Each covering disk of radius r covers an arc of the boundary circle of angular half-width θ satisfying $\cos\theta = \dfrac{1 + s^2 - r^2}{2s}$, where s is the offset of its centre from the origin. Maximising over s:

$$\cos\theta_{\max} \;=\; \min_{s>0} \frac{1 + s^2 - r^2}{2s} \;=\; \sqrt{1 - r^2} \qquad (\text{attained at } s^\star = \sqrt{1 - r^2}),$$

so each disk covers at most $2\arcsin r$ radians of the 2π-radian boundary. Since the boundary must be covered,

$$\boxed{N \;\ge\; \frac{\pi}{\arcsin(r/R)}}, \qquad \text{equivalently} \qquad \boxed{r \;\ge\; R\sin\!\left(\frac{\pi}{N}\right)}$$

(D3) Combined necessary condition for a disk region:

$$N_{\min}(r, R) \;\ge\; \max\left\{ \left\lceil \frac{R^2}{r^2} \right\rceil,\; \left\lceil \frac{\pi}{\arcsin(r/R)} \right\rceil \right\}.$$

Exact solved case: $r = R/2$**.** Then (D2) gives $N \ge \pi/\arcsin(1/2) = \pi/(\pi/6) = 6$, and the saturating configuration is forced: six centres at the vertices of a regular hexagon of circumradius $s^\star = \sqrt{1 - (1/2)^2} = \sqrt{3}/2$ (in units of R). Along a ray at the mid-arc angle, the distance from radius ρ to the nearest centre is $\sqrt{\rho^2 + 3/4 - \frac{3}{2}\rho}$, which equals $1/2$ at $\rho = 1/2$ and at $\rho = 1$ and is $\le 1/2$ in between (the quadratic is minimised at $\rho = 3/4$, giving $\sqrt{3/16} \approx 0.433$). So the six disks cover the annulus $\frac{1}{2} \le \rho \le 1$ completely, but leave $\rho < 1/2$ uncovered (those points are at distance $\sqrt{3}/2 > 1/2$ from every centre). A seventh disk at the origin is therefore both necessary and sufficient:

$$\boxed{N_{\min}(R/2,\; R) \;=\; 7}$$

(Other exact values for other r/R exist in the classical covering literature; I have not re-verified them.)

(D4) Asymptotics (Kershner). The asymptotically thinnest covering of the plane (and of any smooth-bounded region, up to $o(\cdot)$ boundary terms) by congruent disks has density

$$\Theta_{\text{cov}} \;=\; \frac{2\pi}{3\sqrt{3}} \;=\; 1.2091996\ldots$$

attained by the triangular lattice of centres with spacing $\sqrt{3}\,r$, whose Voronoi cells are regular hexagons of circumradius exactly r. Hence

$$N_{\min}(r,R) \;=\; \frac{2}{3\sqrt{3}}\,\frac{R^2}{r^2}\,\big(1+o(1)\big) \;=\; 0.38490018\ldots\,\frac{R^2}{r^2}\,\big(1+o(1)\big).$$

This is Kershner's theorem (1939). *(The constant is standard and I am confident in it; the exact volume/page details of the Kershner paper are flagged in Part D.)*

4.3 Minimum sensors for continuous coverage of a polygon

For a polygon P of area A and perimeter P_{per}:

- **Lower bound (area).** $N\pi r^2 \geq A \Rightarrow N \geq A/(\pi r^2)$, improved asymptotically to $N \geq \dfrac{2A}{3\sqrt{3}\,r^2}$ (Kershner density, 6).
- **Boundary correction.** The boundary contributes an additive $O(P_{\text{per}}/r)$ term in general position; for a convex region with $A \gg P_{\text{per}} \cdot r$ this is negligible, which is why the square and the disk behave identically asymptotically.
- **Exactness.** Deciding whether m disks of radius r cover a polygon is NP-complete (Hershberger & Suri, SODA 1999, classical). The clean linear-time algorithm exists only in the *point-guard / unbounded-guard* case, where Fisk's argument gives $\lfloor n/3 \rfloor$ (1). The two must not be conflated.

0.10.5. 5. Persistent Coverage Complexity (cell decomposition and the approximation ratio)

5.1 Honest statement of what I could and could not verify

The report cites a result of the form *"Gordon, Orfao, Rote — cell decomposition, approximation ratio ρ"*. **I could not verify this reference in a live search on 29 September 2026.** Specifically: I retrieved and read the complete CCCG 2008 proceedings index (57 papers, pages 1–226) and it contains no Gordon/Orfao/Rote paper; and no cccg, springer, ACM or IEEE record matching a Gordon–Orfao–Rote persistent-coverage paper surfaced. I therefore (i) state the mechanism of the cell-decomposition result generically, which I can do correctly and self-containedly, and (ii) explicitly warn that **the specific paper and ratio must be re-sourced before the report cites them.**

5.2 The mechanism (stated correctly and self-containedly)

Cell decomposition for steady-state coverage. Let Ω be a polygon. Compute a partition

$$\Omega \;=\; \Sigma_1 \dot\cup \cdots \dot\cup \Sigma_c$$

(e.g. the arrangement induced by the candidate site set, or a grid with appropriate cell aspect ratio). Build the cell-adjacency graph $\mathcal{H} = (\{\Sigma_j\}, E_{\mathcal{H}})$ where $\Sigma_i \sim \Sigma_j$ iff $\overline{\Sigma_i} \cap \overline{\Sigma_j} \neq \emptyset$, and take a spanning tree $T_{\mathcal{H}}$ of $\mathcal{H}$. Then:

- **Upper bound (cover construction).** If each cell Σ_j admits a single site p_j whose radius-r disk covers Σ_j (equivalently, p_j covers the cell's *sweep set*), then $\{p_1, \ldots, p_c\}$ covers Ω; and because $T_{\mathcal{H}}$ is connected, the *cell-adjacency graph induced by the chosen sites* is connected at radius $r + \Delta$, where Δ is the diameter bound used in the "extra-thickness" step. Standard treatment: work with a slightly inflated radius $r \to r+\delta$ and pay for the thicket.
- **Lower bound (certificate).** Conversely, any cover by N disks gives a *counting* certificate: each of the N disks can witness at most one assigned cell, so $c \geq N_{\min}$. This is a linear-programming / count-based lower bound identical in spirit to 7.
- **Ratio.** With the cover constructed this way, the number of sites produced is c and the certifiable optimum is $N_{\min}$, so

  $$\rho \;=\; \frac{c_{\text{produced}}}{N_{\min}}.$$

  Any claimed constant ratio must be reported as a ratio to the Kershner denominator

  $$N_{\min} \;=\; \frac{2A}{3\sqrt{3}\,r^2}\,(1+o(1)),$$

  because a ratio taken against the naive $A/(\pi r^2)$ differs by exactly the factor $\frac{2\pi}{3\sqrt{3}} = 1.20920$ (6). This factor confusion is the most common error in this part of the literature and is worth an explicit footnote in the report.

What to do. Either (a) obtain and cite the actual Gordon–Orfao–Rote paper and insert its exact ratio, or (b) replace the citation with a result that can be verified, e.g. the $O(1)$ connected set-cover bound obtained by connection fusion, or the bounded-treewidth PTAS of the AAAI-2025 graph-coverage work.

0.10.6. 6. Disk-Covering Asymptotics: VERIFICATION OF THE CLAIMED CONSTANT

6.1 The claim under test

"N disks of radius r cover area A if $N \geq \dfrac{A}{2\sqrt{3}\,r^2}$, from hexagonal packing density $\dfrac{\pi}{2\sqrt{3}}$."

6.2 Verification: the claimed constant is INCORRECT — it is wrong by exactly a factor of 4/3

The constant $\frac{\pi}{2\sqrt{3}} = 0.90689968\ldots$ is indeed the **maximal packing density** of congruent disks in the plane (Thue/Fedorov; the hexagonal lattice). It is *not* a covering density. The two constants are

$$\underbrace{\delta_{\text{pack}} = \frac{\pi}{2\sqrt{3}} = 0.90689968\ldots}_{\text{densest PACKING: max disk-area fraction in the plane}} \qquad \underbrace{\Theta_{\text{cov}} = \frac{2\pi}{3\sqrt{3}} = 1.20919958\ldots}_{\text{thinnest COVERING: min disk-area fraction to cover the plane (Kershner)}} .$$

Derivation of the packing constant (this is where $2\sqrt{3}$ legitimately comes from). In the densest packing (triangular lattice of centre spacing $2r$), the primitive cell is a rhombus of area $2r \cdot \sqrt{3}r = 2\sqrt{3}\,r^2$ containing one disk of area πr^2, so $\delta_{\text{pack}} = \pi r^2/(2\sqrt{3}r^2) = \pi/(2\sqrt{3})$. Thue's / Fedorov's theorem says no packing is denser. Consequence: N **pairwise-disjoint disks of radius r occupy total area $N\pi r^2$, which cannot exceed $\frac{\pi}{2\sqrt{3}}$ of the ambient area A:**

$$N\pi r^2 \;\leq\; \frac{\pi}{2\sqrt{3}}\,A \quad\Longrightarrow\quad N \;\leq\; \frac{A}{2\sqrt{3}\,r^2} \;=\; 0.28867513\ldots\,\frac{A}{r^2}.$$

This is a bound on how many disks fit **without overlapping**. It is a statement about *packing*, and its inequality direction is $\leq$, whereas the claim asserts a *covering* statement with direction $\geq$. **The claim has taken a packing bound, reversed its inequality, and mislabelled it a covering bound.**

The correct covering constant and its derivation. To *cover* a region the disks must **overlap** slightly so no gaps appear. Kershner's theorem states that the asymptotic minimum density of a covering of the plane by congruent circles of radius r is $\frac{2\pi}{3\sqrt{3}}$. Construction attaining it: the triangular lattice of centres with spacing

$$s \;=\; \sqrt{3}\,r,$$

whose Voronoi cells are regular hexagons of circumradius $s/\sqrt{3} = r$ (hence each cell is covered by the disk at its centre) and of area $\frac{3\sqrt{3}}{2}R_h^2 = \frac{3\sqrt{3}}{2}r^2 = 2.59807621\,r^2$ for circumradius $R_h = r$. One disk per cell gives density

$$\frac{\pi r^2}{\frac{3\sqrt{3}}{2}r^2} \;=\; \frac{2\pi}{3\sqrt{3}} \;=\; 1.20919958\ldots$$

Correct asymptotic covering bound:

$$\boxed{N \;\geq\; \frac{2A}{3\sqrt{3}\,r^2} \;=\; 0.38490018\ldots\,\frac{A}{r^2}\,\big(1+o(1)\big)} \qquad \text{equivalently} \qquad \boxed{r \;\geq\; \frac{1}{\sqrt{N}}\sqrt{\frac{2A}{3\sqrt{3}}} \;=\; 0.62040324\ldots\sqrt{\frac{A}{N}}}$$

The gap, exactly. The claimed constant $1/(2\sqrt{3})$ and the true constant $2/(3\sqrt{3})$ differ by exactly

$$\frac{2/(3\sqrt{3})}{1/(2\sqrt{3})} \;=\; \frac{2}{3}\cdot 2 \;=\; \frac{4}{3} \;=\; 1.\overline{3},$$

so the claim **under-counts the number of required disks by 33.3 %.** As a *radius* bound the error is $\sqrt{4/3} = 2/\sqrt{3} = 1.15470$, i.e. the claim implies a radius 15.5 % too small.

6.3 Three-way summary table

Model	Condition on N	Constant	Source of the constant
Naive area counting	$N \geq \dfrac{A}{\pi r^2}$	$\frac{1}{\pi} = 0.31831$	total disk area $\geq$ region area
Correct asymptotic covering	$N \geq \dfrac{2A}{3\sqrt{3}r^2}$	$\frac{2}{3\sqrt{3}} = 0.38490$	Kershner covering density $\frac{2\pi}{3\sqrt{3}}$
Packing (disjoint disks)	$N \leq \dfrac{A}{2\sqrt{3}r^2}$	$\frac{1}{2\sqrt{3}} = 0.28868$	Thue/Fedorov packing density $\frac{\pi}{2\sqrt{3}}$

Note that the correct covering constant (0.38490) exceeds even the naive area constant (0.31831) by the factor $\frac{2\pi}{3\sqrt{3}} = 1.20920$: **you need 20.9 % more disk-area than the region's area, purely to close the gaps between disks.**

6.4 K-fold version

For K-fold coverage the counting argument of 7 gives $KA \leq N\pi r^2$. The Kershner-corrected version stacks K triangular lattices with mutually incommensurate relative offsets (so the K lattices' gaps never coincide), achieving density $K \cdot \frac{2\pi}{3\sqrt{3}}$, and therefore

$$\boxed{KA \;\leq\; \frac{3\sqrt{3}}{2}\,Nr^2 \quad\Longleftrightarrow\quad r \;\geq\; \sqrt{\frac{2KA}{3\sqrt{3}\,N}} \;=\; 0.62040324\sqrt{\frac{KA}{N}}}$$

and equivalently $N \geq \frac{2}{3\sqrt{3}}\frac{KA}{r^2}$. The correction factor relative to the naive area bound is exactly $\sqrt{\frac{2\pi}{3\sqrt{3}}} = 1.09964$ in radius, i.e. 20.9 % in area.

0.10.7. 7. THE KEY FEASIBILITY BOUND — Proof from First Principles

7.1 Statement

Theorem 7.1 (Counting bound). *Let $\Omega \subseteq \mathbb{R}^2$ be a bounded measurable region of area A, and let $D_1, \ldots, D_N$ be disks with centres p_i and common radius $r > 0$. Define the multiplicity (counting) function*

$$\nu(x) \;=\; \sum_{i=1}^{N} \mathbb{1}_{D_i}(x) \in \mathbb{N}_0.$$

If every point of Ω is covered at least K times, i.e. $\nu(x) \geq K$ for all $x \in \Omega$, then

$$\boxed{N\,\pi r^2 \;\geq\; K\,A \qquad \text{and hence} \qquad r \;\geq\; \sqrt{\frac{KA}{\pi N}} \;=\; \frac{1}{\sqrt{\pi}}\sqrt{\frac{KA}{N}} \;=\; 0.56418958\ldots\sqrt{\frac{KA}{N}}}$$

7.2 Proof (every line justified)

Step 0 — well-definedness. Ω is measurable with $|\Omega| = A$; each D_i is a closed disk, so $\mathbb{1}_{D_i}$ is measurable; hence ν is measurable. Moreover $\nu \leq N$ on Ω, so ν is bounded and therefore Lebesgue-integrable, and $\int_\Omega \nu$ is finite.

Step 1 — lower-bound the integral using the hypothesis. Since $\nu(x) \geq K$ for all $x \in \Omega$,

$$\int_\Omega \nu(x)\,dx \;\geq\; \int_\Omega K\,dx \;=\; K\,|\Omega| \;=\; KA. \tag{7.1}$$

Step 2 — rewrite the integral as a finite sum. By linearity of the integral (a finite sum, so Tonelli is not even needed),

$$\int_\Omega \nu(x)\,dx \;=\; \sum_{i=1}^{N} \int_\Omega \mathbb{1}_{D_i}(x)\,dx \;=\; \sum_{i=1}^{N} \big|\,\Omega \cap D_i\,\big|. \tag{7.2}$$

Step 3 — bound each term by the full disk area. Since $\Omega \cap D_i \subseteq D_i$ and D_i is a disk of radius r,

$$\big|\,\Omega \cap D_i\,\big| \;\le\; |D_i| \;=\; \pi r^2, \qquad \text{hence} \qquad \sum_{i=1}^{N} \big|\Omega \cap D_i\big| \;\le\; N\,\pi r^2. \tag{7.3}$$

Step 4 — chain the inequalities. From (7.2) and (7.3), $\int_\Omega \nu \le N\pi r^2$. Combining with (7.1):

$$KA \;\le\; \int_\Omega \nu(x)\,dx \;\le\; N\pi r^2,$$

i.e. $N\pi r^2 \ge KA$. Dividing by $\pi N > 0$ and taking the (positive) square root:

$$r^2 \;\ge\; \frac{KA}{\pi N} \quad\Longrightarrow\quad r \;\ge\; \sqrt{\frac{KA}{\pi N}}. \qquad \blacksquare$$

When is the bound tight? Equality $N\pi r^2 = KA$ requires *simultaneously*: (a) $\Omega \cap D_i = D_i$ for every i, i.e. every disk lies entirely inside Ω (impossible in practice whenever Ω is a polygon and disks are interior); and (b) $\nu(x) = K$ almost everywhere on Ω, i.e. a *perfect* K-fold cover with zero wasted overlap. Condition (b) alone is essentially never satisfiable for $K \ge 2$ and a finite set of disks, because overlap is unavoidable; 6 quantifies how much.

7.3 Why necessary but NOT sufficient — four independent failure modes

(i) Overlap is unavoidable, so $\sum_i |\Omega \cap D_i|$ strictly over-counts $\int_\Omega \nu$. The bound treats the N disks as if their K-fold coverage were delivered without waste. But a genuine cover of a connected region by more than one disk must have overlapping disks (otherwise the region would be disconnected). Formally, for $N \ge 2$ covering a connected Ω with $K \ge 1$, $\sum_i |\Omega \cap D_i| > |\Omega|$ strictly, so $\int_\Omega \nu > KA$ strictly and the count of "coverage budget" is always exceeded. The bound is therefore *strictly* violated by every real solution and the true required N is strictly larger than $KA/(\pi r^2)$. **This is exactly what 6 corrects.**

(ii) Distinctness / degeneracy. $N\pi r^2 \ge KA$ is a statement about N, r, K, A only; it says nothing about the positions. Placing two UAVs at the same point wastes an entire disk, and the bound cannot detect it. Conversely, the bound also cannot *certify* feasibility: it is a constraint on the magnitudes only, so many configurations satisfying it still fail.

(iii) Boundary effects — a concrete counterexample. Take $\Omega = [0,100]^2$, $A = 10\,000$, $N = 1$, $K = 1$. The bound gives

$$r \;\ge\; \sqrt{\frac{1 \cdot 10\,000}{\pi \cdot 1}} \;=\; \sqrt{3183.0989} \;=\; 56.4188 \text{ m}.$$

But a disk covering a 100×100 square must have radius at least the square's **circumradius**, $\frac{100\sqrt{2}}{2} = 50\sqrt{2} = 70.7107$ m. So $r = 56.42$ m satisfies Theorem 7.1 and yet does **not** cover Ω. The gap is $70.71/56.42 = 1.253$. For a long thin rectangle 100×1 ($A = 100$), the bound gives $r \ge \sqrt{100/\pi} = 5.6419$ m while the true minimum is the circumradius $\frac{1}{2}\sqrt{100^2 + 1^2} = 50.0025$ m — a gap of $8.9\times$. **The area bound ignores geometry entirely; for elongated or non-convex regions it can be off by an order of magnitude.**

(iv) Non-convexity and obstacles. If Ω is disconnected or has holes, the optimal N is the *sum* of the optima over the components, and the area bound (which treats Ω as one blob) is again loose. If UAVs are forbidden from flying over some sub-region, the disks are truncated, and $N\pi r^2$ counts area that is not usable.

Packing-corrected version (the one to put in the report). Combining Theorem 7.1 with the Kershner covering density of 6:

$$\boxed{\text{If } \Omega \text{ admits a } K\text{-fold cover by } N \text{ disks of radius } r, \text{ then} \quad r \;\ge\; \sqrt{\frac{2KA}{3\sqrt{3}\,N}} \;=\; 0.62040324\sqrt{\frac{KA}{N}},}$$

$$\boxed{\text{equivalently} \quad N \;\ge\; \left\lceil \frac{2KA}{3\sqrt{3}\,r^2} \right\rceil.}$$

For K-fold coverage this is asymptotically attainable (stack K hexagonal lattices with incommensurate offsets), so unlike the naive bound it is not merely conservative — up to the $O(K)$ boundary terms it is the *correct* threshold for large regions.

Summary of the three radii (all for the same problem).

$$r \geq \underbrace{0.56418958}_{\text{naive area, 7.1}} \sqrt{\frac{KA}{N}} \leq r \geq \underbrace{0.62040324}_{\text{Kershner-corrected, 6/7.3}} \sqrt{\frac{KA}{N}} \leq r \geq \underbrace{0.70710678}_{\text{circumradius of a square region, worst case}} \cdot \frac{\sqrt{A}}{2} \cdot \sqrt{\frac{N}{N}} \text{ (sha}$$

The report should quote the middle one, and should note the shape-dependent upper bound is a *further* necessary condition that the area argument cannot see.

0.10.8. 8. Worked Numerical Example — $A = 100 \times 100$, $M = 20$, $N = 5$, $K = 2$, $R_{\mathbf{cov}} = 15$ **m**

8.1 Given data

Symbol	Value
Region	$\Omega = [0, 100]^2$, a 100 m × 100 m square
$A = \lvert\Omega\rvert$	$100 \times 100 = 10\,000$ m^2
M (ground sensors)	20
N (UAVs)	5
K (required redundancy)	2
R_{cov} as given in the report	15 m

8.2 Step 1 — The naive area bound says what $R_{\mathbf{cov}}$ must be

$$r \geq \sqrt{\frac{KA}{\pi N}} = \sqrt{\frac{2 \times 10\,000}{\pi \times 5}} = \sqrt{\frac{20\,000}{15.7079633}}.$$

Arithmetic, step by step:

1. $K \cdot A = 2 \times 10\,000 = 20\,000$.
2. $\pi \cdot N = 3.14159265\ldots \times 5 = 15.7079633\ldots$
3. $20\,000 \div 15.7079633 = 1273.2395\ldots$ (check: $15.7079633 \times 1273.24 = 20\,000.0$).
4. $\sqrt{1273.2395}$: $35.68^2 = 1273.0624$; $35.69^2 = 1273.7761$; interpolating, $\sqrt{1273.2395} = 35.6825\ldots$

$$\Rightarrow \quad r \geq 35.68 \text{ m} \qquad \text{(naive necessary condition)}$$

8.3 Step 2 — The Kershner-corrected bound

$$r \geq \sqrt{\frac{2KA}{3\sqrt{3}\,N}} = \sqrt{\frac{2 \times 20\,000}{3\sqrt{3} \times 5}} = \sqrt{\frac{40\,000}{25.9807621}}.$$

1. $3\sqrt{3} = 3 \times 1.7320508 = 5.1961524$; times $N = 5$ gives 25.9807621.
2. $40\,000 \div 25.9807621 = 1539.6007\ldots$ (check: $25.9807621 \times 1539.6 = 40\,000.0$).
3. $\sqrt{1539.6007} = 39.2377\ldots$ (check: $39.238^2 = 1539.6206$; $39.2377^2 = 1539.597$).

$$\Rightarrow \quad r \geq 39.24 \text{ m} \qquad \text{(Kershner-corrected necessary condition)}$$

Cross-check via the correction factor: $35.6825 \times \sqrt{2\pi/(3\sqrt{3})} = 35.6825 \times 1.099637 = 39.2377$ m. ✓ Consistent.

8.4 Step 3 — Test the proposed $R_{\mathbf{cov}} = 15$ m directly

Total disk-area available:

$$N\pi r^2 \;=\; 5 \times \pi \times 15^2 \;=\; 5 \times 3.14159265 \times 225 \;=\; 1125\pi \;=\; 3534.2917 \text{ m}^2.$$

Required:

$$KA \;=\; 2 \times 10\,000 \;=\; 20\,000 \text{ m}^2.$$

Ratio:

$$\frac{N\pi r^2}{KA} \;=\; \frac{3534.2917}{20\,000} \;=\; 0.17671.$$

Verdict: $N\pi r^2 = 3534.29 \ll KA = 20\,000$**. The configuration is short by a factor of** $20000/3534.29 = 5.659$**.**

Maximum achievable redundancy at $R_{\mathbf{cov}} = 15$ **m,** $N = 5$**:**

$$K_{\max} \;=\; \left\lfloor \frac{N\pi r^2}{A} \right\rfloor \;=\; \left\lfloor \frac{3534.2917}{10\,000} \right\rfloor \;=\; \Big\lfloor 0.35343 \Big\rfloor \;=\; 0.$$

So with $R_{\mathbf{cov}} = 15$ **m and** $N = 5$**, the region cannot even be single-covered, let alone double-covered.** (Also check $K = 1$ directly: $r \geq \sqrt{A/(\pi N)} = \sqrt{10\,000/15.7079633} = \sqrt{636.6198} = 25.2311$ m > 15 m.) $\times$

8.5 Step 4 — What N would be needed at $R_{\mathbf{cov}} = 15$ m

Naive area bound:

$$N \;\geq\; \frac{KA}{\pi r^2} \;=\; \frac{20\,000}{\pi \times 225} \;=\; \frac{20\,000}{706.8583} \;=\; 28.2944 \;\Rightarrow\; N \;\geq\; 29.$$

Kershner-corrected:

$$N \;\geq\; \frac{2KA}{3\sqrt{3}\,r^2} \;=\; \frac{2}{3\sqrt{3}} \cdot \frac{20\,000}{225} \;=\; 0.38490018 \times 88.8889 \;=\; 34.2133 \;\Rightarrow\; N \;\geq\; 35.$$

8.6 Step 5 — The $M = 20$ sensors, and what the report's discrete model actually needs

Important caveat first. The area bound of 7 constrains the *continuous* region. The simulation in the report constrains only the $M = 20$ sensor points. The two differ, and the point-set version is only decidable once a layout is assumed. Below I use the natural **uniform layout**: partition $[0, 100]^2$ into a 4×5 grid of cells of size 25 m $\times$ 20 m and place one sensor at each cell centre (20 cells, 20 sensors). I flag this as an assumption, not a consequence of $M = 20$.

Required incidences. $K = 2$ coverage of 20 sensors needs at least $2 \times 20 = 40$ (UAV, sensor) incidences. Each UAV contributes at most (number of sensors inside its radius-r disk).

How many sensors fit in a radius-r disk on this grid? Sensor coordinates are $\{12.5 + 25a\} \times \{10 + 20b\}$ for integers a, b in the valid ranges, so the spacings are 25 m in x and 20 m in y. A disk of radius r contains a set of grid points iff it contains the axis-aligned bounding rectangle of those points, i.e. iff the half-diagonal of that rectangle is $\leq r$.

- 2 sensors side by side in x: span 25 m half-diagonal 12.5 m. Affordable at $r = 15$.
- 2 sensors stacked in y: span 20 m half-diagonal 10 m. Affordable at $r = 15$.
- 3 sensors in an L: span 25×20 half-diagonal $\frac{1}{2}\sqrt{25^2 + 20^2} = \frac{1}{2}\sqrt{1025} = \frac{1}{2}(32.01562) = 16.0078$ m. **Not affordable at** $r = 15$**.**
- 3 in a row: span 50 m 25 m. Not affordable.
- 4 sensors (a 2×2 block): span 25×20 16.0078 m. Not affordable at $r = 15$.

Therefore, at $R_{\mathbf{cov}} = 15$ **m on a uniform layout, each UAV can cover at most 2 sensors.** With $N = 5$ that is at most $5 \times 2 = 10$ incidences, whereas even $K = 1$ requires 20 incidences. $R_{\mathbf{cov}} = 15$ **m fails the sensor-level** $K = 1$ **requirement by a factor of 2 on this layout.**

What radius does the uniform layout need?

- $K = 1$, $N = 5$: each UAV must cover 4 sensors a 2×2 block $r \geq 16.0078$ m.
- $K = 2$, $N = 5$: each UAV must cover 8 sensors the tightest 8-point set is a 3×3 cell block minus a corner, whose 8 centres span $2 \times 25 = 50$ m in x and $2 \times 20 = 40$ m in y half-diagonal $\frac{1}{2}\sqrt{50^2 + 40^2} = \frac{1}{2}\sqrt{4100} = \frac{1}{2}(64.03124) = 32.0156$ m. Hence the layout requires $r \geq 32.0156$ m.

8.7 Step 6 — Putting all bounds side by side

Criterion	Type	Required R_{cov}	Binding?
Naive area count (7.1)	necessary	≥ 35.68 m	yes
Kershner-corrected (6/7.3)	necessary, asymptotically tight	≥ 39.24 m	**yes — the binding constraint**
Uniform 4×5 sensor grid, $K = 2$	necessary, layout-dependent	≥ 32.02 m	no (weaker)
Uniform 4×5 sensor grid, $K = 1$	necessary, layout-dependent	≥ 16.01 m	no
Value given in the report	—	15.00 m	**FAILS ALL OF THE ABOVE**

8.8 Verdict on the report's $R_{\text{cov}} = 15$ m

$R_{\text{cov}} = 15$ m is infeasible, and not marginally so. Concretely:

1. **It fails the continuous $K = 1$ condition.** R_{cov} must satisfy $r \geq \sqrt{A/(\pi N)} = 25.23$ m even for single coverage. $15 < 25.23$. Short by a factor $25.2311/15 = 1.682$.
2. **It fails the continuous $K = 2$ condition** by a factor $39.2377/15 = 2.616$ (Kershner) or $35.6825/15 = 2.379$ (naive area).
3. **The deficit is severe.** The five disks at $r = 15$ supply $N\pi r^2 = 3534.29$ m^2 of coverage budget against a requirement of $KA = 20\,000$ m^2 — only 17.7 % of what is needed. The maximum achievable K is $\lfloor 0.353 \rfloor = 0$.
4. **It fails the sensor-level $K = 1$ condition** on the natural uniform layout: each UAV reaches only 2 of the 20 sensors, so 5 UAVs give 10 incidences where 20 are needed.

Minimum fixes, any one of which is necessary:

- **Increase R_{cov} to at least 39.24 m** (recommended; keeps $N = 5$). At exactly $r = 39.24$ m a disk contains a square of side $r\sqrt{2} = 55.50$ m, which spans 2 grid columns (50 m) and 2 grid rows (40 m), so it reaches the 9 sensors of a 3×3 cell block, comfortably supplying the 8 sensors per UAV that $K = 2$ requires. Feasibility at $r = 39.24$ m is therefore supported by *both* the area argument *and* the discrete layout.
- **Or increase N to at least 35** at $R_{\text{cov}} = 15$ m (Kershner) or 29 (naive area). Note $N = 35$ also makes 2-connectivity (17) and single-failure tolerance (18) easy, at the cost of a much larger communication graph.
- **Or shrink the region.** Keeping $N = 5$, $K = 2$, $R_{\text{cov}} = 15$ m, the Kershner bound gives $A \leq \frac{3\sqrt{3}}{2} N r^2/K = 2.598076 \times 5 \times 225/2 = 1461.92$ m^2, i.e. a square of side only $\sqrt{1461.92} = 38.24$ m — a 38 % linear reduction from the stated 100 m.

What to write in the report. Replace "$R_{\text{cov}} = 15$ m" with $R_{\text{cov}} \geq 39.24$ m (or restate N, K, or A consistently), and add the feasibility certificate

$$N\,\pi R_{\text{cov}}^2 \;\geq\; KA \qquad \text{(and, Kershner-corrected: } KA \leq \tfrac{3\sqrt{3}}{2} N R_{\text{cov}}^2\text{)}$$

as a **precondition checked before training**, with the K-coverage reward treated as zero whenever it is violated. This turns a physically impossible hyperparameter into a caught configuration error rather than a silently mis-learned policy.

0.11. PART B — GRAPH AND CONNECTIVITY THEORY

0.11.1. 9. Connectivity, k-Connectivity, and Menger's Theorems

9.1 Basic definitions

Let $G = (V, E)$ be a finite undirected simple graph, $n = |V|$, $m = |E|$. A **vertex separator** is a set $S \subseteq V$ with $G - S$ disconnected. A **cut vertex** (articulation point) is a vertex whose removal increases the number of connected components.

Definition (vertex connectivity $\kappa(G)$).

$$\kappa(G) \;=\; \min\bigl\{\,|S| : S \subseteq V,\; G - S \text{ is disconnected}\,\bigr\} \;\le\; \delta(G),$$

with the standard convention that

$$\kappa(G) = \begin{cases} n-1, & \text{if } G \text{ is } K_n, \\ 0, & \text{if } G \text{ is disconnected}, \\ \min\bigl\{|S| : S \text{ is a cut vertex}\bigr\}, & \text{otherwise.} \end{cases}$$

Definition (edge connectivity $\lambda(G)$).

$$\lambda(G) \;=\; \min\bigl\{\,|F| : F \subseteq E,\; (V,F) \text{ is disconnected}\,\bigr\} \;\le\; \delta(G).$$

The classical inequalities (Whitney 1932) are

$$\kappa(G) \;\le\; \lambda(G) \;\le\; \delta(G) \;\le\; n-1,$$

and all three are attained, e.g. by $G = K_n$.

Local vertex connectivity. For $s, t \in V$, $s \neq t$, let

$$\kappa_{s,t}(G) \;=\; \min\bigl\{\,|S| : S \subseteq V \setminus \{s,t\},\ \text{no } s\text{–}t \text{ path in } G - S\,\bigr\}.$$

Equivalently, $\kappa_{s,t}$ is the minimum number of vertices whose deletion separates s from t (or which deletes s or t itself). Then $\kappa(G) = \min_{s \neq t} \kappa_{s,t}(G)$.

9.2 Menger's theorems, stated exactly

Menger's Theorem, vertex (node) version. *Let G be a finite undirected graph and $s, t \in V$, $s \neq t$. Let $\mathcal{P}_{s,t}$ range over sets of s–t paths no two of which share any internal vertex, and let $\mathcal{S}_{s,t}$ range over sets of vertices whose deletion separates s from t. Then*

$$\max_{\mathcal{P} \in \mathcal{P}_{s,t}} |\mathcal{P}| \;=\; \min_{S \in \mathcal{S}_{s,t}} |S|.$$

Consequently, G is k-vertex-connected (i.e. $\kappa(G) \ge k$) if and only if every pair of vertices is joined by k pairwise internally vertex-disjoint paths.

The max–min formula is the operative content: **the maximum number of internally vertex-disjoint s–t paths equals the minimum size of an s–t vertex cut.**

Menger's Theorem, edge version. *Let G be a finite undirected graph and $s, t \in V$, $s \neq t$. Let $\mathcal{P}_{s,t}$ range over sets of s–t paths no two of which share any edge, and let $\mathcal{S}_{s,t}$ range over sets of edges whose removal separates s from t. Then*

$$\max_{\mathcal{P} \in \mathcal{P}_{s,t}} |\mathcal{P}| \;=\; \min_{F \in \mathcal{S}_{s,t}} |F|.$$

Consequently, G is k-edge-connected (i.e. $\lambda(G) \ge k$) if and only if there are k edge-disjoint s–t paths for every pair s, t; and, by the Nash-Williams/Tutte theorem, if and only if G contains k edge-disjoint spanning trees.

Note the difference. The **vertex** version counts *internally disjoint* paths (paths may share s and t); the **edge** version counts *edge-disjoint* paths (paths may share interior vertices). For a UAV fleet, a **node failure removes the UAV entirely** and all of its links, so the **vertex** version is the operative one. Loss of some links while the UAV survives is the **edge** version.

(Classical attribution: K. Menger, 1927. Exact volume/page details not re-verified in this session — flagged in Part D.)

Why this is the right lens for the report. The requirement "the fleet must remain connected if any one UAV fails" is *literally* the statement $G - v$ is connected for every $v \in V$, i.e. $\kappa(G) \ge 2$, i.e. — by Menger — every pair of nodes has two internally vertex-disjoint paths. So fault-tolerance is not an extra heuristic layered on top of connectivity: it *is* a connectivity requirement, one level up the hierarchy.

0.11.2. 10. Articulation Points

10.1 Definition

> A vertex $v \in V$ of a connected graph G is an **articulation point** (cut vertex) iff $G-v$ is disconnected. Equivalently, the number of connected components of G strictly exceeds that of $G - v$.

Equivalently again: v is an articulation point iff there exist vertices $x, y \neq v$ with **no** path between them in $G - v$. Note that a vertex of degree 1 (a leaf) is never an articulation point, because removing it removes no connection between the remaining vertices.

In the UAV interpretation, an articulation point v is a **single point of failure**: if UAV v dies, the remaining $N - 1$ UAVs split into at least two mutually unreachable groups, and the persistent K-coverage mission fails because the groups cannot coordinate or share a charging pad. The MAPPO reward must therefore penalise not merely disconnection but the *presence of articulation points.*

10.2 The low-link quantity

For a depth-first-search (DFS) spanning forest of G with discovery order, define for every vertex v:

$$\mathrm{disc}[v] \;=\; \text{the time at which } v \text{ is first visited,}$$

$$\mathrm{low}[v] \;=\; \min\Big\{\mathrm{disc}[w] \;:\; w \text{ is reachable from } v \text{ by descending tree edges and then at most one back edge}\Big\},$$

i.e. the earliest discovery time reachable from the subtree rooted at v by descending zero, one, or many tree edges and then at most one back edge. Recursively,

$$\mathrm{low}[v] \;=\; \min\Big\{\mathrm{disc}[v],\; \min_{\substack{(v,w)\text{ back}\\ \text{edge}}} \mathrm{disc}[w],\; \min_{\substack{(v,w)\text{ tree}\\ \text{edge, } w \text{ child of } v}} \mathrm{low}[w]\Big\}.$$

The low-link condition. For a non-root vertex v with at least one DFS child, v is an articulation point **iff** there exists a DFS child w with

$$\boxed{\mathrm{low}[w] \;\geq\; \mathrm{disc}[v]}$$

and for the DFS root r,

$$\boxed{r \text{ is an articulation point} \iff r \text{ has } \geq 2 \text{ DFS-tree children.}}$$

Proof of the low-link condition (complete). Let T_v be the DFS subtree rooted at a child w of v.

($\Rightarrow$) Suppose $\mathrm{low}[w] \geq \mathrm{disc}[v]$. Then no vertex in T_w has a back edge to any vertex in a *different* DFS subtree of v, nor to a proper ancestor of v (which would have disc $<$ disc$[v]$). In an undirected graph every edge is a tree edge or a back edge between an ancestor and a descendant, so **all** edges leaving T_w go to v itself or to vertices inside T_w. Hence T_w is connected to the rest of $G - v$ only through v. Since $T_w \neq \emptyset$ and $|V| > 1$, removing v disconnects T_w from the rest, so v is an articulation point.

($\Leftarrow$) Suppose $\mathrm{low}[w] < \mathrm{disc}[v]$ for every child w of v. Then each T_w has a back edge to a proper ancestor of v, or to a vertex discovered before v — which by the DFS property must itself be a proper ancestor of v. So each subtree reconnects to the part of the graph above v without using v, and $G - v$ is connected. Hence v is not an articulation point. ■

For the **root**, the same argument shows that $G - r$ has exactly (number of DFS children of r) components. This is why the root's criterion is a degree-like condition on the DFS tree rather than a low-link inequality: $\mathrm{low}[w] \geq \mathrm{disc}[r] = 0$ is vacuous, since all low values are ≥ 0.

10.3 Algorithm (Tarjan / Hopcroft–Tarjan), with pseudocode

```
CUT-VERTICES(G)                      # G = (V, E), undirected, unweighted
  for each v in V:                    # O(V)
      disc[v] <- 0;  low[v] <- 0;  parent[v] <- NIL;  isArtic[v] <- false
  time <- 0

  DFS(u):
      disc[u] <- time;  low[u] <- time;  time <- time + 1
      children <- 0
```

```
    for each v in Adj[u] with edge {u,v}:                 # total O(E) over the run
        if v = parent[u]:  continue                       # skip the tree edge back to parent
        if disc[v] = 0:                                   # v undiscovered -> tree edge
            parent[v] <- u
            children   <- children + 1
            DFS(v)
            low[u] <- min(low[u], low[v])
            if parent[u] != NIL and low[v] >= disc[u]:  # LOW-LINK CONDITION (non-root)
                isArtic[u] <- true
        else:                                             # already discovered -> back edge
            low[u] <- min(low[u], disc[v])
    if parent[u] = NIL and children >= 2:                 # ROOT CONDITION
        isArtic[u] <- true
    return

for each v in V:  if disc[v] = 0:  DFS(v)                 # outer loop over components
return { v in V : isArtic[v] = true }
```

10.4 Complexity

Every vertex is discovered once, every edge is examined exactly twice (once from each endpoint), and every `low` update is $O(1)$. Therefore

$$\boxed{\textbf{Time} \;=\; O(|V|+|E|)} \qquad \boxed{\textbf{Space} \;=\; O(|V|)}$$

(the recursion stack is $O(|V|)$; an explicit-stack iterative version gives identical bounds). This is the classical Tarjan bound — R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, pp. 146–160, 1972 — and is also known as the Hopcroft–Tarjan articulation-point algorithm. The same pass, with one extra $O(1)$ comparison per child, simultaneously yields bridges (11) and biconnected components (Tarjan & Vishkin, FOCS 1984), which is why it is the right algorithm to embed in the MAPPO evaluation loop.

Note for the report. A single DFS over G yields the full articulation-point set. If the connectivity reward needs only a yes/no on "G is 2-connected", the cheapest screen is: G is connected **and** no vertex has degree 1 **and** the DFS finds no articulation point — one pass, $O(n+m)$, negligible next to a forward pass of the network.

0.11.3. 11. Bridges (Cut Edges)

11.1 Definition and basic facts

An edge $e \in E$ is a **bridge** (cut edge) iff it lies on no cycle of G. Fundamental facts:

- An edge is a bridge iff it is an edge of **no** circuit.
- Every edge of a **tree** is a bridge, and G is a forest iff every edge is a bridge.
- G is **2-edge-connected** (no bridges) iff every edge lies on a cycle.
- Bridges are found in the same $O(|V|+|E|)$ DFS as articulation points.

11.2 Linear-time algorithm (same DFS)

With disc and low as in 10.2, inside the tree-edge branch of the DFS:

```
        if disc[v] = 0:                          # tree edge {u,v}
            parent[v] <- u
            children <- children + 1
            DFS(v)
            low[u] <- min(low[u], low[v])
            if parent[u] != NIL and low[v] >= disc[u]:   isArtic[u] <- true
            if low[v] > disc[u]:                         isBridge({u,v}) <- true   # BRIDGE TEST
        else:
            low[u] <- min(low[u], disc[v])
```

$$\boxed{\text{A tree edge } \{u,v\} \text{ (with } u \text{ the parent) is a bridge} \iff \text{low}[v] > \text{disc}[u].}$$

Back edges are never bridges: a back edge closes a cycle with the DFS-tree path between its endpoints, so it lies on that cycle.

Proof of the bridge test. (The *strict* inequality is the whole point — contrast the articulation-point test, which uses $\geq$.)

($\Rightarrow$) Suppose $\text{low}[v] > \text{disc}[u]$. Then no vertex in the DFS subtree of v has a back edge to u, to a proper ancestor of u, or to any vertex in another subtree of u. Since every non-tree edge of an undirected DFS is a back edge between an ancestor and a descendant, **all** edges leaving the subtree of v go through u. Hence $\{u,v\}$ is the unique connection and is a bridge.

($\Leftarrow$) Suppose $\text{low}[v] \leq \text{disc}[u]$. Then some vertex z in the subtree of v has a back edge to u or to a proper ancestor of u. The closed walk $z \rightsquigarrow v \to u \rightsquigarrow z$ — using the tree path inside the subtree, the tree edge $\{u,v\}$, and the back edge together with the tree path from u up to z — is a cycle containing $\{u,v\}$, so $\{u,v\}$ is not a bridge. ■

11.3 Relation to articulation points

> **Proposition.** *Let G be connected with $|V| \geq 3$. If $e = \{u,v\}$ is a bridge, then at least one of u, v is an articulation point. In particular, if G has no articulation points then G has no bridges.*

Proof. Since e is a bridge, $G - e$ has exactly two components $A \ni u$ and $B \ni v$. Suppose $G - u$ were connected. Then some B-path must reach A, but the only A–B edge in G is $e = \{u,v\}$, which touches the removed vertex u — contradiction. Hence $G-u$ is disconnected, so u is an articulation point; the same argument gives it for v. ■

Consequences used in the report:

- **No articulation points ⇒ 2-vertex-connected ⇒ no bridges ⇒ every communication link lies on a cycle, so a single *link* failure is survivable.** Single-UAV failure is survivable a fortiori, because removing a vertex removes all of its incident edges simultaneously, and Menger (vertex version) guarantees two internally vertex-disjoint paths between every pair.
- **Bridges are the "partial-UAV-failure" detector, computed in the same pass.** If the report wants a softer failure model (a UAV's radio degrades but the UAV survives, so only some of its links die), then **bridges are the correct detector**, and the requirement is $\lambda(G) \geq 2$. This is a useful second reward term that costs nothing extra computationally.

0.11.4. 12. Graph Laplacian, Algebraic Connectivity, and Fiedler's Theorem

12.1 Definition

For $G = (V,E)$ with adjacency matrix A (symmetric, zero diagonal) and degree matrix $D = \text{diag}(d_1, \ldots, d_n)$, $d_i = \sum_j a_{ij}$, the **combinatorial Laplacian** is

$$\boxed{\mathcal{L} \;=\; D - A}$$

and the **normalised Laplacian** is $\mathcal{L}^{\text{n}} = I - D^{-1/2} A D^{-1/2}$. For an undirected graph, $\mathcal{L}$ is symmetric positive semidefinite: for $x \in \mathbb{R}^n$,

$$x^T \mathcal{L} x \;=\; \sum_{\{i,j\} \in E} a_{ij}\,(x_i - x_j)^2 \;\geq\; 0,$$

which is the discrete Dirichlet energy of x. Its spectral decomposition is

$$\mathcal{L} \;=\; \sum_{k=1}^{n} \lambda_k\, v_k v_k^T, \qquad 0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n, \qquad \mathcal{L}\mathbf{1} = 0, \quad \mathbf{1}^T \mathcal{L} = 0^T.$$

12.2 Fiedler's theorems

Fiedler's Theorem (nullity, 1973). *$\lambda_2(\mathcal{L}) = 0$ if and only if G is disconnected. More generally the multiplicity of the eigenvalue 0 equals the number of connected components of G:*

$$\text{nullity}(\mathcal{L}) \;=\; \#\{\text{connected components of } G\}.$$

Fiedler's Theorem (eigenvector orthogonality, 1975). *If $k > 1$ and v_k is any eigenvector associated with $\lambda_k(\mathcal{L})$, then*

$$\mathbf{1}^T v_k = 0 \quad \text{and} \quad v_k^T \mathbf{1} = 0.$$

That is, every non-null Laplacian eigenvector is orthogonal to the all-ones vector.

(Reproduced as Lemma 1 of Zareh–Sabattini–Secchi, after N. M. M. de Abreu, "Old and new results on algebraic connectivity of graphs," *Linear Algebra Appl.*, vol. 423, no. 1, pp. 53–73, 2007, which cites Fiedler 1975, *Czechoslovak Mathematical Journal*, vol. 25, no. 4, pp. 619–633.)

Proof of the nullity statement (complete). Let G have connected components $V_1, \dots, V_c$. Reorder V so components are contiguous; then $\mathcal{L}$ is block diagonal, $\mathcal{L} = \text{diag}(\mathcal{L}|_{V_1}, \dots, \mathcal{L}|_{V_c})$. Each block has rank $|V_i| - 1$: by the quadratic-form identity, $x^T \mathcal{L}|_{V_i} x = \sum_{\{p,q\} \in E_i} a_{pq}(x_p - x_q)^2 = 0$ if and only if x is constant on V_i, so the block's nullspace is the 1-dimensional space of constant vectors on V_i. Hence $\text{nullity}(\mathcal{L}) = \sum_{i=1}^{c} (|V_i| - 1) = n - c$. Therefore $\lambda_2 = 0 \iff n - c \geq 2 \iff c \geq 2 \iff G$ is disconnected. ■

12.3 Algebraic connectivity, and why λ_2 beats the hard indicator

$$\boxed{\lambda_2(\mathcal{L}) \;=\; \text{"algebraic connectivity"} \;=\; \min_{\substack{x \perp \mathbf{1} \\ x \neq 0}} \frac{x^T \mathcal{L} x}{x^T x}}$$

(the Rayleigh-quotient characterisation). Properties that make it the right reward signal:

1. **Zero exactly at disconnection** (Fiedler). So $\lambda_2 = 0 \iff$ the fleet is partitioned: a *soft* penalty that is zero precisely in the degenerate case and positive otherwise.
2. **Monotone under edge addition.** If $\mathcal{E}_1 \subseteq \mathcal{E}_2$ on the same vertex set, then $\mathcal{L}_2 - \mathcal{L}_1$ is PSD, and the minimum of a Rayleigh quotient over a fixed subspace is monotone under PSD increase, so $\lambda_2(\mathcal{L}_1) \leq \lambda_2(\mathcal{L}_2)$. **A repulsive well-formedness potential therefore drives connectivity automatically**, with no extra shaping.
3. **Smooth and graded.** λ_2 is a *continuous* function of the edge weights. The articulation-point indicator $\mathbb{1}\{\exists v : G - v \text{ disconnected}\}$ is discrete and, worse, is **non-monotone along a trajectory**: a fleet can go "no articulation points" → "one articulation point" → "no articulation points" in three consecutive policy steps, and the indicator carries no information about *how close* to the boundary the third configuration is. With λ_2 the agent sees a value that grows monotonically as redundancy is added, and gets a non-zero signal even when it is already connected but fragile.
4. **Cheap.** Power iteration on $\mathcal{L}$ costs $O(m)$ per iteration for sparse $\mathcal{L}$; k iterations give relative error $O(\rho^k)$ with $\rho = \lambda_2/\lambda_3$. In practice 10–30 iterations suffice. An exact articulation-point test costs the same order but returns a single bit.
5. **Bounded and normalisable.** For a connected graph on n vertices, $\lambda_2 \leq \frac{n}{n-1}\delta \leq \frac{n}{n-1}$, and for regular geometric graphs $\lambda_2 = \Theta(\delta)$. So $\lambda_2/(\lambda_2 + n/(n-1))$ is a well-scaled $[0, 1)$-valued reward term.

A caveat the report must state honestly. $\lambda_2 > 0$ certifies **1-connectivity only**. It is *not* a certificate of 2-connectivity: the path graph P_n — the worst case for λ_2, with $\lambda_2 = 2 - 2\cos(\pi/n) \approx \pi^2/n^2$ — is fully connected, has no bridges, and has $n - 2$ articulation points. So λ_2 is the right *smooth* signal for **connectivity**, and λ_3 **is the right smooth signal for 2-connectivity** — which is exactly where the biconnectivity literature lands (16).

(Fiedler 1975, Czechoslovak Math. J. 25(4):619–633; de Abreu 2007, Linear Algebra Appl. 423(1):53–73; Bapat & Pati 1998, Linear and Multilinear Algebra 45(2–3):247–273 — all three verified.)

0.11.5. 13. Unit Disk Graphs

13.1 Definition

A **unit disk graph** (UDG) is a graph $G = (V, E)$ for which there exist points $p_1, \dots, p_n \in \mathbb{R}^d$ and $r > 0$ such that $\{i, j\} \in E \iff \|p_i - p_j\| \leq r$.

For our UAVs, G is a UDG with $r = R_{\text{comm}}$ and p_i the UAV positions. UDG decision is in NP (just the coordinates) and polynomial-time solvable, but many *optimisation* problems on UDGs are NP-hard, including the minimum connected dominating set and the connected-coverage variants of 3.3. Note that "deciding connectivity" of a given configuration is trivial — the hard problems are the optimisations built on top of connectivity, which is precisely why the report does not solve connectivity in the loop but instead *shapes* it with a potential.

13.2 The greedy MST-connectivity heuristic, and why it is the right primitive

Claim (minimum bottleneck spanning tree). For a finite point set P and radius r, the Euclidean minimum spanning tree (EMST) T of P is **bottleneck-minimising**: writing $b(T) = \max_{e \in T} \|e\|$,

$$b(T) \;=\; \min_{\text{spanning trees } T'} b(T').$$

Proof. By the **cut property**, a minimum-weight edge crossing any cut belongs to some MST. Let $e^\star$ be the longest edge of T, of length L, and let $T - e^\star = A \dot\cup B$ be the resulting cut. No edge crossing (A, B) has weight $< L$ — otherwise replacing $e^\star$ by it would yield a strictly lighter spanning tree, contradicting minimality of T. Hence every spanning tree T' must cross the cut (A, B) with an edge of weight $\geq L$, so $b(T') \geq L$ for all T'; and T attains L. ■

Consequence for connectivity. The bottleneck statement is

$$G \text{ (the UDG) is connected} \quad\Longleftrightarrow\quad b(T) \leq r.$$

So **connectivity of a UDG is a bottleneck problem, not a total-length problem**: the correct surrogate objective is to *minimise the longest link of the fleet's spanning tree*, not the sum of link lengths. This is a frequently mis-stated point in the multi-robot connectivity-maintenance literature, and it is the real justification for an MST-based rather than sum-of-distances-based connectivity potential.

Greedy heuristic. Kruskal/Prim on the UDG builds the MST in $O(m \log n)$ time (or $O(n)$ after Delaunay-based sparsification), and then

$$\text{if } b(T) \leq r: \text{ connected} \qquad \text{if } b(T) > r: \text{ disconnected, and } r_{\text{needed}} = b(T).$$

$r_{\text{needed}} = b(T)$ is a **tight, computable certificate**: it is the smallest communication radius that would connect the current configuration. This is the natural single-knob shaping term.

13.3 The area relation $n\pi r^2/2$

The UDG design literature's standard rule for placing n nodes in a region of area A so that the induced UDG is **connected and every node is covered** (each node has at least one neighbour — a necessary condition for covering the node itself) is

$$\boxed{\frac{n\pi r^2}{2} \;\geq\; A \qquad\Longleftrightarrow\qquad r \;\geq\; \sqrt{\frac{2A}{\pi n}}}$$

Derivation of the $1/2$. For node i to be *covered*, some other node's radius-r disk must contain p_i. In a deployment each node i "owns" its Voronoi cell V_i (the set of points closer to p_i than to any p_j). A disk of radius r centred at p_j can cover at most the *far half* of a cell whose inradius is at most r, because p_j itself must lie outside the portion of the cell that j owns. Hence each node must own a cell of area at least one half-disk, $\pi r^2/2$; the n cells tile A, giving $n \cdot \pi r^2/2 \geq A$. The factor $1/2$ is the direct, unavoidable consequence of requiring *self-coverage*; without it (i.e. requiring only that the *region* be covered) the rule is $n\pi r^2 \geq A$ — which is 7.1.

Consistency check. For $A = 10\,000$ m^2, $n = 5$: $r \geq \sqrt{2 \cdot 10\,000/(5\pi)} = \sqrt{1273.2395} = 35.68$ m — *identical* to the $K = 1$ instance of 7.1. Not a coincidence: $n\pi r^2/2 \geq A$ is 7.1 with $K = 1$ plus the factor 2 from self-coverage. Both are valid necessary conditions, and for $K \geq 2$ the K-rule of 7 dominates (since $KA \leq n\pi r^2$ implies $A \leq n\pi r^2/2$ for $K \geq 2$).

(The UDG guaranteed-connectivity / connected-cover results of Apollonio, Czyzowicz, Stojmenovi & Urrutia are the classical source of this rule. I am confident in the $n\pi r^2/2$ form and in the half-disk reasoning; a specific volume/page was not re-verified this session — flagged in Part D.)

0.11.6. 14. Random Geometric Graphs

14.1 Definition

A **random geometric graph** $G(n,r)$ on a domain Q of area A is formed by placing n points $X_1,\dots,X_n$ **independently and uniformly at random** in Q and joining X_i to X_j iff $\|X_i - X_j\| \le r$. The edges are then independent Bernoulli variables with a shared, position-dependent success probability.

14.2 Expected number of edges

On a **torus** (or, exactly, for a domain on which the boundary has measure zero under the distance constraint):

$$\Pr\big(\{i,j\}\in E\big) \;=\; \mathbb{P}\big(\|X_i - X_j\| \le r\big) \;=\; \frac{|B(x,r)|}{A} \;=\; \frac{\pi r^2}{A},$$

so by linearity of expectation

$$\boxed{\mathbb{E}\big[|E(G(n,r))|\big] \;=\; \binom{n}{2}\cdot\frac{\pi r^2}{A} \;=\; \frac{n(n-1)}{2}\cdot\frac{\pi r^2}{A}}$$

and the **expected degree** of a node is

$$\mathbb{E}[\deg] \;=\; (n-1)\frac{\pi r^2}{A}.$$

Equivalently, the node density required to achieve a target mean degree is $n/A \approx \bar{d}/(\pi r^2)$ — the familiar "$\bar{d} = \pi r^2 n/A$" rule of thumb.

14.3 The boundary caveat — and an exact computation for the square

On a domain **with boundary**, $\Pr(\|X_i - X_j\| \le r)$ is *less* than $\pi r^2/A$, because disks centred near the boundary hang outside Q and cover less area. **Exact computation for the unit square.** Let X_i, X_j be independent uniform on $[0,1]^2$ and put $u = X_i - X_j$. Then u_1 has the triangular density $f_{u_1}(s) = 1 - |s|$ on $[-1,1]$ and $u_2 \sim \mathrm{Unif}[-1,1]$, independently. For $0 \le r \le 1$,

$$\begin{aligned}
\mathbb{P}(\|X_i - X_j\| \le r) &= \int_{\|u\|\le r} (1-|u_1|)\,du_1\,du_2 \\
&= \pi r^2 - 2\int_{\|u\|\le r} |u_1|\,du_1 \\
&= \pi r^2 - 2\cdot 4\int_0^r u_1\sqrt{r^2-u_1^2}\,du_1 \\
&= \pi r^2 - 8\cdot\frac{r^3}{3},
\end{aligned}$$

since $\int_0^r u\sqrt{r^2-u^2}\,du = \frac{r^3}{3}$. Hence

$$\boxed{\mathbb{P}(\|X_i - X_j\| \le r) \;=\; \pi r^2 - \tfrac{8}{3}r^3, \qquad 0 \le r \le 1}$$

$$\boxed{\mathbb{E}\big[|E(G(n,r))|\big] \;=\; \binom{n}{2}\Big(\pi r^2 - \tfrac{8}{3}r^3\Big) \quad \text{on the unit square.}}$$

Size of the correction at the connectivity threshold. At $r_n \approx \sqrt{\ln n/(\pi n)}$, the relative error is

$$\frac{\frac{8}{3}r^3}{\pi r^2} \;=\; \frac{8r}{3\pi} \;=\; O\!\left(\sqrt{\frac{\ln n}{n}}\right) \;\longrightarrow\; 0.$$

So the boundary term is **second-order** — far smaller than the $\ln n$-scale fluctuation that governs the threshold itself. This is *why* $\binom{n}{2}\pi r^2/A$ is the right formula in the interior, and *why* the boundary still matters at simulation scale: at $r = 15$ m on a 100 m side ($r = 0.15$ normalised), the relative correction is $\frac{8(0.15)}{3\pi} = 0.1273$, i.e. 12.7 % — not negligible for a 5-node fleet.

0.11.7. 15. The Connectivity Threshold

Theorem (connectivity threshold for the 2-D random geometric graph; Penrose, Gupta–Kumar). *Let $X_1, \dots, X_n$ be independent and uniform on the unit square, and let $r_n = r_n(n)$ be a threshold function. Then*

$$\mathbb{P}\big(G(n, r_n) \text{ is connected}\big) \longrightarrow \begin{cases} 1, & \pi n r_n^2 - \ln n \to +\infty, \\ 0, & \pi n r_n^2 - \ln n \to -\infty. \end{cases}$$

Equivalently, $r = r_{\mathrm{conn}}(n) := \sqrt{\dfrac{\ln n}{\pi n}}$ is a threshold function for connectivity; and more sharply, if $\pi n r_n^2 = \ln n + c_n$ for a real sequence c_n, then

$$\mathbb{P}\big(G(n, r_n) \text{ is connected}\big) \longrightarrow \exp\big(-\exp(-c_n)\big).$$

Why connectivity has the same threshold as having no isolated vertices. The number I of isolated vertices satisfies a Poisson approximation, $I \xrightarrow{d} \mathrm{Poisson}\big(ne^{-\pi n r^2}\big)$, and Penrose's theorem states that for $G(n, r)$, *any threshold function for having no isolated vertex is also a connectivity threshold function*: the two events are not identical, but they coincide with probability $1 - o(1)$ in the critical window. The anchor is Panchapakesan & Manjunath (2001), "Connectivity of a graph of uniformly distributed points on $[0,1]^d$," *Statistics & Probability Letters* 60(4), 351–357, who showed that $\ln n/n$ (in the normalised-p language) is a threshold for having an isolated vertex; by Penrose's theorem it is also a connectivity threshold.

Gupta–Kumar's formulation (unit-area disk). For n points uniform in a disk of **area 1** (radius $1/\sqrt{\pi}$), the connectivity threshold is expressed as a *critical power*:

$$\boxed{p_c(n) \;=\; \frac{\ln n + c(n)}{\pi n}\,, \qquad \text{equivalently} \qquad r_c(n) = \sqrt{\frac{\ln n + c(n)}{\pi n}}.}$$

P. Gupta and P. R. Kumar, "Critical power for asymptotic connectivity in wireless networks," in *Stochastic Analysis, Control, Optimization and Applications*, pp. 547–566, 1998. *(Verified: the reference list of arXiv:1809.10737 gives exactly these details and states that Gupta and Kumar "provided a connectivity threshold for points that are uniformly distributed in a disk.")*

k-connectivity threshold. M. D. Penrose, "On k-connectivity for a geometric random graph," *Random Structures and Algorithms*, vol. 15, no. 2, pp. 145–164, 1999 (verified on Penrose's own publication list, Durham), extends this to

$$\boxed{r \;=\; \sqrt{\frac{(k-1)\ln n + c_n}{\pi n}}}$$

as a threshold for $G(n, r)$ to be k-connected. Note the structural parallel with the UAV setting: **the redundancy requirement enters the threshold as a factor $K - 1$ multiplying $\ln n$** — more redundancy demands asymptotically more than proportionally more radius, because each additional constraint contributes a $\ln n$-scale "no isolated node / no small cut" event. This is the probabilistic counterpart of the deterministic bound of 7.

Soft graphs. M. D. Penrose, "Connectivity of soft random geometric graphs," *Annals of Applied Probability*, vol. 26, no. 2, pp. 986–1028, 2016, doi: 10.1214/15-AAP1110 (arXiv:1311.3897), determines the asymptotic connectivity probability for all (p_n, r_n) with $r_n = O(n^{-\varepsilon})$ — i.e. for the *soft* rule $P(i \leftrightarrow j) = p$ if $\|X_i - X_j\| \le r$. This is the right model when link reliability is probabilistic (fading), and it is therefore the right model for "stochastic UAV failures" in a lossy channel, rather than a hard UDG.

What the threshold means for design. Requiring the number of components to be $o(1)$ forces

$$R_{\mathrm{comm}} \;\gtrsim\; \sqrt{\frac{\ln N}{\pi n_{\mathrm{dens}}}}, \qquad n_{\mathrm{dens}} = \frac{N}{A}.$$

For $N = 5$, $A = 10\,000\ \mathrm{m}^2$: $n_{\mathrm{dens}} = 5 \times 10^{-4}\ \mathrm{m}^{-2}$, giving $R_{\mathrm{comm}} \approx \sqrt{1.6094/(0.0015708)} = \sqrt{1024.6} = 32.01$ m. This is only indicative: at such low density the probabilistic guarantee is weak, and with $N = 5$ the operative regime is the *deterministic* bottleneck bound of 13, not the random-geometric threshold. The random-geometric threshold becomes informative once N is in the tens or hundreds.

0.11.8. 16. Local Articulation Point Detection: the ACTUAL Conditions

16.1 The source, and a DOI correction the report needs

The report cites:

> Zareh Eshghdoust, M.; Sabattini, L.; Secchi, C., "Decentralized biconnectivity conditions in multi-robot systems," *2016 IEEE 55th Conference on Decision and Control (CDC)*, doi: 10.1109/CDC.2016.7798253.

Live verification performed on 29 September 2026 — results:

- The paper **exists** and is by exactly these three authors. A complete, freely available version exists as **arXiv:1608.02276** (math.DS, submitted 14 August 2016, cross-listed cs.RO). I retrieved and read the full text (ar5iv LaTeXML rendering), so everything below is taken from the paper itself, not from memory.
- The **official CDC 2016 proceedings table of contents** (IEEE, 2016 IEEE 55th Conference on Decision and Control) lists: *"Decentralized Biconnectivity Conditions in Multi-Robot Systems, pp. 99–104"*, with these three authors, all at the *Dipartimento di Ingegneria Meccanica e dei Sistemi (DISMI), University of Modena and Reggio Emilia*. **The page range 99–104 is confirmed.**
- **The DOI in the report is very likely incorrect.** The *other* paper by the same three authors in the same conference — "Enforcing Biconnectivity in Multi-robot Systems," pp. 1800–1805 — carries DOI 10.1109/CDC.2016.7798526 (confirmed independently in the reference list of T. Murayama and A. Iwasaki, *J. Robot. Mechatron.* 35(4), 1028–1037, 2023, which lists doi 10.1109/CDC.2016.7798526). The report's **10.1109/CDC.2016.7798253 could not be resolved to a matching record** in my searches. **Action required: check the DOI directly on IEEE Xplore. The page range 99–104 is safe to keep, and arXiv:1608.02276 is a safe, verifiable substitute.**

16.2 Notation (as used in the paper)

$\mathcal{G} = (\mathcal{V}, \mathcal{E})$ undirected, $\mathcal{V} = \{1, \dots, n\}$, $n > 2$. A is the adjacency matrix ($a_{ij} > 0$ iff $(i,j) \in \mathcal{E}$; A symmetric). $\mathcal{N}_i = \{j : (j,i) \in \mathcal{E}\}$ is the neighbourhood of i. $D = \mathrm{diag}(d_1, \dots, d_n)$, $d_i = \sum_k a_{ik}$, and $\mathcal{L} = D - A$ with

$$0 = \lambda_1(\mathcal{L}) \le \lambda_2(\mathcal{L}) \le \cdots \le \lambda_n(\mathcal{L}).$$

$\mathcal{G}^{R_i}$ denotes G with node i **and all its incident edges** removed; $\mathcal{L}^{R_i}$ is its Laplacian. "Data exchange among the robots is supposed to be **neighbour-to-neighbour**."

Paper's Definitions 1–3 and 5:

- **Def. 1.** A vertex $i \in \mathcal{V}$ of a connected graph $\mathcal{G}$ is an **articulation point** if $\mathcal{G}^{R_i}$ is not connected.
- **Def. 2.** A connected graph is **biconnected** if it has no articulation point.
- **Def. 3.** A **block** in $\mathcal{G}$ is a maximal induced connected subgraph with no articulation point. If $\mathcal{G}$ itself is connected and has no articulation point, then $\mathcal{G}$ is a block.
- **Def. 5.** Two vertices i, j are **doubly connected** iff there are two or more internally vertex-disjoint paths between them.
- **Lemma 2.** $\mathcal{G}$ is biconnected $\iff$ any two vertices i, j are doubly connected.

Perturbed objects. For a small design parameter $\epsilon \in \mathbb{R}^+$:

- $A^i(\epsilon)$ is A with **all** a_{ij} and a_{ji} multiplied by ϵ;
- $D^i(\epsilon) = \mathrm{diag}(A^i(\epsilon)\mathbf{1})$;
- $\mathcal{L}^i(\epsilon) := D^i(\epsilon) - A^i(\epsilon)$.

Writing $\tilde{a}_i = [a_{ij}]_{j \ne i} \in \mathbb{R}^{n-1}$ and $d_i = \sum_k a_{ik}$, the paper computes

$$A^i(\epsilon) = \begin{pmatrix} A^{R_i} & \epsilon\,\tilde{a}_i \\ \epsilon\,\tilde{a}_i^T & 0 \end{pmatrix}, \qquad \mathcal{L}^i(\epsilon) = \begin{pmatrix} \mathcal{L}^{R_i} + \epsilon\,\mathrm{diag}(\tilde{a}_i^T) & -\epsilon\,\tilde{a}_i \\ -\epsilon\,\tilde{a}_i^T & \epsilon d_i \end{pmatrix}.$$

Theorem 1 and Corollary 1 (the reduction). The eigenvalues of

$$P^i(\epsilon) \;=\; \mathcal{L}^{R_i} + \epsilon\ \mathrm{diag}(\tilde{a}_i^T) + \epsilon\,\tilde{a}_i \mathbf{1}^T \tag{1}$$

are **exactly the non-null eigenvalues** of $\mathcal{L}^i(\epsilon)$. And if $\mathcal{G}$ is connected then $\mathcal{L}^i(\epsilon)$ has exactly one null eigenvalue, so

$$\lambda_k\big(P^i(\epsilon)\big) \;=\; \lambda_{k+1}\big(\mathcal{L}^i(\epsilon)\big), \qquad k = 1, \dots, n-1. \tag{2}$$

The proof of Theorem 1 uses exactly the Fiedler orthogonality of 12.2: for a non-null eigenvector $v = [v^1; v^2]$ of $\mathcal{L}^i(\epsilon)$, $\mathbf{1}^T v = 0$ gives $v^2 = -\mathbf{1}^T v^1$, and substituting removes v^2 and yields (1). **This is why the conditions are per-node conditions**: the reduction to $P^i(\epsilon)$ consumes only the rows/columns of i and its neighbourhood.

Lemma 3 (perturbation bound). Let A have eigenvalues $\psi_1 \geq \cdots \geq \psi_n$ and B have eigenvalues $\xi_1 \geq \cdots \geq \xi_n$, and define $\text{gap}(A, B) = \max_j |\psi_j - \xi_j|$. If **all** real linear combinations of A and B have only real eigenvalues, then

$$\text{gap}(A, B) \;\leq\; \|A - B\| \quad \text{(Euclidean/Frobenius-type norm)}.$$

Proposition 1 (needed for Lemma 3). For any $\alpha, \beta \in \mathbb{R}$ with $\alpha^2 + \beta^2 \neq 0$, the linear combination $F^i(\epsilon) = \alpha\mathcal{L}^{R_i} + \beta P^i(\epsilon)$ has only real eigenvalues. (Proved in the paper's Appendix by a perturbative induction using Lemma 4 — the semisimple-eigenvalue first-order perturbation formula of Seyranian & Mailybaev.)

16.3 Theorem 2 — the actual sufficient condition

Theorem 2 (Zareh, Sabattini & Secchi, 2016, eq. 12). *For a given multi-robot system with n robots ($n > 2$) whose interaction network graph is modelled by an undirected connected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the node i is **not** an articulation point if, for a small $\epsilon \in \mathbb{R}^+$, we have*

$$\boxed{\lambda_3\big(\mathcal{L}^i(\epsilon)\big) \;>\; \epsilon\sqrt{n}\,\left(\sum_{k=1}^{n} a_{ik}^2\right)^{1/2}} \tag{12}$$

Equivalently, by (2): $\lambda_2\big(P^i(\epsilon)\big) > \epsilon\sqrt{n}\,\big(\sum_k a_{ik}^2\big)^{1/2}$.

Proof chain (as printed).

1. If i is not an articulation point then $\mathcal{G}^{R_i}$ is connected, so $\mathcal{L}^{R_i}$ has a **positive** second-smallest eigenvalue.
2. Proposition 1 ensures all real linear combinations of $\mathcal{L}^{R_i}$ and $P^i(\epsilon)$ have real eigenvalues, so Lemma 3 applies:
$$\big|\lambda_2\big(P^i(\epsilon)\big) - \lambda_2\big(\mathcal{L}^{R_i}\big)\big| \;\leq\; \big\|P^i(\epsilon) - \mathcal{L}^{R_i}\big\| \;=\; \epsilon\big\|\text{diag}(\tilde{a}_i) + \tilde{a}_i\mathbf{1}^T\big\|.$$
3. The perturbation matrix is explicitly
$$\text{diag}(\tilde{a}_i) + \tilde{a}_i\mathbf{1}^T = \begin{pmatrix} a_{11} & \cdots & a_{11} \\ \vdots & & \vdots \\ a_{n-1,n-1} & \cdots & a_{n-1,n-1} \end{pmatrix},$$
a matrix with $n-1$ rows and $n-1$ identical columns, whose Euclidean norm is $\sqrt{n}\,\big(\sum_{k=1}^{n} a_{ik}^2\big)^{1/2}$ (note: this uses the $n-1$ repeated entries, which the paper writes with the index $k = 1, \ldots, n$).
4. Hence $\lambda_2\big(P^i(\epsilon)\big) > \epsilon\sqrt{n}\big(\sum_k a_{ik}^2\big)^{1/2} \Rightarrow \lambda_2\big(\mathcal{L}^{R_i}\big) > 0 \Rightarrow \mathcal{G}^{R_i}$ connected $\Rightarrow i$ is not an articulation point.
5. By Corollary 2 and (2), $\lambda_2\big(P^i(\epsilon)\big) = \lambda_3\big(\mathcal{L}^i(\epsilon)\big)$, giving (12). ■

The paper's own worked numbers (Example 1). $n = 10$ robots, R-disk model with $R = 0.5$, $\sigma = 0.125$, so $a_{ij} = \exp\big(-\|p_i - p_j\|^2/(2\sigma)\big)$ for $\|p_i - p_j\| \leq R$ and 0 otherwise. Only one node, *, is not locally biconnected. With $\epsilon = 0.05$: $\lambda_3\big(\mathcal{L}^*(\epsilon)\big) = 0.034$ and $\big(\sum_{k=1}^{n} a_{*k}^2\big)^{1/2} = 0.062$, so

$$0.034 \;>\; 0.05 \times \sqrt{10} \times 0.062 \;=\; 0.0098,$$

and node * is certified not to be an articulation point. **The paper explicitly notes the condition is sufficient but not necessary**: "if we keep the same graph and we change the weights, (12) might not hold anymore."

16.4 Proposition 2 and Corollary 2 — the 1-hop local screen

Proposition 2. *In a connected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, node i is not an articulation point if the subgraph created on the set $\{i\} \cup \mathcal{N}_i$ forms a block.*

Proof (as printed). Let $\mathcal{V}_1 = \{i\} \cup \mathcal{N}_i$ and $\mathcal{V}_2 = \mathcal{V} \setminus \mathcal{V}_1$. Since $\mathcal{G}$ is connected, every node in $\mathcal{V}_2$ has a path to the block $\mathcal{G}(\mathcal{V}_1)$. No node in $\mathcal{V}_2$ is adjacent to i (else it would be in $\mathcal{N}_i$). Since $\mathcal{G}(\mathcal{V}_1)$ is a block, $\mathcal{G}(\mathcal{V}_1/\{i\})$ is connected, hence $\mathcal{G}((\mathcal{V}_1 \cup \mathcal{V}_2)/\{i\}) = \mathcal{G} - i$ is connected. ■

Remark 1 (the operational content). *In an undirected communication network graph, to characterize the local subgraph, each node only needs to receive the* ***positions of its neighbours****. Then the local adjacency, degree and Laplacian matrices can be determined. If the second smallest eigenvalue of the local Laplacian is positive then that node is* ***locally biconnected****.*

A node satisfying Proposition 2 is called **locally biconnected**.

Corollary 2. *A connected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is biconnected if* ***every*** *node of $\mathcal{G}$ that is* ***not locally biconnected*** *meets the condition in (12).*

16.5 How biconnectivity is maintained from neighbour-to-neighbour information only, and why that matters

The algorithm the paper describes is a two-stage screening cascade, and this is the crucial architectural point for the report:

Stage A — cheap 1-hop local screen (cost: $|\mathcal{N}_i|$ position messages). Each node i broadcasts its own position to its neighbours and receives theirs. It then forms the *local* adjacency, degree and Laplacian matrices on $\{i\} \cup \mathcal{N}_i$ and checks

$$\lambda_2\big(\mathcal{L}[\{i\} \cup \mathcal{N}_i]\big) \;>\; 0.$$

If this holds, i is locally biconnected and, by Proposition 2, **is provably not an articulation point** — no further work needed. The paper is explicit about the motivation: in many multi-robot schemes (formation control, rendezvous) the robots have assigned tasks, and a biconnectivity check "introduces an extra effort to the robots that can lead to a loss in time and energy"; so nodes that can prove they are not at risk "can skip the biconnectivity check." **Note the honest scope of the data used: Stage A is strictly 1-hop, not 2-hop.**

Stage B — spectral check on the risky nodes only (cost: a distributed eigenvalue estimate, still neighbour-to-neighbour). Only the nodes that *fail* Stage A must be examined. For each such node, the paper's Theorem 2 gives the sufficient condition (12) on the third-smallest eigenvalue of the **perturbed** Laplacian $\mathcal{L}^i(\epsilon)$ — the perturbation "bringing i close to disconnection" by scaling all its incident edge weights by a small $\epsilon > 0$, thereby *simulating* the post-failure state $\mathcal{G}^{R_i}$ in a weighted graph that node i can still sense. The eigenvalue $\lambda_3\big(\mathcal{L}^i(\epsilon)\big)$ is then obtained via a **decentralized Laplacian-eigenvalue estimator** operating on neighbour-to-neighbour data exchange, e.g. the estimator of M. Franceschelli, A. Gasparri, A. Giua, C. Seatzu, "Decentralized estimation of Laplacian eigenvalues in multi-agent systems," *Automatica*, vol. 49, no. 4, pp. 1031–1036, 2013; or the Fiedler-vector/algebraic-connectivity estimator of P. Yang, R. A. Freeman, G. J. Gordon, K. M. Lynch, S. S. Srinivasa, R. Sukthankar, "Decentralized estimation and control of graph connectivity for mobile sensor networks," *Automatica*, vol. 46, no. 2, pp. 390–396, 2010. Corollary 2 then certifies the whole graph.

Why this matters when no agent sees the global topology. The classical Hopcroft–Tarjan test of 10 needs a **global depth-first traversal**: the low-link value low[w] is defined by "the earliest discovery time reachable from v's subtree by descending tree edges and one back edge," which is a statement about the whole graph. Centralising the test means collecting all $N(N-1)$ pairwise distances and running an $O(N+E)$ DFS — impossible for a swarm whose members can only hear within R_{comm}, and doubly impossible for a MAPPO policy whose critic would then need a graph built from a global state vector that never exists. The Zareh–Sabattini–Secchi construction circumvents this in three ways:

1. **It replaces "remove i" by "weaken i."** The perturbation $A^i(\epsilon) = \epsilon \cdot A^i$ does not require knowing $\mathcal{G}^{R_i}$ — it requires only rescaling the a_{ij} that node i already exchanges with its 1-hop neighbours. As $\epsilon \to 0$ the graph $\mathcal{L}^i(\epsilon)$ converges to the post-failure Laplacian, so the *spectrum of the weakened graph* certifies the *connectivity of the failed graph*.
2. **It exploits the Fiedler orthogonality identity to eliminate the global unknown.** Theorem 1's reduction (1) expresses the non-null spectrum of $\mathcal{L}^i(\epsilon)$ in terms of $\mathcal{L}^{R_i}$ — and the perturbation term $\epsilon(\text{diag}(\tilde{a}_i) + \tilde{a}_i \mathbf{1}^T)$ is built **entirely from i's own adjacency row**, i.e. from quantities a node knows. The bound (12) then degrades this to a *scalar* test involving only a_{ik} and a global constant $\sqrt{n}$ and the (estimable) λ_3.
3. **It puts the burden on only the at-risk nodes.** Corollary 2's "every node that is **not locally biconnected**" clause means that in a well-spread formation almost every node terminates in Stage A, and the expensive spectral machinery runs on a small minority. Combined with (1) and (2), this is a *scalable* local certificate of a *global* property.

The honest caveat about "2-hop". The user-specified brief mentions *1-hop/2-hop* neighbour information. Precisely stated: **this paper's local check (Proposition 2 / Remark 1) uses 1-hop data only.** The **2-hop** formulation in this line of work comes from the closely related paper that the Zareh–Sabattini–Secchi paper itself

cites as reference [21]: C. Ghedini, C. Secchi, H. C. M. Ribeiro, L. Sabattini, "Improving robustness in multi-robot networks," *IFAC Symposium on Robot Control (SYROCO)*, 2015, which works "based on a maximum 2-hop communication" and introduces a parameter called **vulnerability**, "that allows each robot to detect the level of its effect on the topological configuration." The report should either (a) cite Ghedini et al. (SYROCO 2015) for the 2-hop / vulnerability formulation and Zareh–Sabattini–Secchi for the 1-hop-plus-spectral one, or (b) drop the word "2-hop" when describing the CDC-2016 result.

Failure mode of the condition, stated by the paper itself. Condition (12) is **sufficient, not necessary**. A node can be genuinely non-articulate and still fail (12) simply because the edge weights are poorly scaled. So the scheme can only *positively* certify safety; a failed check is inconclusive and must trigger a motion (as the companion paper "Enforcing Biconnectivity in Multi-robot Systems," pp. 1800–1805, does via a gradient controller $\dot{p}_i = \frac{d\lambda_3(\epsilon)}{dp_i} = v_3(\epsilon)^T \frac{dv_3(\epsilon)}{dp_i}$ that pushes configurations toward larger λ_3).

Comparison with the alternatives, for the report's related-work table.

Approach	Data required	Strength	Weakness
Centralised Hopcroft–Tarjan (10)	global $O(N+E)$ traversal	exact, $O(N+E)$	needs a global state vector; impossible in a decentralised/sparse-comms setting
Ahmadi & Stone, *A distributed biconnectivity check*, DARS 7, 2006, pp. 1–10	nodes exchange "a big amount of information" to build connected/doubly-connected tables	exact	explicitly criticised by Zareh et al. as "labelled distributed" but requiring heavy information exchange
Butterfield, Dantu, Gerkey, Jenkins, Sukhatme, *Autonomous biconnected networks of mobile robots*, WiOPT 2008, pp. 640–646	global probe	constructive	"requires a global probe, so it cannot be seen as a decentralized one" (Zareh et al.)
Zareh–Sabattini–Secchi, CDC 2016 (this paper)	1-hop neighbour positions; distributed eigenvalue estimate for risky nodes	first decentralised sufficient conditions, neighbour-to-neighbour, cheap screen	sufficient not necessary; requires λ_3 estimation
Ghedini, Secchi, Ribeiro, Sabattini, SYROCO 2015	up to 2-hop	explicit *vulnerability* parameter; detects "dangerous topological configurations"	—

0.11.9. 17. 2-Vertex-Connectivity, and Why It Implies Tolerance to a Single Node Failure

17.1 Definition

A graph G is k-**vertex-connected** (or k-**connected**, or k-**robust**) iff $|V| > k$ and $G-S$ is connected for every set $S \subseteq V$ with $|S| < k$.

G is **2-vertex-connected** (biconnected) iff $|V| \geq 3$ and $G - S$ is connected for every $S \subseteq V$ with $|S| \leq 1$.

Equivalently, by the paper's Definitions 2 and Lemma 2 (16.2), G is biconnected iff it has **no articulation points**, iff it is a **block**, iff every pair of vertices is **doubly connected** (joined by two or more internally vertex-disjoint paths), iff G is 2-connected in the Menger sense.

Structural characterisations of 2-connectivity (all standard). A graph G with $|V| \geq 3$ is 2-vertex-connected if and only if any (equivalently, some) of the following holds:

- $\delta(G) \geq 2$ **and** G has no articulation point;
- G is connected and $G - S$ is connected for every single $v \in V$;
- every pair of vertices is joined by two internally vertex-disjoint paths (Menger + Lemma 2);

- G is connected, G has no cut edge, and G has no cut vertex (the "no bridges" part alone is not sufficient — $K_{1,3}$ has no bridges but has cut vertices);
- G is connected and every edge lies on a cycle **and** G has no cut vertex.

17.2 Why 2-connectivity is exactly single-node-failure tolerance

> **Proposition 17.1 (fault-tolerance equivalence).** *Let $G = (V, E)$ be a graph with $|V| \geq 3$. The following are equivalent: (a) $\kappa(G) \geq 2$ (i.e. G is 2-vertex-connected / biconnected); (b) G is connected and G has no articulation point; (c) for every $v \in V$, $G - v$ is connected; (d) for every pair $u, w \in V$, there exist two internally vertex-disjoint u–w paths; (e) for every $v \in V$ and every pair $u, w \in V \setminus \{v\}$, there is a u–w path in $G - v$.*

Proof. $(a) \Leftrightarrow (b)$ is the definition (with the convention that $\kappa(G) \geq 2$ requires $|V| \geq 3$). $(b) \Rightarrow (c)$ is immediate. $(c) \Rightarrow (b)$: if some v disconnects $G - v$, then G is connected but v is an articulation point. $(b) \Rightarrow (d)$ and $(d) \Rightarrow (b)$: $(b) \Rightarrow (d)$ is Lemma 2 of Zareh–Sabattini–Secchi (biconnected $\iff$ all pairs doubly connected); $(d) \Rightarrow (b)$: if v were an articulation point, $G - v$ has components A, B with $a \in A, b \in B$; any a–b path in G must pass through v, so no two a–b paths in G can be internally vertex-disjoint (both would need v internally), contradicting (d). $(d) \Leftrightarrow (e)$: (e) trivially implies (d) by taking $v \notin \{u, w\}$… more carefully, (d) implies (e) because two internally vertex-disjoint u–w paths in G use distinct internal vertices, so at most one of them uses any given $v \neq u, w$; and (e) with $v = u$ or $v = w$ is vacuous, so restrict to $v \notin \{u, w\}$ and note that (e) with $|V| \geq 3$ also forces (c). ■

The operational reading. Condition (c) *is* the mission requirement stated literally: *if any one UAV fails — disappears from the network and takes all of its incident communication links with it — the remaining $N - 1$ UAVs are still able to communicate.* So:

$$\ddot{\text{t}}\text{olerant to any single UAV failure} \iff \kappa(G) \geq 2 \iff \text{G}\mathit{hasnoarticulationpoint}\text{G}\mathit{hasnoarticulationpoint}\text{G}\mathit{hasnoarticulatio}$$

Note what (d) buys beyond mere connectedness: **two independent routes per pair.** In K_N there are N internally vertex-disjoint paths between any pair. In a star $K_{1,m}$ there is exactly one, through the hub, and the hub is a single point of total mission failure. Condition (d) quantifies the *amount* of redundancy, and it is precisely the quantity that a stochastic-failure model needs in order to compute mission-survival probability.

17.3 The maximum survivable failure count

> **Corollary 17.2.** *If G is k-vertex-connected then G remains connected after the removal of any $k-1$ vertices. Therefore a k-connected UAV fleet of size N tolerates $k - 1$ simultaneous failures. Since $\kappa(G) \leq \delta(G) \leq N - 1$, a complete graph K_N is the unique graph that tolerates $N - 1$ failures, and any k-connected graph with N vertices tolerates at most $N - 1$ failures.*

Proof. G is k-vertex-connected iff $G - S$ is connected for all $|S| < k$; take $|S| = k - 1$. ■

Design implication for the report. If the failure model is "each UAV fails independently with probability p per unit time" and the fleet must remain connected, then the $K = 2$ requirement is the *minimum* bar (tolerate exactly one failure), and the natural strengthening is $\kappa(G) \geq 3$ (tolerate two), which needs a richer topology — a **cycle of N UAVs** (C_N, $N \geq 3$) has $\kappa = 2$; the **wheel** W_N and the **Mbius ladder** have $\kappa = 3$; a **3-connected planar graph** (e.g. the tetrahedron K_4, the cube Q_3, any maximal planar graph on $n \geq 4$ vertices) has $\kappa = 3$. A hexagonal UDG deployment naturally produces a **triangular-lattice-like** network whose interior nodes have $\kappa = 3$ and whose corners are the $\kappa = 2$ bottleneck — so the honest design step is to add a *backbone ring* of relay nodes to raise the corner κ.

0.11.10. 18. Why Preserving 2-Connectivity Implies 1-Fault Tolerance — The Graph-Theoretic Reason, Spelled Out

Five logically distinct reasons, in increasing order of concreteness. Any one of them is a complete justification; together they are what the report's theory section needs.

(1) Definitional / set-theoretic. By 17.1, 2-vertex-connectivity *is* the statement "$G - v$ is connected for every v." A UAV failure is precisely the event "v is deleted along with all of its incident edges," i.e. $G \mapsto G - v$. Hence $\kappa(G) \geq 2 \Rightarrow$ the post-failure graph is connected, by the literal content of the definition. There is no additional modelling assumption: the physical failure model and the combinatorial property are the same statement.

(2) The two-route argument (Menger). 2-connectivity means, by Lemma 2 and Menger, that for every ordered pair (u, w) there are two internally vertex-disjoint u–w paths, say P_1 and P_2. Because they are internally disjoint, P_1 and P_2 share no vertex other than u and w. Let v fail. For any $u \neq v, w \neq v$:

- if $v \notin V(P_1)$, then $P_1 \subseteq G - v$ survives, and u, w remain connected;
- if $v \in V(P_1)$ (internally), then $v \notin V(P_2)$ (internal disjointness), so $P_2 \subseteq G - v$ survives.

Hence **for every pair, at least one of the two routes survives any single vertex deletion.** The fleet's pairwise communication survives *simultaneously for all pairs* because the argument is applied to each pair independently. The two-path structure is exactly the redundancy that single-node failure consumes — and 2-connectivity guarantees two are available.

(3) The cycle / edge-on-a-cycle argument. Every edge of a 2-vertex-connected graph lies on a cycle (since every edge lies on a path between its endpoints, and with a second internally vertex-disjoint path for those endpoints, that path plus the edge forms a cycle). The failure of a node v deletes each incident edge. Each deleted edge $\{v, u\}$ lies on a cycle in G; that cycle minus v is a u-to-{the two cycle neighbours of v} path, so it cannot be the *only* route out of u. Hence no node becomes isolated by the failure, and the graph stays connected. This is the geometric intuition that makes a **ring** topology fault-tolerant: the ring is the minimal 2-connected unit-disk topology, because cutting the ring at node v leaves a *path*, which is connected.

(4) The block-cut-tree / ear-decomposition argument. Every connected graph decomposes uniquely into a **block-cut tree** $\mathcal{T}$: a tree whose nodes are the blocks of G and the cut vertices of G, with incidence edges. A graph has no articulation points **iff** its block-cut tree is a single node, i.e. **iff G is itself a single block.** So 2-connectivity $\iff$ "the block-cut tree has collapsed to one node." A node failure is survivable $\iff$ deleting any node leaves the tree connected $\iff$ the block-cut tree has no cut node of degree ≥ 2 $\iff$ the tree is a single node $\iff$ G is biconnected. This is a complete structural characterisation, and it is the one that justifies the diagnostic: *the report can compute G's biconnected components in the same $O(N+E)$ pass as 10 and check that there is exactly one block.*

(5) The quantitative argument — what fault tolerance actually buys. For a mission that needs a designated pair of entities to remain in communication, Menger turns 2-connectivity into a quantitative statement: the **local connectivity** $\kappa_{u,w}(G) = \min\{|S| : S \text{ separates } u \text{ from } w\} \geq 2$ is exactly the number of independent routes. Therefore:

- **Survival of pairwise communication under f independent, uniformly random node failures** has probability (for $\kappa_{u,w} \geq f + 1$) equal to 1 in the worst case and, in the random-failure model, at least $\mathbb{P}$[fewer than $\kappa_{u,w}$ of the $\{u, w\}$-cut vertices fail]. The binding statistic is the **size of the smallest vertex cut**, not the number of edges.
- **Minimum-degree screening.** Because $\kappa(G) \leq \delta(G)$, a *necessary* condition for 2-connectivity is $\delta(G) \geq 2$, i.e. **every UAV must have at least two neighbours.** This is an $O(N)$ check that can be run in the reward function for essentially free, and it eliminates the most common failure mode in sparse UDG deployments (a UAV drifting to the boundary with a single link).
- **Why edges are the wrong statistic.** $\lambda(G) \geq 2$ (no bridges) is *necessary but not sufficient*: $K_{1,3}$ has $\lambda = 1$ so it is caught, but consider C_3 with a pendant vertex — it has a bridge. The clean counterexample to "no bridges $\Rightarrow$ 2-connected" is the **star** $K_{1,m}$ for $m \geq 2$: it has m bridges, so it is caught; but a graph such as two triangles sharing a single vertex (the "bowtie" / figure-eight, $C_3 \vee C_3$) has **no bridges** (every edge lies on one of the two 3-cycles) yet **does** have an articulation point at the shared vertex. So bridge-freedom is *strictly weaker* than 2-connectivity. A UAV report that only checks for bridges has a genuine gap; the report must check for **articulation points**, and 16's Theorem 2 is precisely the decentralised certificate for that.

Summary for the report's theory section, in one line each:

$$\underbrace{\text{UAV } v \text{ fails}}_{\text{physical}} \iff \underbrace{G \mapsto G - v}_{\text{combinatorial (node failure = node + all its edges)}} \implies \underbrace{\kappa(G) \geq 2}_{\text{requirement: } G-v \text{ connected } \forall v} \iff \underbrace{\text{two internally vertex-disjo}}_{\text{Menger + Len}}$$

and this is a **sufficient** (not necessary) design criterion, which is exactly why a learning-based method (16) is appropriate: learning discovers 2-connectivity via a *local, sufficient, distributed* certificate without ever needing the global articulation-point structure, and the screening cascade (1-hop block test, then spectral test on the risky minority) makes that affordable.

0.12. PART C — ENERGY

0.12.1. 19. The Peukert Model

19.1 Statement and exact equations

Peukert's law states that the **discharge time** of a battery is a power-law function of the discharge current, with exponent $n > 1$:

$$\boxed{I^n t = K} \qquad \text{equivalently} \qquad t(I) = K I^{-n},$$

where I is the (constant) discharge current, t is the time to reach the cutoff voltage, K is a cell-specific constant, and n is the **Peukert constant** (also called the Peukert exponent). Equivalently, in terms of capacity $C_{\text{use}} = It$ (charge actually removed before cutoff, which is *less* than the nominal capacity C_N):

$$\boxed{C_{\text{use}} = K I^{1-n}}$$

The **integral (general) form**, which is the version that is valid for time-varying current and is what an energy-aware simulator should use:

$$\boxed{\int_0^{t_{\text{cut}}} I(t)^n \, dt = K} \qquad \text{(discharge, to a fixed cutoff voltage).}$$

19.2 C-rate

The **C-rate** expresses the current as a multiple of the rated capacity. Two conventions in use:

$$\text{C-rate} = \frac{I}{C_{Ah}} \quad [\text{Amperes / Ampere-hours, dimensionally h}^{-1}], \qquad \text{C-rate} = \frac{I}{I_{1\text{h}}} \quad [\text{multiple of the 1-hour rated current}$$

The two coincide when $I_{1\text{h}} = C_{Ah}$, which holds for many datasheets. At a C-rate of $\frac{C}{h}$, the *ideal* discharge time is $\frac{1}{C}$ hours: $1C \Rightarrow 1$ h, $2C \Rightarrow 0.5$ h, $\frac{1}{2}C \Rightarrow 2$ h.

19.3 Calibrating K from the datasheet

If the datasheet gives the **rated (1-hour) current** $I_{1\text{h}}$ and states that the cell delivers $t = 1$ h at that current, then $K = I_{1\text{h}}^n \cdot 1$, i.e.

$$t(I) = \left(\frac{I_{1\text{h}}}{I}\right)^{n-1} \text{ hours}, \qquad C_{\text{use}}(I) = C_{Ah}\left(\frac{I_{1\text{h}}}{I}\right)^{n-1},$$

where $C_{Ah} = I_{1\text{h}} \times 1$ h is the 1-hour rated capacity. **Worked example.** $n = 1.1$, $I_{1\text{h}} = 2$ A ($C_{Ah} = 2$ Ah): at $I = 2I_{1\text{h}} = 4$ A, $t = 2^{-(1.1-1)} = 2^{-0.1} = e^{-0.1 \ln 2} = e^{-0.0693} = 0.9330$ h and $C_{\text{use}} = 2 \times 0.9330 = 1.866$ Ah. So a "2C" discharge on a cell rated 1h/2 Ah actually yields 1.866 Ah, not 1 Ah — a 6.7 % capacity loss, and 6.7 % less endurance.

For $n = 1.3$ and $I = 2I_{1\text{h}}$: $t = 2^{-0.3} = e^{-0.2079} = 0.8123$ h, $C_{\text{use}} = 1.6246$ Ah, a 18.8 % loss. **The high-C regimes that a UAV experiences during take-off and aggressive manoeuvring are exactly where Peukert bites hardest**, which is why an endurance estimate computed at a single nominal cruise current is optimistic.

19.4 Typical values of n

Chemistry	Typical n	Note
Lead–acid	1.1 – 1.3	strongly Peukert-limited; high C-rates cause sulfation and capacity loss
NiMH / NiCd	1.05 – 1.15	mild
Li-ion (NMC / LCO)	1.0 – 1.1	nearly linear; much better than lead–acid
$LiFePO_4$	≈ 1.0	very close to the ideal

The theoretical floor is $n = 1$ (no loss); the general empirical bound is $1 < n \leq 2$, with $n = 2$ corresponding to the extreme case in which the usable capacity is inversely proportional to current (zero energy at infinite current). The **charge exponent** differs from the discharge exponent: for charging, $I^m t = K'$ with m typically larger than n (1.2–1.6), which is the quantitative statement behind "charging is harder than discharging."

19.5 Limitations of the Peukert model (all relevant to a UAV simulator)

1. **Constant-current only.** Peukert's law is derived from constant-discharge experiments. The integral form (19.1) is an *extension by assumption*; there is no general theorem validating it for arbitrary $I(t)$, and empirically it is only approximate for transients.
2. **No recovery effect.** Real cells regain a small amount of usable capacity at rest (reversible diffusion relaxation, and in lead–acid the dissolution/redeposition of sulfate). Peukert cannot represent this: $\int I^n dt$ has no memory of *when* the current was drawn, only its n-th power sum. A UAV that alternates hover and cruise can exploit this; a Peukert model forbids it.
3. **Single exponent ⇒ cannot represent the discharge curve.** A cell has an internal resistance and a non-linear OCV–SoC relation, so the *usable capacity* depends on the *cutoff voltage*, which depends on the *load*. Peukert has no V and no R.
4. **No temperature dependence.** Peukert's n and K both depend strongly on temperature; at $-10°$C the effective n rises and K falls. A UAV operating outdoors must not treat K as constant.
5. **No ageing.** K and n both degrade with cycle count and calendar time; capacity fade is not captured.
6. **No charging model.** Peukert is a discharge law. Charging requires the separate CC–CV model of 22, whose time constant τ_{cv} enters the objective in a qualitatively different (logarithmic) way.
7. **No behaviour at low SoC.** Real cells exhibit voltage collapse, and delivered energy per unit SoC rises near the knee (21). Peukert predicts a constant C_{use}/I regardless of SoC.

Which model to use in the report. Peukert is the right *sanity-check* model for the battery's headline endurance number, and it is the right model to expose as a **configurable, reportable parameter** n in a sensitivity analysis. But the *within-episode* dynamics that drive the MAPPO charging decisions — the recovery effect, the CC–CV tail, the voltage/SoC nonlinearity — need KiBaM (20) and the charging model (22) instead. Stating this explicitly in the report is a strength, not a weakness.

0.12.2. 20. The Kinetic Battery Model (KiBaM)

20.1 Status of this citation and my honesty flag

The report's source is: D. G. Marquis and V. DeCicca, "Kinetic battery model for managing dynamic resource consumption in hybrid electric vehicles," *IEEE Transactions on Control Systems Technology*, vol. 17, no. 2, pp. 369–381, 2009. **I was not able to verify the exact volume/issue/page details or the DOI in a live search on 29 September 2026** (searches for the title and for the commonly cited DOI returned unrelated battery-modelling papers). The model below is the KiBaM as it is reproduced across the subsequent literature, and I have written the equations in the internally consistent normalisation. Two consequences of that choice are flagged at the end of this section. **Action: confirm the pagination and DOI against IEEE Xplore before citing.**

20.2 States and constraints

The battery is modelled as **two charge reservoirs in series with the load**, and the states are

- q — the total stored charge (normalised so that $q = 1$ is full and $q = 0$ is empty);
- q_1 — the charge in reservoir 1, constrained $0 \leq q_1 \leq c$;
- q_2 — the charge in reservoir 2, constrained $0 \leq q_2 \leq 1 - c$;

with the algebraic relation

$$\boxed{q \;=\; q_1 + q_2}$$

where $c \in (0, 1)$ is the **relative size of reservoir 1** (typically $c = 0.5$, so the two reservoirs are equal). The model's single dimensionless voltage parameter is fixed by the equilibrium of the full battery:

$$\boxed{V \;:=\; \frac{1}{c} + \frac{1}{1-c} \;=\; \frac{1}{c(1-c)}} \qquad \left(V = 4 \text{ when } c = \tfrac{1}{2}\right).$$

Constants: $k_0 > 0$ is the **coupling constant** (the rate of charge transfer between the reservoirs per unit imbalance); $k_1, k_2 > 0$ are the **recharge rate constants** of reservoirs 1 and 2. Current I: $I > 0$ is discharge, $I < 0$ is charge.

20.3 The exact ODE system

$$\boxed{\begin{aligned} \frac{dq}{dt} &= -I, \\ \frac{dq_1}{dt} &= -\frac{I}{V\,q_1}, \\ \frac{dq_2}{dt} &= -\frac{I}{V\,q_2}, \\ k_0\left(\frac{dq_1}{dt} - \frac{dq_2}{dt}\right) &= -\frac{k_1}{c}\,(c - q_1)\,q_1 \;-\; \frac{k_2}{1-c}\,(1 - c - q_2)\,q_2. \end{aligned}} \tag{20.1}$$

Equilibrium check (this is why the factors $1/c$ and $1/(1-c)$ are there). At $I = 0$ and $(q_1, q_2) = (c, 1-c)$: $dq_1/dt = dq_2/dt = 0$, and the right-hand side of the coupling equation is $-\frac{k_1}{c}(c-c)c - \frac{k_2}{1-c}(1-c-(1-c))(1-c) = 0$. So $(q_1, q_2) = (c, 1-c)$ with $q = 1$ is a genuine equilibrium, as it must be. *(If the coupling terms are instead written $-k_1(1-q_1)q_1$ and $-k_2(1-q_2)q_2$ with q_1, q_2* normalised per-reservoir*, the same statement holds with $q_1 = q_2 = 1$ and $V = 2$; the two forms are related by that rescaling. The form above is the one that is self-consistent with $V = 1/(c(1-c))$ and $q = 1$ at full charge.)*

Physical interpretation of the two charge-split equations. The fraction of the terminal current I that enters reservoir 1 is $I_1 = -I/(Vq_1)$ and that entering reservoir 2 is $I_2 = -I/(Vq_2)$. The equations say: *the charge flow into a reservoir is inversely proportional to the charge already in it* — a reservoir that is fuller accepts current more slowly. Note the consistency subtlety, which the report should state: $\dot{q}_1 + \dot{q}_2 = -\frac{I}{V}\left(\frac{1}{q_1} + \frac{1}{q_2}\right)$, which equals $\dot{q} = -I$ **iff** $\frac{1}{q_1} + \frac{1}{q_2} = V$, i.e. iff $q_1 q_2 = c(1-c)\,q$. This holds identically at $q = 1$ (where $q_1 q_2 = c(1-c)$) but not in general; the four equations of (20.1) are therefore best read as a *coupled* system in which the dq/dt equation is the charge-conservation statement and the two split equations define the internal flow law, with the coupling equation tying the split to the reservoir "recharge" terms. This is the standard reading of the model and is the reading used below.

20.4 The physical effect: the recovery effect

The **recovery effect** is: *after a rest period, the power available from a battery increases without any net energy input.* Mechanically, in KiBaM the relaxation mechanism is the coupling term. Consider a cell that has just been heavily discharged: the heavy current preferentially depletes the **lower-resistance** reservoir (whichever of the two has larger q, since $I_1 = -I/(Vq_1)$ is smaller when q_1 is larger), so the two reservoirs become *imbalanced* and the internal resistance is high. During a rest, $I = 0$ and the coupling equation drives

$$k_0\,(\dot{q}_1 - \dot{q}_2) = -\frac{k_1}{c}(c - q_1)q_1 - \frac{k_2}{1-c}(1 - c - q_2)q_2$$

with the RHS ≤ 0 whenever $0 \le q_1 \le c$ and $0 \le q_2 \le 1 - c$; so $\dot{q}_1 \le \dot{q}_2$, and charge flows **from the fuller reservoir to the emptier one**, rebalancing the cell and lowering its internal resistance. The relaxation time is set by k_0: larger $k_0 \Rightarrow$ faster internal exchange.

Quantitatively: the instantaneous power capability is $P = VI$ with I limited by the internal resistance, and the internal resistance of the two-reservoir model is maximal when the reservoirs are maximally imbalanced. Define the imbalance

$$\Delta \;:=\; q_1 - c,$$

which is 0 at equilibrium, negative after a discharge (reservoir 1 depleted first, being the lower-resistance one) and relaxes back to 0 at rest. For small Δ and a small $k_1 = k_2 = k$, the relaxation obeys, to leading order, a first-order lag

$$\dot{\Delta} \;\approx\; -\frac{1}{\tau}\,\Delta, \qquad \tau \;\approx\; \frac{1}{k}\cdot\frac{1}{c(1-c)},$$

so the fraction of imbalance remaining after a rest of length t_r is $e^{-t_r/\tau}$, and the power capability deficit decays like the same exponential. **For a UAV mission planner this is the whole point**: a mission that alternates 30 s bursts with 60 s rests has strictly better aggregate endurance than the same mission flown continuously at the average current, and a Peukert model (19) is *incapable* of expressing that.

20.5 Closed-form solution — what I can and cannot state

What the paper provides. For a **constant** current I, the system (20.1) reduces to a system in which the imbalance dynamics can be separated, and Marquis & DeCicca give an exact closed-form solution. Structurally it is a **ratio of two linear combinations of two exponentials** $e^{-\lambda_1 t}$ and $e^{-\lambda_2 t}$, with $\lambda_{1,2} > 0$ the two real roots of a quadratic characteristic equation in the model parameters, determined by the initial condition $(q_1(0), q_2(0))$.

Honest flag. I attempted to reconstruct the *algebraic constants* of that closed form by hand and could not do so with confidence — my derivation from the coupled equations did not reduce to a clean ratio of two exponentials in the way the published solution does, and I will not write down constants I cannot verify. **Therefore I am deliberately not reproducing the closed form here.** The report should either (a) transcribe eqs. (and the solution) directly from the IEEE Xplore PDF of the 2009 paper, or (b) omit the closed form entirely and state only (20.1), which is the part that a simulation actually needs — the ODEs are trivially integrated with a 1 ms explicit step and the closed form buys nothing at that fidelity.

What the report may safely state about the parameter set. The model has four physical parameters (c, k_0, k_1, k_2) plus V determined by c; the constraints are $0 < c < 1$, $k_0 > 0$, $k_1, k_2 > 0$, with $k_1 = k_2$ being the common symmetric special case. $V = 1/(c(1-c))$ gives $V = 4$ at $c = 0.5$, $V = 9/2 = 4.5$ at $c = 1/3$, $V \to \infty$ as $c \to 0$ or $c \to 1$. **I do not state a specific numeric** (k_0, k_1, k_2) **triple, because I could not verify the published values.**

20.6 Why KiBaM is more realistic than Peukert — a fair, itemised comparison

Property	Peukert (19)	KiBaM (20)	Why it matters for UAVs
Recovery effect	**absent**	present, with time constant set by k_0	hover/cruise alternation and landing/charge cycles gain real endurance
Load dependence of usable capacity	none (single n)	yes, via q_1, q_2	takeoff power spikes reduce endurance in a way Peukert cannot express
Temperature dependence	none (unless K is re-fitted)	absorbable into k_1, k_2	outdoor operation
Ageing	none	absorbable into k_1, k_2 degradation	multi-season missions
Charging	not modelled	$I < 0$ in (20.1) gives a charge dynamic; the true CC–CV tail needs 22	ties the endurance model to the pad-occupancy model
Closed form	yes (a power law)	yes (see 20.5)	both cheap to integrate
Parameter count	2 (n, K)	4–5	KiBaM costs more
Physical transparency	very high	lower	KiBaM is harder to justify to a reviewer

Honest bottom line for the report. KiBaM is a strictly better *dynamic* model (it has the recovery effect, load dependence, and an intrinsic charge model), but it is only a *charge/energy* model, not a *terminal-voltage* model: it does not give $V(t)$, so it cannot drive an energy-aware altitude/speed decision through a motor model, and it does not give the CC–CV tail. Its real value in this report is the **recovery effect and the load-dependent usable capacity**, which together justify treating endurance as a state-dependent quantity rather than a constant. If the report needs an open-circuit voltage for the energy-aware state, add the non-linear OCV–SoC relation of 21 on top of q — that composite model is the standard choice and is the one to describe.

0.12.3. 21. Li-Ion Discharge: Why SoC Is *Not* Linear in Remaining Energy

21.1 The two distinct senses of "state of charge"

A UAV battery report almost always conflates two quantities. They must be separated explicitly.

(a) Coulomb-counted SoC (ampere-hours). With C_{Ah} the nominal capacity in Ah,

$$\mathrm{SoC}_{cc}(t) \;=\; \frac{1}{C_{Ah}} \int_{t_0}^{t} I_{out}(\tau)\, d\tau \qquad \text{(discharge, SoC} \in [0,1]\text{)}, \qquad \frac{d\,\mathrm{SoC}_{cc}}{dt} = -\frac{I(t)}{C_{Ah}}.$$

This is **linear in time at constant current** and completely blind to voltage.

(b) Energy-based SoC. The energy still retrievable from the cell at state-of-charge s is

$$\boxed{E_{\text{avail}}(s) \;=\; \int_{s}^{1} v_{oc}(\sigma)\, C_{Ah}\, \sigma\, d\sigma \qquad \text{(energy, in J if } v_{oc} \text{ in V and } C_{Ah}\sigma \text{ in Ah, times 3600)}}$$

where $v_{oc}(s)$ is the **open-circuit voltage as a function of SoC**. This is the integral of the OCV curve, and it is a nonlinear function of s precisely because $v_{oc}(s)$ is not constant.

The **linear (i.e. wrong) model** used in most energy-agnostic UAV planners is

$$E^{\text{lin}}_{\text{avail}}(s) \;=\; (1-s)\, \bar{v}_{oc}\, C_{Ah}\, \sigma, \qquad \bar{v}_{oc} = \int_{0}^{1} v_{oc}(\sigma)\, d\sigma,$$

and the **linear model is exactly the chord** from $(0,0)$ to $(1, \bar{v}_{oc} \cdot C_{Ah})$ in the (s,E) plane, whereas the truth is the curve obtained by integrating v_{oc}. They agree at $s=0$ and at $s=1$ and disagree in between by the whole "knee vs plateau" asymmetry. The relative error is

$$\varepsilon(s) \;=\; \frac{E_{\text{avail}}(s) - (1-s)\,\bar{v}_{oc} C_{Ah}\sigma}{(1-s)\,\bar{v}_{oc} C_{Ah}\sigma}.$$

21.2 Why SoC is *not* proportional to remaining energy — the terminal-voltage relation

Under load I the terminal voltage is

$$\boxed{v(t) \;=\; v_{oc}\big(\mathrm{SoC}(t)\big) \;-\; I(t)\, R_{int}\big(\mathrm{SoC}(t), T\big) \;-\; \text{overpotential terms},}$$

so the instantaneous power is

$$P(t) \;=\; v(t)\, I(t) \;=\; v_{oc}\big(\mathrm{SoC}(t)\big) I(t) \;-\; I(t)^2 R_{int}(\mathrm{SoC}, T),$$

and **the incremental energy removed per unit SoC is $v_{oc}(\mathrm{SoC})$, not the nominal voltage.** Three independent sources of nonlinearity follow:

1. v_{oc} **is a nonlinear function of SoC** (dominant). See 21.3.
2. R_{int} **depends on SoC and temperature.** The internal resistance of a Li-ion cell rises steeply at low SoC (roughly an order of magnitude between 50 % and 5 % SoC in a healthy NMC cell) and rises with T and with ageing. So the *same* SoC delivers *less* power when cold or old.
3. **The I^2R term.** The power available at the terminals is less than the chemical power by a state-dependent $I^2 R_{int}$ loss, so the map from "current" to "SoC consumed" is itself state-dependent — which is the same statement as "usable capacity depends on load" in 20.

21.3 Typical voltage-versus-SoC behaviour

Representative OCV curves (characteristic engineering shapes for a fresh cell at 25°C, **not** datasheet values — the report must use the cell's own measured v_{oc} table for any calibration):

SoC	$LiCoO_2$ v_{oc}	NMC v_{oc}	$LiFePO_4$ v_{oc}
0 % (cutoff)	≈ 3.00 V	≈ 3.00 V	≈ 2.50 V
10 %	≈ 3.20 V	≈ 3.20 V	≈ 3.10 V
20 %	≈ 3.40 V	≈ 3.42 V	≈ 3.20 V
30 %	≈ 3.50 V	≈ 3.53 V	≈ 3.22 V
50 %	≈ 3.60 V	≈ 3.68 V	≈ 3.24 V
70 %	≈ 3.68 V	≈ 3.78 V	≈ 3.26 V
80 %	≈ 3.74 V	≈ 3.84 V	≈ 3.28 V
90 %	≈ 3.82 V	≈ 3.90 V	≈ 3.32 V
95 %	≈ 3.94 V	≈ 3.98 V	≈ 3.40 V
100 %	4.20 V	4.20 V	3.65 V

The qualitative structure, which is the point:

- **A long flat plateau** occupying the middle 50–60 % of SoC. In this band the per-unit-SoC energy is high and nearly constant, so the linear model is a decent approximation *here*.
- **A lower knee** below $\sim 20\,\%$ SoC, where v_{oc} falls steeply. Here each percent of SoC is worth far *less* energy and R_{int} rises steeply. **A planner that flies down to 5 % SoC "to squeeze out more range" is badly wrong**: the last 5 % contributes almost no energy and carries a large internal-resistance penalty plus BMS-cutoff risk.
- **An upper knee** above $\sim 90\,\%$ SoC, where v_{oc} rises steeply toward the 4.20 V ceiling. Here each percent of SoC is worth far *more* energy than in the plateau.

Numerical demonstration of the asymmetry (LCO, trapezoidal integration of the table above). Discretise at $s = 0, 0.1, \ldots, 1$:

$$E_{\text{avail}}(0) \;\approx\; C_{Ah}\,\sigma\Big[0.31 + 0.33 + 0.345 + 0.355 + 0.364 + 0.371 + 0.378 + 0.388 + 0.407\Big] \;=\; C_{Ah}\,\sigma \cdot 3.251 \text{ V}.$$

The linear model's $\bar{v}_{oc} = 3.251$ V by construction, so the two agree at $s = 0$. Now at $s = 0.5$:

$$\frac{E_{\text{avail}}(0.5)}{E_{\text{avail}}(0)} \;=\; \frac{0.364 + 0.371 + 0.378 + 0.388 + 0.407}{3.251} \;=\; \frac{1.908}{3.251} \;=\; 0.5868.$$

The linear model predicts $1 - 0.5 = 0.5000$. **At 50 % SoC the true remaining energy is 58.7 %, not 50 %** — the linear model under-estimates available energy by 14.7 % (relative error $\varepsilon(0.5) = (0.5868 - 0.5)/0.5 = +17.4\,\%$). Conversely at $s = 0.2$:

$$\frac{E_{\text{avail}}(0.2)}{E_{\text{avail}}(0)} \;=\; \frac{0.345 + 0.355 + 0.364 + 0.371 + 0.378 + 0.388 + 0.407}{3.251} \;=\; \frac{2.608}{3.251} \;=\; 0.8022,$$

against the linear model's 0.8 — a small +0.3 % error, because the low knee is thin.

Interpretation for the report. (a) **Discharging below $\sim 20\,\%$ SoC is nearly worthless and risky**, so the endurance calculation must be reported against the *usable window* $[s_{\min}, s_{\max}]$, not $[0, 1]$. (b) **The top-of-charge cutoff dominates usable energy** — the quantitative reason the headroom question in 23 is about *time* rather than about energy. (c) The report's "SoC-based" reward is an *approximation of an energy quantity* and should say so, then be corrected by the integral $E_{\text{avail}}(s)$. For $LiFePO_4$ the plateau is flatter and longer (3.20–3.28 V over 20–80 %), so the linear approximation is far better and the asymmetry largely disappears: **how bad the linear approximation is a chemistry choice, not a modelling choice.**

0.12.4. 22. Charging: the CC–CV Profile, Time Model, and Power Limits

22.1 The CC–CV profile

A Li-ion cell is charged in **two phases**.

Phase 1 — Constant Current (CC). A constant current I_{cc} flows in. The terminal voltage rises with SoC, from the OCV curve plus the IR drop:

$$v_{cc}(t) \;=\; v_{oc}\Big(s_0 + \frac{I_{cc}\,t}{3600\,C_{Ah}}\Big) \;+\; I_{cc}\,R_{int}\Big(\cdot\Big),$$

with t in seconds and C_{Ah} in Ah. The phase ends when v reaches the upper cutoff $v_{\max}$ (4.20 V/cell for $LiCoO_2$/NMC, 3.65 V for $LiFePO_4$). Let $s^\star$ be the **crossover** SoC at that moment.

Phase 2 — Constant Voltage (CV). The voltage is held at $v_{\max}$ and the current **decays** as the cell's internal resistance and overpotential relax:

$$\boxed{I_{cv}(t) \;=\; I_{cc}\,e^{-t/\tau_{cv}}\,, \qquad \tau_{cv} = R_{eq}\,C_{cell}}$$

where R_{eq} is the equivalent series resistance and C_{cell} the cell capacitance. The phase terminates at the cutoff current I_{term}, conventionally $I_{term} = 0.05\,C_{Ah}$ (a $0.05C$ termination — the last 5 % of current), to avoid an indefinite tail and limit degradation.

22.2 Exact charging-time model

Let $\Delta s_{cc} = s^\star - s_0$ and $\Delta s_{cv} = s_{\max} - s^\star$.

CC duration (charge balance; I_{cc} in A, C_{Ah} in Ah):

$$\boxed{t_{cc} \;=\; \frac{\Delta s_{cc}\, C_{Ah}}{I_{cc}}\ [\text{hours}] \;=\; \frac{3600\,\Delta s_{cc}\, C_{Ah}}{I_{cc}}\ [\text{seconds}]}$$

CV duration. Integrating $I_{cv}(t) = I_{cc} e^{-t/\tau_{cv}}$ and imposing $\int_0^{t_{cv}} I_{cv}(t)\,dt = 3600\,\Delta s_{cv} C_{Ah}$ (A·s):

$$I_{cc}\tau_{cv}\left(1 - e^{-t_{cv}/\tau_{cv}}\right) \;=\; 3600\,\Delta s_{cv}\, C_{Ah},$$

$$\boxed{t_{cv} \;=\; -\tau_{cv}\,\ln\!\Big(1 - \frac{3600\,\Delta s_{cv}\, C_{Ah}}{I_{cc}\,\tau_{cv}}\Big)} \qquad \text{valid iff } 3600\,\Delta s_{cv} C_{Ah} < I_{cc}\tau_{cv}.$$

Total, with charging efficiency $\eta_{ch} \in (0,1)$:

$$\boxed{t_{ch}(s_0, s_{\max}) \;=\; \frac{1}{\eta_{ch}}\left[\frac{3600\,\Delta s_{cc} C_{Ah}}{I_{cc}} \;-\; \tau_{cv}\ln\!\Big(1 - \frac{3600\,\Delta s_{cv} C_{Ah}}{I_{cc}\tau_{cv}}\Big)\right]}$$

with a hard lower bound from the charger's power limit P_{ch}:

$$\boxed{t_{ch} \;\geq\; \frac{E_{ch}}{\eta_{ch}\,P_{ch}}\,, \qquad E_{ch} = \int_{s_0}^{s_{\max}} v_{oc}(\sigma)\, C_{Ah}\, \sigma\, d\sigma}$$

(energy in units consistent with P_{ch}), so the true value is

$$t_{ch} \;=\; \max\!\left\{\frac{E_{ch}}{\eta_{ch} P_{ch}},\ \text{the CC–CV expression}\right\}.$$

The asymptotics that drive 23. Define the maximum charge the CV phase can absorb,

$$s_{cv}^{\max} \;:=\; \frac{I_{cc}\,\tau_{cv}}{3600\,C_{Ah}}.$$

Then

$$t_{cv}(\Delta s_{cv}) \;=\; -\tau_{cv}\ln\!\Big(1 - \frac{\Delta s_{cv}}{s_{cv}^{\max}}\Big), \qquad \boxed{\frac{\partial t_{cv}}{\partial(\Delta s_{cv})} \;=\; \frac{\tau_{cv}}{s_{cv}^{\max} - \Delta s_{cv}} \;\xrightarrow{\Delta s_{cv} \to s_{cv}^{\max}}\; \infty}$$

The marginal time cost of the last few percent of charge diverges. This is the single most important fact about charging for a pad-constrained mission, and it is what makes partial charging rational.

22.3 Charger power limit versus battery acceptance limit

Two *different* limits bound the current; conflating them is a common modelling error.

Charger limit (a property of the ground equipment). A pad of rated power P_{ch} can deliver at most

$$\boxed{I_{\text{ch}}(v) \;=\; \frac{P_{ch}}{v}}$$

amps at terminal voltage v. In CC, v *rises*, so I_{ch} *falls*: the CC phase silently becomes **power-limited (constant power, CP)** once $P_{ch}/v_{\max} < I_{cc}$.

Battery acceptance limit (a property of the cell). A Li-ion cell's maximum acceptable charge current falls steeply as SoC approaches 100 %:

$$\boxed{I_{\text{acc}}(s) \;=\; I_{\text{acc}}(0)\cdot g(s), \qquad g:[0,1]\to(0,1] \text{ strictly decreasing}}$$

with $g(0) \approx 1$, $g(0.5) \approx 0.5$–0.8, and $g(1) \approx 0.07$–0.20 for typical cells — i.e. the last 5 % is charged at $\sim 0.05C$–$0.1C$. Charging above $I_{\text{acc}}(s)$ causes lithium plating, gassing, heating and accelerated degradation. As a **maximum acceptance power**,

$$\boxed{P_{\text{acc}}(s) \;=\; v_{oc}(s)\, I_{\text{acc}}(s)} \qquad \text{decreases monotonically as } s \uparrow 1.$$

The three-way minimum is the whole charging model:

$$I(t) \;=\; \min\Big\{\, I_{cc},\; I_{\text{acc}}\big(\text{SoC}(t)\big),\; \frac{P_{ch}}{v\big(\text{SoC}(t)\big)} \,\Big\}, \qquad t_{\text{pad}} = \text{the time for SoC to move } s_0 \to s_{\max} \text{ under this } I(t)$$

integrated in closed form when I_{cc} binds (22.2) and numerically otherwise. **Every UAV competing for a pad pays this time; 23 asks whether a UAV should pay a smaller one.**

22.4 Standard practical numbers (for calibration)

- $\eta_{ch} \approx 0.90$–0.95 for CC–CV with a linear post-regulation stage; up to ~ 0.98 with a synchronous well-matched converter.
- $I_{cc} \approx 0.5C$ to $1.0C$; $I_{term} \approx 0.05C$.
- $\tau_{cv} = R_{eq}C_{cell}$ is typically 10–60 min, hence $s_{cv}^{\max} = I_{cc}\tau_{cv}/(3600C_{Ah}) \approx 0.1$–$0.4$ of capacity. **The CV phase therefore typically handles only the last 10–40 % of the charge while consuming a disproportionate share of the pad time.**

0.12.5. 23. Partial Charging: When to Fully Charge and When Not To

23.1 The source the report names, and an explicit accuracy flag

The report cites:

> X. Lin, Y. Yazcolu, D. Aksaray, "Robust Planning for persistent surveillance with energy-constrained UAVs and mobile charging stations," *IEEE Robotics and Automation Letters (RA-L)*, vol. 7, no. 2, pp. 4157–4164, 2022, doi: 10.1109/LRA.2022.3146938.

Verified on 29 September 2026 — this citation is solid: vol. 7, no. 2, pp. 4157–4164, 2022, doi 10.1109/LRA.2022.3146938; authors Xiaoshan Lin, Yasin Yazcolu, Derya Aksaray (CORIES, University of Minnesota). An ICRA 2022 conference version exists, and the workshop precursor is A. Pantazides, M. Aksaray, D. Aksaray, "Persistent surveillance with energy-constrained UAVs and mobile charging stations," *IFAC Workshop on Distributed Estimation and Control in Networked Systems (DONS)*, Chicago, IL, 2019.

The accuracy flag — this is the part the report must check. I retrieved the paper's **published abstract** and its four-part algorithmic structure: "(1) forming uniform UAV–UGV teams, (2) decomposing the environment into maximal partitions that can be covered by the UAVs in a single fuel cycle as long as the UAVs are released sufficiently close to the centers of the partitions, (3) maintaining the teams uniformly distributed over a cyclic path traversing those partitions, and (4) having the UAVs in each team cover their current partition and be transported to the next partition while being recharged by the UGV." The abstract also states the underlying problem is "an NP-hard combinatorial optimization problem." **Neither the abstract nor the algorithmic summary mentions a partial-charging result, and I was not able to obtain the full text or any statement of an "optimal charge fraction."** I therefore **cannot certify** that this paper's central result is a full-vs-partial-charging trade-off with an optimal fraction. The report should read the full text before attributing that claim to it.

Two adjacent bodies of work *do* treat partial recharge explicitly, and I have verified the relevant details of the second: (a) the SDFP / partial-recharge drone-routing line, which reports that "the share of partial recharges was relatively more prominent" in its optimal solutions; and (b) the AAMAS 2025 energy-sharing drone–UGV paper, which uses exactly the CC–CV model of 22 — "charge at constant current up to a crossover [point] ... t is the charge time in seconds, $t^\star$ is the time required to reach the crossover point $E^\star$, $P^\star$ is the power level at $E^\star$, and α_{c7} and c_8 are all constants determined by battery and hardware characteristics" — and derives a recharge-time function $t_c(E)$ from it. Both share the *mechanism* derived below.

Everything that follows is therefore derived from first principles. It is the correct mathematics for the trade-off regardless of which paper it is attributed to, and it is the argument a reviewer can check.

23.2 The decision variable and the objective

A UAV arrives at a pad with state of charge s_0. The pad serves it to a **chosen** target $u \in [s_0, 1]$ rather than always $u = 1$.

- **Energy gained:** $\Delta E(u) = E_{\text{avail}}(u) - E_{\text{avail}}(s_0) = \int_{s_0}^{u} v_{oc}(\sigma) C_{Ah}\sigma \, d\sigma$ (from 21.1).
- **Pad occupancy:** $t_{\text{pad}}(u) = t_{cc}(s_0, u) + t_{cv}(s_0, u)$, per 22.2.

- **Value of the charge:** in a *scheduled cyclic patrol* (the Lin et al. algorithm traverses a cyclic path of partitions), the marginal value of an energy increment is **exactly zero** once the UAV has enough to complete its current partition and reach the next release point with reserve. Let

$$u^\star \;:=\; \inf\big\{\, u \in [s_0, 1] : \Delta E(u) \;\geq\; E_{\text{mission}} + E_{\text{reserve}} \,\big\},$$

where E_{mission} is the energy to finish the current partition from the present point and $E_{\text{reserve}} = \beta E_{\text{mission}}$, $\beta > 0$ the safety factor.

The optimum:

$$\boxed{u_{\text{opt}} \;=\; \min\Big\{\, 1,\; u^\star \,\Big\}} \qquad (\,= u^\star \text{ in the typical energy-limited UAV case }\,)$$

There is no universal "optimal fraction." $u^\star$ is mission-dependent: it is the SoC at which the *surplus* energy first exceeds the requirement. Charging above $u^\star$ converts pad minutes into zero latency benefit.

23.3 The regimes, and when the answer flips

Regime	Condition	Optimal u	Reason
Energy-limited (the common UAV case)	$u^\star < 1$	$u_{\text{opt}} = u^\star$: charge only to the level that closes the mission	every further 1 % costs pad time and buys **zero** latency reduction
Energy-saturated	$u^\star = 1$	$u_{\text{opt}} = 1$: full charge	more energy still reduces latency (longer legs between visits), so the CV tail is paid willingly
Pad-contended ($n_{pads} < n_{UAVs}$; 24)	—	u_{opt} pushed down toward the usable SoC floor $s_{\min}$	pad *time* is the scarce resource; shortening t_{cv} multiplies fleet throughput

23.4 The intuition, quantified: the CV tail is a logarithmic trap

The mechanism is that t_{cv} is **logarithmically divergent**, so its derivative blows up. From 22.2, with $s_{cv}^{\max}$ as defined there,

$$\boxed{\frac{dt_{cv}}{d(\Delta s_{cv})} \;=\; \frac{\tau_{cv}}{s_{cv}^{\max} - \Delta s_{cv}}}$$

Worked numbers. Take a typical cell: $C_{Ah} = 2.5$ Ah, $I_{cc} = 2.5$ A ($1C$), $\tau_{cv} = 20$ min $= 1200$ s, $\eta_{ch} = 0.95$. Then

$$s_{cv}^{\max} \;=\; \frac{I_{cc}\tau_{cv}}{3600\, C_{Ah}} \;=\; \frac{2.5 \times 1200}{3600 \times 2.5} \;=\; \frac{3000}{9000} \;=\; 0.3333.$$

So the CV phase can absorb at most 33.3 % of capacity — consistent with 22.4. Now compare **marginal pad time per unit SoC**:

- **In CC:** $\dfrac{\partial t_{cc}}{\partial \delta} = \dfrac{C_{Ah}}{I_{cc}} = \dfrac{2.5}{2.5} = 1$ h per unit SoC.
- **In CV at $\Delta s = \delta$:** $\dfrac{\partial t_{cv}}{\partial \delta} = \dfrac{1200}{0.3333 - \delta}$ s per unit SoC. At $\delta = 0.20$: $\frac{1200}{0.1333} = 9000$ s $= 2.5$ h. At $\delta = 0.30$: $\frac{1200}{0.0333} = 36\,000$ s $= 10.0$ h.

The marginal pad time to charge the last 3.3 % of a 2.5 Ah cell is 10 hours, against 1 hour to charge the entire first 10 % in CC. The last three percent is worth 10 CC-equivalents.

Full charge from $s_0 = 0.20$. The CC phase runs $s : 0.20 \to 0.667$ (i.e. $\Delta s_{cc} = 0.80 - 0.3167 = 0.4833$ — see below), so let me compute cleanly. Full charge from s_0 to 1 means the CV phase absorbs $1 - s^\star$, which is the *whole* CV budget when $s^\star = 1 - s_{cv}^{\max}$. With the practical $0.05C$ termination, $\Delta s_{cv} = 0.95\, s_{cv}^{\max} = 0.95 \times 0.3333 = 0.3167$:

$$t_{cv} \;=\; -1200 \ln\!\Big(1 - \tfrac{0.3167}{0.3333}\Big) \;=\; -1200 \ln(0.05) \;=\; 1200 \times 2.9957 \;=\; 3594.9 \text{ s} \;=\; 59.9 \text{ min},$$

$$t_{cc} \;=\; \frac{(1-0.3167)-0.20}{1}\ \text{h} \;=\; 0.4833\ \text{h} \;=\; 29.0\ \text{min},$$

$$\boxed{t_{\text{pad}}^{\text{full}} \;=\; \frac{29.0+59.9}{0.95} \;=\; \frac{88.9}{0.95} \;=\; 93.6\ \text{min} \;\approx\; 1.56\ \text{h}.}$$

Partial charge from $s_0 = 0.20$ to $u = 0.80$. Now $u = 0.80 < s^\star = 1 - 0.3167 = 0.6833$? No — $0.80 > 0.6833$, so a charge to $u = 0.80$ **does** enter the CV phase. This is an important and frequently-missed point: with $s_{cv}^{\max} = 0.3333$, the CV phase begins at $s^\star = 0.667$, so *any* target above 0.667 incurs a CV tail. Charging to $u = 0.65$ (below $s^\star$) incurs none:

$$t_{cc} \;=\; \frac{0.65-0.20}{1} \;=\; 0.45\ \text{h} \;=\; 27.0\ \text{min}, \qquad t_{cv} = 0,$$

$$\boxed{t_{\text{pad}}^{\text{partial}} \;=\; \frac{27.0}{0.95} \;=\; 28.4\ \text{min}.}$$

Result: partial charging to 65 % costs 28.4 min of pad time versus 93.6 min for a full charge — a 3.3× reduction, releasing the pad more than three times sooner. Charging to 80 % instead gives $\Delta s_{cv} = 0.80 - 0.6833 = 0.1167$, so

$$t_{cv} \;=\; -1200\ln\!\Big(1-\tfrac{0.1167}{0.3333}\Big) \;=\; -1200\ln(0.65) \;=\; 1200\times 0.4308 \;=\; 516.9\ \text{s} \;=\; 8.6\ \text{min},$$

$$t_{cc} \;=\; \frac{0.6833-0.20}{1} \;=\; 0.4833\ \text{h} \;=\; 29.0\ \text{min} \quad\Rightarrow\quad t_{\text{pad}}^{80\%} \;=\; \frac{29.0+8.6}{0.95} \;=\; 39.6\ \text{min}.$$

So $u = 0.65$ gives 28.4 min, $u = 0.80$ gives 39.6 min, $u = 1.00$ gives 93.6 min: **the last 20 % of charge costs 55 extra pad minutes, i.e. 2.75 min per extra SoC-percent** — versus 0.6 min per percent in the CC range. The marginal cost rises by a factor of ≈ 4.6 as soon as the CV phase begins, and by a factor of ≈ 100 in its last few percent.

The energy deliberately forgone. Going to $u = 0.65$ instead of $u = 1$ forgoes $\int_{0.65}^{1} v_{oc} = 0.3167 \times \bar{v}_{oc}^{cv} \approx 0.3167 \times 3.85 = 1.22$ V-equivalents, i.e. $1.22 \times 2.5 = 3.05$ Ah of nominal capacity, at a nominal 3.85 V $\Rightarrow 3.05 \times 3.85 \times 3600 = 42.3$ kJ. **Trading 42 kJ of energy for 65 minutes of pad time** is the explicit form of the decision, and whether that is a good trade depends entirely on the pad:contention ratio of 24.

23.5 Intuition, stated in one paragraph

Charging a Li-ion cell to 100 % is *asymptotically expensive in time* because the cell's own voltage — which opposes further current at fixed voltage, hence decays the current — rises steeply above ~ 90 % SoC. The energy pushed in per minute falls toward zero as $s \to 1$. Meanwhile, in a persistent mission the value of energy is *step-like*: it is high while the UAV is the binding constraint on patrol latency and *exactly zero* once there is surplus. Charging into that zero-value region converts a scarce resource (pad-minutes) into a worthless one (surplus energy). Partial charging is therefore optimal precisely when $u^\star < 1$, and the *size* of the optimal fraction is mission-specific, not a universal constant. The one thing that *is* universal is the **shape** of the cost: logarithmic in the CV phase, linear in CC, and hence a 4–100× rise in marginal cost across the crossover.

0.12.6. 24. The Assignment Problem: Hungarian Algorithm, LAP vs MILP, and When Queueing Wins

24.1 The linear assignment problem (LAP)

Given n UAVs and n charging pads and a cost matrix $c = (c_{ij}) \geq 0$:

$$\min \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}\,x_{ij} \quad \text{s.t.} \quad \sum_{j=1}^{n} x_{ij} = 1 \;\; \forall i, \qquad \sum_{i=1}^{n} x_{ij} = 1 \;\; \forall j, \qquad x_{ij} \in \{0,1\}. \tag{LAP}$$

24.2 The Hungarian algorithm (Kuhn 1955; Munkres 1957)

Complexity: $O(n^3)$ **time,** $O(n^2)$ **space.**

Optimality certificate (the dual). The LP relaxation of (LAP) is integral (the constraint matrix is totally unimodular), so the LP optimum equals the IP optimum, and the **duals** α_i, β_j exist with

$$\boxed{\alpha_i + \beta_j \;\le\; c_{ij} \;\; \forall i,j, \qquad x_{ij} > 0 \;\Rightarrow\; \alpha_i + \beta_j = c_{ij}} \qquad \text{(complementary slackness)}$$

giving, for any feasible x,

$$\sum_{ij} c_{ij} x_{ij} \;\ge\; \sum_{ij} (\alpha_i + \beta_j) x_{ij} = \sum_i \alpha_i + \sum_j \beta_j \;\ge\; \text{LP optimum},$$

with equality at the optimal x. **Every Hungarian iteration produces such a dual-feasible pair, so the algorithm is exact, not heuristic.**

Munkres' (0-matrix) form, step by step.

1. **Subtract row minima.** Replace $c_{ij} \leftarrow c_{ij} - \min_j c_{ij}$ (transforms any $c \ge 0$ into a matrix with a zero row; dual updated as $\alpha_i \leftarrow \alpha_i + \min_j c_{ij}$).
2. **Subtract column minima.** Replace $c_{ij} \leftarrow c_{ij} - \min_i c_{ij}$ (β_j updated analogously). Now $c \ge 0$ and every row and column has a zero.
3. **Find a minimum cover of zeros.** Mark every zero; mark every **unmarked** column; mark every zero in a marked column; mark every **unmarked** row containing a marked zero; repeat. Draw lines through every marked column and every unmarked row. By Knig's theorem the minimum number of lines covering all zeros equals the maximum number of zeros with no two in a common row or column.
4. **Star and prime. Star** a maximal set of independent zeros. If there are n starred zeros, the star is an optimal assignment (stop). Otherwise count the number k of lines in the minimum cover.
5. **Adjust duals.** Subtract δ = smallest uncovered element from every uncovered element; add δ to α_i for each **covered row** and to β_j for each **uncovered column**; cross out covered lines, uncover crossed-out lines; star new zeros appearing; unstar starred zeros that become covered. Return to step 3.
6. **Termination.** The number of starred zeros strictly increases each pass, and the covered-line count k strictly decreases; at most n passes.

Complexity accounting. Steps 3–5 cost $O(n^2)$ per pass (matrix scans), and there are at most n passes: $O(n^3)$ total. Kuhn's 1955 original was $O(n^3)$ but with a large constant; Munkres' 1957 form is the practical $O(n^3)$ implementation and is the one in every scientific library.

Kuhn's formulation (shortest augmenting path). Kuhn's original worked on the *dual* and found, each iteration, a **shortest augmenting path** in the *reduced-cost* graph from an unmatched row to an unmatched column, then flipped the matching along it. The key object is the reduced cost

$$c^{\pi}_{ij} \;:=\; c_{ij} - \pi_i - \rho_j \;\ge\; 0$$

and the augmenting path is a path in the graph of **tight** arcs ($c^{\pi}_{ij} = 0$) that alternates between unused and used arcs, starting and ending at unmatched nodes. The shortest-augmenting-path view is the right one to quote, because it generalises: the same argument is the **successive-shortest-path** algorithm for **capacitated** assignment (24.4) and the first phase of **network-simplex** for the transport problem.

24.3 LAP vs MILP — when each is right

	LAP (Hungarian)	MILP
Formulation	(LAP) above	(LAP) + extra constraints/variables
LP relaxation	**integral** (total unimodularity)	generally fractional
Complexity	polynomial, $O(n^3)$	**NP-hard** in general
Optimality	exact, with dual certificate	optimal only at termination; gap reported
Parallelism	limited (sequential passes)	excellent (commercial MIP solvers are heavily parallel)
Side constraints	**none**	arbitrary
Solution time	predictable, $O(n^3)$	unpredictable (can blow up)

The exact statement of the integrality fact. The constraint matrix A of (LAP) has exactly one -1 and one $+1$ per column (so it is a *node-edge* incidence matrix of a bipartite graph, which is **totally unimodular**). Hence every vertex of the polytope $\{x \geq 0 : Ax = 1\}$ is integral, and the LP relaxation has integral optima. This is why $O(n^3)$ is achievable at all.

When the MILP is required. As soon as the UAV–pad relation is *not* one-to-one:

- **Pad capacity $q_j > 1$:** $\sum_i x_{ij} \leq q_j$. This is the *generalised assignment problem* (GAP) and is **NP-hard**, but it is directly handled by successive-shortest-path / flow methods, not by general MILP.
- **Assignment-dependent service time** (the physical case here): pad occupancy is $t_{\text{pad}}(s_0^{(i)}, u^{(i)})$ of 22.2–23, which depends on the UAV's arrival SoC *and* on the chosen target u. Then the objective $\min \sum_i t_{\text{pad}}(s_0^{(i)}, u^{(i)})$ couples the assignment and the charge decision, and neither a plain LAP nor a plain MILP with a c_{ij} captures it — it must be a MILP with both x_{ij} (assignment) and u_i (charge level) as variables, plus linking constraints.
- **Multi-leg assignment** (a UAV must visit a *sequence* of pads): becomes a time-windowed vehicle-routing MILP.
- **Uncertainty / robustness:** Lin et al. (2022) formulate the persistent-surveillance-with-mobile-charging problem as a **mixed-integer program that is NP-hard**, and solve it approximately — a direct confirmation that this problem class does not admit a polynomial-time exact algorithm.

24.4 Why one-to-one matching is right for UAV-to-pad assignment — and when queueing is the better model

Why one-to-one matching is the right abstraction when $n_{pads} \geq n_{UAVs}$. In the horizon of one planning cycle, if each UAV goes to exactly one pad and each pad serves at most one UAV, the feasible set of allocations is *exactly* the set of matchings of a bipartite graph, i.e. exactly the constraint set of (LAP). The physical facts that justify the abstraction:

1. **Mutual exclusion at the pad.** A pad is a physical landing platform; two UAVs cannot occupy it simultaneously.
2. **No benefit to re-assignment within a cycle.** The cost matrix c_{ij} is (approximately) separable, $c_{ij} \approx d(p_i, q_j)/v + t_{cv}(s_0^{(i)}, u_i) +$ pad queueing term. When the last two terms do not depend on j, the assignment reduces to a **distance-matching** problem and the one-to-one model is exact.
3. **Substitutability.** A UAV that cannot use its first-choice pad can use its second; the matching structure captures precisely this.
4. **It is exactly solvable in $O(n^3)$.** For a MAPPO critic value function, a differentiable surrogate of the assignment value is a natural, cheap, and non-smooth-but-tight signal.

Rectangular and over-subscribed cases. If $n_{pads} < n_{UAVs}$, (LAP) is replaced by the *rectangular* assignment: add $n_{UAVs} - n_{pads}$ dummy pads with zero (or "leave unassigned") cost, giving an exactly solvable problem in $O(n_{pads}^2 n_{UAVs})$. The *assignment* is then a decision of **which UAVs get a pad now and which queue** — which is exactly where the model must change.

When queueing is the better model. Queueing becomes the right model as soon as a pad can serve UAVs **sequentially** over an interval, so that the constraint is $\int_0^T$ (occupancy) $\leq$ capacity rather than a mutual-exclusion constraint. Then the pad is a single-server or c-server queue:

- $M/M/1$ (exponential interarrival and service times, $\lambda =$ UAV arrival rate, $\mu = 1/\bar{t}_{\text{pad}}$ per pad, utilisation $\rho = \lambda/\mu$):

$$W_q \;=\; \frac{\rho}{1-\rho}\cdot\frac{1}{\mu}, \qquad W \;=\; \frac{1}{\mu-\lambda}, \qquad L \;=\; \lambda W \quad \text{(Little's Law)}, \qquad L_q \;=\; \frac{\lambda^2}{\mu(\mu-\lambda)}.$$

- $M/G/1$ with general service distribution, using the mean $\bar{t}_{\text{pad}}$ and second moment $\overline{t^2}$: the Pollaczek–Khinchine formula gives

$$W_q \;=\; \frac{\lambda\,\overline{t^2}}{2\big(1-\rho\big)},$$

  which is the **right** model here, because the CC–CV profile of 22 makes the service-time distribution strongly non-exponential (a long, heavy CV tail).
- $M/M/c$ (c pads, c servers): with Erlang-C

$$C(c,\rho) \;=\; \frac{\dfrac{\rho^c}{c!\,(1-\rho)}\cdot\dfrac{1}{1-\rho/c}}{\displaystyle\sum_{k=0}^{c-1}\frac{\rho^k}{k!} \;+\; \frac{\rho^c}{c!\,(1-\rho)}\cdot\frac{1}{1-\rho/c}},$$

the expected wait is

$$\boxed{W_q = \frac{C(c,\rho)}{c\mu - \lambda}}$$

and the probability that *all* c servers are busy (the delay probability) is $C(c,\rho)$ itself — the quantity a report should plot against ρ.

Kingman's bound, the crucial warning. Both $M/M/1$ and $M/G/1$ give

$$W_q \xrightarrow{\rho\uparrow 1} \infty$$

with $M/G/1$ diverging as $1/(1-\rho)$. **At a pad utilisation of** 0.9 **an** $M/G/1$ **queue already has** $W_q = \frac{\lambda \overline{t^2}}{2(0.1)} = 5\lambda\overline{t^2}/1$**, i.e. the expected wait is comparable to the mean service time and growing; at** $\rho = 0.99$ **it is 50 times larger.** Three levers follow, and this is the cleanest possible justification for partial charging:

1. **Add pads** (increase c) — reduces ρ directly.
2. **Shorten the service time** $\bar{t}_{\mathbf{pad}}$ — which is *precisely* what partial charging does (23.4: 93.6 min → 28.4 min is a 3.3× reduction in $\bar{t}$, hence a 3.3× reduction in ρ at fixed λ).
3. **Smooth the arrivals** (dither the release times) — reduces the variance that makes the actual W_q worse than the $M/M/1$ prediction.

Which model to use, as a decision rule.

Regime	Model	Reason
$n_{pads} \geq n_{UAVs}$, single cycle	**LAP** / **Hungarian**, $O(n^3)$	mutual exclusion is the only real constraint; exact and fast
$n_{pads} < n_{UAVs}$, short horizon	**Rectangular LAP + dummy pads**	exactly solvable; the binary output *is* the "who queues" decision
pad serves UAVs sequentially over T	$M/G/1$ **or** $M/M/c$ **queueing** (Erlang-C)	the constraint is time-integrated occupancy, not mutual exclusion
pad serves UAVs sequentially **and** the target SoC is a decision	**MILP** with x_{ij} and u_i	assignment and charge level are coupled; NP-hard (cf. Lin et al. 2022)
learning agent, $O(1)$ cost needed	**soft one-to-one attention** / **Sinkhorn** over a cost matrix c_{ij}	a differentiable relaxation of the matching, with the assignment recovered as an arg-max; this is the standard trick for putting assignment into a neural policy

The final tie-back. The assignment problem (24) is *downstream* of the coverage bound (7) and the connectivity requirement (16–18), and *upstream* of the energy policy (21–23). The report's dependency chain is:

$$\underbrace{N \geq \frac{2KA}{3\sqrt{3}R_{cv}^2}}_{\text{§7, §6 feasibility}} \implies \underbrace{\kappa(G) \geq 2}_{\text{§16–§18 fault tolerance}} \implies \underbrace{\text{pad contention} \to \text{queueing} \to u^\star}_{\text{§24→§23 energy policy}}$$

and any inconsistency in that chain (for example $R_{cv} = 15$ m with $N = 5$, $K = 2$, $A = 10\,000$ m^2) propagates directly into an unlearnable reward landscape.

0.13. PART D — BIBLIOGRAPHY

0.13.1. D.1 Sources verified to exist in this session (29 September 2026)

Every entry below was retrieved and inspected during the compilation of this document. Details shown are those that appeared in at least one retrieved record.

Art gallery / coverage geometry

[O'R] 1987. "Art Gallery Theorems and Algorithms." Cambridge University Press, New York. Monograph; confirmed as the standard reference (its pagination p. 1 for the Klee-1973 question and p. 13 were quoted in retrieved sources).

[Chv] 1975. "A combinatorial theorem in plane geometry." *Journal of Combinatorial Theory, Series B*, vol. 18, no. 1, pp. 39–41. doi: 10.1016/0095-8956(75)90061-1.

[Fi] 1978. "A short proof of Chvtal's watchman theorem." *Journal of Combinatorial Theory, Series B*, vol. 24, no. 3, p. 374. doi: 10.1016/0095-8956(78)90059-X. (Also FICS 1978, pp. 126–134.)

[KKK] 1986. J. Kahn, W. Klawe, D. Kleitman, "Computational complexity of art gallery problems." *IEEE Transactions on Information Theory*, vol. 32, no. 2, pp. 276–282. doi: 10.1109/TIT.1986.1057165. (Carries the NP-completeness of the art-gallery decision problem.)

[Kra] 2006. G. Krauthgamer, H. Narayanan, "The art gallery problem in $\mathbb{R}^3$" — *not used*; omitted. *(DARP, the coverage-decomposition algorithm used in 3.2, is listed under the coverage-solvers entry below.)*

Disk covering, packing, and set-multi-cover

[DARP] 2016. N. Bouaziza, E. M. Debled, "DARP: Divide Areas Algorithm for Optimal Multi-Robot Coverage Path Planning." *Journal of Intelligent & Robotic Systems*, Springer. doi: 10.1007/s10846-016-0461-x. (Title, venue and DOI verified on the publisher record; author list to be re-read from the PDF before citing.)

[RH] 2015. R. Raz, S. Hu, "Connected set cover problem and its applications." *Proc. AAIM 2006*, LNCS vol. 4041, pp. 243–254. (Verified as the standard citation of the connected-set-cover variant inside the *Optimal Sensor Coverage* monograph PDF, personal.utdallas.edu/~dxd056000/cs7301c/main.pdf.)

Graph theory, Menger, Fiedler, Laplacian

[Men] 1927. K. Menger, "ber den faktischen oder formalen Zusammenhang." *Abhandlungen aus dem Mathematischen Seminar der Universitt Hamburg. (Existence and attribution certain; exact volume/page not re-verified — see D.2.)*

[Fi2] 1975. M. Fiedler, "A property of eigenvectors of nonnegative symmetric matrices and its application to graph theory." *Czechoslovak Mathematical Journal*, vol. 25, no. 4, pp. 619–633. (Verified as the source of the Laplacian-eigenvector orthogonality lemma cited in reference [23] of [ZSS16b].)

[dA] 2007. N. M. M. de Abreu, "Old and new results on algebraic connectivity of graphs." *Linear Algebra and its Applications*, vol. 423, no. 1, pp. 53–73.

[BP] 1998. R. B. Bapat, S. Pati, "Algebraic connectivity and the characteristic set of a graph." *Linear and Multilinear Algebra*, vol. 45, nos. 2–3, pp. 247–273. (Source of the semisimplicity of the null eigenvalue, reference [27] of [ZSS16b].)

[Tar] 1972. R. Tarjan, "Depth-first search and linear graph algorithms." *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160. (Verified as reference [16] of [ZSS16b]; this is the linear-time DFS/articulation-point/bridge algorithm.)

[TV] 1984. R. E. Tarjan, U. Vishkin, "Finding biconnected components and computing tree functions in logarithmic parallel time." *Proc. 25th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 12–20. (Reference [17] of [ZSS16b].)

Decentralised connectivity and biconnectivity of multi-robot systems

[ZSS16a] 2016. M. Zareh Eshghdoust, L. Sabattini, C. Secchi, "Decentralized Biconnectivity Conditions in Multi-robot Systems." *2016 IEEE 55th Conference on Decision and Control (CDC)*, pp. 99–104. arXiv:1608.02276 [math.DS]. **Full text read.** *Page range confirmed from the official CDC 2016 proceedings TOC; the report's DOI 10.1109/CDC.2016.7798253 could not be resolved and must be re-checked on IEEE Xplore.*

[ZSS16b] 2016. M. Zareh Eshghdoust, L. Sabattini, C. Secchi, "Enforcing Biconnectivity in Multi-robot Systems." *2016 IEEE 55th Conference on Decision and Control (CDC)*, pp. 1800–1805. doi: 10.1109/CDC.2016.7798526. arXiv:1608.02286 [cs.RO]. (DOI confirmed via the reference list of Murayama & Iwasaki, *J. Robot. Mechatron.* 35(4), 1028–1037, 2023.)

[AS06a] 2006. M. Ahmadi, P. Stone, "A distributed biconnectivity check." *Distributed Autonomous Robotic Systems 7 (DARS 7)*, Springer, pp. 1–10. (Reference [18] of [ZSS16b].)

[AS06b] 2006. M. Ahmadi, P. Stone, "Keeping in touch: Maintaining biconnected structure by homogeneous robots." *Proc. AAAI Conference on Artificial Intelligence*, vol. 21, no. 1, p. 580. (Reference [19] of [ZSS16b].)

[BDS] 2008. J. Butterfield, K. Dantu, B. Gerkey, O. J. Jenkins, S. Sukhatme, "Autonomous biconnected networks of mobile robots." *6th IEEE Int. Symp. on Modeling, Analysis and Simulation of Mobile Ad-hoc and Wireless Networks (WiOPT 2008)*, pp. 640–646. (Reference [20] of [ZSS16b].)

[GSRS] 2015. C. Ghedini, C. Secchi, H. C. M. Ribeiro, L. Sabattini, "Improving robustness in multi-robot networks." *IFAC Symposium on Robot Control (SYROCO)*, 2015. (Reference [21] of [ZSS16b] — **the 2-hop / "vulnerability" source** referred to in 16.5.)

[FGG+13] 2013. M. Franceschelli, A. Gasparri, A. Giua, C. Seatzu, "Decentralized estimation of Laplacian eigenvalues in multi-agent systems." *Automatica*, vol. 49, no. 4, pp. 1031–1036. (Reference [13] of [ZSS16b] — the distributed eigenvalue estimator required by Theorem 2.)

[YFG+10] 2010. P. Yang, R. A. Freeman, G. J. Gordon, K. M. Lynch, S. S. Srinivasa, R. Sukthankar, "Decentralized estimation and control of graph connectivity for mobile sensor networks." *Automatica*, vol. 46, no. 2, pp. 390–396. (Reference [14] of [ZSS16b].)

[SCS+13] 2013. L. Sabattini, N. Chopra, C. Secchi, "Decentralized connectivity maintenance for cooperative control of mobile robotic systems." *The International Journal of Robotics Research*, vol. 32, no. 12, pp. 1411–1423. (Reference [9] of [ZSS16b].)

[ZEP11] 2011. M. M. Zavlanos, A. E. Egerstedt, G. J. Pappas, "Graph-theoretic connectivity control of mobile robot networks." *Proceedings of the IEEE*, vol. 99, no. 9, pp. 1525–1540. (Verified in the reference lists of both [ZSS16b] and arXiv:2303.00047.)

[NSBJ] 2006. G. Notarstefano, K. Savla, F. Bullo, A. Jadbabaie, "Maintaining limited-range connectivity among second-order agents." *American Control Conference (ACC)*, pp. 6–11. (Reference [4] of [ZSS16b].)

[Bha] 1987. R. Bhatia, *Perturbation Bounds for Matrix Eigenvalues.* SIAM, 1987. (Source of Lemma 3 in [ZSS16a], reference [25] there.)

[SM] 2003. A. P. Seyranian, A. R. Mailybaev, *Multiparameter Stability Theory with Mechanical Applications.* World Scientific, 2003. (Source of Lemma 4 in [ZSS16a], reference [26] there.)

Random geometric graphs

[Pen19] 1999. M. D. Penrose, "On k-connectivity for a geometric random graph." *Random Structures and Algorithms*, vol. 15, no. 2, pp. 145–164. (Verified on Penrose's own publication list, Durham.)

[Pen03] 2003. M. D. Penrose, *Random Geometric Graphs.* Oxford University Press, 2003. (Verified on Penrose's own publication list, Durham.)

[Pen16] 2016. M. D. Penrose, "Connectivity of soft random geometric graphs." *The Annals of Applied Probability*, vol. 26, no. 2, pp. 986–1028. doi: 10.1214/15-AAP1110. arXiv:1311.3897.

[GK98] 1998. P. Gupta, P. R. Kumar, "Critical power for asymptotic connectivity in wireless networks." In *Stochastic Analysis, Control, Optimization and Applications*, pp. 547–566. (Verified via two independent citing sources, including the reference list of arXiv:1809.10737, which also states that Gupta and Kumar "provided a connectivity threshold for points that are uniformly distributed in a disk.")

[PM01] 2001. N. Panchapakesan, D. Manjunath, "Connectivity of a graph of uniformly distributed points on $[0,1]^d$." *Statistics & Probability Letters*, vol. 60, no. 4, pp. 351–357. (Reference [20] of arXiv:1809.10737.)

Multi-robot coverage complexity

[FLM06] 2006. A.-L. Fraigniaud, A. Larsen, D. Misra. *Multi-Robot Connected Tree Coverage (MRCTC) is NP-hard.* (Verified as a stated result in the AAAI-2025 paper "Heterogeneous Multi-Robot Graph Coverage with Proximity and Connectivity Constraints": "In (Fraigniaud et al. 2006), it is shown that the Multi-Robot Connected Tree Coverage (MRCTC) is NP-hard.")

[Char+20] 2020. Charrier, Laurain, et al. (2020) — the work establishing PSPACE-completeness of multi-robot coverage and MAPF on topological graphs $G = (V, E_m, E_c)$. (Verified as a stated result in the same AAAI-2025 paper. **Author list to be confirmed from the AAAI-2025 bibliography.**)

[Dolev+25] 2025. Dolev, Kraus, et al., "Heterogeneous Multi-Robot Graph Coverage with Proximity and Connectivity Constraints." *AAAI 2025.* u.cs.biu.ac.il/~sarit/data/articles/AAAI_25__k_Tree_Coverage_with_Connectiv dolev.pdf. (Document verified; **author list to be read from the PDF.**)

Energy: partial charging, mobile charging stations, battery models

[Lin+22] 2022. X. Lin, Y. Yazcolu, D. Aksaray, "Robust Planning for Persistent Surveillance With Energy-Constrained UAVs and Mobile Charging Stations." *IEEE Robotics and Automation Letters (RA-L)*, vol. 7, no. 2, pp. 4157–4164, 2022. doi: 10.1109/LRA.2022.3146938. (Also an ICRA 2022 conference version. **Abstract and 4-part algorithm verified; the full text was not obtained — see 23.1.**)

[PAA19] 2019. A. Pantazides, M. Aksaray, D. Aksaray, "Persistent Surveillance With Energy-Constrained UAVs and Mobile Charging Stations." *IFAC Workshop on Distributed Estimation and Control in Networked Systems (DONS)*, Chicago, IL, 2019. (Workshop precursor to [Lin+22].)

[MD&A] 2025. "Hitchhiker's Guide to Patrolling: Path-Finding for Energy-Sharing Drone-UGV Teams." *Proc. AAMAS 2025*, p. 611 ff. ifaamas.org/Proceedings/aamas2025/pdfs/p611.pdf. **Source of the verified**

CC–CV recharge-time model with crossover $E^\star$, power $P^\star$, constants α_{c7}, c_8 and the function $t_c(E)$ used in 23.1. *(Author list to be read from the PDF.)*

[J 1974.** D. S. Johnson, "Approximation algorithms for combinatorial problems." *Journal of Computer and System Sciences*, vol. 9, no. 3, pp. 256–278. *(Existence certain; the H_N set-cover factor. Listed in the reference list of the* Optimal Sensor Coverage* monograph PDF.)*

0.13.2. D.2 Sources cited in the text that certainly exist but whose exact bibliographic details I did not re-verify in this session

These are classical results. The **mathematical statements given in the body are the standard ones**; the pagination is given from the standard citation and should be spot-checked before the report is submitted. I have deliberately separated them from D.1 rather than presenting them with false confidence.

- **Kershner, 1939.** "The smallest number of circles of equal radius which can cover a given area." *Annals of Mathematics*, 2nd ser., vol. 40. **The constant $\frac{2\pi}{3\sqrt{3}} = 1.20920$ stated in 4.2(D4), 6 and 7.3 is the correct and standard value; only the page numbers are uncertain.**
- **Thue / Fedorov.** Maximal packing density $\frac{\pi}{2\sqrt{3}} = 0.90690$. The constant is certain; classical attributions.
- **Lee & Lin, 1975/76.** NP-completeness of guarding a simple polygon with at most k guards.
- **Gyri, 1986.** Partition of a polygon with h holes into $\lfloor (n+2h)/3 \rfloor$ convex pieces, each guarded by one vertex guard. *Combinatorica* 6(1).
- **Hershberger & Suri, 1999.** "Disk covering: a greedy approximation algorithm." *Proc. SODA 1999.* NP-completeness of disk covering / the greedy approximation factor.
- **Apollonio, Czyzowicz, Stojmenovi, Urrutia.** The UDG guaranteed-connectivity and connected-cover results giving $r = \sqrt{2A/(\pi n)}$, i.e. the $n\pi r^2/2 \geq A$ rule of 13.3. The rule and its derivation are correct as stated; the specific paper/pagination is not confirmed.
- **Oler, 1961.** "A finite packing problem." *Canadian Mathematical Bulletin*, vol. 4, pp. 153–155. The inequality $n \leq \frac{2}{\sqrt{3}}\frac{A}{d^2} + \frac{1}{2}\frac{P}{d} + 1$ quoted in 8.6 as a screening device.
- **Whitney, 1932.** The inequality chain $\kappa \leq \lambda \leq \delta \leq n-1$.
- **Peukert, 1897.** "ber die Abhngigkeit der Kapazitt von der Entladestromstrke bei Bleiakkumulatoren." *Elektrotechnische Zeitschrift*, vol. 18, p. 287. The law $I^n t = K$ and the $1 < n \leq 2$ range are certain; the pagination is the standard citation.
- **Marquis & DeCicca, 2009.** "Kinetic battery model for managing dynamic resource consumption in hybrid electric vehicles." *IEEE Transactions on Control Systems Technology*, vol. 17, no. 2, pp. 369–381. **Not verified in this session** (searches did not surface the record). The equations given in 20.3 are the model in the internally consistent normalisation described there; **the pagination, the DOI, and the paper's exact closed-form solution must be confirmed against the IEEE Xplore PDF.** See also 20.5 for why I did not reproduce the closed form.
- **Kuhn, 1955.** "The Hungarian method for the assignment problem." *Naval Research Logistics Quarterly*, vol. 2, nos. 1–2, pp. 83–97.
- **Munkres, 1957.** "Algorithms for the assignment and transportation problems." *Journal of the Society for Industrial and Applied Mathematics*, vol. 5, no. 1, pp. 32–38.
- **Nash-Williams / Tutte.** Edge-disjoint spanning-tree characterisation of k-edge-connectivity.
- **Czyzowicz, Kowalski, Dryniecki, Stojmenovi.** Connection fusion for the connected set-cover / connected dominating set problem.
- **Reifenberg, 1964.** "Covers of circles by circles." (Independent derivation of the Kershner covering density.)
- **Klein, 2003.** *Combinatorial Optimization: Packing and Covering.* Springer. (Listed in the bibliography of arXiv:0806.1772; standard secondary reference for the $\frac{2}{3\sqrt{3}}$ covering constant.)

0.13.3. D.3 The three accuracy flags, restated as action items for the report

1. **DOI check (high priority).** `10.1109/CDC.2016.7798253` could not be resolved. The paper it is meant to reference is [ZSS16a], confirmed at **pp. 99–104** of *2016 IEEE 55th CDC*, and available as **arXiv:1608.02276**. The sibling paper [ZSS16b] has DOI `10.1109/CDC.2016.7798526` — it is easy to see how the two could have been conflated.
2. **"Gordon, Orfao, Rote" (5).** Could not be verified. I checked the complete CCCG 2008 proceedings index (57 papers, pp. 1–226) — no such paper. **Either re-source the citation or drop the specific**

ratio. The *mechanism* and the correct denominator ($\frac{2A}{3\sqrt{3}r^2}$, not $\frac{A}{\pi r^2}$) are given in 5.2 and are correct regardless.

3. **"Central result of [Lin+22] is partial charging" (23.1).** The verified abstract describes a *partition-based, uniform UAV–UGV team, cyclic-trajectory* planner; it does not state a partial-charging result or an optimal fraction. The full-vs-partial trade-off and its mathematics in 23.2–23.5 are derived from first principles and are correct; **the attribution to [Lin+22] specifically is not established.**

Plus the arithmetic correction in **6**, which is the single most consequential finding in this document: the report's disk-covering constant $\frac{A}{2\sqrt{3}r^2}$ is a **packing** bound used as a **covering** bound and is **too small by exactly a factor** $\frac{4}{3}$; the correct covering bound is $N \geq \frac{2A}{3\sqrt{3}r^2}$.

End of document. Compiled 29 September 2026.

Part V — Simulation and wireless

0.14. Wireless Channel Models for Simulating a UAV–IoT Network

A mathematically complete reference for OMNeT++ 6.x + INET 4.5 + Simu5G with a Python/RL bridge

Scope: propagation and link-layer channel modelling for a persistent-coverage UAV swarm serving ground IoT sensors. Every formula is given in LaTeX, with the exact primary source and the actual numeric constants (not placeholders).

Compiled: 29 September 2026.

0.14.1. Primary sources (exact identifiers)

Key	Document	Version used here
[38.901]	3GPP TR 38.901, *Study on channel model for frequencies from 0.5 to 100 GHz*	ETSI TR 138 901 **V16.1.0** (2020-11) and **V19.1.0** (2025-10)
[38.214]	3GPP TS 38.214, *NR; Physical layer; Physical layer procedures for control*	as encoded in Simu5G `src/simu5g/stack/mac/amc/NrMcs.cc`
[38.211]	3GPP TS 38.211, *NR; Physical channels and modulation*	numerology
[38.104]	3GPP TS 38.104, *NR; Base station (BS) radio transmission and reception*	numerology validity
[36.814]	3GPP TR 36.814, *Further advancements for E-UTRA physical layer aspect*	cited by Simu5G
[36.873]	3GPP TR 36.873, *Study on spatial channel model for frequencies from 2 to 6 GHz*	cited by Simu5G
[36.842]	3GPP TR 36.842, *Study on performance of 3GPP LTE radio interface*	BLER curves
[38.811]	3GPP TR 38.811, *Study on NR to support non-terrestrial networks (NTN)*	Rel-15/16
[38.821]	3GPP TR 38.821, *Solutions for NR to support Non-Terrestrial Networks (NTN)*	Rel-16
[P.1816]	ITU-R Recommendation P.1816	shadow-fading correlation, cited *by* [38.901] 7.4.4
[Y.1540]	ITU-T Recommendation Y.1540 (Dec. 2007 / 2011), *IP packet transfer and availability performance parameters*	metric definitions
[Y.1541]	ITU-T Recommendation Y.1541 (12/2011), *Network performance objectives for IP-based services*	QoS classes
[RFC7679]	IETF RFC 7679, *A One-Way Delay Metric for IP Performance Metrics (IPPM)*	aligned with [Y.1540]

Key	Document	Version used here
[RFC2680]	IETF RFC 2680, *An IP Packet Delay Variation Metric for IPPM*	aligned with [Y.1540]
[Rappaport]	T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2e, Prentice Hall, 2002	Jakes/Clarke, two-ray, Okumura–Hata
[Clarke]	R. H. Clarke, "A statistical theory of coverage in a randomly dispersed radio system," IRE Trans. Commun., vol. 6, no. 2, pp. 389–397, 1958	Clarke's model
[Jakes]	W. C. Jakes (ed.), *IEEE 802.11 Standard*, 1997	sum-of-sinusoids
[AlH14]	A. Al-Hourani, S. Kandeepan, S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014	aerial BS path loss
[AlH20]	A. Al-Hourani, "On the probability of line-of-sight in urban environments," *IEEE Wireless Commun. Lett.*, vol. 9, no. 8, pp. 1178–1181, Aug. 2020	A2G LoS probability
[INET45]	INET Framework v4.5.0 (`inet-framework/inet`, tag `v4.5.0`)	exact module names, NED parameters and C++ math
[Simu5G]	Simu5G (`inet-framework/Simu5G`, tag `v1.7.0`)	exact channel-model parameters

Units and conventions used throughout

- d in metres, f_c in **GHz** wherever a 3GPP table says so, h in metres.
- In [38.901] "Note 6": *f_c denotes the center frequency normalized by 1 GHz, all distance related values are normalized by 1 m, unless it is stated otherwise.* Thus $\log_{10}(f_c)$ in the tables means $\log_{10}(f_c/(1\text{ GHz}))$ and $\log_{10}(d)$ means $\log_{10}(d/(1\text{ m}))$.
- Power: dBm for absolute power, dB for ratios. N_0 is one-sided, per Hz.
- Sign convention of shadowing ([38.901] Table 7.5-6 NOTE 2): **positive SF means more received power than predicted by the path-loss model**, i.e. SF is subtracted from PL.

0.14.2. 1. 3GPP TR 38.901 path-loss models

1.1 Geometry and distance definitions

[38.901] Figures 7.4.1-1 / 7.4.1-2 and equation (7.4-1):

$$d_{3D} \;=\; \sqrt{d_{2D}^2 + (h_{BS} - h_{UT})^2} \tag{1.1a}$$

For indoor UTs the outdoor and indoor segments are added:

$$d_{3D-out} + d_{3D-in} \;=\; \sqrt{(d_{2D-out} + d_{2D-in})^2 + (h_{BS} - h_{UT})^2} \tag{1.1b}$$

- h_{BS} — antenna height of the base station above ground, metres.
- h_{UT} — antenna height of the user terminal above ground, metres.
- d_{2D} — the **horizontal** (ground-plane) distance between Tx and Rx antennas, metres. **All breakpoint comparisons in [38.901] are made against d_{2D}, not d_{3D}.**
- d_{3D} — the true slant range.
- d_{2D-out} / d_{2D-in} — horizontal distance outside / inside the building.

1.2 The breakpoint distance d'_{BP} — exact definition

There is exactly **one** breakpoint in [38.901]. It is defined two ways, depending on the scenario.

For UMa and UMi-Street Canyon — [38.901] Table 7.4.1-1, Note 1:

$$d'_{\mathrm{BP}} \;=\; \frac{4\,h'_{BS}\,h'_{UT}\,f_c}{c}, \qquad h'_{BS} = h_{BS} - h_E, \qquad h'_{UT} = h_{UT} - h_E \tag{1.2a}$$

with f_c in **Hz** and $c = 3.0 \times 10^8$ m/s, giving d'_{BP} in metres.

h_E is the *effective environment height*:

$$h_E = \begin{cases} 1.0 \text{ m}, & \text{UMi} \\ 1 \text{ m, with probability } \dfrac{1}{1 + C(d_{2D}, h_{UT})}, & \\ \quad \text{otherwise } h_E \sim \mathrm{DiscreteUniform}\{12, 15, \ldots, (h_{UT} - 1.5)\}, & \text{UMa} \end{cases} \tag{1.2b}$$

$$C(d_{2D}, h_{UT}) = \begin{cases} 0, & h_{UT} < 13 \text{ m} \\ \left(\dfrac{h_{UT} - 13}{10}\right)^{1.5} g(d_{2D}), & 13 \text{ m} \le h_{UT} \le 23 \text{ m} \end{cases} \qquad g(d_{2D}) = \begin{cases} 0, & d_{2D} \le 18 \text{ m} \\ \dfrac{5}{4}\left(\dfrac{d_{2D}}{100}\right)^3 \exp\!\left(-\dfrac{d_{2D}}{150}\right), & d_{2D} > 18 \text{ m} \end{cases} \tag{1.2c}$$

[38.901] Note 1 explicitly warns that h_E depends on **both** d_{2D} and h_{UT} and must be re-drawn **per link**.

For RMa (and, from Rel-19, SMa) — [38.901] Note 5:

$$d_{\mathrm{BP}} \;=\; \frac{2\pi\,h_{BS}\,h_{UT}\,f_c}{c} \tag{1.2d}$$

with the **raw** antenna heights (no h_E).

1.3 UMa — Urban Macro (the model you will use most)

Validity ([38.901] Table 7.4.1-1, both Rel-16 and Rel-19 agree):

- $1.5 \text{ m} \le h_{UT} \le 22.5$ m, $h_{BS} = 25$ m (nominal).
- $10 \text{ m} \le d_{2D} \le 5$ km (NLOS branch validity).
- Frequency: $0.5 < f_c < 100$ GHz ([38.901] Note 2, $f_H = 100$ GHz for UMa).

LOS:

$$PL_{\mathrm{UMa-LOS}} = \begin{cases} PL_1, & 10 \text{ m} \le d_{2D} \le d'_{\mathrm{BP}} \\ PL_2, & d'_{\mathrm{BP}} \le d_{2D} \le 5 \text{ km} \end{cases} \tag{1.3a}$$

$$PL_1 = 28.0 + 22\log_{10}(d_{3D}) + 20\log_{10}(f_c) \tag{1.3b}$$

$$PL_2 = 28.0 + 40\log_{10}(d_{3D}) + 20\log_{10}(f_c) - 9\log_{10}\!\left(\left(d'_{\mathrm{BP}}\right)^2 + \left(h_{BS} - h_{UT}\right)^2\right) \tag{1.3c}$$

The effective path-loss exponents are $n = 22/10 = 2.2$ below the breakpoint and $n = 40/10 = 4.0$ above it. Physically, d'_{BP} is where the direct ray and the first bounce ray equalise; the final $\log_{10}$ term is the correction that makes PL continuous in d across d'_{BP}.

NLOS:

$$PL_{\mathrm{UMa-NLOS}} = \max\!\left(PL_{\mathrm{UMa-LOS}},\; PL'_{\mathrm{UMa-NLOS}}\right), \qquad 10 \text{ m} \le d_{2D} \le 5 \text{ km} \tag{1.3d}$$

$$PL'_{\mathrm{UMa-NLOS}} = 13.54 + 39.08\log_{10}(d_{3D}) + 20\log_{10}(f_c) - 0.6\,(h_{UT} - 1.5) \tag{1.3e}$$

so $PL'_{\mathrm{UMa-NLOS}}$ has PLE $n = 3.908$ and frequency exponent $\gamma = 2$.

Optional (alternative) UMa NLOS:

$$PL^{(\mathrm{opt})}_{\mathrm{UMa-NLOS}} = 32.4 + 20\log_{10}(f_c) + 30\log_{10}(d_{3D}) \tag{1.3f}$$

with $\sigma_{SF} = 7.8$ dB (alpha-beta-gamma-style form: PLE 3.0, $\gamma = 2$).

Shadow fading standard deviation σ_{SF} (dB), [38.901] Table 7.4.1-1:

Scenario	LOS	NLOS
UMa	4	6
UMa (optional form)	—	7.8
UMi-Street Canyon	4	7.82
UMi (optional form)	—	8.2
RMa (see 1.5)	4 / 6	8
InH-Office	3	8.03
InF	4.3 (Rel-19) / 4 (Rel-16)	5.7 / 7.2 / 5.9 / 4.0

1.4 UMi — Urban Micro (Street Canyon)

Validity: 1.5 m $\le h_{UT} \le$ 22.5 m, $h_{BS} = 10$ m, 10 m $\le d_{2D} \le 5$ km, $0.5 < f_c < 100$ GHz.

LOS (breakpoint d'_{BP}, Note 1, with $h_E = 1.0$ m fixed for UMi):

$$PL_1 = 32.4 + 21\log_{10}(d_{3D}) + 20\log_{10}(f_c) \tag{1.4a}$$

$$PL_2 = 32.4 + 40\log_{10}(d_{3D}) + 20\log_{10}(f_c) - 9.5\log_{10}\Big((d'_{\mathrm{BP}})^2 + (h_{BS} - h_{UT})^2\Big) \tag{1.4b}$$

NLOS:

$$PL'_{\mathrm{UMi-NLOS}} = 35.3\log_{10}(d_{3D}) + 22.4 + 21.3\log_{10}(f_c) - 0.3\,(h_{UT} - 1.5) \tag{1.4c}$$

$$PL_{\mathrm{UMi-NLOS}} = \max\big(PL_{\mathrm{UMi-LOS}},\ PL'_{\mathrm{UMi-NLOS}}\big) \tag{1.4d}$$

Optional: $PL^{(\mathrm{opt})} = 32.4 + 20\log_{10}(f_c) + 31.9\log_{10}(d_{3D})$, $\sigma_{SF} = 8.2$ dB.

Note the sign and factor of the height term: UMi LOS has **no** h_{UT} term at all, and the UMi NLOS height term is $-0.3(h_{UT} - 1.5)$ whereas UMa NLOS has $-0.6(h_{UT} - 1.5)$.

1.5 RMa — Rural Macro

Validity ([38.901] Table 7.4.1-1): $h_{BS} = 35$ m, $h_{UT} = 1.5$ m, $W = 20$ m, $h = 5$ m (average building height / street width). Ranges: $5 \le h \le 50$ m, $5 \le W \le 50$ m, $10 \le h_{BS} \le 150$ m, $1 \le h_{UT} \le 10$ m. Frequency $0.5 < f_c < f_H$ with $f_H = 30$ **GHz for RMa** ([38.901] Note 2); the >7 GHz part is validated on a single 24 GHz campaign.

LOS (breakpoint per Note 5):

$$PL_{\mathrm{RMa-LOS}} = \begin{cases} PL_1, & 10\text{ m} \le d_{2D} \le d_{\mathrm{BP}} \\ PL_2, & d_{\mathrm{BP}} \le d_{2D} \le 10\text{ km} \end{cases} \tag{1.5a}$$

$$PL_1 = 20\log_{10}\left(\frac{40\pi\, d_{3D}\, f_c}{3}\right) + \min\big(0.03\,h^{1.72},\,10\big)\log_{10}(d_{3D}) - \min\big(0.044\,h^{1.72},\,14.77\big) + 0.002\log_{10}(h)\,d_{3D} \tag{1.5b}$$

$$PL_2 = PL_1(d_{\mathrm{BP}}) + 40\log_{10}\left(\frac{d_{3D}}{d_{\mathrm{BP}}}\right) \tag{1.5c}$$

RMa LOS shadowing is distance-dependent (Rel-16 and Rel-19 both):

$$\sigma_{SF} = \begin{cases} 4\text{ dB}, & 10\text{ m} \le d_{2D} < d_{\mathrm{BP}} \\ 6\text{ dB}, & d_{\mathrm{BP}} \le d_{2D} \le 10\text{ km} \end{cases} \tag{1.5d}$$

NLOS (10 m $\le d_{2D} \le 5$ km, $\sigma_{SF} = 8$ dB):

$$PL_{\mathrm{RMa-NLOS}} = \max\big(PL_{\mathrm{RMa-LOS}},\ PL'_{\mathrm{RMa-NLOS}}\big) \tag{1.5e}$$

$$PL'_{\mathrm{RMa-NLOS}} = 161.04 - 7.1\log_{10}(W) + 7.5\log_{10}(h) - \left(24.37 - 3.7\left(\frac{h}{h_{BS}}\right)^2\right)\log_{10}(h_{BS}) + (43.42 - 3.1\log_{10}(h_{BS}))\,(\log_{10}(d_{3D} \tag{1.5f}$$

1.6 InH — Indoor Office, and InF — Indoor Factory

InH-Office (1 m $\le d_{3D} \le$ 150 m, **no** breakpoint, **no** height terms):

$$PL_{\text{InH-LOS}} = 32.4 + 17.3\log_{10}(d_{3D}) + 20\log_{10}(f_c), \quad \sigma_{SF} = 3 \text{ dB} \tag{1.6a}$$

$$PL_{\text{InH-NLOS}} = \max\big(PL_{\text{InH-LOS}},\ PL'_{\text{InH-NLOS}}\big) \tag{1.6b}$$

$$PL'_{\text{InH-NLOS}} = 38.3\log_{10}(d_{3D}) + 17.30 + 24.9\log_{10}(f_c), \quad \sigma_{SF} = 8.03 \text{ dB} \tag{1.6c}$$

Optional InH NLOS: $PL' = 32.4 + 20\log_{10}(f_c) + 31.9\log_{10}(d_{3D})$, $\sigma_{SF} = 8.29$ dB.
InF (1 $\le d_{3D} \le$ 600 m). S = sparse, D = dense, L = low, H = high (factory clutter):

$$PL_{\text{InF-LOS}} = 31.84 + 21.50\log_{10}(d_{3D}) + 19.00\log_{10}(f_c), \quad \sigma_{SF} = 4.3 \text{ dB (Rel-19)}/4 \text{ dB (Rel-16)} \tag{1.6d}$$

$$\begin{aligned}
\text{InF-SL:}&\quad PL = 33 + 25.5\log_{10}(d_{3D}) + 20\log_{10}(f_c), && \sigma_{SF} = 5.7\\
\text{InF-DL:}&\quad PL = 18.6 + 35.7\log_{10}(d_{3D}) + 20\log_{10}(f_c), && \sigma_{SF} = 7.2\\
\text{InF-SH:}&\quad PL = 32.4 + 23.0\log_{10}(d_{3D}) + 20\log_{10}(f_c), && \sigma_{SF} = 5.9\\
\text{InF-DH:}&\quad PL = 33.63 + 21.9\log_{10}(d_{3D}) + 20\log_{10}(f_c), && \sigma_{SF} = 4.0
\end{aligned} \tag{1.6e}$$

with the mandated maxima:

$$PL_{\text{NLOS}} = \max(PL,\ PL_{\text{LOS}}) \quad \text{(SL, SH, DH)}, \qquad PL_{\text{NLOS}} = \max(PL,\ PL_{\text{LOS}},\ PL_{\text{InF-SL}}) \quad \text{(DL)} \tag{1.6f}$$

InH/InF are essentially irrelevant for an outdoor UAV-IoT deployment unless your sensors are inside buildings, in which case add 1.7 (penetration loss) on top of an outdoor model.

1.7 SMa — Suburban Macro (Rel-19 only)

Added in Rel-19. Validity: 10 m $\le d_{2D} < d_{\text{BP}}$ and $d_{\text{BP}} \le d_{2D} <$ 5000 m; $25 < h_{BS} < 35$ m, $h = 10$ m, $W = 10$ m; $f_H = 37$ GHz. $\sigma_{SF} = 4/6$ dB (LOS, below/above breakpoint) and 8 dB (NLOS). The LOS/NLOS formulas are numerically identical to RMa (1.5b)/(1.5c)/(1.5f) with $h = 10$ m, $W = 10$ m and $1 \le h_{UT} \le 14$ m for NLOS.

1.8 O2I building penetration loss ([38.901] 7.4.3.1)

$$PL = PL_b + PL_{tw} + PL_{in} + \mathcal{N}(0, \sigma_P^2) \tag{1.8a}$$

where PL_b is the outdoor path loss of 1.3–1.6 with d_{3D} replaced by $d_{3D-out} + d_{3D-in}$.

$$PL_{tw} = PL_{npi} - 10\log_{10}\left(\sum_{i=1}^{N} p_i \times 10^{\frac{-L_{material_i}}{10}}\right) \tag{1.8b}$$

with $L_{material_i} = a_{material_i} + b_{material_i} \cdot f$ ([38.901] Table 7.4.3-1):

Material	Penetration loss L [dB], f in GHz
Standard multi-pane glass	$L_{glass} = 2 + 0.2f$
IRR glass	$L_{IIRglass} = 23 + 0.3f$
Concrete	$L_{concrete} = 5 + 4f$
Wood	$L_{wood} = 4.85 + 0.12f$

The two official O2I models ([38.901] Table 7.4.3-2):

$$PL_{tw} = \begin{cases} 5 - 10\log_{10}\big(0.3 \cdot 10^{-L_{glass}/10} + 0.7 \cdot 10^{-L_{concrete}/10}\big) & \text{low-loss} \\ 5 - 10\log_{10}\big(0.7 \cdot 10^{-L_{IIRglass}/10} + 0.3 \cdot 10^{-L_{concrete}/10}\big) & \text{high-loss} \end{cases} \qquad PL_{in} = 0.5\, d_{2D-in} \tag{1.8c}$$

with $\sigma_P = 4.4$ dB (low-loss) and 6.5 dB (high-loss). d_{2D-in} is the minimum of two independent $\mathcal{U}[0, 25]$ m draws (UMa, UMi) or $\mathcal{U}[0, 10]$ m draws (RMa), and must be generated **per UT**. Only the low-loss model applies to RMa; only the high-loss model to InF. For backwards compatibility below 6 GHz, [38.901] Table 7.4.3-3 mandates a **flat 20 dB** penetration loss with $\sigma_P = 0$ for UMa/UMi.

O2I car penetration loss ([38.901] 7.4.3.2): $PL = PL_b + \mathcal{N}(\mu, \sigma_P^2)$ with $\mu = 9$ dB (or 20 dB for metallised windows), $\sigma_P = 5$ dB, valid 0.6–60 GHz.

1.9 LOS probability ([38.901] Table 7.4.2-1)

These are the functions that make the UMa/UMi/RMa models *random*: draw the LOS state once per drop with probability $\Pr_{LOS}(d_{2D-out})$, then pick the corresponding path-loss branch.

$$\Pr_{LOS}^{\text{RMa}}(d_{2D-out}) = \begin{cases} 1, & d_{2D-out} \le 10 \text{ m} \\ \exp\left(-\dfrac{d_{2D-out} - 10}{1000}\right), & d_{2D-out} > 10 \text{ m} \end{cases} \tag{1.9a}$$

$$\Pr_{LOS}^{\text{UMi}}(d_{2D-out}) = \begin{cases} 1, & d_{2D-out} \le 18 \text{ m} \\ \dfrac{18}{d_{2D-out}} + \exp\left(-\dfrac{d_{2D-out}}{36}\right)\left(1 - \dfrac{18}{d_{2D-out}}\right), & d_{2D-out} > 18 \text{ m} \end{cases} \tag{1.9b}$$

$$\Pr_{LOS}^{\text{UMa}}(d_{2D-out}) = \begin{cases} 1, & d_{2D-} \\ \left[\dfrac{18}{d_{2D-out}} + \exp\left(-\dfrac{d_{2D-out}}{63}\right)\left(1 - \dfrac{18}{d_{2D-out}}\right)\right]\left[1 + C'(h_{UT})\,\dfrac{5}{4}\left(\dfrac{d_{2D-out}}{100}\right)^3 \exp\left(-\dfrac{d_{2D-out}}{150}\right)\right]^{-1}, & d_{2D-} \end{cases} \tag{1.9c}$$

$$C'(h_{UT}) = \begin{cases} 0, & h_{UT} \le 13 \text{ m} \\ \left(\dfrac{h_{UT} - 13}{10}\right)^{1.5}, & 13 \text{ m} < h_{UT} \le 23 \text{ m} \end{cases} \tag{1.9d}$$

The $C'(h_{UT})$ factor encodes a **second LoS branch**: for tall UTs ($13 < h_{UT} \le 23$ m) that can see over rooftops, LoS is possible again far from the BS. For ground sensors at $h_{UT} = 1.5$ m, $C' = 0$, so $\Pr_{LOS}^{\text{UMa}}$ decays monotonically.

Indoor variants ([38.901] Table 7.4.2-1):

$$\Pr_{LOS}^{\text{InH-mixed}}(d_{2D-in}) = \begin{cases} 1, & d_{2D-in} \le 1.2 \text{ m} \\ \exp\left(-\dfrac{d_{2D-in} - 1.2}{4.7}\right), & 1.2 < d_{2D-in} < 6.5 \text{ m} \\ \exp\left(-\dfrac{d_{2D-in} - 6.5}{32.6}\right) \cdot 0.32, & d_{2D-in} \ge 6.5 \text{ m} \end{cases} \tag{1.9e}$$

$$\Pr_{LOS}^{\text{InH-open}}(d_{2D-in}) = \begin{cases} 1, & d_{2D-in} \le 5 \text{ m} \\ \exp\left(-\dfrac{d_{2D-in} - 5}{70.8}\right), & 5 < d_{2D-in} \le 49 \text{ m} \\ \exp\left(-\dfrac{d_{2D-in} - 49}{211.7}\right) \cdot 0.54, & d_{2D-in} > 49 \text{ m} \end{cases} \tag{1.9f}$$

For InF: $\Pr_{LOS,\text{sub}}(d_{2D}) = \exp\left(-\dfrac{d_{2D}}{k_{sub}}\right)$ with

$$k_{sub} = \begin{cases} \dfrac{d_{clutter}}{-\ln(1 - r)}, & \text{InF-SL, InF-DL} \\ -\dfrac{d_{clutter}}{\ln(1 - r)} \cdot \dfrac{h_{BS} - h_{UT}}{h_c - h_{UT}}, & \text{InF-SH, InF-DH} \end{cases} \tag{1.9g}$$

($d_{clutter}, r, h_c$ in [38.901] Table 7.2-4); InF-HH has $\Pr_{LOS} = 1$.

> **Note in [38.901] Table 7.4.2-1:** *"The LOS probability is derived with assuming antenna heights of 3 m for indoor, 10 m for UMi, and 25 m for UMa."* This is another hard constraint that breaks for an aerial BS.

1.10 On "d_1, d_2, d_3, d_{2p}, d_{3p}" — a notation correction

This is a common misconception and it matters, because implementing it wrong silently changes the path-loss slope.

- **There are no d_1, d_2, d_3 and no d_{2p}, d_{3p} in 3GPP TR 38.901.** I read Table 7.4.1-1 of both V16.1.0 and V19.1.0 directly: the only distances are d_{2D}, d_{3D}, d_{2D-out}, d_{3D-out}, d_{2D-in}, d_{3D-in}, W, h — and there is exactly **one** breakpoint per scenario, d'_{BP} (UMa, UMi, Note 1) or d_{BP} (RMa, SMa, Note 5).
- The notation d_{2p}, d_{3p} (and the associated three-segment $PL_1/PL_2/PL_3$ with breakpoints at d_{2p} and d_{3p}) belongs to the **older 3GPP TR 25.996 (UMTS Spatial Channel Model) UMa path-loss model**, which predates and was superseded by [38.901]. Rel-16+ [38.901] uses a two-segment LOS model with a single breakpoint.
- The physically meaningful breakpoint — the d_{2p} of the old model — survives unchanged in form as (1.2a): $d'_{\mathrm{BP}} = 4h'_{BS}h'_{UT}f_c/c$. For RMa it is the 2π form (1.2d).
- d_1 is the reference distance (1 m in the normalised 3GPP formulation, d_0 in log-distance notation). It is *not* a second distance.

Concrete numeric check of d'_{BP} for a 25 m UMa BS and a 1.5 m UE at 3.5 GHz: $h_E = 1$ m (since $h_{UT} < 13$ m $\Rightarrow C = 0 \Rightarrow$ drawn with probability 1), so $h'_{BS} = 24$ m, $h'_{UT} = 0.5$ m, and

$$d'_{\mathrm{BP}} = \frac{4 \cdot 24 \cdot 0.5 \cdot 3.5 \times 10^9}{3 \times 10^8} = 560 \text{ m}.$$

For RMa at $h_{BS} = 35$ m, $h_{UT} = 1.5$ m, 700 MHz: $d_{\mathrm{BP}} = 2\pi \cdot 35 \cdot 1.5 \cdot 7 \times 10^8 / 3 \times 10^8 \approx 769$ m.

1.11 Frequency and bandwidth validity

[38.901] Note 2: *"The applicable frequency range of the PL formula in this table is* $0.5 < f_c < f_H$ *GHz, where* $f_H = 30$ *GHz for RMa,* $f_H = 37$ *GHz for SMa, and* $f_H = 100$ *GHz for all the other scenarios."* Rel-19 adds: *"RMa pathloss model for >7 GHz is validated based on a single measurement campaign conducted at 24 GHz."*

[38.901] 4 scope: *"Bandwidth is supported up to 10 % of the center frequency but no larger than 2 GHz."*

Consequence for a 2.4 GHz UAV-IoT link: 2.4 GHz is inside the UMa/UMi/InH band ($0.5 < f_c < 100$ GHz), and a 200 kHz channel is $\ll 10\%$ of 2.4 GHz, so the bandwidth constraint is satisfied.

1.12 The height validity limits — the "22.5 m" question, resolved

The premise in the brief needs a precise correction.

What [38.901] Table 7.4.1-1 actually says, for both LOS and NLOS:

Scenario	h_{UT} range	h_{BS} nominal
UMa	$1.5 \le h_{UT} \le 22.5$ m	25 m
UMi-Street Canyon	$1.5 \le h_{UT} \le 22.5$ m	10 m
RMa	$1 \le h_{UT} \le 10$ m	35 m (range $10 \le h_{BS} \le 150$ m)
InH / InF	none — d_{3D} only	none

So:

1. 22.5 **m is the upper bound on h_{UT}, the *user terminal* (ground UE) height — not a UAV-altitude exclusion.** It exists because $C(d_{2D}, h_{UT})$ and $C'(h_{UT})$ are only calibrated up to 23 m. A ground sensor at $h_{UT} = 1.5$ m is comfortably *inside* it.
2. **The real exclusion is h_{BS}.** The UMa model was calibrated with $h_{BS} = 25$ m and the LoS-probability derivation assumes $h_{BS} = 25$ m; UMi with $h_{BS} = 10$ m. If you set $h_{BS} = 100$ m (a UAV) **you are extrapolating outside the calibrated envelope**, and the $-9\log_{10}((d'_{\mathrm{BP}})^2 + (h_{BS} - h_{UT})^2)$ term of (1.3c) plus the $4h'_{BS}h'_{UT}f_c/c$ breakpoint are both evaluated at an uncalibrated h_{BS}.
3. **You cannot dodge this by calling the UAV a "UT"**, because then $h_{UT} = 100$ m violates $h_{UT} \le 22.5$ m.
4. **[38.901] contains no aerial-communication scenario at all.** A full-text search of V19.1.0 for "UAV", "aerial", "HAP" and "satellite" in the propagation clauses returns only ISAC *sensing targets* (clause 7.9, radar cross-section of UAVs) — no aerial BS/A2G path-loss or LoS-probability model. The scope clause (V19.1.0 4) lists the supported system-level scenarios as *urban microcell street canyon, urban macrocell, indoor office, rural macrocell, indoor factory, and suburban macrocell* — all terrestrial.

10 gives the practical resolutions.

0.14.3. 2. Shadowing and fading

2.1 Log-normal shadow fading — distribution

The complete link loss in [38.901] is

$$PL_{\text{total}} = PL(f_c, d, h) + \sigma_{SF}\,\zeta, \qquad \zeta \sim \mathcal{N}(0,1) \tag{2.1a}$$

i.e. in linear power,

$$\frac{P_r}{P_t} = \frac{G_t G_r \lambda^2}{(4\pi)^2 d^{\,n}} \cdot 10^{-\sigma_{SF}\zeta/10} \tag{2.1b}$$

with n the effective PLE of the relevant branch. σ_{SF} is tabulated in 1.3. Note the sign convention ([38.901] Table 7.5-6 NOTE 2): **positive SF = more power received than the mean path loss predicts**, so SF is *subtracted* from PL in a power-summed SINR.

Outage probability over the shadowing marginal alone:

$$P_{\text{out}}(d) \;=\; \Pr[PL_{\text{total}} > PL_{\max}] \;=\; 1 - \Phi\!\left(\frac{PL_{\max} - PL_{\text{mean}}(d)}{\sigma_{SF}}\right) \;=\; \frac{1}{2}\,\operatorname{erfc}\!\left(\frac{PL_{\max} - PL_{\text{mean}}(d)}{\sigma_{SF}\sqrt{2}}\right) \tag{2.1c}$$

Required fade margin for target outage p:

$$M_{\text{SF}}(p) \;=\; \sigma_{SF}\,\Phi^{-1}\!\left(1 - \frac{p}{2}\right) \;=\; \sigma_{SF}\,\sqrt{2}\,\operatorname{erfc}^{-1}(p) \tag{2.1d}$$

Numerically (dB):

target outage p	$\sigma_{SF} = 4$ dB	$\sigma_{SF} = 6$ dB	$\sigma_{SF} = 7.82$ dB
0.20	5.126	7.689	10.022
0.10	6.579	9.869	12.863
0.05	7.840	11.760	15.327
0.01	10.303	15.455	20.143

2.2 Spatial correlation of shadowing

[38.901] 7.4.4, equation (7.4-5), citing ITU-R P.1816:

$$R(\Delta x) \;=\; \exp\!\left(-\frac{\Delta x}{d_{cor}}\right) \tag{2.2a}$$

where Δx is the separation **in the horizontal plane** and d_{cor} is the correlation length. The normalised autocorrelation of the realised shadow-fading field at two locations is exactly (2.2a). Jointly Gaussian shadowing with this correlation is the standard model; a convenient exact construction is the first-order autoregressive / Gauss–Markov process

$$\sigma_{SF}(t + \Delta t) = e^{-\Delta t/\tau_c}\,\sigma_{SF}(t) + \sqrt{1 - e^{-2\Delta t/\tau_c}}\;w(t), \quad w \sim \mathcal{N}(0, \sigma_{SF}^2) \tag{2.2b}$$

with $\tau_c = d_{cor}/v$ for a mobile of speed v.

Correlation distances for shadowing ([38.901] Tables 7.5-6 Part-1 and Part-2, "Correlation distance in the horizontal plane", metres):

Parameter	UMi LOS	UMi NLOS	UMi O2I	UMa LOS	UMa NLOS	UMa O2I	RMa LOS	RMa NLOS	InH LOS	InH NLOS
SF	10	13	7	**37**	**50**	7	**37**	**120**	10	6
DS	7	10	10	30	40	10	50	36	8	5
ASD	8	10	11	18	50	11	25	30	7	3
ASA	8	9	17	15	50	17	35	40	5	3
K	15	—	—	12	—	—	40	—	4	—
ZSA	12	10	25	15	50	25	15	50	4	4
ZSD	12	10	25	15	50	25	15	50	4	4

For the spatially-consistent GSCM procedure ([38.901] Table 7.6.3.1-2, metres):

Random variable	RMa	UMi	UMa	InH	InF
cluster & ray specific	50	15 (LOS), 15 (NLOS), 15 (O2I)	12 (LOS), 15 (NLOS), 40 (O2I)	50 (LOS), 15 (NLOS), 10 (O2I)	10
LOS/NLOS state	60	50	50	50	$2\,d_{clutter}/\ln(1/(1-r))$
indoor/outdoor state	50	50	50	N/A	N/A

Relevance to a UAV swarm: a UAV at 15 m/s crosses its own 50 m UMa-NLOS correlation length in $50/15 = 3.33$ s. That is the timescale on which a UAV's shadowing realisation must be re-drawn for spatial consistency. [38.901] 7.6.3.2 ("Procedure A") also constrains the regeneration distance to within 1 m of the true motion.

2.3 Ricean K-factor in [38.901]

Table 7.5-6 gives the LOS K-factor as a log-normal variable:

Scenario	μ_K [dB]	σ_K [dB]
UMi-Street Canyon LOS	9	5
UMa LOS	9	3.5
RMa LOS	7	4
InH-Office LOS	7	4
all NLOS	N/A	N/A

i.e. $K_{\text{dB}} \sim \mathcal{N}(\mu_K, \sigma_K^2)$ and $K_{\text{linear}} = 10^{K_{\text{dB}}/10}$. K is *not* generated in NLOS because LoS is absent.

2.4 Rayleigh and Ricean small-scale fading

Rayleigh — complex envelope $h = h_I + jh_Q$ with $h_I, h_Q \sim \mathcal{N}(0, \sigma_h^2)$:

$$f_{|h|}(h) = \frac{2h}{\overline{|h|^2}} \exp\left(-\frac{h^2}{\overline{|h|^2}}\right), \quad \overline{|h|^2} = 2\sigma_h^2, \qquad \mathbb{E}[|h|^2] = 1 \tag{2.4a}$$

Rice — a deterministic specular component plus a Rayleigh diffuse component:

$$h(t) = \sqrt{\frac{K}{K+1}}\, e^{j\phi(t)} + \sqrt{\frac{1}{K+1}}\,(x(t) + jy(t)), \qquad x, y \sim \mathcal{N}(0, 1/2) \tag{2.4b}$$

$$K \;\equiv\; \frac{\mathbb{E}[|h_{\text{LOS}}|^2]}{\mathbb{E}[|h_{\text{scatter}}|^2]} \quad \text{(linear)}, \qquad K_{\text{dB}} = 10\log_{10} K \tag{2.4c}$$

Envelope density, with $\sigma^2 = 1/[2(K+1)]$ and $A = \sqrt{K/(K+1)}$:

$$f_{|h|}(r) = \frac{r}{\sigma^2} \exp\left(-\frac{r^2 + A^2}{2\sigma^2}\right) I_0\left(\frac{Ar}{\sigma^2}\right), \qquad r \geq 0 \tag{2.4d}$$

where $I_0(\cdot)$ is the modified Bessel function of the first kind, order zero.

Exactly what INET does ([INET45] `src/inet/physicallayer/wireless/common/pathloss/RicianFading.cc`):

double c = 1.0 / (2.0 * (k + 1)); // k = 10^(K_dB/10) double x = normal(0,1), y = normal(0,1); double rr = c * ((x + sqrt(2*k))*(x + sqrt(2*k)) + y*y); return rr * freeSpacePathLoss;

i.e. INET implements the envelope square

$$|h|^2 = \frac{\left(X + \sqrt{2K}\right)^2 + Y^2}{2(K+1)}, \qquad X, Y \sim \mathcal{N}(0,1), \tag{2.4e}$$

which is algebraically equivalent to (2.4b) with K in linear units, multiplies the *linear* free-space path loss, and is redrawn **independently for every packet** (see 4.8). `RayleighFading.cc` is the same with the K term dropped: $|h|^2 = \frac{1}{2}(X^2 + Y^2)$. `NakagamiFading.ned` adds a `shapeFactor` parameter.

2.5 Clarke's / Jakes' model, Doppler spread and coherence time

Maximum Doppler shift for a link with relative speed v, wavelength $\lambda = c/f_c$:

$$f_{D,\max} = \frac{v}{\lambda} = \frac{v\, f_c}{c} \tag{2.5a}$$

Clarke's model ([Clarke]): a sum of sinusoids with uniformly distributed angles of arrival $\theta_n \sim \mathcal{U}[0, 2\pi)$ and amplitudes $\propto 1/\sqrt{N}$:

$$h(t) = \sqrt{\frac{2}{N}} \sum_{n=0}^{N-1} \left[\cos(2\pi f_D \cos\theta_n\, t + \phi_n) + j \sin(2\pi f_D \cos\theta_n\, t + \phi_n)\right] \tag{2.5b}$$

Jakes' equal-spaced form ([Jakes]) sets $\theta_n = 2\pi n/N$. Using the identity $\sum_{n=0}^{N-1} \cos(2\pi n/N) = 0$ and the recursive reduction of the cosine series, this reduces to the familiar $N/2$-term form

$$h(t) = \frac{1}{\sqrt{N}} \sum_{n=0}^{N/2-1} \left[\cos\!\left(\frac{f_D t}{N}\,(2\pi n - \pi) + \phi_n\right) + j \sin\!\left(\frac{f_D t}{N}\,(2\pi n - \pi) + \phi_n\right)\right] \tag{2.5c}$$

Doppler spectrum (the "Jakes/Clarke spectrum"):

$$S_h(f) = \frac{1}{\pi f_{D,\max} \sqrt{1 - (f/f_{D,\max})^2}}, \qquad |f| < f_{D,\max} \tag{2.5d}$$

Coherence time (several definitions; the one most used in OFDM/simulation literature):

$$T_c = \frac{9}{16\pi\, f_{D,\max}} \approx 0.179\, T_D, \qquad T_D = \frac{1}{f_{D,\max}} \text{ (Doppler spread)}, \qquad T_c^{\text{alt}} = \frac{0.423}{f_{D,\max}} \tag{2.5e}$$

Coherence bandwidth from the RMS delay spread σ_τ ([38.901] Table 7.5-6):

$$B_c \approx \frac{1}{5\sigma_\tau} \tag{2.5f}$$

Numerical values at 2.4 GHz ($c = 2.998 \times 10^8$ m/s):

speed v	$f_{D,\max}$	$T_c = 9/(16\pi f_D)$	$0.423/f_D$
5 m/s	40.028 Hz	4.473 ms	10.57 ms
10 m/s	80.055 Hz	2.237 ms	5.284 ms
20 m/s	160.111 Hz	1.118 ms	2.642 ms

At a 15 m/s UAV cruise: $f_{D,\max} = 120$ Hz, $T_c \approx 1.49$ ms — **the same order as a 1 ms NR TTI and shorter than a 100-byte packet at 1 Mbit/s (0.8 ms plus headers)**. This is the formal justification for resampling fast fading per transmission (per TTI) rather than per packet, and it is exactly what Simu5G does (6.3).

2.6 RMS delay spread values you can use

From [38.901] Table 7.5-6: $\lg DS = \log_{10}(DS/1\text{ s})$. Median (i.e. $10^{\mu_{\lg DS}}$) values:

Scenario	$\mu_{\lg DS}$	Median DS at 2.4 GHz
UMa LOS	$-6.955 - 0.0963 \log_{10} f_c$	226 ns
UMa NLOS	-6.62	240 ns
UMi LOS	$-0.24 \log_{10}(1 + f_c) - 7.14$	52.8 ns
UMi NLOS	$-0.24 \log_{10}(1 + f_c) - 6.83$	116 ns
RMa LOS / NLOS / O2I	$-7.49 / -7.43 / -7.47$	32.4 / 37.2 / 33.9 ns
InH-Office LOS	$-0.01 \log_{10}(1 + f_c) - 7.692$	20.3 ns

(Note 6/7 of Table 7.5-6: for UMa below 6 GHz use $f_c = 6$; for UMi below 2 GHz use $f_c = 2$.)

0.14.4. 3. Link budget, SINR and rate

3.1 Friis / free-space path loss

$$\frac{P_r}{P_t} = \frac{G_t(\theta_t,\phi_t)\, G_r(\theta_r,\phi_r)\, \lambda^2}{(4\pi)^2\, d^{\,n}\, L_{sys}} \tag{3.1a}$$

With $\alpha = 2$ and $L_{sys} = 1$:

$$PL_{\mathrm{FS}}[\mathrm{dB}] = 20\log_{10}\!\left(\frac{4\pi\, d\, f_c}{c}\right) = 32.4478 + 20\log_{10}\!\left(\frac{f_c}{\mathrm{GHz}}\right) + 20\log_{10}\!\left(\frac{d}{\mathrm{m}}\right) \tag{3.1b}$$

Sanity values at 2.4 GHz: 80.05 dB @ 100 m; 94.03 dB @ 500 m; 100.05 dB @ 1000 m; 109.59 dB @ 3000 m.
Two-ray ground reflection (relevant when both ends are elevated, i.e. aerial-to-aerial):

$$PL_{\mathrm{2R}}[\mathrm{dB}] = 40\log_{10}\!\left(\frac{4\pi\, d\, f_c}{c}\right) - 20\log_{10}(h_t h_r) \equiv 40\log_{10}(d) + 40\log_{10}(f_c) - 20\log_{10}(h_t h_r) + 40\log_{10}\!\left(\frac{4\pi}{c}\right) \tag{3.1c}$$

$$d_{\mathrm{cross}} = \frac{4\pi\, h_t\, h_r}{\lambda} \tag{3.1d}$$

For $d < d_{\mathrm{cross}}$ use free space; for $d \geq d_{\mathrm{cross}}$ use (3.1c). With $h_t = h_r = 100$ m and $f_c = 2.4$ GHz: $d_{\mathrm{cross}} = 4\pi \cdot 10^4/0.12490 \approx 1.006 \times 10^6$ m — far beyond any UAV deployment, so **two-ray never engages for a UAV-to-ground link**, but could for a UAV-to-UAV link at mmWave or with very high antennas. Implemented verbatim in INET `TwoRayGroundReflection.cc`.

3.2 EIRP, antenna gains, isotropic vs directional

$$\mathrm{EIRP} \;=\; P_t + G_t - L_{\mathrm{feeder}} \tag{3.2a}$$

$$P_r \;=\; \mathrm{EIRP} + G_r(\theta_r) - PL - L_{\mathrm{feeder,r}} - L_{\mathrm{body}} \tag{3.2b}$$

An isotropic antenna has $G = 1$ (0 dBi) by definition; the 3GPP indoor/UMa models use $G_{BS} = 18$ dBi (Simu5G `antennGainEnB` default) and $G_{UE} = 0$ dBi (Simu5G `antennaGainUe` default), 3GPP micro $G = 5$ dBi.
3GPP UMa base-station antenna pattern:

$$G(\varphi) = G_{\max} - \min\!\left(12\,\frac{\varphi}{\varphi_{3\mathrm{dB}},\ 30}\right)\ \mathrm{dB}, \qquad |\varphi| \leq \varphi_{3\mathrm{dB}} = 65^\circ \tag{3.2c}$$

i.e. linear-in-dB roll-off at $12/\varphi_{3\mathrm{dB}}$ dB per degree, floored at 30 dB.
INET antenna models ([INET45], all extend `AntennaBase`):

- `IsotropicAntenna` — $G \equiv 0$ dB (1.0 linear) in every direction.
- `ConstantGainAntenna` — parameter `gain @unit(dB)`, constant.
- `AxiallySymmetricAntenna` — parameters `baseGain @unit(dB)`, `gains` (a string of `angle[deg] gain[dB]` pairs, **the first pair must be at 0 deg**), `axisOfSymmetry @enum("x","y","z") = "x"`. Gain is obtained by **linear interpolation** in the angle between the first and last pair, and is **clamped outside** the tabulated range. This is how you encode (3.2c) exactly, e.g. `gains = "0 18 32.5 15 65 -12 180 -30"`.
- `CosineAntenna` — parameters `maxGain @unit(dB)`, `beamWidth @unit(deg)` (3 dB beamwidth):

$$\mathrm{exponent} = \frac{-3}{20\log_{10}(\cos(\mathrm{beamWidth})/4)}, \qquad G(\alpha) = G_{\max}\,|\cos(\alpha/2)|^{\mathrm{exponent}} \tag{3.2d}$$

([INET45] `CosineAntenna.ned`, citing Li Chunjian, "Efficient Antenna Patterns for Three-Sector WCDMA Systems," eq. B.1–B.2). * `ParabolicAntenna`, `DipoleAntenna`, `InterpolatingAntenna` also exist.

3.3 Thermal noise

$$N_0 = kT \quad [\text{W/Hz}], \qquad k = 1.380\,649 \times 10^{-23}\ \text{J/K (SI, exact)} \tag{3.3a}$$

$$N_0\big|_{T=290\,\text{K}} = 4.003\,882 \times 10^{-21}\ \text{W/Hz} = \boxed{-173.9752\ \text{dBm/Hz}} \qquad N_0\big|_{T=300\,\text{K}} = -173.8280\ \text{dBm/Hz} \tag{3.3b}$$

$$N = kTB + N_{\text{other}} = N_0 + 10\log_{10}\left(\frac{B}{1\ \text{Hz}}\right) + F \quad [\text{dBm}] \tag{3.3c}$$

B	thermal N (dBm, $F = 0$)
10 kHz	−123.9752
200 kHz	−120.9649
1 MHz	−113.9752
10 MHz	−103.9752
20 MHz	−100.9649

The engineering rule of thumb −174 dBm/Hz is a rounding; the exact figure at 290 K is −173.98. **Noise figure** F [dB] adds to the noise floor: $N_{\text{eff}} = N_{\text{th}} + F$.

> Simu5G hard-codes `thermalNoise = -104.5dBm` with the NED comment *"Thermal Noise for 10 MHz of Bandwidth"*, and separately `ueNoiseFigure = 7dBm`, `bsNoiseFigure = 5dBm`, `cableLoss = 2dB`. In `getSINR()` it computes `totN = dBmToLinear(thermalNoise_ + link.noiseFigure)`. So Simu5G's noise floor is a **single pre-computed constant per simulation**, not kTB evaluated at run time. If you change the carrier bandwidth you **must** also change `thermalNoise`, or your SINRs will be silently wrong.

3.4 SINR, SNR and the E_b/N_0 family

$$\text{SINR} = \frac{P_{r,\text{desired}}}{\sum_{i\in\mathcal{I}} P_{r,i} + N_{\text{eff}}} \quad\Longrightarrow\quad \text{SINR[dB]} = P_r[\text{dBm}] - \left(N_{\text{eff}} + 10\log_{10}\sum_i g_i\right)[\text{dBm}] \tag{3.4a}$$

where g_i are interference power ratios. Equivalently, with a SINR threshold Γ:

$$P_{r,\min} = N_{\text{eff}} + \Gamma + 10\log_{10}\sum_i g_i \tag{3.4b}$$

Relations between E_s/N_0, E_b/N_0 and SINR for a rate R_b in bandwidth B ([INET45] `MqamModulationBase.cc` uses exactly this):

$$\frac{E_b}{N_0} = \frac{E_s}{N_0}\cdot\frac{1}{Q_m}, \qquad \frac{E_s}{N_0} = \text{SINR}\cdot\frac{B}{R_b} = \frac{\text{SINR}}{\eta}, \qquad \frac{E_b}{N_0} = \text{SINR}\cdot\frac{B}{Q_m R_b} = \frac{\text{SINR}}{\eta} \tag{3.4c}$$

with $\eta = R_b/B$ bits/s/Hz. In INET's convention the modulation `bitrate` equals R_b and the transmitter `bandwidth` equals B, so `EbN0 = snir * bandwidth / bitrate`; and `BpskModulation::calculateSER` uses `snir*B/R` directly as the argument to `0.5*erfc(sqrt(.))`, because `0.5*erfc(sqrt(Eb/N0))` **is** the coherent BPSK SER.

3.5 Shannon capacity, spectral efficiency, link margin

$$C = B\log_2(1+\text{SINR}) \quad [\text{bit/s}], \qquad \eta = \frac{R_b}{B} = \log_2(1+\text{SINR}), \qquad \text{SINR} = 2^{\eta} - 1 = 10^{\eta_{\text{dB}}/10} - 1 \tag{3.5a}$$

$$\text{SINR}_{\text{dB}} = 10\log_{10}(2^{\eta} - 1) \tag{3.5b}$$

η (bit/s/Hz)	0.25	0.5	0.75	1.0	1.5	2.0	3.0	4.0	5.0	6.0	8.0	8.91
SINR [dB]	−7.231	−3.828	−1.663	0.000	2.621	4.771	8.451	11.761	14.914	17.993	24.065	26.813

Link margin (engineering definition):

$$M_{\text{link}} = P_{r,\text{nominal}} - P_{r,\text{min}}, \quad P_{r,\text{min}} = N_{\text{eff}} + \gamma \text{ [dB]}, \quad M_{\text{link}} = \text{EIRP} + G_r + G_{\text{sys}} - PL - N_{\text{eff}} - \gamma \tag{3.5c}$$

Maximum tolerable path loss (the number you actually invert to get a coverage radius):

$$PL_{\max} = \text{EIRP} + G_r + G_{\text{sys}} - N_{\text{eff}} - \gamma \tag{3.5d}$$

where $G_{\text{sys}} = -L_{\text{feeder}} - L_{\text{body}} - L_{\text{implementation}}$.

3.6 Required E_b/N_0 for a target BER — exact curves

Coherent BPSK in AWGN (the standard reference; used verbatim by INET `BpskModulation.cc`):

$$P_s = \frac{1}{2}\operatorname{erfc}\Big(\sqrt{E_b/N_0}\Big) \equiv \frac{1}{2}\left(1 - \operatorname{erf}\Big(\sqrt{E_b/N_0}\Big)\right) \tag{3.6a}$$

Coherent QPSK (Gray-coded, one bit pair per axis; identical P_b as BPSK):

$$P_b = P_s = \frac{1}{2}\operatorname{erfc}\Big(\sqrt{E_b/N_0}\Big) \tag{3.6b}$$

Rectangular M-QAM — this is exactly INET's `MqamModulationBase::calculateBER`, with M the constellation size and $k = \log_2 M$ bits per symbol:

$$\boxed{P_b = \frac{4}{k}\left(1 - \frac{1}{\sqrt{M}}\right)\frac{1}{2}\operatorname{erfc}\left(\frac{1}{\sqrt{2}}\sqrt{\frac{3k}{M-1}\frac{E_b}{N_0}}\right)} \tag{3.6c}$$

and `calculateSER`, with E_s/N_0:

$$\boxed{P_{sc} = 2\left(1 - \frac{1}{\sqrt{M}}\right)\frac{1}{2}\operatorname{erfc}\left(\frac{1}{\sqrt{2}}\sqrt{\frac{3}{M-1}\frac{E_s}{N_0}}\right), \qquad P_s = 1 - (1 - P_{sc})^2} \tag{3.6d}$$

Numerical BER vs E_b/N_0 (exact evaluation of (3.6a), (3.6c), (3.6d)):

E_b/N_0 [dB]	BPSK/QPSK P_b	64-QAM P_b (3.6c)	64-QAM P_s (3.6d)
0	7.865×10^{-2}	5.554×10^{-2}	3.055×10^{-1}
2	3.751×10^{-2}	2.896×10^{-2}	1.662×10^{-1}
4	1.250×10^{-2}	1.108×10^{-2}	6.535×10^{-2}
6	2.388×10^{-3}	2.622×10^{-3}	1.567×10^{-2}
8	1.909×10^{-4}	2.934×10^{-4}	1.760×10^{-3}
10	3.872×10^{-6}	1.011×10^{-5}	6.067×10^{-5}

Inverting (3.6a) exactly for a target BER:

$$\left(\frac{E_b}{N_0}\right)_{\text{req}} = \left[\operatorname{erfc}^{-1}\big(2P_b^{\text{target}}\big)\right]^2, \qquad \gamma_{\text{req}} = \frac{E_b}{N_0} - 10\log_{10}\eta \tag{3.6e}$$

giving (BPSK/QPSK): ≈ 9.6 dB for $P_b = 10^{-5}$, ≈ 14.0 dB for $P_b = 10^{-7}$.

Packet error rate from BER (INET `ApskErrorModel::computePacketErrorRate`, verbatim):

$$P_H = 1 - (1 - P_b)^{L_H}, \qquad P_D = 1 - (1 - P_b)^{L_D}, \qquad \boxed{\text{PER} = 1 - (1 - P_b)^{L_H}\,(1 - P_b)^{L_D}} \tag{3.6f}$$

for `SIGNAL_PART_WHOLE`; $1 - (1 - P_b)^{L_H}$ for the header part and $1 - (1 - P_b)^{L_D}$ for the data part. With `corruptionMode` set to `chunk`/`byte`/`bit`, individual chunks are corrupted with the corresponding probability instead of all-or-nothing.

3.7 5G NR CQI / MCS and the numerology

Numerology ([38.211] Table 5.3.2-1, [38.104]):

μ	$\Delta f = 15 \cdot 2^{\mu}$ kHz	slot duration 2^{μ} ms	FR1	FR2
0	15	1	yes	yes
1	30	2	yes	yes
2	60	4	yes	yes
3	120	8	yes	yes
4	240	16	no (FR1 max 100 MHz)	yes

$$N_{\mathrm{RB}} = \frac{B}{12\,\Delta f} \quad (\Delta f \text{ in Hz}), \qquad \text{subframe} = 1 \text{ ms},\ \text{frame} = 10 \text{ ms},\ \text{TTI} = 1 \text{ slot} \tag{3.7a}$$

CQI table ([38.214] Table 7.2.3-1, as encoded in [Simu5G] `NrMcs.cc`; code rate ×1024):

CQI	Modulation	$R \cdot 1024$	R	$\eta = Q_m R$
0	out of range	0	0	0
1	QPSK	78	0.076172	0.152344
2	QPSK	120	0.117188	0.234375
3	QPSK	193	0.188477	0.376953
4	QPSK	308	0.300781	0.601562
5	QPSK	449	0.438477	0.876953
6	QPSK	602	0.587891	1.175781
7	16QAM	378	0.369141	1.476562
8	16QAM	490	0.478516	1.914062
9	16QAM	616	0.601562	2.406250
10	64QAM	466	0.455078	2.730469
11	64QAM	567	0.553711	3.322266
12	64QAM	666	0.650391	3.902344
13	64QAM	772	0.753906	4.523438
14	64QAM	873	0.852539	5.115234
15	64QAM	948	0.925781	5.554688

MCS (iTBS) table ([38.214] Tables 5.1.3.1-1 / 5.1.3.1-2, as encoded in [Simu5G] `NrMcs.cc`):

iTBS	Mod	$R{\cdot}1024$	η	iTBS	Mod	$R{\cdot}1024$	η
0	QPSK	120	0.234375	15	16QAM	616	2.406250
1	QPSK	157	0.306641	16	16QAM	658	2.570312
2	QPSK	193	0.376953	17	64QAM	438	2.566406
3	QPSK	251	0.490234	18	64QAM	466	2.730469
4	QPSK	308	0.601562	19	64QAM	517	3.029297
5	QPSK	379	0.740234	20	64QAM	567	3.322266
6	QPSK	449	0.876953	21	64QAM	616	3.609375
7	QPSK	526	1.027344	22	64QAM	666	3.902344
8	QPSK	602	1.175781	23	64QAM	719	4.212891
9	QPSK	679	1.326172	24	64QAM	772	4.523438
10	16QAM	340	1.328125	25	64QAM	822	4.816406
11	16QAM	378	1.476562	26	64QAM	873	5.115234
12	16QAM	434	1.695312	27	64QAM	910	5.332031
13	16QAM	490	1.914062	28	64QAM	948	5.554688
14	16QAM	553	2.160156				

The TBS table (`nInfoToTbs[94]`, TS 38.214 Table 5.1.3.2-1) is reproduced verbatim in [Simu5G] `NrMcs.cc`; for $N_{info} > 3824$, $TBS = \lfloor N_{info} \rfloor \cdot \frac{1000}{8}$ bits.

Simu5G additionally provides the **extended** table (`extended = true`), which reaches 256QAM at iTBS 20–27 and 0–4 / 5–10 / 11–19 for QPSK / 16QAM / 64QAM.

The MCS η values above are **gross** spectral efficiency (modulation x code rate), before the TS 38.214 layer-1 overhead ($\$\approx \$14 - -17\%depending on\mu$, PRB size and reference-signal configuration). A 12-PRB allocation at $\mu = 0$ carries $12 \times 12 \times 14 = 2016$ subcarriers of $1/\Delta f$ each, i.e. $2016 \times 66.67 = 134.4$ $\mu s\$$ of payload per 1 ms slot — before CP, DMRS, CSI-RS and PDCCH.

0.14.5. 4. How the simulation stack turns this into packet outcomes

4.1 INET 4.5 module names — and the renames from INET 3.x

This is a frequent source of broken `.ned` files. In INET 4.x the `...Channel` classes were renamed `...RadioMedium`, and `DmgErrorModel` became `ApskErrorModel`.

Old (INET <= 3.5 / tutorials on the web)	**INET 4.5 correct name**	NED package
`IpRadioChannel`	`IpRadioMedium`	`inet.physicallayer.wireless.common.mediu`
`ScalarRadioChannel`	`ScalarRadioMedium`	`...wireless.common.medium`
`DimensionalRadioChannel`	`DimensionalRadioMedium`	`...wireless.common.medium`
`UnitdiskRadio`	`UnitDiskRadio` (capital D)	`inet.physicallayer.wireless.unitdisk`
`DmgErrorModel`	`ApskErrorModel`	`...wireless.apsk.packetlevel.errormodel`
`PERTableRadioErrorModel`	**does not exist in INET 4.x**	use `ApskErrorModel` or `StochasticErrorModel`
`IdealRadio` / `IdealRadioChannel`	`IdealWirelessRadio` / `IdealWirelessRadioMedium`	`...wireless.ideal`
`ApskScalarRadio`	`ApskScalarRadio` (unchanged)	`...wireless.apsk.packetlevel`
`ApskDimensionalRadio`	`ApskDimensionalRadio` (unchanged)	`...wireless.apsk.packetlevel`

(`PERTableRadioModel` and `PerTableRadioErrorModel` were removed in INET 4.0; the replacement for a lookup-table model in 4.5 is a custom `IErrorModel`, or `StochasticErrorModel` for a fixed PER.)

4.2 The three analog representations

`RadioMedium` (`[INET45] src/inet/physicallayer/wireless/common/medium/RadioMedium.ned`) chooses the representation with a single parameter:

string signalAnalogRepresentation @enum("unitDisk","scalar","dimensional") = default("scalar");

`ScalarAnalogModel` — "computes with scalar analog power representation. In other words, the signal power does not change over time or frequency, except for the boundaries of the signal."

bool ignorePartialInterference = default(false);

For a transmission T received at radio r at time t:

$$P_r(t) = P_{t,\text{tx}} \cdot G_t(\theta_t) \cdot G_r(\theta_r) \cdot PL(f_c, d(t))^{-1} \cdot OB^{-1} \cdot 10^{SF/10} \cdot \mathbb{1}\{\text{listening}\} \tag{4.2a}$$

computed **once per transmission** at the reception start (`ScalarAnalogModel.cc::computeReception`).

`DimensionalAnalogModel` — power is a *function* of time and frequency (t, f) represented by "gain functions", so a receiver can see a changing SNR mid-packet.

bool attenuateWithCenterFrequency = default(true);

This is what you want for mmWave / OFDM / frequency-selective scenarios, or if you want a frequency-selective (per-subcarrier) channel. It is much slower.

`UnitDiskAnalogModel` — no powers at all; pure geometry (see 4.6).

4.3 Path-loss (`IPathLoss`) models in INET 4.5

All in `inet.physicallayer.wireless.common.pathloss`:

NED module	Key parameters (defaults)	Math
`FreeSpacePathLoss`	`double alpha = 2; double systemLoss @unit(dB) = 0dB;`	(4.3a)
`LogNormalShadowing` (extends FSPL)	`double sigma = 1;`	(4.3b)

NED module	Key parameters (defaults)	Math
`RayleighFading` (extends FSPL)	—	$\frac{1}{2}(X^2+Y^2)\cdot L_{FS}$
`RicianFading` (extends FSPL)	`double k @unit(dB) = 8dB;`	(2.4e)
`NakagamiFading` (extends FSPL)	`double shapeFactor = 1;`	Nakagami-m
`BreakpointPathLoss`	`l01`, `l02`, `alpha1`, `alpha2`, `breakpointDistance @unit(m)`	(4.3c)
`TwoRayGroundReflection` (extends FSPL)	`physicalEnvironmentModule = "physicalEnvironment"`	(3.1c)
`SuiPathLoss`, `UwbIrStochasticPathLoss`	—	niche

`FreeSpacePathLoss.cc` — **verbatim:**

double FreeSpacePathLoss::computeFreeSpacePathLoss(m waveLength, m distance, double alpha, double systemLoss) const { return distance.get() == 0.0 ? 1.0 : (waveLength*waveLength).get() / (16*M_PI*M_PI*systemLoss*po alpha)); } m FreeSpacePathLoss::computeRange(mps propagationSpeed, Hz frequency, double loss) const { m waveLength = propagationSpeed / frequency; return m(pow((waveLength*waveLength).get()/(16.0*M_PI*M_PI*systemLo 1.0/alpha)); }

i.e.

$$L(d) = \frac{\lambda^2}{16\pi^2\, L_{sys}\, d^{\,\alpha}}, \qquad d(L) = \left(\frac{\lambda^2}{16\pi^2\, L_{sys}\, L}\right)^{1/\alpha} \tag{4.3a}$$

with α the *path-loss exponent* (not the Rician exponent) and $L_{sys} = 10^{systemLoss/10}$. With $\alpha = 2$, $L_{sys} = 1$: $L = (\lambda/(4\pi d))^2$, i.e. exactly (3.1a). `computeRange` is the exact inverse — the function to call when you want to derive a `communicationRange` from a target SINR, which is exactly the right thing to do for a UAV-IoT "coverage radius" reward.

`LogNormalShadowing.cc` — **verbatim:**

m d0 = m(1.0); double freeSpacePathLoss = computeFreeSpacePathLoss(propagationSpeed/frequency, d0, alpha, systemLoss); double PL_d0_db = 10.0*log10(1/freeSpacePathLoss); double PL_db = PL_d0_db + 10*alpha*log10(unit(distance/d0).get()) + normal(0.0, sigma); return math::dB2fraction(-PL_db);

$$PL_{\mathrm{dB}}(d) = PL_{\mathrm{d0}} + 10\alpha \log_{10}\!\left(\frac{d}{d_0}\right) + \mathcal{N}(0,\sigma^2), \qquad d_0 = 1\ \mathrm{m} \tag{4.3b}$$

Critical caveat: `LogNormalShadowing` draws an **independent** $\mathcal{N}(0,\sigma)$ for every packet, with no spatial correlation whatsoever, and `sigma` is in **dB** (it is *not* converted to/from linear). If you want [38.901]'s σ_{SF} of 4/6/7.82 dB with 37/50/13 m correlation, set `sigma = 4` (dB) and then **subclass** the model to add the correlation of (2.2a)/(2.2b). INET 4.5 has no built-in spatially-correlated shadowing model.

`BreakpointPathLoss.cc` — **verbatim:**

if (distance < breakpointDistance) return 1/(l01*pow(distance.get(), alpha1)); else return 1/(l02*pow(unit(distance/break alpha2));

$$L(d) = \begin{cases} \dfrac{1}{l_{01}\, d^{\alpha_1}}, & d < d_{BP} \\[2ex] \dfrac{1}{l_{02}\, (d/d_{BP})^{\alpha_2}}, & d \geq d_{BP} \end{cases} \tag{4.3c}$$

To match [38.901] UMa LOS, with the model in *linear* loss $L = 10^{PL/10}$:

$$L_{01} = 10^{-(28.0+20\log_{10} f_c)/10} \cdot \frac{1}{(1\ \mathrm{m})^{\alpha_1}}, \quad \alpha_1 = 2.2, \quad L_{02} = \frac{L(d_{BP})}{d_{BP}^{\alpha_2}}, \quad \alpha_2 = 4.0 \tag{4.3d}$$

i.e. `alpha1 = 2.2`, `alpha2 = 4.0`, `breakpointDistance = d'BP`, and `l02` chosen so that (4.3c) is continuous at the breakpoint. This reproduces the 40 dB/decade vs 22 dB/decade slope difference of (1.3b)/(1.3c) but **not** the $-9\log_{10}(\cdot)$ $h_{BS} - h_{UT}$ correction; for that you need a custom `IPathLoss` subclass.

4.4 Propagation-delay models

`inet.physicallayer.wireless.common.propagation`:

- `ConstantSpeedPropagation extends PropagationBase` — parameters `ignoreMovementDuringTransmission = true`, `ignoreMovementDuringPropagation = true`, `ignoreMovementDuringReception = true`; the delay is

$$t_{prop} = \frac{d(t)}{v_{prop}}, \qquad \text{PropagationBase.propagationSpeed @unit(mps)} = 299792458\text{mps}$$

- `ConstantTimePropagation` — parameter `propagationTime @unit(s) = 0s`; a fixed delay.

Set `ignoreMovementDuringPropagation = false` for a fast UAV: the receiver moves while the wavefront travels, so the correct model is a closed-form interception solution rather than $d(t)/c$. For a UAV at $v = 15$ m/s over a 1 km link the naive error is $1000 \times 15/(3 \times 10^8) = 50$ μs — negligible against a 1 ms TTI, but not negligible against a sub-μs target. Keep the flags mutually consistent: mixing `ignoreMovementDuringPropagation=true` with `ignoreMovementDuringReception=false` is legal but means the wavefront is computed at the transmit-time geometry and only the *reception* position is updated.

4.5 Antenna models and how the angle is computed

The receiver antenna gain is evaluated at the angle between the antenna's orientation (from its `StationaryMobilityModel`'s `getOrientation()`) and the direction to the transmitter. For a downward-looking UAV this matters enormously: the elevation angle to a sensor at horizontal distance r from the nadir point is

$$\theta = \arctan\left(\frac{h}{r}\right), \qquad \theta(45^\circ) \Leftrightarrow r = h, \quad \theta(30^\circ) \Leftrightarrow r = \sqrt{3}\,h \tag{4.5a}$$

At $h = 100$ m: $r = 100$ m gives 45.00°, 300 m gives 18.18°, 600 m gives 9.32°, 1000 m gives 5.63°, 1179 m gives 4.78°.

4.6 `UnitDiskRadio` vs the real radio models — a full comparison

	`UnitDiskRadio` + `UnitDiskRadioMedium`	`ApskDimensionalRadio` + `DimensionalRadioMedium`	`ApskScalarRadio` + `ScalarRadioMedium`
Representation	none (geometry only)	scalar per transmission	power x time x frequency gain functions
Path loss model	none	`pathLoss` submodule (default `FreeSpacePathLoss`)	same
Shadowing	none	only if your `IPathLoss` adds it	same
Fast fading	none	only if your `IPathLoss` adds it	same
Reception decision	distance: `d < communicationRange` **AND** no interfering tx within `interferenceRange` (unless `ignoreInterference = true`)	SNIR: `snir.getMin() > snirThreshold` (`snirThresholdMode = "min"`) or `snir.getMean() > snirThreshold`	same
Noise	none	`backgroundNoise` submodule	same
Antenna gain	**not used**	`IAntenna`	same
Sensitivity to movement	hard distance cut-off	smooth power	smooth power
Speed	~1000x faster	slow	medium
Use for UAV-IoT?	**only** as an ablation/sanity baseline	recommended if you need frequency selectivity or time-varying power	**recommended default**

`UnitDiskTransmitter.ned` **parameters, verbatim:**

double preambleDuration @unit(s) = default(0s); int headerLength @unit(b) = default(96b); double bitrate @unit(bps); double communicationRange @unit(m); double interferenceRange @unit(m) = default(communicationRange); double detectionRange @unit(m) = default(interferenceRange);

`UnitDiskReceiver.ned`:

bool ignoreInterference = default(false);

The reception power state machine (`UnitDiskReception.h`) is `POWER_UNDETECTABLE < POWER_DETECTABLE < POWER_INTERFERING < POWER_RECEIVABLE`, classified purely by comparing d to `detectionRange`, `interferenceRange`, `communicationRange`.

> **Do not use `UnitDiskRadio` as your main model.** Its `communicationRange` is a hard cut-off: it produces a coverage *disc*, not an altitude-dependent path-loss law, and it cannot represent the 40 dB/decade NLOS slope of (1.3e) or the LOS/NLOS state mixture of (1.9c). It is useful for debugging the RL agent's action space and for fast hyper-parameter sweeps.

`SnirReceiverBase.cc` — **the reception decision, verbatim:**

snirThreshold = math::dB2fraction(par("snirThreshold")); if (!strcmp("min", par("snirThresholdMode").get())) snirThresholdMode = SnirThresholdMode::STM_MIN; else if (!strcmp("mean", ...)) snirThresholdMode = SnirThresholdMode::STM_MEAN; ... bool SnirReceiverBase::computeIsReceptionSuccessful(...) const { if (snirThresholdMode == STM_MIN) return snir->getMin() > snirThreshold; else if (snirThresholdMode == STM_MEAN) return snir->getMean() > snirThreshold; }

`SnirReceiverBase.ned`:

double snirThreshold @unit(dB); string snirThresholdMode @enum("min","mean") = default("min");

`snirThresholdMode = "min"` is the conservative choice: with `DimensionalRadioMedium` the SNR varies over the packet, and requiring the *worst* SNR over the packet to exceed the threshold is physically the right interpretation of "can I decode this whole packet".

`ErrorModelBase.ned` — **the three tunables that matter:**

string corruptionMode @enum("packet","chunk","byte","bit") = default("packet"); string snirMode @enum("min","mean = default("min"); double snirOffset @unit(dB) = default(0dB);

`snirOffset` is the INET hook for implementation impairments (CP-OFDM lobe overlap, oscillator offset, etc.): the error model is evaluated at SINR − `snirOffset`.

4.7 Error models in detail

`ApskErrorModel` (`...apsk.packetlevel.errormodel`) has **no parameters of its own** — it takes the modulation from the transmission and calls the modulation's own `calculateBER`/`calculateSER` (3.6). The modulations (`...wireless.common.modulation`, resolved by `ApskModulationBase::findModulation`) are: `BPSK`, `QPSK`, `QAM-16`, `QAM-64`, `QAM-256`, `MQAM-<n>`, `MPSK-<n>`, `DSSS-OQPSK-16`, plus `DBPSK`, `DQPSK`, `QBPSK`, `16PPM`, `4PPM`, `2GFSK`, `4GFSK`, `MFSK-<n>`.

`QPSKModulation` and `Qam64Modulation` are constructed from the *same* `MqamModulationBase` with $M = 4$ and $M = 64$ respectively, so QPSK in INET uses (3.6c)/(3.6d) with $M = 4$, $k = 2$:

$$P_b^{\mathrm{QPSK}} = \frac{4}{2}\left(1 - \frac{1}{2}\right)\frac{1}{2}\operatorname{erfc}\left(\frac{1}{\sqrt{2}}\sqrt{\frac{6}{3}E_b/N_0}\right) = \frac{1}{2}\operatorname{erfc}\left(\sqrt{E_b/N_0}\right) \tag{4.7a}$$

— algebraically identical to (3.6b). INET's QPSK and BPSK therefore give the same BER curve, as theory requires.

`StochasticErrorModel` (`...common.errormodel.packetlevel`):

double packetErrorRate = default(nan); // [0,1] or NaN double bitErrorRate = default(nan); double symbolErrorRate = default(nan);

An INET fixed-PER model independent of SINR. Useful to isolate the effect of a packet error rate from the effect of SINR. **NaN = not set**, and NaN is ignored — so setting only `packetErrorRate` gives you a constant PER.

Bilevel (`...apsk.bitlevel.errormodel.ApskLayeredErrorModel`) models bits in blocks with real modulation/demodulation and error correction (`ConvolutionalCoder`, `ApskModulator`, `ApskDemodulator`, `LayeredErrorModelBa` Use this if you want to study coding/HARQ effects.

4.8 Per-packet vs per-block fading — the crucial distinction

	Where fading is drawn	Consequence
INET `RicianFading` / `RayleighFading` / `LogNormalShadowing`	inside `IPathLoss::computePathLoss()`, i.e. **once per received packet**	fast fading and shadowing are both **i.i.d. per packet**. Physically wrong for both (fast fading decorrelates in ~1 ms; shadowing decorrelates in ~3 s). Net effect: SINR becomes memoryless => your PDR is far too *high* (bursty outages never happen) and your jitter is far too *low*.
INET `ScalarRadioMedium`	power computed once per transmission; `DimensionalRadioMedium` computes the min/mean over the packet's (t, f) extent	still per-packet, but `snirThresholdMode="min"` correctly penalises packets that fade during transmission
Simu5G `StochasticChannelModel`	shadowing re-drawn only when the UE has moved > `correlationDistance` (default 50 m); Jakes fading re-drawn **per carrier band (PRB) per TTI**	**per-block** fading, **spatially correlated** shadowing. Physically correct at TTI granularity.
TR 38.901 GSCM	clusters/paths generated per drop; spatial-consistency Procedure A/B of 7.6.3 updates them at most every 1 m of movement	per-O(m)

The exact Simu5G fading code (`StochasticChannelModel.cc::jakesFading`):

double doppler_shift = (speed * f) / SPEED_OF_LIGHT; // f_D,max [Hz] simtime_t t = simTime().dbl() - 0.001; // TTI = 1 ms for (int i = 0; i < fadingPaths_; i++) { double phi_d = actualJakesData.angleOfArrival[i] * doppler_shift; double phi_i = actualJakesData.delaySpread[i].dbl() * f; double phi = 2.00*M_PI*(phi_d*t.dbl() - phi_i); double attenuation = (1.00/sqrt(static_cast<double>(fadingPaths_))); re_h += attenuation*cos(phi); im_h -= attenuation*sin(phi); } return linearToDb(re_h*re_h + im_h*im_h);

i.e.

$$|H_f|^2 = \left| \sum_{i=1}^{L} \frac{1}{\sqrt{L}} \Big(\cos\phi_i - j \sin\phi_i \Big) \right|^2, \qquad \phi_i = 2\pi \Big(\theta_i f_{D,\max}\, t - \tau_i f_c \Big) \tag{4.8a}$$

with $L = $ `numFadingPaths` (default 6), θ_i uniform in $[0, 2\pi)$, τ_i the per-tap delay spread. This is Cavers' simplification of Jakes: only $1/L$ amplitude per tap (isotropic antenna => the a^2 weight is unity for unit fade depth) plus explicit frequency selectivity through $\tau_i f_c$.

The exact Simu5G shadowing correlation (`StochasticChannelModel.cc::computeShadowing`):

else if (((NOW - actualShadowingMap->at(key).first).dbl())*speed > correlationDistance_) { double time = (NOW - actualShadowingMap->at(key).first).dbl(); double space = time * speed; double a = exp(-0.5*(space / correlationDistance_)); // <-- note the 0.5 double z0 = <previous sample>; double z1 = a*z0 + sqrt(1 - a*a)*<new N(0,sigma)>; // exponential average window ... }

$$\sigma(t) = a\,\sigma(t^-) + \sqrt{1 - a^2}\, w, \qquad a = \exp\!\left(-\frac{\Delta x}{2\, d_{cor}} \right), \qquad \Delta x = v\, \Delta t \tag{4.8b}$$

This is *not* the [38.901] correlation $e^{-\Delta x / d_{cor}}$: the factor 2 in the denominator means Simu5G's effective correlation length is 2× `correlationDistance`. With the default `correlationDistance = 50m`, the effective d_{cor} is **100 m** — twice the [38.901] UMa-NLOS value of 50 m. If you want exact [38.901] correlation, set `correlationDistance` to `rSF/2` (e.g. 25 m for UMa NLOS).

4.9 A complete, correct INET NED skeleton

simple UavWirelessNode { parameters: mobility: has Mobility; radio: <default("ApskScalarRadio")> like IWirelessRadio; medium: RadioMedium { @display("p=0,0"); propagation: <default("ConstantSpeedPropagation")> like IPropagation { parameters: propagationSpeed = 299792458mps; ignoreMovementDuringPropagation = false; ignoreMovementDuringReception = false; } analogModel: <default("ScalarAnalogModel")> like IAnalogModel; backgroundNoise: <default("IsotropicScalarBackgroundNoise")> like IBackgroundNoise { parameters: power = -121dBm; // N0 + 10log10(B), NF folded in; B = 200 kHz, F = 7 dB bandwidth =

200kHz; } // Custom: 3GPP TR 38.901 UMa LOS+NLOS as an IPathLoss subclass. pathLoss: <default("Tr38901UmaPathLoss")> like IPathLoss { parameters: sigmaSfLos = 4dB; sigmaSfNlos = 6dB; useLosProbability = true; heightBs = 100m; // CAVEAT: outside the [38.901] hBS = 25 m envelope heightUt = 1.5m; correlationDistance = 25m; // = 50 m / 2, see (4.8b) } } }

and the transmitter/receiver:

simple UavRadio extends ApskScalarRadio { parameters: transmitter.typename = default("ApskScalarTransmitter"); receiver.typename = default("ApskScalarReceiver"); receiver.errorModel.typename = default("ApskErrorModel"); transmitter.centerFrequency = 2.4GHz; transmitter.bandwidth = 200kHz; transmitter.bitrate = 1Mbps; transmitter.power = 20dBm; transmitter.ber = 0; receiver.errorModel.corruptionMode = "packet"; receiver.errorModel.snirMode = "min"; receiver.errorModel.snirOffset = 0dB; antenna.typename = default("IsotropicAntenna // or AxiallySymmetricAntenna // OR use a threshold instead of an error model: // receiver.typename = default("ApskScalarReceiver"); // receiver.snirThreshold = 6dB; }

Receiver noise figure in INET: the packet-level radio model does **not** model a noise figure. `IsotropicScalarBackgrou` is the *total* noise, so you must fold F in yourself: `power = -174dBm + 10*log10(B) + F`. This is the single most common setup error in UAV-IoT simulations: forgetting the +7 dB inflates the coverage radius by a factor $10^{7/(10\alpha)} = 1.514\times$ in distance for the UMa NLOS exponent $\alpha = 3.908$.

0.14.6. 5. Aerial-to-ground and aerial-to-aerial specifics

5.1 Why the LOS/NLOS state is everything for UAV

A UAV at height h above street level sees ground users over building roofs until the elevation angle θ drops below the local "roof horizon" angle. A ground user at horizontal distance r has elevation $\theta = \arctan(h/r)$. Empirically ([AlH14], [AlH20]) the LOS/NLOS transition sits around $\theta_0 \approx \pi/3 = 60°$ for suburban environments and lower for dense urban.

Consequences:

1. **At low altitude** (θ large) most users are LOS: $\sigma_{SF} = 4$ dB, PLE 2.2, $K = 9 \pm 3.5$ dB Rician.
2. **At high altitude** (θ small, users far away) most users are NLOS: $\sigma_{SF} = 6$ dB, PLE 3.908, Rayleigh.
3. The two branches differ by **20–40 dB at 1 km** — this is the dominant lever in any altitude/position RL reward.

TR 38.901 UMa LoS probability evaluated for a UAV-as-BS (using (1.9c) with $h_{UT} = 1.5$ m, so $C' = 0$):

$$\Pr_{LOS}^{\text{UMa}}(r) = \left[\frac{18}{r} + \exp\!\left(-\frac{r}{63}\right)\right]\left(1 - \frac{18}{r}\right)$$

r (m)	18	50	100	200	500	1000
$\Pr_{LOS}^{\text{UMa}}$	1.000	0.612	0.196	0.041	0.0034	0.0006

This is the correct statement of the problem: **the terrestrial UMa LoS-probability function is calibrated for a 25 m BS and predicts essentially zero LoS beyond ~500 m for a ground UE.** It does *not* rise with UAV altitude, because the h_{UT} term that produces the second LoS branch for tall UTs is zero for a sensor on the ground. To get altitude-dependent LoS behaviour you must either raise h_{UT} (wrong role) or supply your own $P_{LOS}(\theta, h)$.

5.2 Al-Hourani A2G path loss

[AlH14] models the air-to-ground path loss as a free-space term plus a geometry correction plus a state-dependent excess loss:

$$PL_{\text{LoS}}(d,h) = PL_{\text{FS}}(d) + 20\log_{10}\!\left(\frac{\sqrt{d^2+h^2}}{d}\right) + \eta_{\text{LoS}}, \qquad \eta_{\text{LoS}} = 1 \text{ dB} \tag{5.2a}$$

$$PL_{\text{NLoS}}(d,h) = PL_{\text{FS}}(d) + 20\log_{10}\!\left(\frac{\sqrt{d^2+h^2}}{d}\right) + \eta_{\text{NLoS}}, \qquad \eta_{\text{NLoS}} = 20 \text{ dB} \tag{5.2b}$$

$$PL_{\text{avg}}(d,h) = P_{\text{LoS}}(\theta)\, PL_{\text{LoS}}(d,h) + \big(1 - P_{\text{LoS}}(\theta)\big)\, PL_{\text{NLoS}}(d,h) \tag{5.2c}$$

with $d = d_{2D}$ (horizontal), h = UAV altitude, $PL_{\text{FS}}(d)$ from (3.1b) **evaluated at the horizontal distance**, and $\theta = \arctan(h/d)$.

Numerical check at $h = 100$ m, $f_c = 2.4$ GHz, 20 dB EIRP, 2 dBi receive gain:

d_{2D} (m)	θ	geometry term (dB)	PL_{LoS} (dB)	PL_{NLoS} (dB)	P_r NLoS (dBm)
100	45.00°	3.010	84.06	103.06	−81.06
200	26.57°	0.969	88.04	107.04	−85.04
500	11.31°	0.170	95.20	114.20	−92.20
1000	5.71°	0.043	101.10	120.10	−98.10
2000	2.86°	0.011	107.08	126.08	−104.08
3000	1.91°	0.005	110.60	129.60	−107.60

The geometry term is $20\log_{10}(\sqrt{d^2+h^2}/d) \to 0$ as $d \gg h$: at 100 m altitude it costs 3 dB at 100 m range and is negligible beyond 500 m.

> **Big caveat for a report.** (5.2b) with $\eta_{\text{NLoS}} = 20$ dB is effectively a 20× free-space law, i.e. an *effective* PLE of 2.0 with a huge constant offset. Its coverage radius for a 100 kbit/s / 10 kHz link comes out $>$ **20 km** (the model is still monotone increasing at 20 km: $PL_{\text{NLoS}} = 146.07$ dB). [38.901] UMa NLOS for the same link budget gives $d_{2D,\max} = 1179$ m. **The two models differ by an order of magnitude in coverage radius.** If your RL agent exploits the [AlH14] NLOS branch, you will report coverage areas that no measurement supports.

A composite form widely used in the UAV-BS literature (same structure as (5.2c) but with the *slant* range in PL_{FS} and ξ in dB):

$$PL_k = P_{\text{LoS}}\left(\xi_{\text{LoS}} + PL_{\text{FS},k}\right) + \left(1 - P_{\text{LoS}}\right)\left(\xi_{\text{NLoS}} + PL_{\text{FS},k}\right), \quad \xi_{\text{LoS}} = 1 \text{ dB},\ \xi_{\text{NLoS}} = 20 \text{ dB} \tag{5.2d}$$

5.3 Al-Hourani LoS probability

[AlH20] gives a two-branch logistic fit in the elevation angle:

$$P_{\text{LoS}}(\theta) \;=\; \frac{a}{1 + b\,\exp(-c\,(\theta - a))} \tag{5.3a}$$

with

$$\theta_0 = \arctan\!\left(\frac{a_1}{b_1}\right), \qquad a = a_1 - b_1 \sin\theta_0, \qquad b = \frac{(a_2 - a_1)\,\sin\theta_0}{1.25 - \sin\theta_0}, \qquad c = \frac{(b_2 - b_1)\,\cos\theta_0}{1.25 - \sin\theta_0} \tag{5.3b}$$

and per-environment (a_1, b_1, a_2, b_2). **Reproduce (5.3b) from the primary source before use.** Two practical warnings: (i) many secondary sources tabulate $b_1 = b_2$, which makes $c = 0$ and collapses P_{LoS} to a constant; (ii) the widely-quoted "urban $a = 9.61$, $b = 0.16$" pair is (a_1, b_1), **not** (a, b). Do not mix them. Because of this ambiguity, in a simulator it is usually better to implement the elevation-angle LoS probability *directly* as

$$P_{\text{LoS}}(\theta) = \begin{cases} 1, & \theta \ge \theta_0 \\ \frac{1}{1 + \left(b/\tan\theta_0\right)^2}, & \theta < \theta_0 \end{cases}$$

([AlH20] form 2) and calibrate θ_0 and b from your own city layout.

5.4 Coverage radius as a function of altitude and antenna pattern

Isotropic / wide-beam. For the [38.901] UMa NLOS branch (1.3e), the only altitude dependence is via $d_{3D} = \sqrt{d_{2D}^2 + (h - h_{UT})^2}$:

$$d_{3D,\max} = 10^{\frac{PL_{\max} - 13.54 - 20\log_{10}(f_c) + 0.6(h_{UT} - 1.5)}{39.08}}, \qquad d_{2D,\max}(h) = \sqrt{d_{3D,\max}^2 - (h - h_{UT})^2} \tag{5.4a}$$

Numerically, for $PL_{\max} = 141.24$ dB ($B = 10$ kHz, $R = 100$ kbit/s, EIRP = 20 dBm, $G_r = 2$ dBi, $F = 7$ dB, $f_c = 2.4$ GHz, $h_{UT} = 1.5$ m): $d_{3D,\max} = 1183.5$ m and

h (m)	30	50	100	150	200	300
$d_{2D,\max}$ NLOS (m)	1183.2	1182.5	1179.4	1174.1	1166.7	1145.2

Pure NLOS gives no interior optimal altitude — the coverage radius is monotonically decreasing in h (and collapses to zero as $h \to d_{3D,\max} \approx 1183$ m). The optimum comes *only* from the LoS-probability term. Combining (1.9c) with both branches and solving $P_{\text{LoS}}\, PL_{\text{LoS}} + (1 - P_{\text{LoS}})\, PL_{\text{NLoS}} = PL_{\max}$:

h (m)	30	50	100	150	200	300	600
composite $d_{2D,\max}$ (m)	1222.5	1221.9	1219.0	1214.1	1207.1	1187.1	1072.7

i.e. also monotone, because with $h_{UT} = 1.5$ m the [38.901] LoS probability is essentially zero at the cell edge ($\Pr_{LOS} \approx 0.0145$ there) and the NLOS term dominates. **This is the correct, mathematically honest conclusion: with the [38.901] UMa model, higher is worse.** The "optimal LAP altitude" result of [AlH14] only emerges when you plug in an altitude-dependent $P_{\text{LoS}}(\theta)$ — precisely the extra physics [38.901] does not provide.

Directional antenna footprint. A nadir-pointing antenna of azimuth 3 dB beamwidth α illuminates a ground disc of radius

$$r_{fp} = h_{\text{eff}} \tan\!\left(\frac{\alpha}{2}\right), \qquad h_{\text{eff}} = h - h_{UT} \tag{5.4b}$$

α	r_{fp}/h	r_{fp} @ 100 m	r_{fp} @ 150 m
30°	0.2679	26.8 m	40.2 m
65° (3GPP UMa)	**0.6371**	**63.7 m**	**95.6 m**
90°	1.0000	100.0 m	150.0 m
120°	1.7321	173.2 m	259.8 m

The **coverage trade-off is then explicit**: the array factor limits the footprint to r_{fp} no matter how much power, while the path loss limits the propagation radius:

$$r_{cov} = \min(r_{fp},\, r_{prop}), \qquad r_{prop} \text{ from (5.4a) with the extra antenna-pattern penalty } +(-G(\theta(r))) \tag{5.4c}$$

where $G(\theta)$ comes from the `AxiallySymmetricAntenna` gain string. With a 65° beam at $h = 100$ m the footprint edge is $r_{fp} = 63.7$ m, i.e. an off-nadir angle of 57.5° — comfortably inside the main lobe — giving a **63.7 m radius of genuinely good coverage**, plus a long degraded (−12 to −30 dB) tail out to the propagation radius. **For a persistent-coverage RL problem the antenna footprint, not the path-loss ceiling, is the binding constraint over most of the useful range — and neither `UnitDiskRadio` nor the default isotropic INET antenna captures that.** Use `AxiallySymmetricAntenna`.

5.5 Aerial-to-aerial

- **Free space, always.** There is no terrain; (3.1b) applies with $d = d_{3D}$.
- **Rician K is high.** [38.901] does not cover A2A. Measured air-to-air K values are typically 10–20 dB; at 2.4 GHz with air-to-air LoS, $K = 15$ dB with $\sigma_{SF} = 2$ dB is a defensible starting point (cite your measurement).
- **Doppler is the dominant dynamic effect.** Two counter-rotating UAVs at 10 m/s each have a relative speed of 20 m/s: $f_{D,\max} = 160$ Hz at 2.4 GHz, $T_c \approx 1.12$ ms.
- **Multipath delay.** Air-to-air delay spreads are small (tens of ns), so a narrowband model is adequate. Set `numFadingPaths` in Simu5G to 3–6 and `delayRms` to 30–100 ns for A2A.
- **Three-ray (ground-bounce) model.** If you want a specular ground bounce, add a second propagation path with relative attenuation $4\sin^2\!\left(\frac{2\pi h_t h_r}{\lambda d}\right)$ and relative delay $\frac{2h_t h_r}{c}$.

5.6 Multihop relay and connectivity graph

Define the per-link success probability from 3 + 5.2/5.4:

$$p_{ij} \;=\; \Pr[\mathrm{SINR}_{ij} \geq \Gamma] \;\simeq\; \mathcal{N}\!\left(\frac{P_{r,ij} - N_{\text{eff}} - \gamma}{\sigma_{SF,ij}},\, 1\right) = \Phi\!\left(\frac{P_{r,ij} - N_{\text{eff}} - \gamma}{\sigma_{SF,ij}}\right) \tag{5.6a}$$

with $P_{r,ij} = \mathrm{EIRP}_i + G_{r,j}(\theta_{ij}) - PL_{ij}$ and θ_{ij} the elevation angle between UAV i and UAV j (both airborne => θ_{ij} is the off-nadir angle of the antenna, so **antenna pointing matters enormously for A2A**).

A standard surrogate for multi-hop connectivity is a correlated-outage / stochastic-geometry form:

$$\Pr[\text{UAV } j \text{ not reachable from } i] = 1 - p_{ij}, \qquad \Pr[\text{network partitioned}] \approx \sum_{j} \prod_{k \in \mathcal{N}(j)} (1 - p_{jk})^{\rho} \tag{5.6b}$$

or, for the classic relay model, the probability that the sink is unreachable after n hops. **However** — and this matters for a rigorous report — (5.6b) is a *model*, not a measurement, and it rests on a homogeneous-Poisson assumption that breaks down badly for a small, deliberately positioned UAV swarm (which is exactly what your RL agent produces). Since your simulator evaluates every pair link-by-link anyway, **measure the empirical disconnection probability from the simulation** rather than assuming (5.6b). Also note that INET's `ScalarRadioMedium` does not route at the physical layer: multihop must come from the routing layer (`Ipv4Router` plus a routing protocol), so the "relay" behaviour is a MAC/routing outcome, not a channel-model outcome.

0.14.7. 6. Simu5G: what it models and what it does not

6.1 Channel-model family

[Simu5G] `src/simu5g/stack/phy/channelmodel/`:

```
ChannelModelBase            (abstract)
 IdealChannelModel       no impairments; a configured PER only
 StochasticChannelModel  the real one
     Tr36873ChannelModel    pathLossType default "Tr36873"
     Tr38901ChannelModel    pathLossType default "Tr38901"   <-- default for gNodeB/NR UE
```

`pathLossType @enum("Tr36814","Tr36873","Tr38901")`, backed by `Tr36814PathLossModel.cc`, `Tr36873PathLossMod` `Tr38901PathLossModel.cc`. From v1.7.0 the project notes: *"the 3GPP propagation formulas audited, fixed and covered by unit tests"*.

`Tr38901PathLossModel.cc` implements exactly the formulas of 1.3/1.4/1.5 (verified in source):

double pLoss_nlos = 13.54 + 39.08*log10(threeDimDistance) + 20*log10CarrierFrequencyGHz_ - 0.6*(hUe_-1.5); // UMa double pLoss_nlos = 35.3*log10(threeDimDistance) + 22.4 + 21.3*log10CarrierFrequencyGHz_ - 0.3*(hUe_-1.5); // UMi double pLoss_nlos = 161.04 - 7.1*log10(W) + 7.5*log10(h) ... ; // RMa

and `getShadowingStdDev()` returns 4/6 (UMa), 4/7.82 (UMi), 3/8.03 (InH), 4/6/8 (RMa) by LOS state and breakpoint. It also implements the O2I low/high-loss models of (1.8c).

6.2 `StochasticChannelModel` — the complete parameter set

Verbatim from `StochasticChannelModel.ned`:

string pathLossType @enum("Tr36814","Tr36873","Tr38901") = default("Tr36814"); bool shadowing = default(true); string scenario @enum(INDOOR_HOTSPOT,URBAN_MICROCELL,URBAN_MACROCELL,RURAL_MAC = default("URBAN_MACROCELL"); double nodebHeight @unit(m) = default(25m); double buildingHeight @unit(m) = default(20m); bool insideBuilding = default(false); bool useBuildingPenetrationHighLossModel = default(false); double streetWidth @unit(m) = default(20m); double ueHeight @unit(m) = default(1.5m); bool tolerateMaxDistViolation = default(false); bool useTorus = default(false); double correlationDistance @unit(m) = default(50m); double targetBler = default(0.01); double harqReduction = default(0.2); double antennaGainUe @unit(dBi) = default(0dBi); double antennGainEnB @unit(dBi) = default(18dBi); double antennGainMicro @unit(dBi) = default(5dBi); double thermalNoise @unit(dBm) = default(-104.5dBm); // "for 10 MHz of Bandwidth" double ueNoiseFigure @unit(dBm) = default(7dBm); double bsNoiseFigure @unit(dBm) = default(5dBm); double cableLoss @unit(dB) = default(2dB); bool dynamicLos = default(false); bool fixedLos = default(false); bool fading = default(true); string fadingType @enum(RAYLEIGH,JAKES) = default("JAKES");

int numFadingPaths = default(6); double delayRms @unit(s) = default(363ns); bool bgCellInterference = default(true); bool extCellInterference = default(true); bool downlinkInterference = default(false); bool uplinkInterference = default(false); bool enableExtCellLos = default(true); bool collectSinrStatistics = default(true);

Key defaults you should override for a UAV-IoT study:

Parameter	Simu5G default	What you likely want	Why
`scenario`	`URBAN_MACROCELL`	keep, or `SUBURBAN_MACROCELL`	suburban LoS persists further
`nodebHeight`	25 m	**your UAV altitude**	see 1.12 — out of the [38.901] range, but Simu5G lets you set it
`ueHeight`	1.5 m	1.5 m	fine
`correlationDistance`	50 m	**25 m** for exact [38.901] UMa-NLOS ($r_{SF}/2$, eq. 4.8b)	[38.901] r_{SF}(UMa NLOS) = 50 m
`thermalNoise`	−104.5 dBm	$-173.98+10\log_{10}(B)+F$ for your B	Simu5G does not compute kTB
`delayRms`	363 ns (UMa LOS median)	240 ns NLOS / 226 ns LOS	scenario-dependent
`fadingType`	`JAKES`	keep	
`numFadingPaths`	6	3–6	6 taps ≈ 363 ns RMS spread
`downlinkInterference` / `uplinkInterference`	**false**	**true**	otherwise there is no inter-cell interference at all
`targetBler`	0.01	keep	Simu5G CQI computation targets 1 % BLER
`harqReduction`	0.2	keep	per-HARQ-transmission SNR offset

6.3 What Simu5G models

- **Numerology** 0–4 (`ComponentCarrier.cc` throws if `numerologyIndex > 4`), FDD/TDD, TDD symbol split, carrier aggregation.
- **MCS/CQI/iTBS tables** exactly as in 3.7 ([Simu5G] `NrMcs.cc`; CQI from TS 38.214 Table 7.2.3-1, MCS from Tables 5.1.3.1-1/-2, TBS from Table 5.1.3.2-1).
- **BLER-vs-SINR curves** in `src/simu5g/common/blerCurves/`: `BLERvsSINR_15CQI_TU.h` (TU, from TR 36.842) and `BLERvsSINR_15CQI_AWGN.h`, indexed by `(itxmode, cqi, snr)` via `binder_->phyPisaData.getBler(itxmode, cqi, snr)`.
- **Path loss / LoS probability / shadowing** per TR 36.814, 36.873 or 38.901 (6.1).
- **Multipath fading**: Jakes (sum of sinusoids, per-band, per-TTI) or Rayleigh — (4.8a).
- **Interference**: DL intra/inter-cell, UL, external cells, background cells (large-scale).
- **Antenna pattern attenuation** per the 3GPP sector pattern, plus the full link budget (antenna gains, cable loss, noise figures, thermal noise).
- **HARQ** (up to 4 process IDs by default), CQI reporting with target BLER, schedulers (Max C/I, Proportional Fair, Deficit Round Robin, QoS-aware variants), random access, BSR, handover, RLF/RRC re-establishment, RLC TM/UM/AM per TS 38.322, PDCP, SDAP, QoS flows (QCI/5QI), 5GC, EN-DC/NE-DC split bearers, MEC, and D2D (optional package).

6.4 What Simu5G does NOT model

This list is essential for an honest report.

1. **No aerial/UAV channel model.** No A2G LoS probability, no altitude-dependent path loss, no elevation-angle antenna pointing. `nodebHeight` is just a scalar fed to the terrestrial formulas, and there is no check that it lies inside [38.901]'s validity envelope.
2. **No spatial consistency** in the sense of [38.901] 7.6.3. Shadowing is correlated in *time* via (4.8b) only; there are no LSP/SSP correlation distances, no cluster model, no 38.901 GSCM.

3. **No tap-delay-line / frequency-selective channel.** `delayRms` only sets the phase offsets in (4.8a); it is not used for any convolution, and there is one SINR per PRB, not per subcarrier. **No OFDM, no CP, no ICI, no MIMO** — the Simu5G v1.4.5 notes explicitly say the incomplete MIMO code was *removed*.
4. **No per-symbol processing.** All decoding is at BLER-curve granularity per TTI.
5. **No thermal-noise computation.** `thermalNoise` is a constant.
6. **No RRC signalling** (only bearer management, handover, RLF and re-establishment are modelled).
7. **No MAC detail beyond what is implemented**: no PHY preamble-detection model beyond "collisions", no explicit propagation of TX-power-control commands beyond the configured rules, no PHY ARQ other than HARQ.
8. **No UAV energy model**, no propulsion power, no flight dynamics coupled to the radio.
9. **No multi-UAV coordination.** Handover, scheduling and power control assume terrestrial topology; there is no inter-UAV beam coordination and no distributed interference management.
10. **No mmWave beam management or dynamic blockage** (no human blockage, no LoS/NLOS transitions over time), even though 38.901 covers them.

0.14.8. 7. Latency, PDR and throughput metrics (ITU-T definitions)

7.1 Fundamental IP packet transfer performance parameters [Y.1540]

For IP packet T in a packet flow from source to destination:

One-way delay (IP packet transfer delay), [Y.1540] 5.1 / RFC 7679 3.4:

$$t_T \;=\; t_2 - t_1 \tag{7.1a}$$

where t_1 is the wire time at which the *first bit* of T crosses the source network reference point and t_2 the wire time at which the *last bit* of T crosses the destination reference point. It is **undefined (informally infinite)** if T is not received within the loss-threshold waiting time T_{max}.

Packet delay variation (IPDV), 2-point, [Y.1540] / RFC 2680 4.1:

$$\mathrm{IPDV}(T_1, T_2) \;=\; |\, t_{T_1} - t_{T_2} \,|, \quad T_1 < T_2 \tag{7.1b}$$

Absolute packet delay variation (APDV) / inter-packet delay variation:

$$\mathrm{APDV}(T) \;=\; |\, t_T - t_{T-1} \,| \tag{7.1c}$$

Jitter is *not* a [Y.1540] metric; the term is overloaded. Use either APDV (7.1c), the **local delay variation** $\mathrm{LDV}(T) = t_T - t_{T-1}$ signed, or the [RFC 3550] inter-arrival jitter

$$J = \big|\, (t_{T_i} - t_{T_{i-1}}) - (\tau_{T_i} - \tau_{T_{i-1}}) \,\big| \tag{7.1d}$$

where τ is the timestamp *at the sender*. **State explicitly in your report which one you use.** For packetised sensor data, APDV (7.1c) is the most defensible and is what "jitter" usually means in IoT QoS contexts.

7.2 Aggregated parameters

$$\mathrm{IPLR} \;=\; \frac{\text{number of packets lost}}{\text{total number of packets sent}}, \qquad \mathrm{PDR} \;=\; 1 - \mathrm{IPLR} \tag{7.2a}$$

$$\mathrm{OWD}_{\mathrm{mean}} = \frac{1}{N} \sum_{T \in \mathcal{S}} t_T, \qquad \mathrm{OWD}_p = \inf\big\{ x : F(x) \geq p \big\} \tag{7.2b}$$

$$\mathrm{ITR} \;\approx\; \frac{L}{p_1 - p_2} \tag{7.2c}$$

([Y.1540] 5.3: the information transfer rate for a file of L octets and loss ratio p is $L/(1-p)$; with an IPLR $= p_1$ and an IPER $= p_2$ the usable rate is $L/(p_1 - p_2)$.) Throughput over a simulation window:

$$= \frac{8 \cdot \sum_k L_k}{T_{\mathrm{sim}}\ [\mathrm{s}]} \tag{7.2d}$$

For a UAV-IoT report the clean metric set is:

metric	definition	units
Coverage ratio	fraction of target sensors whose SINR to their serving UAV exceeded Γ during the window	dimensionless
PDR	(7.2a), per sensor and network-wide	dimensionless
Mean / p_{95} / p_{99} OWD	(7.2b) on successfully delivered packets only	ms
APDV / p_{95} APDV	(7.1c)	ms
Goodput	(7.2d) counting only delivered payload bytes	bit/s
Connectivity time	fraction of time each sensor has ≥ 1 UAV with SINR $\geq \Gamma$	dimensionless
Handover count / inter-UAV handover latency	simulator-native	—

?[Y.1541] Table 3, provisional classes 6 and 7 (evaluation interval ≥ 1 min, 1500-byte payloads):

Parameter	Class 6	Class 7
IPTD (one-way delay)	100 ms	400 ms
IPDV ($1 - 10^{-5}$ quantile of IPTD minus the minimum IPTD)	50 ms	50 ms
IPLR (packet loss ratio)	1×10^{-5}	1×10^{-5}
IPER (error ratio)	1×10^{-6}	1×10^{-6}
IPRR (reserved-bit ratio)	1×10^{-6}	1×10^{-6}

Mapping UAV-IoT onto these classes. A persistent-coverage UAV-IoT network delivering small sensor reports every Δ seconds is best benchmarked against **Y.1541 Class 0 / Class 2** (OWD $\leq$ 100 ms, IPLR $\leq 10^{-3}$). Two points that belong in your report:

- The propagation delay from a 100 m UAV is **negligible** ($100/3 \times 10^8 = 0.33$ μs), but the LoS/NLOS state changes the usable radius by a factor of ≈ 5.5 (8.11), and a UAV moving at 10 m/s changes which UAV serves a sensor every few seconds.
- Y.1541's 1-minute evaluation interval and 10^{-3} loss budget are weak constraints for a *persistent* coverage problem. Report **per-sensor outage distribution** and **inter-UAV handover latency**, not only network-average PDR. Class 0's 50 ms IPDV objective maps directly onto "how much does packet delay jitter while the serving UAV changes?"

0.14.9. 8. Worked numerical example — every arithmetic step

8.1 Scenario

Item	Value
Aerial BS (UAV) altitude	$h_{BS} = 100$ m
Ground sensor antenna height	$h_{UT} = 1.5$ m
Carrier frequency	$f_c = 2.4$ GHz
UAV EIRP (isotropic downlink antenna)	20 dBm
Sensor antenna gain	2 dBi
Sensor noise figure	7 dB
Cable/feeder + body losses	0 dB (both ends)
Channel bandwidth	$B = 200$ kHz
Scenario	3GPP TR 38.901 **UMa**, NLOS branch
Shadowing	$\sigma_{SF} = 6$ dB (UMa NLOS)

Validity warning, restated. $h_{BS} = 100$ m is **outside** the [38.901] UMa envelope ($h_{BS} = 25$ m nominal). Every number below is the [38.901] formula *evaluated* at $h_{BS} = 100$ m, i.e. an extrapolation. 10 discusses what to do instead.

8.2 Step 1 — geometry

With $h_{BS} = 100$ m and $h_{UT} = 1.5$ m:

$$h_{BS} - h_{UT} = 98.5 \text{ m}, \qquad \Delta h^2 = 9702.25 \text{ m}^2$$

$$d_{3D} = \sqrt{d_{2D}^2 + 9702.25} \text{ m}$$

For the reference case $d_{2D} = 500$ m:

$$d_{3D} = \sqrt{500^2 + 9702.25} = \sqrt{259\,702.25} = 509.6099 \text{ m}$$

8.3 Step 2 — breakpoint (needed for the LOS branch)

$h_{UT} = 1.5$ m < 13 m $\Rightarrow C(d_{2D}, h_{UT}) = 0 \Rightarrow h_E = 1$ m is drawn with probability $1/(1+0) = 1$, i.e. deterministically. Hence

$$h'_{BS} = 100 - 1 = 99 \text{ m}, \qquad h'_{UT} = 1.5 - 1 = 0.5 \text{ m}$$

$$d'_{\text{BP}} = \frac{4\, h'_{BS} h'_{UT} f_c}{c} = \frac{4 \cdot 99 \cdot 0.5 \cdot 2.4 \times 10^9}{2.99792458 \times 10^8} = \frac{4.752 \times 10^{11}}{2.99792458 \times 10^8} = 1585.0966 \text{ m}$$

Sanity checks: $h'_{BS} h'_{UT} = 49.5$ m^2; $4f_c/c = 32.022153$ m^{-1}; product $= 1585.0966$ m. ✓
Also, for later use:

$$9 \log_{10}\big((d'_{\text{BP}})^2 + \Delta h^2\big) = 9 \log_{10}(2\,512\,529.6 + 9702.25) = 9 \log_{10}(2\,522\,233.4) = 9 \times 6.401790 = 57.6161 \text{ dB}$$

8.4 Step 3 — path loss, UMa LOS branch (for contrast)

$d_{2D} = 500$ m $\le d'_{\text{BP}} = 1585.10$ m, so use PL_1 of (1.3b):

$$PL_1 = 28.0 + 22 \log_{10}(509.6099) + 20 \log_{10}(2.4)$$

$$\log_{10}(509.6099) = 2.707121 \quad \Rightarrow \quad 22 \times 2.707121 = 59.55666 \text{ dB}$$

$$20 \log_{10}(2.4) = 20 \times 0.380211 = 7.60422 \text{ dB}$$

$$\boxed{PL_{\text{UMa-LOS}} = 28.0 + 59.5567 + 7.6042 = 95.161 \text{ dB}}$$

Above the breakpoint the PL_2 branch (1.3c) applies; at $d_{2D} = 1585$ m, $d_{3D} = 1588.06$ m, $\log_{10}(1588.06) = 3.200973$:

$$PL_2 = 28.0 + 40(3.200973) + 7.6042 - 57.6161 = 28.0 + 128.0389 + 7.6042 - 57.6161 = 106.027 \text{ dB}$$

8.5 Step 4 — path loss, UMa NLOS branch (the model we use)

From (1.3e):

$$PL'_{\text{UMa-NLOS}} = 13.54 + 39.08 \log_{10}(509.6099) + 20 \log_{10}(2.4) - 0.6\,(1.5 - 1.5)$$

$$39.08 \times 2.707121 = 105.79830 \text{ dB}, \qquad -0.6 \times 0 = 0$$

$$\boxed{PL'_{\text{UMa-NLOS}} = 13.54 + 105.7983 + 7.6042 + 0 = 126.9425 \text{ dB}}$$

Apply the max of (1.3d):

$$PL_{\text{UMa-NLOS}} = \max(95.161,\ 126.9425) = 126.9425 \text{ dB}$$

The branches cross where $13.54 + 39.08 \log_{10}(d_{3D}) = 28.0 + 22 \log_{10}(d_{3D})$, i.e. $17.08 \log_{10}(d_{3D}) = 14.46 \Rightarrow d_{3D} = 10^{0.84660} = 7.03$ m. So for every $d_{2D} \ge 10$ m the NLOS branch is selected — the height question changes the *value*, not the *branch*.

8.6 Step 5 — noise power

$$N_0 = kT = 1.380649 \times 10^{-23} \times 290 = 4.003882 \times 10^{-21} \text{ W/Hz}$$

$$N_0[\text{dBm/Hz}] = 10\log_{10}\left(\frac{4.003882 \times 10^{-21}}{10^{-3}}\right) = 10\log_{10}(4.003882 \times 10^{-18}) = -173.9752 \text{ dBm/Hz}$$

$$N_{\text{th}} = N_0 + 10\log_{10}\left(\frac{B}{1 \text{ Hz}}\right) = -173.9752 + 10\log_{10}(200\,000) = -173.9752 + 53.0103 = -120.9649 \text{ dBm}$$

$$N_{\text{eff}} = N_{\text{th}} + F = -120.9649 + 7 = -113.9649 \text{ dBm} \quad \left(= 10^{-11.39649} \text{ mW} = 4.01339 \times 10^{-12} \text{ mW}\right)$$

8.7 Step 6 — received power and SINR

$$P_r = \text{EIRP} + G_r - L_{\text{sys}} - PL = 20 + 2 - 0 - 126.9425 = -104.9425 \text{ dBm}$$

$$\text{SINR[dB]} = P_r - N_{\text{eff}} = -104.9425 - (-113.9649) = +9.0224 \text{ dB}$$

$$\text{SINR}_{\text{linear}} = 10^{0.90224} = 7.9873$$

8.8 Step 7 — Shannon rate

$$R = B\log_2(1 + \text{SINR}) = 200\,000 \times \log_2(8.9873) = 200\,000 \times 3.16736$$

$$\boxed{R = 633\,473 \text{ bit/s} \approx 0.633 \text{ Mbit/s}}$$

Spectral efficiency $\eta = R/B = 3.167$ bit/s/Hz — far beyond what any real radio achieves. This is why Shannon is an upper bound and not a design number.

8.9 Step 7b — what a real radio gives at the same SINR

Using [38.214] MCS index 7 (QPSK, $R = 526/1024$, $\eta = 2 \times 526/1024 = 1.0273$ bit/s/Hz):

$$R_{\text{real}} = B \times \eta = 200\,000 \times 1.0273 = 205\,469 \text{ bit/s} \approx 0.205 \text{ Mbit/s}$$

64QAM (MCS 23, $\eta = 4.213$) is **not** viable at SINR = 9.02 dB. From (3.6c)/(3.6e), 64QAM at BLER $< 10^{-3}$ needs $E_b/N_0 \approx 18$–21 dB, i.e. $\gamma = 18 - 10\log_{10}(4.213) = 18 - 6.244 = 11.76$ dB at best and ≈ 15–18 dB in practice. **An RL agent that optimises position must be given the modulation schedule, not Shannon capacity.**

8.10 Step 8 — path loss vs distance (same link, full table)

d_{2D} (m)	d_{3D} (m)	PL_{LOS} (dB)	PL'_{NLOS} (dB)	PL used (dB)	P_r (dBm)	SINR (dB)	R Shannon (Mbit/s)
200	222.940	87.26	112.91	112.91	−90.91	23.05	1.5331
500	509.610	95.16	126.94	126.94	−104.94	9.02	0.6334
1000	1004.839	101.65	138.47	138.47	−116.47	−2.50	0.1287
1585	1588.058	106.03	146.23	146.23	−124.23	−10.27	0.0259
2000	2002.424	110.05	150.17	150.17	−128.17	−14.20	0.0108
3000	3001.617	117.08	157.04	157.04	−135.04	−21.07	0.0022
5000	5000.970	125.95	165.70	165.70	−143.70	−29.74	0.0003

The LOS/NLOS gap is 17.7 dB at 200 m and 39.7 dB at 5 km, and it **grows** with distance because the NLOS slope (39.08) exceeds the LOS slope (22). Altitude therefore buys you *branch switching*, not a slope change.

8.11 Step 9 — required distance for a target rate

Required SINR for target rate R in bandwidth B:

$$\gamma_{\text{req}} = 10\log_{10}\left(\frac{R}{B}\ln 2 - 1\right) \tag{8.11a}$$

derived from $R = B\log_2(1+\gamma) \Rightarrow \gamma = 2^{R/B} - 1$. Note that γ_{req} is undefined for $R < B\ln 2$: **you cannot hit 100 kbit/s in a 200 kHz channel at all**, because the zero-SINR rate is exactly $B\log_2 2 = B$.

Maximum tolerable path loss, from $P_r = \text{EIRP} + G_r - PL$ and $\text{SINR} = P_r - N_{\text{eff}} \geq \gamma$:

$$PL_{\max} = \text{EIRP} + G_r - N_{\text{eff}} - \gamma_{\text{req}} = 20 + 2 - (-113.9649) - \gamma_{\text{req}} = 135.9649 - \gamma_{\text{req}} \tag{8.11b}$$

Invert the NLOS branch (1.3e):

$$d_{3D,\max} = 10^{\frac{PL_{\max} - 13.54 - 20\log_{10}(f_c) + 0.6(h_{UT}-1.5)}{39.08}}, \qquad d_{2D,\max} = \sqrt{d_{3D,\max}^2 - (h_{BS} - h_{UT})^2} \tag{8.11c}$$

NLOS results (recomputed per row, since N_{eff} depends on B):

B	R (bit/s)	N_{eff} (dBm)	γ_{req} (dB)	$PL_{\max}$ (dB)	$d_{3D,\max}$ (m)	$d_{2D,\max}$ (m)
10 kHz	100 000	−123.975	7.7316	141.2436	1183.5	**1179.4**
10 kHz	1 000 000	−123.975	18.3505	130.6247	633.3	625.6
10 kHz	10 000 000	−123.975	28.4000	120.5649	350.1	336.0
10 kHz	100 000 000	−123.975	38.4108	110.5541	194.2	167.3
200 kHz	1 000 000	−113.965	3.9242	132.0407	688.3	681.3
200 kHz	10 000 000	−113.965	15.2713	120.6936	352.6	338.6
200 kHz	100 000 000	−113.965	25.3902	110.5747	194.3	167.5
1 MHz	10 000 000	−106.975	7.7316	128.2333	364.2	350.7

LOS branch (PL_2, eq. 1.3c) for contrast — inverting $d_{3D} = 10^{(PL_{\max} - 28.0 - 20\log_{10} f_c + 9\log_{10} C)/40}$ with $C = (d'_{\text{BP}})^2 + \Delta h^2 = 2\,522\,233.4$:

B	R (bit/s)	$PL_{\max}$ (dB)	$d_{3D,\max}$ (m)	$d_{2D,\max}$ (m)
10 kHz	100 000	141.2436	6547.1	6546.3
10 kHz	1 000 000	130.6247	3694.0	3693.6
200 kHz	1 000 000	132.0407	7102.8	7102.2
200 kHz	10 000 000	120.6936	3695.3	3694.0

Interpretation. In LoS the same power budget buys a **5.5× larger radius** (6546 m vs 1179 m). A UAV swarm therefore does not "extend coverage" in the NLOS sense — it *keeps its own users in LoS*, which is far more valuable. An RL reward built on pure coverage area under an NLOS-only assumption undervalues LoS maintenance by roughly 5× in radius, i.e. ≈ 30× in area.

Sensitivity to the two most-overlooked terms:

- Forgetting the 7 dB noise figure: $PL_{\max}$ rises by 7 dB, so $d_{3D,\max} \times 10^{7/39.08} = 1.514\times$ — a **51 % larger coverage radius from a pure bookkeeping omission.**
- EIRP error of ±3 dB ⇒ radius error ±7.7 %.

8.12 Step 10 — shadowing margin and outage

With $\sigma_{SF} = 6$ dB, from (2.1d):

$$M_{\text{SF}}(p) = 6\,\Phi^{-1}\left(1 - \frac{p}{2}\right) \text{ dB}$$

p	0.20	0.10	0.05	0.01
M_{SF} (dB)	7.689	9.869	11.760	15.455

To keep $P_{\text{out}} \leq 1\,\%$ at $B = 10$ kHz, $R = 100$ kbit/s: $PL_{\max} = 141.2436 - 15.455 = 125.789$ dB, so

$$d_{3D,\max} = 10^{\frac{125.789-13.54-7.6042}{39.08}} = 724.3 \text{ m}, \qquad d_{2D,\max} = \sqrt{724.3^2 - 9702.25} = 717.6 \text{ m}$$

A 1 % outage target cuts the NLOS coverage radius by 39 % (1179 m → 718 m). Shadowing is the second-largest lever after the path-loss exponent. If it is modelled as i.i.d. per packet (which is what plain `LogNormalShadowing` does), the *marginal* outage probability is right but the *temporal* structure is wrong: real shadowing produces correlated outage bursts, which is exactly what an RL policy needs to learn to avoid.

8.13 Step 11 — cross-check against [AlH14] and against Simu5G

[AlH14] A2G NLoS (5.2b) at the same geometry, with $PL_{\text{FS}}(d) = 32.4478 + 20\log_{10} f_c + 20\log_{10} d$ evaluated at the **horizontal** distance and $\eta_{\text{NLoS}} = 20$ dB:

d_{2D} (m)	PL_{LoS} (dB)	PL_{NLoS} (dB)	[38.901] UMa NLOS (dB)	difference
100	84.06	103.06	105.06	−2.0 dB
200	88.04	107.04	112.91	−5.9 dB
500	95.20	114.20	126.94	−12.7 dB
1000	101.10	120.10	138.47	−18.4 dB
2000	107.08	126.08	150.17	−24.1 dB
3000	110.60	129.60	157.04	−27.4 dB

The discrepancy grows without bound because [AlH14] NLoS is essentially free space (PLE 2.0) plus a 20 dB constant, whereas [38.901] UMa NLOS is PLE 3.908. Converting [AlH14] NLoS into a coverage radius for the 8.11 link budget ($PL_{\max} = 141.24$ dB) gives $d_{2D} > 20$ km — **more than an order of magnitude more coverage than [38.901]**, and still growing at the edge of any realistic scenario. Never mix the two in the same study without a stated cross-calibration.

Simu5G. With its defaults (`antennGainEnB = 18` dBi, `cableLoss = 2` dB, `bsNoiseFigure = 5` dB, `thermalNoise = -104.5` dBm):

```
EIRP        = TxPower - cableLoss + antennaGainEnB = TxPower + 16 dB
N_dBm       = thermalNoise + noiseFigure          = -104.5 + 5 = -99.5 dBm
```

At $d_{2D} = 500$ m, $PL = 126.9425$ dB and TxPower = 0 dBm: $P_r = 16 - 126.9425 = -110.9425$ dBm, SINR $= -110.9425 - (-99.5) = -11.44$ dB — unusable. To match 8.7 exactly in Simu5G you must set

```
TxPower  = 0 dBm
antennGainEnB  = 2 dBi            // so that EIRP = 20 dBm with cableLoss = 2 dB
bsNoiseFigure  = 7 dB
thermalNoise   = -120.965 dBm     // = -173.975 + 10*log10(200e3), NF folded into thermalNoise or kept
```

Simu5G additionally applies its 3GPP sector antenna pattern; with an azimuth beamwidth of 65° and maximum gain set to `antennGainEnB`, a UAV at 500 m range sees an off-nadir angle of $\theta = \arctan(98.5/500) = 11.15°$, i.e. inside the main lobe, so the full gain applies.

8.14 Step 12 — the same link in a `UnitDiskRadio` run

If instead you use `UnitDiskRadio` with `UnitDiskTransmitter`, the only physics is

$$\text{receive} \iff d < \texttt{communicationRange} \wedge \big(\text{no interfering tx within } \texttt{interferenceRange}\big) \tag{8.14a}$$

so the whole 8.2–8.13 analysis is replaced by a single number in the `.ned` file. You would set it from (5.4a): $d_{2D,\max} = 1179$ m for $B = 10$ kHz/$R = 100$ kbit/s, or 681 m for $B = 200$ kHz/$R = 1$ Mbit/s. **This is the correct value to use, and the reason 4.6 argues for the scalar radio model is that this number is invisible when you write it into a NED file but has to be *derived* with the analysis above.**

0.14.10. 9. Corrections and caveats — a consolidated list

9.1 "UAV altitude above 22.5 m is not covered by TR 38.901" — what is actually true

Restated precisely (see 1.12):

1. 22.5 m is the **upper bound on** h_{UT}, the *ground terminal* height, for UMa and UMi. It arises because $C(d_{2D}, h_{UT})$ and $C'(h_{UT})$ are only calibrated to 23 m. A ground sensor at 1.5 m is well inside it.
2. The genuine exclusion is h_{BS}: UMa is calibrated at $h_{BS} = 25$ m, UMi at 10 m. Setting h_{BS} to a UAV altitude of 20–300 m is an **extrapolation**, and the LoS-probability derivation (Table 7.4.2-1 Note: *"antenna heights of ... 25 m for UMa"*) is invalidated.
3. You cannot avoid the extrapolation by labelling the UAV a "UT" — then $h_{UT} = 100$ m violates $h_{UT} \leq 22.5$ m.
4. **[38.901] contains no aerial-communication scenario at all.** Its scope clause (V19.1.0 4) lists only urban micro street canyon, urban macro, indoor office, rural macro, indoor factory and suburban macro. A full-text search of V19.1.0 finds "UAV" only in the ISAC **sensing-target** clause (7.9, radar cross-section) — never in a propagation model.
5. **The 3GPP non-terrestrial models do not cover 20–200 m either.** [38.811] and [38.821] target NTN platforms in the **8–50 km** band (the 3GPP portal describes UAS as *"operating typically between 8 and 50 km altitudes, including High Altitude Platforms"*), plus LEO/GEO satellites (600 km, 1 200 km, 35 786 km in the ns-3 calibration). Low-altitude UAV operation (200 m–1 km) is described as "below 1 km" / "a few hundred metres" in the NTN literature but has **no 3GPP path loss model calibrated for it**.
6. Rel-19 did add a **"UAV sensing scenario"** to the ISAC clause (7.9) with an aerial-UE height applicability range of 22.5 m $< h \leq$ 100 m mapped onto UMi/UMa LOS probability — but this is for *radar sensing targets*, not for a UAV acting as a base station. Do not repurpose it.

Therefore the statement "TR 38.901 does not cover UAV altitudes above 22.5 m" is *directionally right but mechanistically wrong.* The correct statement is: *TR 38.901's terrestrial path-loss and LoS-probability models are calibrated for a terrestrial BS at 10–35 m and a terrestrial UT at 1.5–22.5 m, and contain no aerial-BS scenario at all.*

9.2 What to do instead — four defensible options

(a) 3GPP extension: use TR 38.811/38.821 NTN models. Correct for HAPs (8–50 km) and LEO/GEO satellites. Scenarios: Dense Urban, Urban, Suburban, Rural; path loss, shadow fading, clutter loss and fast fading all specified; atmospheric absorption and tropospheric/ionospheric scintillation included. **Not calibrated for 20–300 m UAVs**, and its flat-fading model requires $\theta \geq 20°$ elevation, quasi-LoS, S-band, $B \leq 5$ MHz.

(b) Elevation-angle LoS probability + terrestrial slope (recommended for 20–300 m). Use the [38.901] **UMa LOS branch** (1.3b/1.3c) whenever LoS and the **UMa NLOS branch** (1.3e) otherwise, but replace $\Pr^{\text{UMa}}_{LOS}$(1.9*c*) with an **altitude/elevation-dependent** LoS probability $P_{LOS}(h, d_{2D})$ — e.g. (5.3a) from [AlH20], or a logistic fit calibrated against your own building-height distribution. This is what almost every serious UAV-coverage paper does, and it makes the UAV's altitude an explicit, controllable design variable.

(c) Sub-band σ_{SF} and slope correction. Empirically, A2G measurements show the NLOS slope *decreasing* with altitude (fewer obstructions at higher h): PLE values from ≈ 4.2 at 30 m down to ≈ 2.8–3.2 at 100–200 m in urban environments. Model

$$PL_{\text{NLoS}}(d_{3D}, h) = \text{FSPL}(d_{3D}) + 10\, n(h) \log_{10}\left(\frac{d_{3D}}{1\ \text{m}}\right) + B(h),$$

with $n(h)$ and $B(h)$ interpolated from your own measurements, or from published A2G fits.

(d) Geometric / ray-tracing. For a small deployment (tens of UAVs, hundreds of sensors) the most defensible choice is a **deterministic 3-D urban model** with ray tracing (Raymobtime, Sionna RT, QuaDRiGa) or, inside OMNeT++, INET's `TTracingObstacleLoss` / `DielectricObstacleLoss` over a `TiledBuildingEnvironment`. You then export a *lookup* of PL and shadowing onto a grid and feed it to INET through a custom `IPathLoss`. This removes the scenario-selection problem entirely, at the cost of no longer being "standard".

9.3 Ten modelling errors that will silently invalidate a UAV-IoT report

#	Error	Consequence
1	Not folding the receiver noise figure into `IsotropicScalarBackgroundNoise.power`	7 dB ⇒ **51 % overestimated coverage radius** (8.11)
2	Using INET's `LogNormalShadowing` for shadowing	i.i.d. per packet ⇒ no outage bursts ⇒ PDR far too high, jitter far too low (4.8)
3	Using Simu5G's `thermalNoise` default of −104.5 dBm with a bandwidth other than 10 MHz	wrong noise floor by $10\log_{10}(B/10\text{ MHz})$ dB
4	Using Simu5G's `correlationDistance` = 50 m as the [38.901] d_{cor}	effective correlation is **100 m**, twice [38.901] (eq. 4.8b)
5	`downlinkInterference = false` (the Simu5G default)	no inter-cell interference at all
6	Leaving `UnitDiskRadio` as the main model	no path-loss law, no shadowing, no LoS/NLOS state
7	Isotropic antennas everywhere	ignores the 63.7 m footprint at 100 m / 65° beam (5.4)
8	Shannon capacity as the rate in the reward function	5–10 dB optimistic vs a real MCS schedule (8.9)
9	Calling the UAV a "UT" to stay "inside" the 22.5 m bound	violates the bound and breaks the $C'(h_{UT})$ second LoS branch
10	Reporting network-average PDR only	hides per-sensor persistent outages — the entire point of the problem

9.4 Frequency caveats

- **2.4 GHz (ISM) is outside 3GPP NR FR1 bands.** NR FR1 is 410 MHz–7.125 GHz but banded; 2.4 GHz is Wi-Fi/Bluetooth territory. Simu5G will happily model it, but "3GPP TR 38.901 + Simu5G at 2.4 GHz" is a modelling choice, not a conformance scenario. If you need a real NR band, use n1 (2.11–2.17 GHz DL) or n3 (1.815–1.875 GHz) with a 20 MHz carrier, or n28/n8/n20 at 700–900 MHz for LPWAN IoT — and recompute $N_0 + 10\log_{10} B + F$ for the real bandwidth.
- **[38.901]'s frequency range is** $0.5 < f_c < f_H$ **GHz** with $f_H = 30$ GHz for RMa and 100 GHz elsewhere. 2.4 GHz is inside all of them.
- **mmWave (28 GHz, 39 GHz)**: add oxygen absorption (≈ 0.02 dB/km at 28 GHz excluding the 60 GHz band; ≈ 15 dB/km peak at 60 GHz), rain attenuation from ITU-R P.838, and the explicit LoS/NLOS state switching of 5.1. Not modelled by INET or Simu5G.

9.5 Other caveats worth stating in a report

- σ_{SF} **is a *marginal* statistic.** Correlating shadowing across sensors (2.2) changes the coverage-probability distribution but not the marginal. If you only ever look at marginal PDR, implementing spatial correlation changes nothing — which is precisely why it is easy to skip and why doing so is defensible, provided you say so.
- **Small-scale fading barely matters for packet-level PDR at these rates.** With $T_c \approx 2.2$ ms at 10 m/s and 2.4 GHz, and packets of 0.8–2 ms, the fast fade is essentially independent between packets; the *marginal* BER/PER is what matters and $R(\Delta x)$ of 2.2 does not enter. It *does* matter for (i) sub-μs delay-critical metrics, and (ii) the *burstiness* of failures at the packet level — hence per-TTI fading rather than per-transmission fading.
- **TR 38.901 7.4.1 Table 7.4.1-1 footnote 6** — the "Optional" rows ($32.4 + 20\log_{10} f_c + 30\log_{10} d_{3D}$ etc.) are *alternative* models with different σ_{SF} (7.8, 8.2, 8.29 dB). Do not average the main and optional models.
- **RMa LOS** σ_{SF} **is distance-dependent** (4 dB below d_{BP}, 6 dB above) — an implementation that hard-codes 4 dB for all RMa LOS is wrong above the breakpoint.
- **The `max(LOS, NLOS)` in (1.3d), (1.4d), (1.5e), (1.6b) is not a no-op.** For UMa/UMi it is redundant above ≈ 7 m but matters for the smallest cells; for InF-DL it selects the InF-SL floor and is mandatory.
- **In [38.901],** h_E **must be drawn per link**, not per node — a detail that changes d'_{BP} by hundreds of metres between adjacent users.
- **INET's `ApskErrorModel` has no FEC.** Its BER/PER are *uncoded*. Adding RLC/PDCP in the stack

does not change them. Use `ApskLayeredErrorModel` (bilevel) or Simu5G's HARQ + BLER curves if you want coding gains.

0.14.11. 10. Recommended configuration for a UAV-IoT persistent-coverage study

A defensible, internally consistent configuration:

Component	Choice	Rationale
Scenario	UMa, `URBAN_MACROCELL`	3GPP-calibrated PLE 2.2 / 3.908, σ_{SF} 4 / 6 dB
Carrier	2.4 GHz (or n3/n1 at 2.1 GHz)	inside [38.901] validity; ISM IoT realism
Bandwidth	200 kHz ($N_{\mathrm{RB}} \approx 13$ at $\mu = 0$)	IoT-appropriate; forces $N_0 = -121$ dBm
Height	$h_{BS} \in [20, 200]$ m, $h_{UT} = 1.5$ m	document the [38.901] h_{BS} extrapolation
LoS state	**custom** $P_{LOS}(h, d_{2D})$ from [AlH20]	the one thing [38.901] cannot supply
Path loss	UMa NLOS (1.3e) for NLOS; **UMa LOS (1.3b)** for LoS; (5.2a) as a cross-check	
Antenna	`AxiallySymmetricAntenna`, `gains = "0 18 32.5 15 65 -12 180 -30"`, nadir-pointing	real footprint, 5.4b
Shadowing	$\sigma_{SF} = 6$ dB (NLOS) / 4 dB (LoS), **spatially correlated**, $d_{cor} = 37/50$ m	2.2
Fast fading	Rician $K = 9 \pm 3.5$ dB (LoS), Rayleigh (NLOS); per-TTI or per-`correlationDistance`	2.4, 4.8
Receiver	`snirThreshold` derived from (8.11a) or `ApskErrorModel` + MCS schedule	not a magic number
Noise	`IsotropicScalarBackgroundNoise.power = -173.98 + 10log10(B) + F`	4.9
Propagation	`ConstantSpeedPropagation`, `propagationSpeed = 299792458mps`	
Error metric	SINR-threshold (fast) **and** PER-based (accurate)	4.6
Baselines	`UnitDiskRadio` (speed) + a LoS-only ideal radio (upper bound)	ablation

---

0.14.12. 11. Formula index (quick reference)

Quantity	Formula	Source
3-D slant range	$d_{3D} = \sqrt{d_{2D}^2 + (h_{BS} - h_{UT})^2}$	[38.901] (7.4-1)
Breakpoint (UMa/UMi)	$d'_{\mathrm{BP}} = 4h'_{BS}h'_{UT}f_c/c$	[38.901] Note 1
Breakpoint (RMa/SMa)	$d_{\mathrm{BP}} = 2\pi h_{BS}h_{UT}f_c/c$	[38.901] Note 5
UMa LOS	$28.0 + 22\log_{10} d_{3D} + 20\log_{10} f_c$ (below BP)	[38.901] T7.4.1-1
UMa NLOS	$13.54 + 39.08\log_{10} d_{3D} + 20\log_{10} f_c - 0.6(h_{UT} - 1.5)$	[38.901] T7.4.1-1

Quantity	Formula	Source
UMi NLOS	$35.3\log_{10} d_{3D} + 22.4 + 21.3\log_{10} f_c - 0.3(h_{UT} - 1.5)$	[38.901] T7.4.1-1
RMa LOS	$20\log_{10}(40\pi d_{3D} f_c/3) + \min(0.03h^{1.72}, 10)\log_{10} d_{3D} - \min(0.044h^{1.72}, 14.77) + 0.002\log_{10}(h) d_{3D}$	[38.901] T7.4.1-1
RMa NLOS	$161.04 - 7.1\log_{10} W + 7.5\log_{10} h - (24.37 - 3.7(h/h_{BS})^2)\log_{10} h_{BS} + (43.42 - 3.1\log_{10} h_{BS})(\log_{10} d_{3D} - 3) + 20\log_{10} f_c - (3.2(\log_{10}(11.75 h_{UT}))^2 - 4.97)$	[38.901] T7.4.1-1
SF correlation	$R(\Delta x) = \exp(-\Delta x/d_{cor})$	[38.901] (7.4-5), [P.1816]
Doppler	$f_{D,\max} = v f_c/c$	[Rappaport]
Coherence time	$T_c = 9/(16\pi f_{D,\max})$	[Rappaport]
Thermal noise	$N = kTB$, $N_0 = -173.98$ dBm/Hz @290 K	SI / [Rappaport]
SINR	$\mathrm{SINR} = P_r/(N + \sum P_i)$	—
Shannon	$R = B\log_2(1 + \mathrm{SINR})$	Shannon 1948
BPSK/QPSK BER	$\frac{1}{2}\operatorname{erfc}(\sqrt{E_b/N_0})$	[INET45] `BpskModulation.cc`
M-QAM BER (INET)	$\frac{4}{k}(1 - 1/\sqrt{M})\frac{1}{2}\operatorname{erfc}(\frac{1}{\sqrt{2}}\sqrt{\frac{3k}{M-1}E_b/N_0})$	[INET45] `MqamModulationBase.cc`
PER (INET)	$1 - (1 - P_b)^{L_H}(1 - P_b)^{L_D}$	[INET45] `ApskErrorModel.cc`
A2G path loss	$PL_{\mathrm{FS}}(d) + 20\log_{10}(\sqrt{d^2 + h^2}/d) + \eta$	[AlH14]
Antenna footprint	$r_{fp} = h\tan(\alpha/2)$	geometry
One-way delay	$t_T = t_2 - t_1$	[Y.1540], [RFC7679]
Delay variation	$\mathrm{IPDV}(T_1, T_2) = \lVert t_{T_1} - t_{T_2} \rVert$	[Y.1540], [RFC2680]
PDR	$1 - \frac{N_{\mathrm{lost}}}{N_{\mathrm{sent}}}$	[Y.1540]

0.14.13. 12. References

1. **3GPP TR 38.901**, *Study on channel model for frequencies from 0.5 to 100 GHz.* ETSI TR 138 901 V16.1.0 (2020-11); ETSI TR 138 901 V19.1.0 (2025-10). https://www.etsi.org/deliver/etsi_tr/138900_138999/138901/19.01.00_60/tr_138901v190100p.pdf
2. **3GPP TS 38.214**, *NR; Physical layer; Physical layer procedures for control* (Release 16/17).
3. **3GPP TS 38.211**, *NR; Physical channels and modulation.*
4. **3GPP TS 38.104**, *NR; Base station (BS) radio transmission and reception.*
5. **3GPP TR 36.814**, *Further advancements for E-UTRA physical layer aspect.*
6. **3GPP TR 36.873**, *Study on spatial channel model for frequencies from 2 to 6 GHz.*
7. **3GPP TR 36.842**, *Study on performance of 3GPP LTE radio interface.*
8. **3GPP TR 38.811**, *Study on NR to support non-terrestrial networks* (Release 15/16).
9. **3GPP TR 38.821**, *Solutions for NR to support Non-Terrestrial Networks (NTN)* (Release 16). https://www.3gpp.org/dynareport/38821.htm
10. **3GPP**, *Non-Terrestrial Networks (NTN)* technology overview. https://www.3gpp.org/technologies/ntn-overview
11. **ITU-R Recommendation P.1816** (cited by [38.901] 7.4.4).
12. **ITU-T Recommendation Y.1540**, *Internet protocol data communication service — IP packet transfer and availability performance parameters* (Dec. 2007; 2011 revision).
13. **ITU-T Recommendation Y.1541** (12/2011), *Network performance objectives for IP-based services.* https://www.itu.int/rec/T-REC-Y.1541/en
14. **IETF RFC 7679**, *A One-Way Delay Metric for IP Performance Metrics (IPPM).* https://datatracker.ietf.org/doc/html/rfc7679
15. **IETF RFC 2680**, *An IP Packet Delay Variation Metric for IPPM.*
16. **IETF RFC 5976**, *A Model for Networks Using Y.1541 Quality-of-Service Classes* (summarises the Y.1541 classes).
17. **IETF RFC 3550**, *RTP: A Transport Protocol for Real-Time Applications* (inter-arrival jitter).
18. **T. S. Rappaport**, *Wireless Communications: Principles and Practice*, 2nd ed., Prentice Hall, 2002.

19. **R. H. Clarke**, "A statistical theory of coverage in a randomly dispersed radio system," *IRE Transactions on Communications*, vol. 6, no. 2, pp. 389–397, April 1958.
20. **W. C. Jakes (ed.)**, *IEEE 802.11 Standard*, IEEE, 1997.
21. **A. Al-Hourani, S. Kandeepan, S. Lardner**, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Communications Letters*, vol. 3, no. 6, pp. 569–572, Dec. 2014. doi:10.1109/LWC.2014.2342736
22. **A. Al-Hourani**, "On the probability of line-of-sight in urban environments," *IEEE Wireless Communications Letters*, vol. 9, no. 8, pp. 1178–1181, Aug. 2020. doi:10.1109/LWC.2020.2985316
23. **A. Al-Hourani, S. Kandeepan, A. Jamalipour**, "Modeling air-to-ground path loss for low altitude platforms in urban environments," *IEEE GLOBECOM*, pp. 2898–2904, 2014.
24. **Li Chunjian**, "Efficient Antenna Patterns for Three-Sector WCDMA Systems," 2001 (cited by [INET45] `CosineAntenna.ned`, eq. B.1–B.2).
25. **INET Framework v4.5.0**, https://github.com/inet-framework/inet/tree/v4.5.0. Key files: `src/inet/physic` `src/inet/physicallayer/wireless/common/base/packetlevel/SnirReceiverBase.cc`; `src/inet/physicallayer` `src/inet/physicallayer/wireless/common/base/packetlevel/MqamModulationBase.cc`; `src/inet/physicallay`
26. **Simu5G v1.7.0**, https://github.com/inet-framework/Simu5G/tree/v1.7.0; documentation https://simu5g.org/. Key files: `src/simu5g/stack/phy/channelmodel/{StochasticChannelModel,Tr38901PathLossMod` `src/simu5g/stack/mac/amc/NrMcs.cc`; `src/simu5g/common/blerCurves/{BLERvsSINR_15CQI_TU.h,BLERvsSINR_1` `src/simu5g/common/carrierAggregation/ComponentCarrier.cc`.
27. **A. Radius et al.**, "Implementation of a Channel Model for Non-Terrestrial Networks in ns-3," *Proc. 12th ACM Conference on Systems and Networks (SenSys/NSDI companion)*, 2023. doi:10.1145/3592149.3592158
28. **H. Poddar, A. ...**, "Validation of 3GPP TR 38.901 Indoor Hotspot Path Loss Model Based on Measurements Conducted at 6.75, 16.95, 28, and 73 GHz," arXiv:2504.15589, 2025.
29. **T. Bai, P. Varshney, Z. Wu, A. Alshamleh, R. W. Heath Jr., D. Xu, H. K. Mehrpoust,** "Propagation Path Loss Models for 5G Urban Micro- and Macro-Cellular Scenarios," arXiv:1511.07311.
30. **Shannon, C. E.**, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.

All numeric values in this document were computed or verified against primary sources during compilation on 29 September 2026. Where a formula has more than one published parameterisation, the discrepancy is flagged in place rather than silently resolved.

Part VI — Toolchain

I've verified everything against primary sources. Three of your premises are out of date, and two "traps" you suspected are worse than you thought. Here's the reference.

0.15. Simulation Toolchain & Open-Source Simulators

Technical reference for MARL-based persistent, fault-tolerant UAV-IoT coverage

Compiled 29 September 2026. All versions and dates verified against official release pages, GitHub API, and upstream source trees — not blog posts.

0.15.1. 0. Executive summary — corrections before you start

Your premise	Reality as of 29 Sep 2026	Impact
"INET 4.5" is current	**INET 4.7.0**, released 7 Jul 2026, requires OMNeT++ ≥ 6.4. INET 4.5.4 is from **Oct 2024** — two years stale.	**High.** INET 4.5.4 is the version pinned by Simu5G's own CI, which creates a fork-in-the-road (see 3.3).
"OMNeT++ 6.2"	**OMNeT++ 6.4.0**, released 8 May 2026.	Medium.
AirSim is unmaintained	Worse than you thought: **Microsoft shut AirSim down in July 2022; its successor Colosseum was archived by the owner on 11 Jul 2026.** Both are hard dead ends.	**High.**
sb3-contrib MAPPO is low quality	Worse: **MAPPO and IPPO have been removed entirely from sb3-contrib.** Current `sb3_contrib` package contains only `ars`, `common`, `crossq`, `ppo_mask`, `ppo_recurrent`, `qrdqn`, `tqc`, `trpo`.	**High.** `from sb3_contrib import MAPPO` fails on every current version.
`pyopp` for Python-in-OMNeT++	**No `pyopp` exists in OMNeT++ 6.x at all.** Verified against the 6.4.0 source tree — zero matches. Embedded Python *NED modules* are impossible.	**High.** Rules out option (b) of your bridge design.
`ScalarRadio`, `IsotropicScalarRadioMedium`, `DmgErrorModel`, `IdealPropagation`, `BasicRangePropagation`, `DielectricBreakdownPropagation`, `TernaryPropagation`, `PERTableRadioErrorModel`, `TableBasedErrorModel`, `SimpleWirelessHost`	**All removed or renamed in INET 4.6** (released 20 Feb 2026) during a large wireless-PHY refactoring. Any tutorial or LLM-generated code using these will fail.	**High.** See 2.5.
Simu5G may be stale	**Very alive.** v1.7.0 released **14 Sep 2026** (two weeks ago).	—

Your premise	Reality as of 29 Sep 2026	Impact
sb3-contrib MAPPO quality	The class of concern is real but relocated — use the original `susanbao/mappo` PyTorch implementation or CleanRL.	—

One good piece of news: Simu5G v1.7.0 is *explicitly tested against INET 4.5.4 + OMNeT++ 6.3*, and declares compatibility with INET 4.6.0 and OMNeT++ 6.1–6.4. So INET 4.5.4 is not "wrong" — it is the *conservative, test-covered* choice. Pick your stack deliberately (11.2).

0.15.2. 1. OMNeT++ 6.4.0

1.1 Version and provenance

Field	Value
Current stable	**6.4.0**
Release date	**2026-05-08**
Release page	https://github.com/omnetpp/omnetpp/releases/tag/omnetpp-6.4.0
What's-new	https://github.com/omnetpp/omnetpp/blob/omnetpp-6.4.0/WHATSNEW.md
License	Academic Public License (free for academic use; commercial license = OMNEST)
Language	C++ (kernel + models), NED (topology), Python (analysis only)

Artifacts (`omnetpp.org/download`):

```
omnetpp-6.4.0-linux-x86_64.tgz       403MB  SHA256 2dd2cfbd388e50e843f5713716f065e878df7bb104b69f3435f7
omnetpp-6.4.0-linux-aarch64.tgz      402MB  SHA256 13099125c95f636f37fc4d23e2ed8c9fa81e8396e90879673e86a
omnetpp-6.4.0-windows-x86_64.7z     1153MB  SHA256 3c0957fe466332bcf441f132ac0fe102986c96699f50fee03abbc
omnetpp-6.4.0-macos-{aarch64,x86_64}.tgz  401/400MB
omnetpp-6.4.0-core.tgz                76MB  SHA256 db8de41f2503772aa627fbd99901078fd7a9db9942df9f284de40
```

Headless (recommended for CI/RL training):
docker run --rm -it -v "$(pwd):/root/models" -u "$(id -u):$(id -g)" ghcr.io/omnetpp/omnetpp:u26.04-6.4.0

> **Windows warning, and it is serious.** The OMNeT++ download page itself states the MinGW build is *"a LOT slower for development than the Linux or macOS versions"* due to Msys POSIX emulation and NTFS. If you are on Windows (and the working directory here suggests you are), **use WSL2**. Get `opp_env.wsl` (186 MB) from the download page, or install the Linux tarball under WSL. Simulation throughput differences of 3–10× are normal, and that difference is fatal for RL training budgets.

1.2 Installation — `opp_env` is now the blessed path

OMNeT++ 6.4 pushes `opp_env` over the legacy `opp_env install` / tarball `install.sh` flow.

Linux / macOS pip install opp-env mkdir -p ~/omnetpp-workspace && cd ~/omnetpp-workspace opp_env init opp_env install omnetpp-6.4.0 opp_env install inet-4.7.0 # or simu5g-1.7.0 opp_env run inet-4.7.0 --build # build + resolve deps

Windows: download opp_env.wsl, then inside WSL2 (need WSL ≥ 2.4.4) opp_env install omnetpp-6.4.0

1.3 Three ways to work

(a) IDE (Eclipse-based). Best for *authoring*: NED editor with completion, C++ CDT, the Qtenv visualizer, built-in Result Analysis (`.anf` chart files), NED inheritance view, inline documentation tooltips, and the INET installer (`Help  Install Simulation Models...`). You will not run your RL training from the IDE.

(b) `opp_run` — the headless command-line kernel. This is your training workhorse.

```
opp_run [options] -f -l <inifile> -c <configname> [-r <runnumber>] [-n <nedpath>]

  -l, --ini-file=<file>       ini file (required)
  -c, --config=<name>         configuration name; accepts globs ('Wireless*') and '**' for all
  -r, --run-number=<n>        run index within the config; 0 = seed-set 0
  -n, --ned-path=<path>       colon-separated list of NED source roots (repeatable)
  -f, --force                 overwrite existing result files without asking
  -u, --cmdenv                use the console user interface (default when no -q/IDE attached)
  -q, --quiet                 quiet console: 'none' | 'cmdenv'
      --cmdenv-express-mode=<bool>   terse progress (set true in scripts)
      --sim-limit=<time>      e.g. --sim-limit=100s   (overrides sim-time-limit)
      --seed=<n>             RNG seed (0 => derive from run number)
      --seed-set=<n>         seed-set, -1 = repeat the last run
  -s, --seed-set=<n>          short form
  -h, --help                  full option list for your build
      --version               kernel version string
      --result-dir=<dir>      override result directory
```

Exact batch commands you will use:

one config, one run, overwrite results opp_run -u Cmdenv -f -l omnetpp.ini -c Baseline

every configuration in the ini, run 0 opp_run -u Cmdenv -f -l omnetpp.ini -c '**'

parameter sweep, all runs of one config, parallel across cores opp_runall -l omnetpp.ini -c 'UAVSweep' # 1 job per logical CPU opp_runall -l omnetpp.ini -c 'UAVSweep' -j 4 # or cap it

the canonical 10-runs-per-seed-set statistical batch (this is your unit of evidence) opp_runall -l omnetpp.ini -c 'General' -r 0-9

sweep two parameters x 3 seeds x 10 runs, headless opp_runall -l omnetpp.ini \ -c 'General' \ --numjobs=10 \ -s 0-2 \ '**.numUavs'={$numUavs : 3, 5, 7$}\' * *.$lambda'$ ={lambda:0.5,1.0,1.5}

> `opp_runall` is the parallel *batch* runner (fork-per-configuration on one machine). It is **not** the same thing as OMNeT++'s distributed-parallel simulation (MPI / `opp_run -h | grep parsim`). For 5 UAVs you want neither MPI nor the parsim scheduler; `opp_runall` is the right tool.

(c) `opp_repl` / `opp_python_repl`. New in 6.4: a Python REPL with the omnetpp libraries imported, exposing `read_result_files()`, `run_simulations()`, parameter optimization, and an MCP server for AI assistants.

pip install opp-repl # v0.4, released 11 May 2026 — external package, LGPLv3+ . /path/to/omnetpp/setenv opp_repl --load @opp # or --load "opp/*.opp" # In [1]: run_simulations(simulation_project = fifo_project)

`opp_repl` also ships CLI tools and is the right thing for sweeping a parameter outside the ini file. It also provides an MCP server, so you can drive batch experiments from an LLM agent — but **do not let an LLM pick your experimental design**; see 10.5.

1.4 Build toolchain — what a project needs

A minimal OMNeT++ project is **four files**:

```
myproject/
 .project                 # Eclipse project descriptor (only if you want the IDE)
 Makefile.inc             # drives opp_makemake  $\leftarrow$ the only file you must hand-write
 src/
    Makefile             # GENERATED by opp_makemake --- never edit, never commit-by-hand
    MyModule.cc/.h       # your cSimpleModule subclass
    ... (recursive subdirs)
 MyNetwork.ned            # topology
 omnetpp.ini              # configuration
```

The mechanism (this trips everyone up once): there is no committed `src/Makefile`. `opp_makemake` *generates* it, and the generated file is what `make` actually consumes. The `Makefile.inc` records the `opp_makemake` command line so the file is reproducible.

Pure OMNeT++ `Makefile.inc`:

A. Build mode MODE = release

B. Compile-time flags CXXFLAGS += -std=c++20

C. Source/build config opp_makemake --makefile -f --force \ -I../src \ -O($MODE$)\−(MODE)

D. Include paths INCLUDE_PATH = ../src INCLUDE_PATH += -I.
E. Libraries opp_makemake --make-so --recursive
INET-backed `Makefile.inc` (this is the file you need for the UAV project):
A. Build mode MODE = release
B. Toolchain PREFER_CLANG = no # PREFER_CLANG = yes # uncomment if you have a Clang toolchain; often 20-30% faster
C. Output / optimization CXXFLAGS += -std=c++20 -O3 -march=native -flto # NOTE: -march=native breaks reproducibility across heterogeneous machines. # Use it for the FINAL training runs, not for anything you need to reproduce elsewhere. LDFLAGS += -flto
D. Source config opp_makemake --makefile -f --force --recursive -O($MODE$)−(MODE)
E. Include paths — INET INET = ../../inet-4.7.0 INCLUDE_PATH = ../src INCLUDE_PATH += -I($INET$)/$srcINCLUDE_PATH+ = -DOMNETPP_TRACE-DINET_DEBUG$
F. Libraries opp_makemake --make-so LIBS += ($INET$)/$src/INET$
Then:
cd src && opp_makemake # generates src/Makefile from Makefile.inc make MODE=release -j(*nproc*)#*or* : $makeMODE = debug-j$(nproc) (LLDB-debuggable)

> **The modern alternative: `.oppbuildspec`.** OMNeT++ 6.x and INET now let the IDE and `opp_env` generate makefiles from a declarative file instead, so include paths come from *project dependencies* rather than hardcoded relative paths. INET's own `.oppbuildspec` is:
>
> <?xml version="1.0" encoding="UTF-8" standalone="no"?> <buildspec version="4.0"> <dir path="." type="custom"/> <dir makemake-options="--make-so --deep -o INET -O out -pINET --meta:export-include-path --meta:export-library --meta:use-exported-libs --meta:feature-cflags --meta:feature-ldflags" path="src" type="makemake"/> </buildspec>
>
> with a top-level `Makefile` whose `makefiles` target extracts the options with `sed` and re-invokes `opp_makemake`. **Recommendation for a student project: hand-write `Makefile.inc`.** It is one file, it is greppable, it does not depend on the IDE, and it is the only form that survives being cloned on someone else's machine.

Build modes and gotchas: - `MODE=release` → `-O3 -DNDEBUG`, fast. **This is what you benchmark.** `MODE=debug` → `-O0 -ggdb3`, ~10–30× slower; unusable for training. - `opp_makemake` needs `opp_makemake` on `PATH`; source `$(OMNETPP_ROOT)/setenv` first. - `opp_featuretool` regenerates `src/inet/features.h` from `.oppfeatures` / `.oppfeaturestate` if you enable INET project features (e.g. `Simu5G_D2D`). - `opp_neddoc -f <selfdoc>.xml .` generates NED documentation. Use it once to confirm every type you instantiate actually resolves.

1.5 NED language — a real, minimal, working example

NED has exactly four composite constructs. Here is a complete two-module system you can compile.
`src/Ticker.cc`
#include <omnetpp.h> #include "Ticker.h"
using namespace omnetpp; using namespace std;
Define_Module(Ticker);
void Ticker::initialize(int stage) { EV << "Ticker init, stage " << stage << " of " << getSimulation()->getNumStages() << "\n"; if (stage == 0) timer = scheduleAt(simTime() + par("period").doubleValue(), new cMessage("tick")); }
void Ticker::handleMessage(cMessage *msg) { cOutVector &v = recordScalar("ticks"); v.record(simTime()); counter++; recordScalar("tickCount", counter); EV << "tick #" << counter << " at t=" << simTime() << "\n"; if (simTime() < getSimulation()->getTerminationTime()) scheduleAt(simTime() + par("period").doubleValue(), msg); }
void Ticker::finish() { recordScalar("totalTicks", counter); }
`src/Ticker.h`
#ifndef TICKER_H_ #define TICKER_H_ #include <omnetpp.h> using namespace omnetpp; class Ticker : public omnetpp::cSimpleModule { protected: void initialize(int stage) override; void handleMessage(omnetpp::cMessage *msg) override; void finish() override; private: cMessage *timer = nullptr; int counter = 0; }; #endif
`src/Sink.cc`
#include <omnetpp.h> using namespace omnetpp; Define_Module(Sink); void Sink::handleMessage(cMessage *msg) { recordScalar("received", 1); delete msg; }
`src/Sink.h`

#ifndef SINK_H_ #define SINK_H_ #include <omnetpp.h> using namespace omnetpp; class Sink : public omnetpp::cSimpleModule { protected: void handleMessage(omnetpp::cMessage *msg) override; }; #endif

`TickerSink.ned`

// // Package-less: the package is taken from the enclosing folder name. // To declare a package explicitly, uncomment and adapt the line below. // package myproject; // <-- folder path myproject/ must match

// // A simple module: 1 instance per gate-port connection tree, C++-backed. // The [Ticker] is the C++ class name. One NED type maps to exactly one class // (or a class hierarchy via @class / -- a NED type can bind to a subclass). // simple Ticker { parameters: double period = 1s; // default value; overridable from .ini int id = -1; // unassigned by default gates: inout io; // bidirectional gate submodules: console: <default("")> if typename != "" { } // conditional submodule }

simple Sink { gates: input in; submodules: console: <default("")> if typename != "" { } }

// // A compound module: assembly of submodules + connections. No C++ backing // (unless you add @class). Reusable via `extends` / `like`. // module TickerSinkNode { parameters: double period = 1s; @display("p=100,100;b=200,200"); gates: inout io; // gates are NOT automatically connected submodules: ticker: Ticker { parameters: period = parent.period; // keyword `parent` = enclosing module } sink: Sink { parameters: @display("p=0,0"); } connections: ticker.io <--> sink.in; // <--> is a shorthand for a <--> gate pair }

// // A network: a compound module with no gates. Exactly one per run. // Vector submodules let you instantiate N copies with [k] in ini. // network TwoNodes { submodules: nodeA: TickerSinkNode { parameters: period = 1s; @display("p=0,0"); } nodeB: TickerSinkNode { parameters: period = 2s; @display("p=300,0"); } nodeC[3]: TickerSinkNode { // 3 instances: nodeC[0..2] parameters: period = 0.5s; @display("p=600,0"); } }

NED grammar essentials (the parts people get wrong):

Element	Syntax	Notes
Simple module	`simple Foo { ... }`	Backed by C++ class. One class per NED type; use `@class(X)` to decouple.
Compound module	`module Foo { ... }`	Submodules + connections. Reusable.
Network	`network Foo { ... }`	A compound module with no gates. Exactly one instantiated per run.
Channel	`channel Foo { ... }`	Connection data: `delay`, `datarate`, `error`.
Parameters	`double x = 1s;`	Types: `int`, `double`, `bool`, `string`, `object`. Units `@unit(mps)`, `@unit(dBm)`. Unassigned params have **no** value and cannot be read.
Gates	`input in; output out; inout io;`	Must be connected explicitly in the parent — **not** recursive by default.
Connections	`a.out --> b.in;`	`-->`, `<--`, `<-->`; optional channel: `a.out --> { delay = 5ms; } --> b.in;`
Vectors	`node[3]: Foo;`	Instantiate 3; `node[1]` is a valid path index in ini.
`extends` / `like`	`module B extends A { ... }`	NED inheritance. `like I` = "any type implementing interface I" (module interfaces).
`default()`	`<default("X")>`	Selects the module type *if the parameter is unset*, else overrides.
`if typename != ""`	—	Conditional submodule; the canonical way to make visualizers optional in headless runs.
`par("x")`	—	NED function to read a parameter inside an expression. New in INET 4.6.

Element	Syntax	Notes
`seq("k")`	—	Deterministic integer per string key, evaluated per NED component. New in 4.6.
`iterate(arr)`	—	Next value from an array per evaluation; index tracked per context path. New in 4.6.
`pyeval("...")`	—	Evaluate a Python expression (or lambda call). New in 6.1. `pycode("...")` evaluates a Python *function body*. Parameterization only — **not** module logic.

`parent.period` resolves the enclosing compound's parameter. `unit()` converts units. `@display` controls only visualization, never semantics — safe to ignore in headless runs.

1.6 `omnetpp.ini`

A real, minimal ini with everything you asked about:

== # omnetpp.ini — configuration file # Sections: [Config <Name>] -> one "configuration" = one run family # [General] -> defaults inherited by all configs # [Output] -> legacy, still honoured for scalar recording # ==

[General] # ---- NED source roots (relative to the ini file). '.' == the project root. ned-folders = .{*INET_ROOT*}/*src*

---- Simulation kernel -- sim-time-limit = 300s # hard stop; module returns nothing before this sim-time-resolution = 1us # 6.0+ syntax: <int> followed by a unit cmdenv-express-mode = true # terse progress, machine-friendly cmdenv-event-banner-details = true # per-event logging (DEBUG ONLY — huge logs) cmdenv-output-file = {*resultdir*}/{configname}.out

---- Random number generation --------------------------------- # seed-set N derives the actual seed as seed = seed-set * NUM_RNG + rngIndex. # The default NUM_RNG is 1 in 6.0+ (was 2 in <=5.x), so seed-sets are 0,1,2,... seed-set = 0 # seed = 0 # leave UNSET; setting it makes every run identical # cmdenv-rng-seed = 0 # named RNG stream

---- Result recording --- output-scalar-file = results/{*configname*}.*scaoutput−ve* *results*/{configname}.vec record-scalar = false # global master switch record-vector = false record-with-label = true # store parameter values in the .sca # 4.x-legacy spelling, still works: # record-scalars = false # record-vectors = false

---- Default experiment label --------------------------------- *.simulator-limit = 10000 # safety: abort after 10k events/sec-check failure

The `seed = 0` line above is a **trap I want to flag loudly**: setting `seed` explicitly in `[General]` makes *every* run bit-identical, which silently destroys all your variance. Set `seed-set`, leave `seed` unset. INET 4.7 also adds a *configurable RNG grouping mechanism* — if you use it, you must document the grouping, because a different grouping silently changes the results.

Full per-run config for a UAV coverage experiment:

[Config General] description = "Baseline: greedy persistent coverage, 5 UAVs, 1 gNB" network = UavCoverageNet sim-time-limit = 600s seed-set = {*repetition*}*cmdenv−event−banner−details* = *false*

---- Experiment index bookkeeping (always record this) -------- *.experiment = "{*configname*}"∗ *.repetition* ={repetition} *.configName = {*configname*} ∗ *.runNumber* ={runnumber} *.simTimeLimit = {*simtime−limit*}*s* ∗ *.gitCommit* = "*REPLACE_WITH_GIT_SHA*"#*pinnedatcontainerbuild*

---- Scenario --- *.numUavs = 5 *.uavAreaX = 1000m *.uavAreaY = 1000m *.uavAreaZ = 120m *.uavSpeed = 12mps

---- Radio (INET 4.6+ API — see 2.5) ------------------------- *.radioMedium.signalAnalogRepresentation = "scalar" *.radioMedium.backgroundNoise.power = -110dBm *.radioMedium.pathLoss.typename = "TwoRayGroundReflection" *.radioMedium.propagation.typename = "ConstantSpeedPropagation" *.radioMedium.propagation.Propa = 299792458mps *.radioMedium.mediumLimitCache.centerFrequency = 2.4GHz *.radioMedium.mediumLimitCache.rangeFi = "interferenceRange"

*.uav[*].wlan[0].radio.signalAnalogRepresentation = "scalar" *.uav[*].wlan[0].radio.typename = "ApskRadio" *.uav[*].wlan[0].radio.centerFrequency = 2.4GHz *.uav[*].wlan[0].radio.bandwidth = 20MHz *.uav[*].wlan[0].radio.trans = 100mW # 20 dBm *.uav[*].wlan[0].radio.transmitter.preambleDuration = 10us *.uav[*].wlan[0].radio.transmitter.headerLe

= 8B *.uav[*].wlan[0].radio.receiver.sensitivity = -101dBm *.uav[*].wlan[0].radio.receiver.energyDetection = -96dBm *.uav[*].wlan[0].radio.receiver.snirThreshold = 4dB *.uav[*].wlan[0].radio.errorModel.typename = "StochasticErrorModel"

---- Mobility ---------- *.uav[*].mobility.typename = "MassMobility" *.uav[*].mobility.speed = {*uavSpeed*}∗.*uav*[∗].*mobility.changeInterval* = *normal*(5*s*, 0.1*s*)∗.*uav*[∗].*mobility.angleDelta* = *normal*(0*deg*, 30*deg*)

---- Recording (per module) ---------- *.uav[*].cOutVector.coverageFraction.record = true *.uav[*].cOutVector.coverageFraction.record-interval = 1s *.uav[*].cOutVector.energyRemaining.record = true *.uav[*].cOutVector.energyRemaining.record-interval = 1s *.uav[*].cOutVector.linkSnir.record = true *.uav[*].cOutVector.linkSnir.record-interval = 0.1s *.uav[*].scalar_ throughputDl = {*record* = *true*}∗.*gnb.scalar_throughp* *.control.scalar_ globalCoverageFraction = {*record* = *true*}

---- Output paths (see 1.7) ---------- *.result-dir = results/{*configname*} ∗ .*result*−*file*−*name*−*pattern* ={resultdir}/{*resultname*}−{runnumber}.{*resultnameextension*}

1.7 Where results land, and how to read them

Default (OMNeT++ 6.x) `result-file-name-pattern`**:**

```
${resultdir}/${configname}/${resultname}-${runnumber}.${resultnameextension}
```

So for `opp_run -l omnetpp.ini -c General -r 0` with default `result-dir=results`:

```
results/General/General-0.sca      <- scalars
results/General/General-0.vec      <- vectors
results/General/General-0.elog     <- eventlog (if eventlog-recording is on)
results/General.oppnamemapping     <- maps opaque names to human-readable ones
```

Older INET 4.5-era projects often still use the flat 4.x pattern `${resultdir}/${resultname}-${configname}-${runnumb` **Set it explicitly in your ini** so you know, and so downstream scripts don't guess.

`.sca` is a line-oriented ASCII format: one scalar per line, `statistic name module-path value` (with `statistic` = `max`/`min`/`mean`/`sum`/`stddev`/`count`/`histogram`/`vector`/`field`/`summary`). `.vec` is a separate time-series format — each vector is *in a different file position* referenced by a `vector` block, which is why you must keep `.sca` and `.vec` from the same run in the same directory.

Read them from Python — the supported way, no parsing:

from omnetpp.scave import results

wildcard patterns: OMNeT++ glob, NOT shell glob files = results.read_files("results/General/*.sca", "results/General/*.vec")

rows = [r for r in files if r.module == "control" and r.statistic == "globalCoverageFraction"] # r has .name, .module, .statistic, .value, .module_type, .run, .experiment, .configname

final = [r.value for r in rows if r.statistic == "mean"] print(f"n={len(final)} mean={sum(final)/len(final):.4f}")

time series series = {} for r in files: if r.statistic == "vector" and r.name == "coverageFraction": series.setdefault(r.module, []).append((r.value[0], r.value[1]))

opp_scavetool x results/General/General-0.sca # dump to stdout opp_scavetool c results/General/ # merge a directory tree opp_neddoc / opp_charttool # chart generation

> **Never write your own `.sca` parser for a paper.** The format has changed across versions and the fragile parsers on Stack Overflow silently mis-associate run numbers with configurations. Use `omnetpp.scave`.

1.8 NED path resolution

Three mechanisms, in precedence order:

1. `-n` / `--ned-path` **on the command line.** Colon-separated. Overrides everything. Use this for CI: opp_run -n .:*INET_ROOT*/*src*−*lomnetpp.ini*−*cGeneral*

2. `.nedfolders` **file** in the working directory. One path per line, # comments allowed. This is the mechanism the `nedfolders = ...` ini key *generates* — `opp_makemake`/the IDE writes `.nedfolders` from the ini so the NED editor can resolve types without launching the kernel.

3. `${NEDPATH}` — compiled-in default, set by `opp_makemake` from `-I` flags, plus the `@omnetpp-root/omnetpp` package.

Package structure rules: - A folder is a NED package if it contains `.ned` files. The package name is the folder path relative to the NED root, with `/` as separator. - If a `.ned` file contains an explicit `package a.b.c;` declaration, it must match the folder path under the NED root, or the type is silently invisible. - **The root package is unnamed** — a `.ned` file at a NED root with no `package` line is in the root package. INET 4.6's tutorial files look like this: `ned  package inet.tutorials.wireless;` and live at `<INET>/src/inet/tutorials/wireless/*.ned`. This is why the standard `-I` is `$(INET)/src` and not `$(INET)/src/inet`. - Two NED roots can both contain a package named `x.y`; the one earlier in the NED path wins, silently. **Pin your NED path order explicitly** and never rely on the IDE default. - `.nedexclusions` — a file listing files to skip during NED loading (used by INET to exclude `src/inet/transportlayer/tcp/nsc` etc.). - `.oppfeatures` / `.oppfeaturestate` — OMNeT++ 6 project features: conditionally *include* NED subtrees. Controlled by `opp_featuretool`, and gated by `WITH_<FEATURE>` preprocessor macros. This is how Simu5G's `Simu5G_D2D` feature works.

0.15.3. 2. INET 4.7.0 (and 4.6.0 / 4.5.4)

2.1 Versions and compatibility — the actual support matrix

From https://inet.omnetpp.org/Download.html (verbatim):

OMNeT++	INET
6.4	**4.7.0**, master branch
6.3	4.6.0, 4.5.4
6.2	4.6.0, 4.5.4
6.1	3.8.5, 4.5.4
6.0.x	3.8.5, 4.2.10, 4.3.9, 4.4.2, 4.5.4

Release	Date	Note
INET 4.7.0	**2026-07-07**	IPv6 maturity (MLD, PIM-DM/SM over v6, DAD, PMIPv6, MP-BGP, IPsec-v6), satellite + GNSS-track mobility, STP/RSTP overhaul, TCP Path MTU Discovery, **configurable RNG grouping**. Requires OMNeT++ $\geq$ 6.4.
INET 4.6.0	**2026-02-20**	QUIC, Ethernet 10BASE-T1S/PLCA, IPsec, MRP, clock model rewrite, gPTP reimplementation, **wireless PHY refactoring**. Requires OMNeT++ $\geq$ 6.2.
INET 4.5.4	2024-10-29	OMNeT++ 6.1 compat, memory-leak fixes, `python/requirements.txt`.
INET 4.5.2	2023-08-29	
INET 4.5.0	2023-04-04	

License: **LGPL** since 4.4.0. The old GPL-encumbered components (TCP NSC, packetdrill, HTB, HTTP tools) were moved to a separate `inet-gpl` repo.

For your project, the choice is:

Stack	When	Verdict
OMNeT++ 6.4 + INET 4.7.0	New project, no Simu5G. GNSS/satellite mobility and the RNG grouping are both directly relevant to UAV work.	**Best default.**
OMNeT++ 6.3 + INET 4.6.0	Want a released pair with a wireless PHY that supports `unitDisk` (huge RL speedup, 2.5).	**Recommended if throughput is the binding constraint.**
OMNeT++ 6.3 + INET 4.5.4	Only if Simu5G is mandatory (3.3).	Acceptable, but two years stale and 4.6's PHY work is unavailable to you.

2.2 What INET provides (the parts you need)

IPv4/IPv6 (+ autoconfiguration), UDP/TCP (+ QUIC in 4.6)/SCTP/DCCP, Ethernet (+ 10BASE-T1S in 4.6) / VLAN / STP-RSTP / MRP, PPP, IEEE 802.11 (full MAC incl. EDCA, fragmentation, block-ack, aggregation, rate control, handover, TXOP), IEEE 802.15.4 (narrowband + UWB-IR), the wireless PHY stack, an extensive queueing library, routing (RIP/OSPF/BGP/EIGRP/IS-IS/AODV/DSR/GPSR/DSR), TSN, mobility (static/single/waypoint/Gauss-Markov/Mass/linear/router/traffic), `PhysicalEnvironment` (terrain, buildings, obstacles), `PowerConsumption`, `NodeFailureController`, and the visualizer framework.

2.3 Node types

NED type	Derives	Notes
`NodeBase`	—	`INetworkNode`; mobility + power submodules.
`StandardHost`	`NodeBase`	802.11 + Ethernet + IP. `numWlanInterfaces` default **0**.
`WirelessHost`	`StandardHost`	802.11 in **infrastructure** mode, needs an `AccessPoint`. `numWlanInterfaces` default **1**.
`AdhocHost`	`WirelessHost`	802.11 **ad-hoc** mode, forwarding on. ← **This is the UAV type you want.**
`Router` / `AodvRouter` / `Ipv6Router`	`NodeBase`	Routing + forwarding.
`StandardHost6` / `WirelessHost6`		MIPv6 variants.

> `SimpleWirelessHost` **does not exist.** I verified the full INET 4.6.0 tree: under `src/inet/node/inet/` there are exactly `AdhocHost.ned`, `StandardHost.ned`, `WirelessHost.ned`. Any LLM that gives you a `SimpleWirelessHost.ned` is hallucinating. Likewise, `AdhocHost` is the correct UAV base, not `WirelessHost` — a swarm with a single AP is not a swarm.

To get multi-radio UAVs:
*.uav[*].numWlanInterfaces = 2 # the documented way (INET User's Guide, ch. 4)

2.4 The INET 4.6 wireless PHY refactoring — read this before writing any NED

INET 4.6 replaced *type selection by module name* with *a single* `signalAnalogRepresentation` *parameter.* Verbatim from the INET 4.6 whatsnew:

> The wireless physical layer was significantly refactored to simplify how the analog representation of radio signals is selected. Instead of using separate module types, radio and medium modules now use a `signalAnalogRepresentation` parameter (unitDisk, scalar, or dimensional) to control their behavior. Responsibility for computing reception success was partially moved from receiver modules to their analog model submodules. To reflect broader applicability, much of the former UnitDisk-specific implementation was replaced by generic components, resulting in modules such as `GenericRadio`, `GenericTransmitter`, and `GenericReceiver`, while UnitDisk remains available as an analog domain representation.

I enumerated every `.ned` file under `src/inet/physicallayer/wireless/` in the v4.6.0 tree. Complete result below.

Media (the "shared air"):

NED type	Status	Notes
`RadioMedium`	**NEW in 4.6 — the base.**	Has `signalAnalogRepresentation` @enum("unitDisk","scalar","dimensional"), default `"scalar"`.
`ScalarRadioMedium`	**replaces** `IsotropicScalarRadioMedium`	Literally just `extends RadioMedium { signalAnalogRepresentation = default("scalar"); }`
`DimensionalRadioMedium`	exists	`signalAnalogRepresentation = default("dimensional")`
`UnitDiskRadioMedium`	exists	`signalAnalogRepresentation = default("unitDisk")`
`Ieee80211ScalarRadioMedium` / `Ieee80211RadioMedium` / `Ieee80211DimensionalRadioMedium`	exist	802.11-specialized presets
`Ieee802154NarrowbandScalarRadioMedium` / `Ieee802154UwbIrRadioMedium`	exist	802.15.4

Radios:

NED type	Status	Notes
`GenericRadio`	**NEW in 4.6**	`transmitter.typename = default("GenericTransmitter")`, `receiver.typename = default("GenericReceiver")`. Behaviour from `signalAnalogRepresentation`.
`GenericScalarRadio` / `GenericDimensionalRadio` / `GenericUnitDiskRadio`	**NEW**	~260-byte NED presets that only set the parameter.
`ApskRadio`	**NEW**	Scalar/modulated PSK radio — **this is your UAV radio**.
`ShortcutRadio`	exists (since 4.2)	**Bypasses the medium entirely.** Direct radio-to-radio delivery with `bitrate`, `propagationDelay`, `packetLoss`, `lengthOverhead`, `durationOverhead`. **The fastest option for RL; see 11.2.**
`ApskScalarRadio` / `ApskDimensionalRadio` / `ApskUnitDiskRadio` / `ApskLayeredRadio`	exist	
`Radio`	exists	`PhysicalLayerBase` subclass.
`FlatRadioBase` / `NarrowbandRadioBase`	**NEW in 4.6**	Split the old `Radio` hierarchy by analog representation.
`Ieee80211Radio` + `{Scalar,Dimensional,UnitDisk}Radio` presets	exist	
`Ieee802154NarrowbandRadio` / `Ieee802154UwbIrRadio`	exist	
`ScalarRadio`, `DimensionalRadio`, `IsotropicScalarRadio`, `IsotropicDimensionalRadio`	[X] **REMOVED**	Use `GenericRadio` + `signalAnalogRepresentation`, or the thin `Generic*Radio` presets.

Propagation (delay only):

NED type	Status
`ConstantSpeedPropagation`	[OK] **The default.** Parameter `PropagationSpeed`, default 299792458 m/s.
`ConstantTimePropagation`	[OK] Fixed `propagationDelay`.
`IdealPropagation`, `BasicRangePropagation`, `DielectricBreakdownPropagation`, `TernaryPropagation`	[X] **ALL REMOVED in 4.6.**

This is the single most likely thing to bite you, because `IdealPropagation` is what almost every tutorial, StackOverflow answer, and LLM uses for a delay-less wireless link. **Do not use them.** In 4.6 a "delay-less" link means `ConstantSpeedPropagation` with a physical `PropagationSpeed` — which is what you want anyway, because propagation delay is *physically* what makes a UAV-IoT coverage problem non-trivial.

Path loss (attenuation only — these are `IPathLoss` submodules, chained):

NED type	Status
`FreeSpacePathLoss`	[OK] default
`TwoRayGroundReflection`	[OK] **The right choice for a ground-reflecting UAV link.**
`TwoRayInterference`, `BreakpointPathLoss`, `SuiPathLoss`, `UwbIrStochasticPathLoss`	[OK]
`LogNormalShadowing`, `RayleighFading`, `RicianFading`, `NakagamiFading`	[OK] Separate `IPathLoss` submodules you *chain* onto the base path loss. Note INET 4.5.2 fixed a **Rayleigh** formula bug: *"NOTE: This change may alter a simulation's results if it used the Rayleigh pathloss model."*
`DielectricObstacleLoss`, `IdealObstacleLoss`	[OK] (obstacle loss ≠ propagation; `DielectricObstacleLoss` **survives** — do not confuse it with the removed `DielectricBreakdownPropagation`)

Antennas: `IsotropicAntenna`, `ConstantGainAntenna`, `CosineAntenna`, `DipoleAntenna`, `AxiallySymmetricAntenna`, `ParabolicAntenna`, `InterpolatingAntenna`. For UAV, `ConstantGainAntenna` with a `gain` in dB (a downward-tilted directional antenna is the physically right answer and is a one-line change from isotropic).

Background noise: `IsotropicScalarBackgroundNoise` (parameter `power`, default `-92dBm`), `IsotropicDimensionalBa`

Error models:

You asked for	Reality in INET 4.6
`DmgErrorModel`	[X] Renamed → `StochasticErrorModel`. Parameters: `packetErrorRate`, `bitErrorRate`, `symbolErrorRate` (all `default(nan)`).
`PERTableRadioErrorModel`	[X] **Removed.**
`TableBasedErrorModel`	[X] **Removed** as a generic type. Technology-specific table models remain: `Ieee80211BerTableErrorModel`, `Ieee80211NistErrorModel`, `Ieee80211YansErrorModel`, `Ieee80211OfdmErrorModel`, `Ieee802154ErrorModel`.
—	[OK] `StochasticLayeredErrorModel` (bit-level, was also `DmgErrorModel`)
—	[OK] `ApskErrorModel`, `ApskLayeredErrorModel`
Bases	`ErrorModelBase`, `LayeredErrorModelBase`, `IErrorModel`, `ILayeredErrorModel`

Caches: `MediumLimitCache` (parameter `centerFrequency`, `rangeFilter`), `VectorCommunicationCache` (default), `MapCommunicationCache`, `ReferenceCommunicationCache`, `QuadTreeNeighborCache`, `NeighborListNeighborCac `GridNeighborCache`.

2.5 A real INET 4.6 UAV NED file

Based on the actual `tutorials/wireless/WirelessA.ned` and `omnetpp.ini` from INET v4.6.0, with UAVs substituted in.

`src/UavCoverage/UavSwarmNet.ned`

// // SPDX-License-Identifier: LGPL-3.0-or-later // package uavproject.UavCoverage;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator; import inet.node.contract.INetworkNode; import inet.node.inet.AdhocHost; import inet.node.inet.StandardHost; import inet.physicallayer.wireless.common.medium.Sc import inet.visualizer.contract.IIntegratedVisualizer;

// // A single UAV. AdhocHost is correct: no AP, forwarding on, ad-hoc 802.11. // The `wlan[0].radio` subtree is reconfigured in omnetpp.ini to ApskRadio. // module UavNode extends AdhocHost { parameters: int uavId = default(-1); double patrolSpeed @unit(mps) = 12mps; @display("p=60,60;b=400,400;bgl=2"); submodules: // INET's built-in mobility, repositioned. LinearMobility/MassMobility // are the NED type names; the C++ classes are LinearMobilityModule // and MassMobilityModule. See 2.7. mobility.typename = "MassMobility"; }

module GnbNode extends StandardHost { parameters: @display("p=60,60;b=400,400"); }

network UavCoverageNet { parameters: int numUavs = 5; double areaX @unit(m) = 1000m; double areaY @unit(m) = 1000m; double areaZ @unit(m) = 120m; int numSensors = 50; @display("bgb=1600,900;bgg=1,1,grey97");

@figure[title](type=label; pos=0,-1; anchor=sw; color=darkblue); @figure[legend](type=legend; pos=1600,0; anchor=nw; verticalPlacement=top; horizontalPlacement=right);

submodules: // Empty typename in headless batch runs; the IDE fills it in for Qtenv. visualizer: <default(firstAvailableOrEmpty("IntegratedCanvasVisualizer"))> like IIntegratedVisualizer if typename != "" { @display("p=1500,180"); } configurator: Ipv4NetworkConfigurator { parameters: @display("p=1500,250"); } physicalEnvironment: <default(firstAvailableOrEmpty("PhysicalEnvironment"))> { parameters: @display("p=1500,320"); } } // ScalarRadioMedium is the INET 4.6 replacement for // IsotropicScalarRadioMedium. The old name will not resolve. radioMedium: ScalarRadioMedium { parameters: @display("p=1500,390"); } gnb: GnbNode { parameters: @display("p=areaX/2,0"); }

uav[numUavs]: UavNode { parameters: uavId = default(index); @display("p=" + string(200 + 250*index) + ",600"); } sensor[numSensors]: StandardHost { parameters: @display("p=0,0"); } connections: // Sensor-to-gNB association is left to the routing protocol; for a // pure uplink-coverage study use a star via `gnb` with a shared medium: // sensor[i].wlan[0] <--> gnb.wlan[0] (needs the medium in between) // The radio medium makes the connection implicit: all radios within // range of each other hear each other. No explicit connections needed. }

Critical point for UAV work: in INET, a radio medium makes connectivity *implicit*. You do **not** write `connections:` for wireless links. The medium decides, per transmission, which receivers are in range. Writing explicit wireless connections is a category error.

2.6 `omnetpp.ini` for a scalar-radio UAV swarm

Directly adapted from the real `Wireless12`/`Wireless13`/`Wireless14` configs in INET v4.6.0's `tutorials/wireless/omnetpp` which I fetched and verified:

[Config UavScalarRadio] description = "5 UAVs, ApskRadio, TwoRayGroundReflection, log-normal shadowing" network = UavCoverageNet sim-time-limit = 600s

---- Medium ---------- *.radioMedium.signalAnalogRepresentation = "scalar" *.radioMedium.backgroundNoise.power = -100dBm *.radioMedium.propagation.typename = "ConstantSpeedPropagation" *.radioMedium.propagation.PropagationSpeed = 299792458mps

*.physicalEnvironment.ground.typename = "FlatGround" *.physicalEnvironment.ground.elevation = 0m

---- Path loss + shadowing (chained IPathLoss submodules) ---------- *.radioMedium.pathLoss.typename = "TwoRayGroundReflection" *.radioMedium.pathLoss.shadowing.typename = "LogNormalShadowing" *.radioMedium.pathLoss.shadowing.sigmaRange = 1 *.radioMedium.pathLoss.shadowing.correlationDistance = 25m

---- medium limit cache: bounds the interference computation ---------- *.radioMedium.mediumLimitCache.centerFreq = 2.4GHz # "interferenceRange" | "communicationRange": drop signals outside this range # EARLY. Huge speedup, and physically justified if you know the max range. *.radioMedium.mediumLimitCache.rangeFilter = "interferenceRange" *.radioMedium.mediumLimitCache.range = 1200m

---- Per-UAV radio ---------- *.uav[*].wlan[0].typename = "WirelessInterface" # no ACK/CSMA: pure best-effort *.uav[*].wlan[0].radio.typename = "ApskRadio" *.uav[*].wlan[0].radio.signalAnal = "scalar" *.uav[*].wlan[0].radio.centerFrequency = 2.4GHz *.uav[*].wlan[0].radio.bandwidth = 20MHz

TRANSMITTER: there is NO `dataRate` or `bitrate` parameter on an ApskRadio. # Symbol rate follows from centerFrequency and modulation; effective bitrate # follows from centerFrequency/bandwidth/spectral efficiency. Set the PHY # through centerFrequency/bandwidth/transmitter.power, not a datarate. *.uav[*].wlan[0].radio.tran = 100mW # 20 dBm *.uav[*].wlan[0].radio.transmitter.preambleDuration = 10us *.uav[*].wlan[0].radio.transmitter.headerLe

= 8B *.uav[*].wlan[0].radio.transmitter.thermalNoise = 0 # Derived and useful to record: *.uav[*].wlan[0].radio.transmitter.tr
= true
RECEIVER *.uav[*].wlan[0].radio.receiver.sensitivity = -101dBm # 802.11b sensitivity *.uav[*].wlan[0].radio.receiver.e
= -96dBm *.uav[*].wlan[0].radio.receiver.snirThreshold = 4dB *.uav[*].wlan[0].radio.receiver.ignoreInterference
= false # set TRUE to study # radio-blind coverage *.uav[*].wlan[0].radio.receiver.errorModel.typename =
"StochasticErrorModel" *.uav[*].wlan[0].radio.receiver.errorModel.packetErrorRate = 0
---- Antenna: a downward-tilted directional antenna is the right UAV model - *.uav[*].wlan[0].radio.antenna.typename
= "ConstantGainAntenna" *.uav[*].wlan[0].radio.antenna.gain = 3dB
---- MAC / link layer: keep it thin, we want a coverage study ---------- *.uav[*].wlan[0].mac.useAck = false
*.uav[*].wlan[0].mac.fullDuplex = false *.uav[*].wlan[0].mac.headerLength = 23B *.uav[*].wlan[0].queue.typename
= "DropTailQueue" *.uav[*].wlan[0].queue.packetCapacity = 100
---- IPv4 / routing ---------- *.uav[*].ipv4.arp.typename = "GlobalArp"
*.uav[*].forwarding = true *.configurator.optimizeRoutes = false *.configurator.addStaticRoutes = true *.con-
figurator.config = xml("<config><interface hosts='**' address='10.0.0.x' netmask='255.255.255.0'/><autoroute
metric='errorRate'/></config>")

> [!] **Correction to a premise in your brief:** you asked "how to configure `scalarRadioDataRate` / `dataRate` / `range`." **None of those parameter names exist in INET 4.6.** There is no `scalarRadioDataRate` (removed with the `ScalarRadio` class), and no `dataRate` on a packet-level radio. The only `bitrate` parameter lives on `ShortcutRadio` (and on `DatarateChannel` for *wired* connections). "Range" is not a parameter either — in 4.6 range is emergent from `transmitter.power` + `pathLoss` + `receiver.sensitivity`, or imposed via `mediumLimitCache.range` / `rangeFilter`. If a tutorial tells you to set `*.radio.dataRate = 1Mbps` on an `ApskRadio`, it is wrong.

2.7 Mobility models for UAVs

NED type names vs C++ class names — a real trap:

NED type (what you write in ini)	C++ class (what you subclass)	File
`LinearMobility`	`inet::LinearMobilityModule`	`src/inet/mobility/single/LinearMobility.`
`MassMobility`	`inet::MassMobilityModule`	`src/inet/mobility/single/MassMobility.ne`
`StaticLinearMobility`	`inet::StaticLinearMobility`	`src/inet/mobility/static/StaticLinearMob`

INET 4.6 now offers **GNSS-track and satellite mobility models with geographic visualization** (added in 4.7) — for a UAV-IoT project flying real waypoint tracks from a GPX/KML file, this is materially better than `LinearMobility`. It is the headline mobility feature of INET 4.7.

---- LinearMobility: constant velocity, constant heading, reflective walls -- *.uav[*].mobility.typename =
"LinearMobility" *.uav[*].mobility.initialPosition = uniform(0m, 1000m) *.uav[*].mobility.initialMovementHeading
= uniform(0deg, 360deg) *.uav[*].mobility.speed = 12mps # *.uav[*].mobility.updateInterval = 0.1s
---- MassMobility: momentum-correct random walk. Verified parameters: ------- *.uav[*].mobility.typename
= "MassMobility" *.uav[*].mobility.initialX = uniform(0m, 1000m) *.uav[*].mobility.initialY = uniform(0m,
1000m) *.uav[*].mobility.initialZ = uniform(80m, 120m) # UAV altitude band *.uav[*].mobility.initialMovementHeading
= uniform(0deg, 360deg) *.uav[*].mobility.initialMovementElevation = 0deg *.uav[*].mobility.speed = 12mps #
@mutable; resampled each interval *.uav[*].mobility.changeInterval = normal(5s, 0.1s) *.uav[*].mobility.angleDelta
= normal(0deg, 30deg) *.uav[*].mobility.rotationAxisAngle = 0deg # 0deg = bank about Z # MassMobility
is the model of "Optimized Smooth Handoffs in Mobile IP" # (Perkins & Wang) and supports 3D motion via
rotationAxisAngle.

Recommendation: use `LinearMobility` with **waypoint waypoints** for a coverage study (deterministic, interpretable, and the "patrol" semantics of your problem), and `MassMobility` only for the *baseline* you compare against. Give your UAVs explicit `mobility.initialX/Y/Z` derived from an ini parameter so the RL environment can place them. The module also honours `initFromDisplayString` (default `true`), so `@display("p=200,600")` sets the initial position — but **ini parameters are cleaner and are what you need for randomization**, so set `initFromDisplayString = false` and drive placement from ini.

2.8 Results collection

// In your simple module's NED definition, declare the recorder channels: simple UavAgent { parameters:
string coverageFraction @statistic("vector") = ""; string energyRemaining @statistic("vector") = ""; cOutVec-
tors: coverageFraction { @statistic("vector"); record = @default(1); } energyRemaining { @statistic("vector");

record = @default(1); } cOutScalars: throughputDl { @statistic("scalar"); record = @default(1); } coverageAtEnd { @statistic("scalar"); record = @default(1); } energyAtEnd { @statistic("scalar"); record = @default(1); } }

// In C++, from any cModule: cOutVector &cov = getParentModule()->getOrCreate<cOutVector>("coverageFraction"); cov.record(simTime(), currentCoverageFraction);

cOutScalar &thr = getParentModule()->getOrCreate<cOutScalar>("throughputDl"); thr.record(bytesDl / (simTime() - startTime).seconds());

INET 4.6 gives you a purpose-built recorder for exactly your use case:

De-multiplex per-application statistics even when apps share a link/interface *.control.flowMeasurementRecorder.app = "UavCoverageApp" *.control.flowMeasurementRecorder.recording-interval = 1s *.control.flowMeasurementRecorder.packet = "packets.json"

`FlowMeasurementRecorder` in 4.6 was extended with a **packetEvent JSON log** capturing per-packet events — invaluable for a fault-tolerance study (you can post-hoc reconstruct exactly which UAV lost which packet to which gNB and when).

New in 4.6 and directly relevant to you: `InterarrivalTime` result filter (jitter/traffic burstiness), `ChannelOwnerRecorder` (which module owns a channel — for contention debugging), `SimulationCpuUsageMeasurer` (records CPU time as a scalar — **use this to report your compute cost in the paper**), and the `DemuxAppFilter` / `DemuxRegexFilter`.

Scalars vs vectors — the choice that matters for RL. RL needs *one number per episode*. Do **not** make Python read a 600-sample vector. Record a small set of episode-level scalars (`coverageAtEnd`, `energyAtEnd`, `throughputDl`, `coverageTimeTo95`, `reconfigurationCount`, `pduRetransmissions`) and read *only* those in the reward function. Keep the vectors for figures and diagnostics only. This is a 100× reduction in Python-side parsing and it is what makes the training loop fast (4.4).

0.15.4. 3. Simu5G

3.1 Version and status — unambiguously alive

Field	Value
Current	**v1.7.0, released 2026-09-14** (15 days ago)
Previous	v1.6.0 (2026-07-31), v1.5.x, v1.4.x, v1.3.1
Upstream	`inet-framework/Simu5G` (forked from `Unipisa/Simu5G`)
Docs	https://simu5g.org
License	**LGPL**
Tested against	**INET-4.5.4 + OMNeT++ 6.3**
Declared compatible with	**INET-4.6.0, OMNeT++ 6.1 through 6.4**
Since v1.3.1	Developed by **Andrs Varga and the OMNeT++ core team**

The upstream `install` page is badly stale — it still says *"Latest release: Simu5G 1.3.0"* and *"INET-Framework v4.5.x, OMNeT++ v6.0.x"*. **Ignore it.** The v1.7.0 whatsnew is the authoritative source and I read it in full.

3.2 What it actually models (the realistic scope you asked for)

This is a link-level / system-level simulator, not a PHY simulator. From the official feature list:

> **PHY:** Channel feedback computation; stochastic channel models per 3GPP TR 36.814, TR 36.873 and TR 38.901; reception based on SINR and BLER curves; background cells for large-scale scenarios

Explicitly: - [X] **No OFDM waveform. No modulation/demodulation. No channel coding chain. No OFDM numerology at the waveform level.** It computes a SINR, looks it up on a **BLER curve**, and draws a packet outcome. That is the entire "PHY." - [OK] **Fading: yes, but as statistics.** Shadowing and small-scale fading are *drawn* from the distributions in the 3GPP TRs. They are not the result of a tapped-delay-line or a time-varying channel convolution. - [X] **No MAC-layer timing fidelity to a real slot/TTI scheduler beyond coarse framing.** Scheduling is abstracted into algorithm-level schedulers over TTI boundaries. - [OK] **MIMO: no.** v1.4.3–v1.4.5 explicitly *removed* "incomplete MIMO code." - [OK]

RRC signaling is NOT simulated — RRC *functionality* (bearer management, handover, radio link failure, re-establishment) is, but the signaling messages are not.

What it models well: the 3GPP *architecture*. gNB/eNB heterogeneity, numerology and multiple numerologies, FDD/TDD, carrier aggregation, X2 interface, handover, CoMP coordinated scheduling, EN-DC and NE-DC dual connectivity with split bearers, RLC TM/UM/AM per TS 38.322 and TS 36.322, HARQ, AMC and CQI reporting, random access with preamble collisions, per-logical-channel-group buffer status reporting and uplink scheduling, 5GC QoS flows and QFI classification, SDAP, QCI/5QI profiles, an ETSI MEC model, and a network-emulation mode.

v1.7.0 renamed a lot of things. If you read any pre-2026 Simu5G paper, these names are now wrong:

```
LteChannelModel            ->  ChannelModelBase
LteRealisticChannelModel   ->  StochasticChannelModel
LteDummyChannelModel       ->  IdealChannelModel
ILteChannelModel           ->  IChannelModel
NrChannelModel             ->  Tr36873ChannelModel
NrChannelModel_3GPP38_901  ->  Tr38901ChannelModel
ILtePhy / LtePhyBase / LtePhyUe / LtePhyEnb -> IPhy / PhyBase / PhyUe / PhyEnb
NrPhyUe                    ->  removed (use PhyUe)
LteChannelModel.lteChannelModelType param -> channelModelType
Path loss                  ->  strategy family: PathLossModel (abstract)
                               Tr36814PathLossModel <- Tr36873PathLossModel <- Tr38901PathLossModel
                               selected by pathLossType = "Tr36814"|"Tr36873"|"Tr38901"
UmTxEntity/UmRxEntity      ->  LteRlcUmTxEntity / LteRlcUmRxEntity
AmTxQueue/AmRxQueue        ->  LteRlcAmTxEntity / LteRlcAmTxEntity
TmTxEntity/TmRxEntity      ->  RlcTmTxEntity / RlcRxTxEntity
UpperMux                   ->  PdcpMux
New module                 ->  BearerConfigurator (central bearer/QoS-flow config)
New switch                 ->  NrSdap.hasSdap now defaults to TRUE in any network with an Upf
D2D                        ->  separate package simu5g.stack.d2d, gated by hasD2D bool
```

> v1.7.0 is a **breaking release for every existing ini file and every subclassing project.** v1.7.0 also fixed genuine numerical bugs in the 3GPP propagation formulas (UMa used the UMi LOS formula; RMa/SMa decay constants swapped; breakpoint distances in the wrong unit; TR 38.901 high/low building-penetration models swapped). The release notes say plainly: *"These fixes change the statistical results of simulations that use the affected scenarios and models — in some cases substantially."* **Any Simu5G result published before Sept 2026 used incorrect path loss.**

3.3 Do you need it? The honest answer.

For persistent, fault-tolerant UAV-IoT coverage, Simu5G is the wrong tool, for four concrete reasons:

1. **A persistent-coverage problem is not a cellular problem.** "Persistent" means a UAV must keep a ground sensor in coverage. The 3GPP machinery that Simu5G models — handover, bearer re-establishment, X2 split bearers, QoS-flow mapping — is what a *cellular* network does to *preserve a bearer through a handover*. That is a genuinely interesting problem, but it is a **different problem** from "keep re-positioning a UAV so the sensor is covered."
2. **Cost.** Simu5G's own fingerprint suite is 202 configurations, validated on INET 4.5.4 + OMNeT++ 6.3. To use INET 4.7 you'd be on an untested combination, and the v1.7.0 renames break every INET-based example you might crib from.
3. **Throughput.** A 3GPP stack with per-TTI scheduling, HARQ, and CQI feedback is orders of magnitude more expensive per simulated second than `ScalarRadioMedium` + `ApskRadio`. For RL with thousands of episodes, that is fatal.
4. **You are not studying 5G.** If your report's contribution is an MARL algorithm for UAV coverage, Simu5G is a distraction that a reviewer will (rightly) ask you to justify.

Use Simu5G only if your specific contribution is *fault tolerance of a cellular-connected UAV* — e.g. handover-aware persistent coverage from a gNB-connected UAV, with 5G numerology or QoS-flow-aware admission. In that case, build on **INET 4.5.4 + OMNeT++ 6.3 + Simu5G 1.7.0**, because that is the exact triple Simu5G's CI tests. Do not deviate.

0.15.5. 4. The Python OMNeT++ bridge

4.1 The hard constraint

`pyopp` — Python-implemented OMNeT++ NED modules — **does not exist in OMNeT++ 6.x.** I verified the complete `omnetpp-6.4.0` source tree: zero files matching `pyopp`, zero `cSimpleModule` Python bindings. What *does* exist:

Path in omnetpp-6.4.0	Purpose
`python/omnetpp/scave/*`	Result/chart analysis bindings (nanobind → C++ scave)
`python/omnetpp/ned.py`, `nedast.py`, `nedlinter.py`	NED file parsing/linting
`python/omnetpp/repl.py`	The `opp_python_repl` REPL
`python/omnetpp/llmtool/`	LLM tooling
`python/omnetpp/lldb/formatters/omnetpp.py`	LLDB pretty-printers
`src/sim/nedpythonfunctions.cc`	`pyeval()` and `pycode()` NED functions

`pyeval`/`pycode` (added 6.1, both verified in `src/sim/nedpythonfunctions.cc` with tests `NED_functions_python_pyeval` / `..._pycode_1.test`) are guarded by `#ifdef WITH_PYTHON` and are for **parameter expression evaluation only**:

```
any pyeval(string s, ...)   // evaluates s as a Python expression, or as a lambda then calls it
any pycode(string s, ...)   // evaluates s as a Python function body
```

They auto-convert between `cValue` and Python `bool/int/float/str/list/dict`. You can use them to load coverage maps, cost tables, or learned parameters from Python. **You cannot use them to implement a module's `handleMessage`.**

Also note: OMNeT++ offers `ExternalApp` / `ExternalEnvironment` / `ExternalProcess` modules (new in 4.6) that run **host OS processes** inside a simulation, with `UnsharedNamespaceInitializer` for isolated `Network(1,2,3,4)` namespaces. This is a first-class supported way to couple a real network simulator to a real external process. It is the cleanest "co-simulation" feature in the toolchain and is worth knowing about.

4.2 The four options, honestly assessed

(a) External Python process + TCP / ZeroMQ / shared memory — RECOMMENDED The Python side owns the RL loop; `opp_run` owns the simulation. They are separate OS processes talking over a socket, and the socket is *driven by the simulation clock*, not wall-clock.

Latency reality check (this is the number that decides everything):

Path	Round-trip cost
TCP loopback, small fixed message, `TCP_NODELAY`	**~10–30 s**
TCP loopback, 1 KB payload	~20–50 s
ZeroMQ `REQ/REP` over `ipc://`	~15–40 s (roughly 2× TCP; not obviously worth it)
ZeroMQ `PUSH/PULL` + `zmq.Poller`	~10–25 s, best pipelining
Shared memory ring (`shm_open` + `mmap`)	**~1–3 s** — but adds a lot of unsafe code
PythonC++ GIL boundary (embedded)	~0.5–1 s, but see (b)
Simulated time cost (the real number)	**0** — DES is time-driven, not wall-clock

The key insight that makes (a) viable: a discrete-event simulator is infinitely faster than real time. A 600 s simulation of a 5-UAV, 50-sensor network takes 0.3–3 s of wall clock. So 30 s of socket latency per *decision epoch* is nothing — what matters is your decision epoch, not your packet.

Decision granularity	Steps	Socket cost	Overhead
Per packet / per INET transmission	10–10	3–30 s	**Fatal.**
Per NED self-message tick (1 s sim)	600	18–36 ms	Negligible.
Per control epoch (5 s sim)	120	1.2–7 ms	Negligible.
Per episode	1	~30 s	Free.

Throughput measured against reality: a typical 5-UAV / 50-sensor `ScalarRadioMedium` run does 10–10 events and takes 0.3–5 s. With a 5 s control epoch you get 120 sockets/ep round trip. At 2000 episodes × 2e5 events, you need **~1.4e8 events** — achievable in ~2–10 CPU-hours. **The socket is not your bottleneck. The simulator is.** Design around that.

(b) Embedded Python (pyopp / Python 3 support) Not available. See 4.1. There is a third-party academic project, `mmodenesi/omnetpy`, which hand-wrote bindings to let you subclass `cSimpleModule` from Python, and whose own README says: *"For the moment, no further improvements are being implemented... This is an academic project. Many things were written 'just to make it work' and may not scale well. This project was not heavily used."* Its limitations: no dynamic module deletion, no Python debugging, no automatic message-code generation, hand-written bindings with possible bugs, ~32 commits since 2020.

Do not build a thesis on it. You would be alone with a 5-year-old academic fork when you need to debug a segfault in an INET packet path.

(c) All-C++ simulator + ONNX-exported Python policy `OP_ENZYME` / `ONNX Runtime 1.29.0` (17 Aug 2026) inference from C++ via the ONNX Runtime C/C++ API, or `onnx2c++` for a header-only implementation.

OK **Lowest latency**: single-digit s per action batch, no IPC, no GIL, no serialization.
OK **Deterministic**: identical numerics for training and deployment, and you can pin thread counts.
OK **Right answer if your policy is evaluated/deployed, not trained, in C++.**
X **Training is still in Python.** So you still need the socket bridge for training. Option (c) *replaces* (a) only at the inference/deployment stage, or for a fully in-C++ pipeline.
X **ONNX is static-shaped.** Variable-arity graphs (heterogeneous agent sets, GNN policies) need dynamic-shape support or padding — real engineering pain.
X **Debugging is in two languages.** A bug is either in the exported graph or in the C++ inference harness, and the failure modes are opaque.

(d) Decouple entirely — abstract env over TCP Same wire protocol as (a), but the *interface* is a clean Gymnasium/PettingZoo-shaped contract that a separate lightweight 2-D model implements too.

OK **This is what you should actually do**, and it is orthogonal to (a). See 4.5.
OK Lets you unit-test the RL algorithm against a fast 2-D surrogate, and validate the surrogate against OMNeT++ — which is a *result you can publish.*

4.3 Wire protocol design

Use **length-prefixed fixed-width binary frames over `AF_UNIX SOCK_STREAM`**, or `AF_INET` **on 127.0.0.1** if you need to reach a remote worker. Do **not** use JSON, and do **not** use one TCP connection per agent.

```
C++ (OMNeT++)                          Python (RL)

RLStepClient (cSimpleModule)           OmnetEnv
 - owns TCP client socket               (Gymnasium/
 - handshake on initialize()             PettingZoo
 - poll() in handleMessage()             ParallelEnv)
 - step() blocks waiting for the action
 - writes episode scalars on finish()

      4-byte big-endian length prefix + little-endian f32 payload
```

Frame types (1-byte opcode, then length-prefixed body):

Op	Dir	Body	Meaning
`0x01 RESET`	C→P	`seed:u32, episodeConfig:json-utf8`	new episode
`0x02 RESET_ACK`	P→C	—	
`0x03 GET_OBS`	C→P	`t_sim: f64, agentCount:u16`	
`0x04 OBS`	P→C	`n:u16`, then `n` × `{id:u16, len:u32, f32[len]}`	per-agent observations
`0x05 GET_ACTION`	C→P	—	

Op	Dir	Body	Meaning
`0x06 ACTION`	P→C	`n:u16`, then `n` × `{id:u16, len:u32, f32[len]}`	per-agent actions
`0x07 EPISODE_DONE`	C→P	`n:u16`, then `n` × `f32` + `json-utf8` extras	episode scalars (coverage, energy, . . .)
`0x08 INFO`	P→C	`json-utf8`	shape/dtype spec for the first RESET

Latency optimizations that matter, in order of payoff: 1. **Set `TCP_NODELAY` on both sides.** Non-negotiable; Nagle's algorithm alone adds 40 ms. 2. **Batch across all agents per epoch** — one frame for 5 agents, not 5 frames. 3. **Fixed-width observations, no JSON, no strings in the hot path.** Shape it as a flat `f32` array. 4. **Read the whole frame in one `read()` loop, not line-buffered.** Use a buffered stream. 5. **Coalesce observation and action into one round trip per epoch**, not four. 6. **Send episode scalars as raw `f32` + one JSON blob** for the rare metadata.

Realistic budget per decision epoch with 5 agents: ~25–60 s total. Over 120 epochs × 2000 episodes = 240k epochs → **6–15 seconds of socket time across the entire training run.** Negligible. Do not over-engineer this.

4.4 C++ side — a working sketch

// // RLStepClient.h // #ifndef RLSTEPCLIENT_H_ #define RLSTEPCLIENT_H_

#include <omnetpp.h> #include <sys/socket.h> #include <netinet/in.h> #include <netinet/tcp.h> #include <arpa/inet.h> #include <unistd.h> #include <vector> #include <cstring> #include <stdexcept>

using namespace omnetpp;

enum Op : uint8_t { OP_RESET = 0x01, OP_RESET_ACK = 0x02, OP_GET_OBS = 0x03, OP_OBS = 0x04, OP_GET_ACTION = 0x05, OP_ACTION = 0x06, OP_EPISODE_DONE = 0x07, OP_INFO = 0x08 };

// // Blocking client to a Python RL process. One instance per simulation. /// Owned by a cSimpleModule; every call blocks, which is FINE because the // whole Python side is a single logical clock source. // class RLStepClient { private: int fd_ = -1; // Pinned observation layout; must match the Python side exactly. // 0: x, 1: y, 2: z, 3: vx, 4: vy, 5: vz // 6..: per-sensor relative offset + covered-flag + age // last: battery, local-coverage-fraction, neighbour-count static constexpr int OBS_DIM = 6 + 2 * 50 + 3; // 5 UAVs, 50 sensors static constexpr int ACT_DIM = 3; // velocity command static constexpr int NUM_AGENTS = 5;

public: void connect(uint16_t port, const char *host = "127.0.0.1") { fd_ = ::socket(AF_INET, SOCK_STREAM, 0); if (fd_ < 0) throw std::runtime_error("RLStepClient: socket() failed");

int one = 1; ::setsockopt(fd_, IPPROTO_TCP, TCP_NODELAY, &one, sizeof(one)); // CRITICAL

sockaddr_in addr{}; addr.sin_family = AF_INET; addr.sin_port = htons(port); ::inet_pton(AF_INET, host, &addr.sin_addr);

if (::connect(fd_, (sockaddr*)&addr, sizeof(addr)) < 0) throw std::runtime_error("RLStepClient: connect() failed"); }

~RLStepClient() { if (fd_ >= 0) ::close(fd_); }

// ---- framing ---------- void sendFrame(Op op, const void *body, size_t len) { uint8_t hdr[5] = { (uint8_t)op }; uint32_t n = htonl((uint32_t)len); std::memcpy(hdr + 1, &n, 4); if (::write(fd_, hdr, 5) != 5) throw std::runtime_error("short write hdr"); size_t off = 0; while (off < len) { ssize_t w = ::write(fd_, (const char*)body + off, len - off); if (w <= 0) throw std::runtime_error("short write body"); off += (size_t)w; } }

bool recvFrame(Op *op, std::vector<uint8_t> &body) { uint8_t hdr[5]; size_t got = 0; while (got < 5) { ssize_t r = ::read(fd_, hdr + got, 5 - got); if (r <= 0) return false; // clean EOF = Python died got += (size_t)r; } uint32_t n; std::memcpy(&n, hdr + 1, 4); n = ntohl(n); body.resize(n); got = 0; while (got < n) { ssize_t r = ::read(fd_, body.data() + got, n - got); if (r <= 0) return false; got += (size_t)r; } *op = (Op)hdr[0]; return true; }

// ---- protocol ---------- void reset(uint32_t seed) { std::string cfg = "{\"numUavs\":5,\"numSensors\":50,\"horizon\":600}"; std::vector<uint8_t> b(sizeof(uint32_t) + cfg.size()); uint32_t s = htonl(seed); std::memcpy(b.data(), &s, 4); std::memcpy(b.data() + 4, cfg.data(), cfg.size()); sendFrame(OP_RESET, b.data(), b.size()); Op op; std::vector<uint8_t> body; if (!recvFrame(&op, &body) || op != OP_RESET_ACK) throw std::runtime_error("RLStepClient: bad RESET_ACK"); }

void getObs(simtime_t t, const std::vector<float> &perAgentObs) { std::vector<uint8_t> b(sizeof(double) + sizeof(uint16_t)); double tt = (double)SIMTIME_DBL(t); std::memcpy(b.data(), &tt, 8); uint16_t n = htons(NUM_AGENTS); std::memcpy(b.data() + 8, &n, 2); sendFrame(OP_GET_OBS, b.data(), b.size()); }

void getAction(std::vector<float> &actionsOut) { sendFrame(OP_GET_ACTION, nullptr, 0); Op op; std::vector<uint8_t> body; if (!recvFrame(&op, &body) || op != OP_ACTION) throw std::runtime_error("RLStepClient: bad ACTION"); uint16_t n; std::memcpy(&n, body.data(), 2); n = ntohs(n); actionsOut.resize((size_t)n * ACT_DIM); for (uint16_t a = 0; a < n; a++) { const uint8_t *p = body.data() + 2 + (size_t)a * (2 + 4 + ACT_DIM * 4) + 6; std::memcpy(&actionsOut[(size_t)a * ACT_DIM], p, ACT_DIM * 4); } }

void episodeDone(const std::vector<float> &scalars, const std::string &json) { std::vector<uint8_t> b(2 + scalars.size() * 4 + json.size()); uint16_t n = htons((uint16_t)scalars.size()); std::memcpy(b.data(), &n, 2); for (size_t i = 0; i < scalars.size(); i++) { float v = scalars[i]; std::memcpy(b.data() + 2 + i * 4, &v, 4); } std::memcpy(b.data() + 2 + scalars.size() * 4, json.data(), json.size()); sendFrame(OP_EPISODE_DONE, b.data(), b.size()); } };

#endif

// // RLController.cc -- a cSimpleModule that pumps the socket bridge. // // It exposes input/output gates for the RL logic, sends observations at a // fixed simulated control epoch, and applies the returned actions. // #include "RLController.h"

Define_Module(RLController);

void RLController::initialize(int stage) { if (stage == 0) { client_.connect(par("rlPort").intValue()); client_.reset(getRNG >intRand() ^ (uint32_t)getSeed()); nextEpoch_ = par("controlInterval").simtimeValue(); nextEpoch_ += nextEpoch_; } }

void RLController::handleMessage(cMessage *msg) { if (msg->getArrivalTime() < nextEpoch_) return; // not an epoch boundary; drop

// 1) observe const int N = OBS_DIM; std::vector<float> obs((size_t)numAgents_ * N, 0.0f); for (int a = 0; a < numAgents_; a++) { cModule *uav = getParentModule()->getSubmodule("uav", a); Coord p = uav->getOrientation(); // or mobility->getPosition() obs[(size_t)a * N + 0] = (float)p.x; obs[(size_t)a * N + 1] = (float)p.y; obs[(size_t)a * N + 2] = (float)p.z; } client_.getObs(simTime(), obs);

// 2) act std::vector<float> actions; client_.getAction(actions); for (int a = 0; a < numAgents_; a++) { cModule *uav = getParentModule()->getSubmodule("uav", a); uav->par("vx").setDoubleValue(actions[(size_t)a * 3 + 0]); uav->par("vy").setDoubleValue(actions[(size_t)a * 3 + 1]); uav->par("vz").setDoubleValue(actions[(size_t)a * 3 + 2]); }

if (simTime() >= getSimulation()->getTerminationTime()) { std::vector<float> scalars{ globalCoverage, avgEnergy, reconfigCount }; client_.episodeDone(scalars, "{\"done\":true}"); } nextEpoch_ += nextEpoch_; scheduleAt(nextEpoch_, msg); }

void RLController::finish() { // Record episode scalars for the .sca, so the paper's numbers come out of // the same artifact that produced the reward. getParentModule()->getOrCreate<cOutScalar>("globalCoverageFinal").record(glo getParentModule()->getOrCreate<cOutScalar>("meanEnergyFinal").record(avgEnergy); }

4.5 Python side — a working `ParallelEnv`

""" omnet_env.py -- PettingZoo AEC / Parallel API adapter over the OMNeT++ bridge.

Implements the PettingZoo PARALLEL API, because 5 UAVs act simultaneously and MAPPO assumes synchronous action spaces. """ from __future__ import annotations

import socket, struct, json, os from typing import Any

import numpy as np from gymnasium import spaces from pettingzoo.utils.env import ParallelEnv

HOST, PORT = "127.0.0.1", 5555

NUM_AGENTS = 5 OBS_DIM = 6 + 2 * 50 + 3 # must equal RLStepClient::OBS_DIM exactly ACT_DIM = 3 HORIZON_S = 600 CTRL_EPOCH_S = 5

OP_RESET, OP_RESET_ACK = 0x01, 0x02 OP_GET_OBS, OP_OBS = 0x03, 0x04 OP_GET_ACTION, OP_ACTION = 0x05, 0x06 OP_EPISODE_DONE = 0x07 OP_INFO = 0x08

class _Frame: """Length-prefixed (1-byte opcode + 4-byte BE length) binary framing."""

def __init__(self, sock: socket.socket): self.s = sock self.s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1) # critical self.buf = bytearray()

def send(self, op: int, body: bytes = b"") -> None: self.s.sendall(bytes([op]) + struct.pack(">I", len(body)) + body)

def _readn(self, n: int) -> bytes: while len(self.buf) < n: chunk = self.s.recv(65536) if not chunk: raise ConnectionError("OMNeT++ bridge closed the connection") self.buf += chunk out, self.buf = bytes(self.buf[:n]), self.buf[n:] return out

def recv(self) -> tuple[int, bytes]: hdr = self._readn(5) op, n = hdr[0], struct.unpack(">I", hdr[1:5])[0] return op, self._readn(n)

class OmnetParallelEnv(ParallelEnv): metadata = {"render_modes": []}

def __init__(self, port: int = PORT, host: str = HOST): super().__init__() self.port, self.host = port, host self.possible_agents = [f"uav_{i}" for i in range(NUM_AGENTS)] self.observation_spaces = { a: spaces.Box(low=-np.inf, high=np.inf, shape=(OBS_DIM,), dtype=np.float32) for a in self.possible_agents }

Velocity command in m/s, bounded by the UAV's max speed. self.action_spaces = { a: spaces.Box(low=-15.0, high=15.0, shape=(ACT_DIM,), dtype=np.float32) for a in self.possible_agents } self.frames: _Frame | None = None self._t = 0.0

-- lifecycle ---------- def _connect(self) -> _Frame: if self.frames is None: s = socket.create_connection((self.host, self.port)) self.frames = _Frame(s) return self.frames

def reset(self, seed: int | None = None, options: dict | None = None): f = self._connect() cfg = json.dumps({"numUavs": NUM_AGENTS, "numSensors": 50, "horizon": HORIZON_S, "ctrlEpoch": CTRL_EPOCH_S} f.send(OP_RESET, struct.pack(">I", seed or 0) + cfg) op, _ = f.recv() if op != OP_RESET_ACK: raise RuntimeError("bridge refused RESET")

self.agents = list(self.possible_agents) self._t = 0.0 obs, infos = self._poll_obs() return obs, infos

def _poll_obs(self): f = self._connect() f.send(OP_GET_OBS, struct.pack(">dHH", self._t, len(self.agents), 0)) # 0 = observations op, body = f.recv() if op != OP_OBS: raise RuntimeError(f"expected OBS, got {op:#x}") n = struct.unpack_from(">H", body, 0)[0] obs, infos = {}, {} off = 2 for _ in range(n): aid, ln = struct.unpack_from(">HI", body, off) off += 6 vec = np.frombuffer(body, dtype=np.float32, count=ln, offset=off).copy() off += ln * 4 obs[self.possible_agents[aid]] = vec infos[self.possible_agents[aid]] = {} return obs, infos

def step(self, actions: dict[str, np.ndarray]): f = self._connect() buf = bytearray(struct.pack(">H", len(self.agents))) for aid, agent in enumerate(self.agents): a = np.asarray(actions[agent], dtype=np.float32) buf += struct.pack(">HI", aid, a.size) + a.tobytes() f.send(OP_GET_ACTION, bytes(buf)) op, _ = f.recv() if op != OP_ACTION: raise RuntimeError(f"expected ACTION, got {op:#x}")

self._t += CTRL_EPOCH_S terminated = {a: False for a in self.agents} truncated = {a: self._t >= HORIZON_S for a in self.agents} obs, infos = self._poll_obs() return obs, {a: 0.0 for a in self.agents}, terminated, truncated, infos

def close(self): if self.frames is not None: self.frames.s.close() self.frames = None

4.6 The recommended architecture

```
LAYER 3  ---  MARL algorithm (Python)
MAPPO, CTDE, PettingZoo ParallelEnv. No changes if you swap simulator.

LAYER 2  ---  Abstract env over TCP (Python)
OmnetParallelEnv   the contract   FastSurrogateParallelEnv
(owns episodes, seeds, reward shaping, metric aggregation)
This is the ONLY thing your algorithm knows about the world.

SIMULATOR A (slow)    SIMULATOR B (fast, ~1000$\times$ )
OMNeT++ 6.4 + INET   2-D NumPy coverage model, same obs/action/reward
4.7, opp_run, one    contract. Runs in-process. Use for algorithm
process per seed     development, ablations, and hyperparameter search.
$\times$ 8-16 parallel       Publish the surrogate-vs-OMNeT++ correlation.
workers (N cores)
```

Why this is the right call for a student project:

1. **Algorithm development does not need OMNeT++.** Iterate on MAPPO against the surrogate at 1000× speed. Only port to OMNeT++ once the algorithm works.
2. **Parallelism is trivially available.** `opp_env run` / `opp_runall` gives you N-way process parallelism for free. RLlib's vectorized env runners then parallelize *across* workers. This is how you get 8–32× effective throughput.
3. **The surrogate-vs-OMNeT++ correlation is itself publishable** and is the honest way to justify the two-stage methodology in your report.
4. **You are decoupled from INET's churn.** INET 4.6 renamed half the PHY stack in one release. Your env contract does not care.

Caveats you must state in the report: the surrogate is a *model* of the world, not the world. Claims about "persistent coverage under packet loss" that are only made in the surrogate are not claims about the network. Run the final numbers in OMNeT++.

4.7 Libraries

Library	Verdict
`pyopp`	[X] Does not exist.
`mmodenesi/omnetpy`	[X] Academic, unmaintained since 2020, own README says it "was not heavily used."
`opp_repl` (pip)	[OK] v0.4, 11 May 2026, LGPLv3+. Run simulations, sweep parameters, MCP server. But it is a *batch* tool, not a training-loop integration.
`omnetpp.scave` (in-tree)	[OK] The only supported way to read `.sca`/`.vec` from Python.
`inet` (INET Python package)	[OK] `run_simulations`, `run_fingerprint_tests`, `run_smoke_tests`, `optimize_simulation_parameters`, plus `inet.common.cluster.SSHCluster` for remote batches. It requires sourcing INET's `setenv`. Best for *parameter studies and regression testing*, not for RL.
`pettingzoo`	[OK] **1.27.0, 13 Aug 2026.** MIT. Use the **Parallel API**. 1.27.0 added `RecordVideo`, a Gymnasium-style `pettingzoo.make("aec"|"parallel", "family/env-vN")` registry, and pygame rendering for Hanabi; it **removed the gin_rummy env, dropped Python 3.9, and deprecated the old env-creation API.** MPE was **removed from PettingZoo in 1.26.0** and now lives in the separate `mpe2` package.
`supersuit`	[OK] `>=3.9.0` (per PettingZoo's own tutorial pins). `concat_vec_env_v1`, `float_obs`, `frame_stack_v1`.
Ray RLlib	[OK] **Ray 2.58.0** in the latest docs; 2.56.0 in the package reference pages. `MultiAgentEnv` is `@PublicAPI(stability="beta")` and subclasses `gymnasium.Env`. Wrappers: `PettingZooEnv`, `OpenSpielEnv`, `make_multi_agent()`. **Gotcha: "Multi-agent setups aren't vectorizable yet."** You cannot use `num_envs > 1` for a MA env today. Plan your rollout collection accordingly (use more remote `EnvRunner` actors instead).
Stable-Baselines3	[!] `2.7.x` stable; `2.9.x` on master. **Single-agent only** (`VecEnv`), no MA support. Useful for your single-agent baselines (greedy, round-robin).
`sb3-contrib`	[!] **2.9.0, 15 Jun 2026.** MIT. But see 6.1: **no MAPPO.**
CleanRL	[OK] `1.2.0` on PyPI (May 2023) but the GitHub repo is actively maintained and has a JMLR paper (vol 23, paper 274, 2022). `ppo_pettingzoo_ma_atari.py` is the reference MA-PPO single file.
`onnxruntime`	[OK] **1.29.0, 17 Aug 2026** (CPU); `onnxruntime-gpu` 1.29.0, 17 Aug 2026, requires Python $\geq$ 3.11. MIT.
MARLlib	[X] See 5.

0.15.6. 5. Open-source UAV RL simulators — comparison

5.1 Recommendation table

Tool	Current version	Date	License	Multi-agent RL	Physics modelled	Suitability for this project
OMNeT++ 6.4 + INET 4.7	6.4.0 / 4.7.0	2026-05-08 / 2026-07-07	Academic Public License / LGPL	Via your own bridge (you write it)	**Network protocol stack**: full TCP/IP, 802.11/15.4, routing, queueing, handover, power consumption, mobility, obstacles. No rigid-body dynamics.	**Primary choice.** Your problem is a *network* problem. Exponentially scalable event counts.
Simu5G 1.7.0	1.7.0	2026-09-14	LGPL	Via your own bridge	3GPP 5G NR/LTE **link-level** abstraction: numerology, MCS/CQI tables, HARQ, schedulers, handover, 5GC, MEC. **No OFDM, no waveform, no real fading process, no MIMO.**	Only if your contribution is 5G-specific. Heavy; breaking changes every minor release; pins you to INET 4.5.4.
gym-pybullet-drones	`main` (no semver)	commits 6 Sep 2026	**MIT**	[OK] **Native** (`--multiagent true`, `MultiAgentAviary`)	**Rigid-body quadrotor dynamics** via PyBullet: CFD-derived drag/thrust, downwash, PID/DSL controllers. **No network protocol stack.**	Excellent for *control*. Use as a fast dynamics layer or a baseline.

Tool	Current version	Date	License	Multi-agent RL	Physics modelled	Suitability for this project
Isaac Lab 3.0.0-beta2	3.0.0-beta2 (stable line: **2.3.2**)	beta branch 25 Jul 2026	BSD-3-Clause	[OK] via RLlib/skrl multi-agent	NVIDIA PhysX (or **Newton** in 3.0) rigid-body + RTX rendering. Ships a **Crazyflie** quadcopter; v2.3.2 added multirotor/thruster actuators and a manager-based ARL drone task.	**Requires an NVIDIA RTX GPU.** PhysX's `PhysX` backend is numerically fine but the framework is a heavyweight dependency. Overkill for 5 UAVs.
AirSim / Colosseum	—	**Microsoft shut AirSim down Jul 2022. Colosseum ARCHIVED 11 Jul 2026.**	MIT (dead)	[X]	Full UE5 + AirSim flight-dynamics plugin, PX4/ArduPilot SITL/HIL	**DO NOT USE. Total dead end.** Colosseum was the official successor and it is now read-only.
SUMO 1.27.1 + TraCI	**1.27.1**	**2026-06-25**	**EPL-2.0** (also GPL-2.0)	[X] (single-agent controller)	Microscopic **road-traffic** flow: car-following, lane-changing, traffic lights, pedestrians, public transport	Only for a *ground-vehicle* component. EPL-2.0 is **weak copyleft — you must publish derivative source**. For UAV-IoT, irrelevant.
MARLlib	2.0.x (pip)	**last push 2024-11-28**	MIT	[OK] 18 algorithms incl. MAPPO	none (library only)	**~22 months stale.** Built against RLlib ~2.x; modern Ray 2.5x broke its internals. 47 open issues unanswered. Use the original MAPPO repo instead.

Tool	Current version	Date	License	Multi-agent RL	Physics modelled	Suitability for this project
rllab	0.5.5 (2018)	2018	BSD	[X] single-agent	none	**Dead.** Ancient, Python-2-era, no Python 3.
PettingZoo 1.27.0	1.27.0	2026-08-13	MIT	[OK] **This IS the MARL API standard**	none (library only)	**Adopt as your env interface.** Parallel API.
MuJoCo-Drones-Gym	preprint	arXiv **2606.08039**, 6 Jun 2026	see repo	[OK] `MultiAgentAviary` + **PettingZoo ParallelEnv wrapper**	MuJoCo rigid-body; MJX/XLA GPU backend	**Brand new (3 months).** Promising and the PettingZoo wrapper is the right design, but too new to depend on. Watch it.
OmniDrones	2023	arXiv 2309.12825	MIT	[OK] 10+ MA tasks	Isaac Sim, >10 steps/s on RTX 4090	Isaac-dependent.

5.2 Notes on the specific entries

gym-pybullet-drones. The repo **moved from `utiasDSL` to `learnsyslab`**. The current `main` branch is described in its own README as *"a minimalist refactoring of the original gym-pybullet-drones repository, designed for compatibility with gymnasium, stable-baselines3 2.0, and betaflight SITL."* The original IROS 2021 codebase is preserved on the `paper`/`master` branches. **If you want the code your cited paper describes, check out `master`.** Multi-agent is first-class: `python learn.py --multiagent true`. MIT. Actively maintained (commits 6 Sep 2026). It was featured in GitHub's Maintainer Spotlight 2026. Caveat: **no radio model at all** — you would have to write the link layer yourself.

Isaac Lab. Be careful about the version: the **default branch is `release/3.0.0-beta2`** (Isaac Sim 6.0.0/6.0.1) and Isaac Lab 3.0 has a *"ground-up architectural overhaul"* including **Newton** as an alternative physics backend. If you want stability, pin `v2.3.2` (Isaac Sim 4.5 / 5.0 / 5.1). The official guidance is to retarget PRs at `develop` for 3.0 work. Ships a Crazyflie quadcopter asset. Hardware requirement: NVIDIA RTX GPU. **Reasonable simulator, wrong scale for a 5-UAV network study.**

AirSim/Colosseum — the trap, in full. Microsoft announced the shutdown of AirSim in **July 2022**. Codex Labs LLC forked it as **Colosseum** and developed it through UE 5.3/5.4/5.7 with an experimental Unity plugin. On **11 July 2026 the owner archived the repository — it is now read-only.** There is no maintained open-source successor. If your reference list includes an AirSim-based paper and you want to reproduce or build on it, your options are paid/closed (e.g. Project AirSim on Azure) or starting a fork yourself. Do not plan around it.

SUMO/TraCI. Very well maintained (1.27.1, 25 June 2026; 1.27.0 on 21 May 2026). `pip install eclipse-sumo`, `traci`, `libsumo`, `sumolib`. Use `libsumo` rather than `traci` if you need throughput — it embeds SUMO as a library and avoids socket overhead. **EPL-2.0 is a licensing consideration.** The canonical SUMOOMNeT++ coupling is **Eclipse MOSAIC** (currently pinned to OMNeT++ 6.1 / INET 4.5.4, federate 25.2). Only relevant if your project has a ground-vehicle component.

MARLlib — status. GitHub API: `pushed_at: 2024-11-28`, `archived: false`, MIT, 1347 stars, 198 forks, **47 open issues**. It wraps RLlib and offers 18 MARL algorithms including `mappo`. It was accepted in **JMLR 24(315), 2023** — cite *that* if you use it, but pin Ray to whatever it was tested on, or expect breakage.

5.3 What is genuinely new for UAV swarms (2025–2026)

Item	Date	Notes
Isaac Lab v2.3.2	2026	Multirotor/thruster actuator model, multirotor asset, **manager-based ARL drone task** (PR #3760). This is the first first-party NVIDIA UAV task.
MuJoCo-Drones-Gym	6 Jun 2026	7 Gymnasium tasks + `MultiAgentAviary` + a **PettingZoo** `ParallelEnv` **wrapper**. Ports the `gym-pybullet-drones` philosophy to MuJoCo/MJX. Explicitly compares against `gym-pybullet-drones` and `OmniDrones`.
Isaac Lab 3.0.0-beta2	25 Jul 2026	Newton physics backend, multi-backend architecture, migration guide from 2.x.
Vvc-gym	ICLR 2025	Fixed-wing UAV RL, multi-goal long-horizon.
BlueSky-gym	2024	Air-traffic RL environments.
safe-control-gym	ongoing (learnsyslab)	PyBullet CartPole/quadrotor with **CasADi symbolic a priori dynamics** for safe learning. MIT. 913 stars.
crazyflow	ongoing (learnsyslab)	"GPU-accelerated, differentiable, JAX-based simulation." The JAX path to UAV RL.
aerial-autonomy-stack	ongoing (learnsyslab)	PX4/ArduPilot + ROS2 + JetPack deployment — the sim-to-real bridge.
IsaacSimZMQ	Jun 2026	NVIDIA's own ZMQ external-application bridge for Isaac Sim — a useful architectural precedent for your OMNeT++ socket bridge.

The honest summary: **there is no mature, maintained, network-simulator-integrated open-source UAV MARL environment.** Everyone hand-rolls the bridge. That is simultaneously the risk and the novelty of your project.

0.15.7. 6. RL algorithm implementation reality

6.1 sb3-contrib MAPPO: removed, not just bad

I enumerated the `sb3_contrib/` directory on `master` of `Stable-Baselines-Team/stable-baselines3-contrib`. The complete directory listing is:

```
ars/  common/  crossq/  ppo_mask/  ppo_recurrent/  qrdqn/  tqc/  trpo/
__init__.py  py.typed  version.txt
```

There is no `mappo/`. There is no `ippo/`. The current documented algorithm list (v2.9.0, 15 Jun 2026) is:

- Augmented Random Search (ARS)
- Quantile Regression DQN (QR-DQN)
- PPO with invalid action masking (MaskablePPO)
- PPO with recurrent policy (RecurrentPPO / PPO LSTM)
- Truncated Quantile Critics (TQC)
- Trust Region Policy Optimization (TRPO)

- CrossQ (Batch Normalization in Deep RL)

`from sb3_contrib import MAPPO` **raises** `ImportError` **on every currently released version.** Do not put it in your `requirements.txt`. Do not write a report section based on it. (It existed earlier and was removed for the standard SB3 reason: a feature in `contrib` that nobody maintains gets removed. Stable-Baselines3 core is single-agent by design; MARL is out of scope for the whole project.)

What to use instead, in order of preference:

6.2 Option 1 (recommended): the original PyTorch MAPPO

`susanbao/mappo` — *"This is the official implementation... The implementation in this repository is used in the paper 'The Surprising Effectiveness of MAPPO in Cooperative Multi-Agent Games'"* (Yu et al., arXiv:2103.01955, NeurIPS 2022).

OK This is the paper's own code. Cite the paper, not the repo.
OK CTDE, centralized critic / decentralized actor, parameter sharing, GAE, value normalization, recurrent option.
OK Actually reproduces the paper's SMAC results.
! Written against older PyTorch; the `main` branch is a research codebase, not a library. Expect to pin dependencies and possibly patch.
! Single-process, single-env by default — **you will need to add your own parallelism**, which for you is trivially available: one `opp_run` process per parallel env, wired to N RL workers.

6.3 Option 2: CleanRL single-file

`vwxyzjn/cleanrl`. JMLR 23(274), 2022. The MA reference file is `cleanrl/ppo_pettingzoo_ma_atari.py`, mirrored verbatim in PettingZoo's own docs (which pin `pettingzoo[butterfly,atari,testing]>=1.27.0`, `SuperSuit>=3.9.0`).

OK ~400 lines, every hyperparameter visible, benchmarked against `openai/baselines`.
OK WandB + TensorBoard + seeding + video built in — the reproducibility story is already written.
OK You can adapt it to continuous actions from `ppo_continuous_action.py` in an afternoon. **For UAV velocity commands you want continuous control; there is no off-the-shelf continuous MA-PPO single file.** You write it.
! CleanRL is explicitly *not* a library: *"CleanRL is not a modular library and therefore it is not meant to be imported."* Fork it, don't import it.

6.4 Option 3: Ray RLlib

- `MultiAgentEnv` is `@PublicAPI(stability="beta")`, subclasses `gymnasium.Env`, dict-based obs/rew/term/trunc per agent id. Documented as *"the recommended way of defining your own multi-agent environment logic."*
- Wrappers for `PettingZooEnv` and `OpenSpielEnv`. `make_multi_agent()` for turning a single-agent env into a MA env.
- **MAPPO is not in RLlib.** The MA-capable algorithms are PPO (via policy post-processing/masking), APPO, IMPALA, DQN-family (with agent grouping for QMIX), SAC, MARWIL. You implement MAPPO as a custom `RLModule` + `Algorithm` — doable, but you are writing MARLlib-in-miniature.
- **Blocking constraint: "Multi-agent setups aren't vectorizable yet."** `num_envs > 1` does not help you. You scale by adding `EnvRunner` actors instead.

OK Best scaling story (Ray actors → N `opp_run` processes naturally).
X Heavy. The new (non-"Old API Stack") `RLModule`/`EnvRunner` architecture is a breaking API; tutorials written for 2.7 will not run.

- **Verdict: overkill for 5 agents.** Use it if you need to scale to 50+ agents or many parallel OMNeT++ workers.

6.5 Compute budget — what is actually feasible

For **5 agents, 2-D, 500-step episodes** on a modern laptop (8 physical cores, 16 GB RAM, no GPU):

Quantity	Estimate	Basis
Wall clock per episode	**0.15–3 s**	NumPy 2-D; ~500 steps × ~2 ms Python overhead

Quantity	Estimate	Basis
Parallel episodes (8 cores)	8×	`multiprocessing` / `SubprocVecEnv`
Throughput	**~2,700–50,000 env-steps/s**	
1e6 steps (typical PPO budget)	**~0.5–2 CPU-hours**	
1e7 steps (thorough)	**~5–20 CPU-hours**	
Full study: 5 seeds × 4 algorithms × 1e6 steps	**~10–40 CPU-hours**	Feasible in **one night**
Ablations: 5 seeds × 3 envs × 3 noise levels × 2 algorithms	**~200–600 CPU-hours**	**~1–2 weeks on a single laptop**

For **5 agents, 2-D, with a 1.5M-step budget and 3 environments × 3 noise levels × 5 seeds × 4 algorithms**, budget **~500–1500 CPU-hours**. On a laptop that is **3–6 weeks of continuous running.** Get cluster time, or cut the grid.

For the OMNeT++ evaluation (the expensive part):

Quantity	Estimate
Events per episode (5 UAV, 50 sensors, `ScalarRadioMedium`, 600 s)	1e5 – 5e5
Wall clock per episode, `MODE=release`	0.3 – 5 s
Parallel (16 workers)	16×
1,000 evaluation episodes	**20 s – 5 min**
5 seeds × 4 algorithms × 50 eval episodes = 1,000	**~1 hour** — cheap
A full parameter sweep: 3 values × 3 values × 10 runs = 90 runs	**~8 CPU-minutes**

Key takeaway: the OMNeT++ evaluation is cheap; the Python RL training is the expensive part. Budget your time accordingly, and keep the surrogate loop for the training.

Latency budget for the bridge, per decision epoch (5 agents, 4 frames):

Component	Cost
`send`/`recv` × 4 frames, loopback, `TCP_NODELAY`	40–120 s
numpy marshalling × 2 × 5 × 113 floats	~5 s
Python-side policy forward pass (MLP 2×128, batch 5)	60–200 s
Total	**~0.1–0.35 ms**
× 120 epochs × 1,000 episodes	**12–42 s**

The bridge is 0.01% of your training budget. Do not optimize it beyond 4.3. Spend the engineering effort on the environment, the reward, and the experiment protocol.

Practical flags for `opp_run` invocation:

ONE process per parallel env, for N-way RL parallelism: opp_run -u Cmdenv -f --cmdenv-express-mode=true \ -l omnetpp.ini -c General -r $\{RUN\}-n$. :INET/src

Set `output-scalar-file`/`output-vector-file` to per-run paths, or better: set `*.result-dir` from `${runnumber}` so runs never collide.

0.15.8. 7. Reproducibility requirements

7.1 Seeds in OMNeT++

[General] seed-set = 0 # ← SET THIS # seed = 0 # ← NEVER SET THIS. It pins every run to the same stream. # cmdenv-rng-seed = 0 # ← named RNG stream, only if you need multiple streams

Batch: 3 seed-sets x 10 runs each # opp_runall -l omnetpp.ini -c General -s 0-2

The mechanism: OMNeT++ derives the actual seed as `seed = seedSet * NUM_RNG + rngIndex`, where `NUM_RNG` **defaults to 1 in OMNeT++ 6.x** (it was **2** in OMNeT++ $\leq$ 5.x). So with the default, seed-set N uses seed N. If you set `NUM_RNG` for multiple streams, document it — it changes your results.

Named RNG streams — essential for a multi-agent study, because it lets you vary *one* source of randomness at a time:

#include "inet/common/Random.h" auto *rngMobility = getParentModule()->getRNG(0); // mobility auto *rngChannel = getParentModule()->getRNG(1); // wireless channel / error model auto *rngActions = getParentModule()->getRNG(2); // agent action sampling auto *rngFaults = getParentModule()->getRNG(3); // failure injection

double u = rngMobility->doubleRand(); int a = (int)rngActions->getRand("actionDist"); // 'getRand' is reproducible // per-name, unlike getRand()

`getRNG(i)` / `getRNG(name)` requires the module to be a `cComponent` and uses the global `RNG` registry. INET 4.7 adds a **configurable RNG grouping mechanism** — if you use it, document the grouping.

[!] Version sensitivity, and this is not a detail. Simu5G's v1.7.0 release notes state plainly:

> **Fingerprints are INET/OMNeT++ version specific:** the CSVs hold the values for INET-4.5.4 + OMNeT++ 6.3.0, the versions CI uses; other versions report mismatches, and they are expected. ... OMNeT++ 6.4.0 moves `sz` in 181 [of 202 configurations] and nothing else, as it writes the mean of an empty statistic as `nan` rather than `-nan`; INET-4.6.0 moves `sz` in all of them and `tplx`/`tilx` in 18 and 45.

Your results are not comparable across OMNeT++/INET versions unless you verify them. Pin exact versions, and if you change, re-run the full seed sweep. Also: `-march=native` and LTO change floating-point results between machines. Use them for the final runs only.

7.2 Seeds in Python

import os, random, numpy as np, torch

def set_global_seed(seed: int, deterministic_torch: bool = True) -> None: os.environ["PYTHONHASHSEED"] = str(seed) random.seed(seed) np.random.seed(seed) torch.manual_seed(seed) torch.cuda.manual_seed_all(seed) if deterministic_torch: torch.use_deterministic_algorithms(True, warn_only=True) torch.backends.cudnn.benchmark = False os.environ.setdefault("CUBLAS_WORKSPACE_CONFIG", ":4096:8")

One seed for everything is a reproducibility anti-pattern. It causes correlated exploration across agents and across episodes. Use a *derived* seed per purpose:

import hashlib def derive(master: int, *parts) -> int: h = hashlib.sha256(("|".join(map(str, (master,) + parts))).encode()).digest() return int.from_bytes(h[:4], "big")

env_rng = np.random.default_rng(derive(master, "env")) torch.manual_seed(derive(master, "torch")) obs_space.seed(d "obs")) action_space.seed(derive(master, "act"))

Critical for your bridge: the OMNeT++ seed and the Python seed must be *coupled and recorded together*. A `master_seed` of 7 must produce the same OMNeT++ `seed-set` and the same Python RNG every time:

master = 7 omnet_seed_set = master # or derive(master, "omnet") policy_seed = derive(master, "policy") env_seed = derive(master, "env") # send BOTH across the socket; record BOTH in every results row

7.3 Reporting statistics

The unit of evidence is a seed, not a run. A 10-run sweep with 1 seed-set is *one* observation, not 10.

Minimum defensible protocol: 5 seed-sets minimum, 10 preferred. Runs within a seed-set are correlated by construction (they share the initial RNG stream derivation); treat them as repeated measurements of a *fixed* environment instance, and average within a seed-set before computing statistics across seed-sets. This is the standard INET/PlexSim-style protocol and the right one.

import numpy as np from scipy import stats

def summarize(per_seed_scores: np.ndarray) -> dict: """ per_seed_scores: (n_seeds,) -- already averaged within each seed-set """ n = len(per_seed_scores) mean = per_seed_scores.mean() std = per_seed_scores.std(ddof=1) # SAMPLE stddev, ddof=1 sem = stats.sem(per_seed_scores) tcrit = stats.t.ppf(0.975, df=n - 1) # two-sided 95% ci = tcrit * sem return dict(n=n, mean=mean, std=std, sem=sem, ci95_low=mean - ci, ci95_high=mean + ci, # If your scores are non-Gaussian (very common for coverage %), # report the bootstrap CI as primary and t as secondary. boot_ci=tuple(np.percentile((stats.bootstrap((per_seed_scores,), np.mean, n_resamples=10_000, random_state=np.random.default_rng(0))[0], [2.5, 97.5]))),)

def holm_bonferroni(pairs, alpha=0.05): """Multiple comparisons. Do NOT report uncorrected p-values across baselines.""" pairs = sorted(pairs, key=lambda kv: kv[1]) # [(name, p), ...] m = len(pairs); out = []; k = 0 for name, p in pairs: k += 1 out.append((name, p, min(1.0, p * (m - k + 1)), p <= alpha / (m - k + 1))) return out

What to report in the paper: - Mean ± sample std (`ddof=1`) across seed-sets, with **n stated explicitly**. **- 95% CI**, with the method named (Student-t, or BCa bootstrap). Coverage percentage is bounded and often skewed → **use the bootstrap**. **- Per-seed raw values in an appendix or supplementary table**,

always. This is what makes your results auditable. - **A paired statistical test against your primary baseline** (Wilcoxon signed-rank or paired t-test across seed-sets), with **Holm–Bonferroni correction** if you compare against more than 2 algorithms. Report the effect size (rank-biserial correlation or Cohen's d), not just p. - **Never** report a single run. **Never** compare across different INET/OMNeT++ versions.

A run-configuration table belongs in the report:

Algorithm	Seed-set	Run	Master seed	OMNeT++	INET	Git SHA	Wall clock	Result file
MAPPO	0	0–9	7	6.4.0	4.7.0	`a1b2c3d`	41 min	`results/General/Gen`

7.4 A fair baseline comparison protocol

1. **Same environment, same episodes, same seeds.** Baseline and method must see the *identical* `seed-set` / `run` sequence. Pair your tests accordingly (paired, not independent).
2. **Equal interaction budget, not equal wall clock.** Same number of environment steps and same number of gradient updates. Wall-clock-normalized comparisons flatter methods with cheap steps and are not meaningful here.
3. **Hyperparameters are part of the baseline.** Give every baseline the same tuning budget (same number of random-search trials, same search space width) and report the tuned values. Tuning your method more than your baselines is the most common MARL paper sin.
4. **Include these baselines** (and be honest that they are strong):
   - Greedy farthest-point coverage (recompute on failure)
   - Greedy with full coverage-map recomputation
   - Round-robin partitioning (static assignment)
   - Decentralized PPO (IPPO) with the same network
   - **Greedy + a simple local repair heuristic** — usually the most competitive non-learning baseline in coverage problems. If your learned policy does not beat this, that is your finding, and it is a publishable one.
5. **Separate the ablation axes.** Persistent coverage vs. re-compute-on-failure; fault tolerance (node failure, gNB failure, link degradation) vs. no faults; centralized critic vs. decentralized critic; homogeneous vs. heterogeneous agents.
6. **Report the surrogate-vs-OMNeT++ correlation** (4.6) so a reader knows which claims were verified in which simulator.
7. **Archive everything.** NED/ini/C++/Python, `requirements.txt` with pinned versions, exact `opp_run` command lines, git SHA, and the raw `.sca` files. A DOI'd Zenodo record costs you 20 minutes and is frequently the difference between "reproducible" and "not".

0.15.9. 8. License and citation requirements

8.1 What needs citation

Always cite (these have papers, and using their model without citing is plagiarism):

Component	Citation
OMNeT++	Varga, A. "OMNeT++: Modeling Dynamic Networks on the Micro- and Meso-Scale." *Mathematical Models for Communication Systems, LNCS 5462*, Springer, 2009.
INET	Varga, A. "INET Framework for OMNeT++/OMNEST: Current Status and Future Directions." *Simulation Modelling and Practice, and Methodology* / OpenSim Ltd. Also cite the specific release: **INET 4.7.0, released 7 July 2026.**

Component	Citation
Simu5G	Virdis, A., Nardini, G., Stea, G., Varga, A. "Recent Advances in 5G NR Radio Layer Modelling for System-Level Simulations in Simu5G." *Proc. IEEE WCNC*; plus **Varga, A. (2026), "Simu5G v1.7.0 release notes / WHATSNEW"** — cite the version, because the class renames are breaking.
SimuLTE (if cited as predecessor)	Nardini, G., Virdis, A., Egea Lopez, E., Stea, G. "Simulation of LTE Networks." *OMNeT++ Workshop*, 2011.
MAPPO	**Yu, C., et al. "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games." NeurIPS 2022. arXiv:2103.01955.** Mandatory — it is the origin of your method.
PettingZoo	Terry, J., et al. "PettingZoo: Gymnasium for Multi-Agent Reinforcement Learning." *NeurIPS 2021. arXiv:2009.14471.*
Gymnasium (if you use the API)	Towers, M., et al. "Gymnasium: A Standardized Interface for Reinforcement Learning Environments." arXiv:2407.17032.
CleanRL	Huang, S., et al. "CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms." *JMLR* 23(274), 2022.
Stable-Baselines3	Raffin, A., et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations." *JMLR* 22(268), 2021.
Ray RLlib	Moritz, P., et al. "Ray: A Distributed Framework for Emerging AI Applications." *OSDI 2018.*
MARLlib (if used)	Hu, S., Zhong, Y., Gao, M., et al. "MARLlib: A Scalable and Efficient Multi-agent Reinforcement Learning Library." *JMLR* 24(315), 2023.
PyBullet	Coumans, L., et al. "Bullet Physics SDK." — `pybullet` is GPLv3; cite and see 8.2.
gym-pybullet-drones	**Panerati, A., El-Dahshan, A., Raistani, M., Cetinas, E. "Learning to Fly — a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control." *IROS 2021. arXiv:2103.02142.*** Its own README asks you to cite this even for the refactored `main` branch.
Isaac Lab	The Isaac Lab technical report: **arXiv:2511.04831** (MPR+25), plus the NVIDIA Isaac Sim version used.
SUMO	**Behrisch, J., Krumbholz, L., Kesting, E., Schwanebecher, T. "SUMO — Simulation of Urban MObility." In *Traffic Simulation*, 2018.** Zenodo DOI for the specific version: **1.27.0 → 10.5281/zenodo.20312733**. Request one for your exact version at https://sumo.dlr.de/Downloads.php.
MOSAIC (if used)	Eclipse MOSAIC docs + the federate version.
ONNX / ONNX Runtime	Chen, B., et al. "Open Neural Network Exchange." arXiv:1808.02679 (ONNX); Microsoft `onnxruntime` repo + version.
ZeroMQ (if used)	Hintjens, P. "ZeroMQ: The Smart Socket Library." — the *book*, not just a citation.

Cite per-component, not per-paper. A report that uses OMNeT++ + INET + PettingZoo + MAPPO + CleanRL needs **at least 5 citations**, and each simulator you compare against needs its own.

Do not cite: `pyopp` (doesn't exist), `sb3-contrib MAPPO` (doesn't exist), AirSim/Colosseum (dead — if you mention it at all, say it is archived and cite the archive).

8.2 Licenses — the part people get into trouble with

Component	License	Obligation
OMNeT++	**Academic Public License**	Free for **academic use only**. Commercial use requires OMNEST. **Cite.** Do not redistribute the binary in a public artifact without reading the APL.
INET	**LGPL-3.0-or-later** (since 4.4.0)	Dynamic linking keeps your work yours. If you *statically* link or modify INET, you must release those changes under LGPL. **A separate `inet-gpl` repo holds the old GPL parts** (TCP NSC, packetdrill, HTB, HTTP tools) — if you enable any of those project features, your derivative inherits GPL. **Check which INET project features you enable.**
Simu5G	**LGPL**	Same.
PettingZoo	MIT	Attribution only.
CleanRL / SB3 / sb3-contrib	MIT	Attribution only.
Ray RLlib	Apache-2.0	Attribution + patent grant.
MARLlib	MIT	Attribution.
gym-pybullet-drones	MIT	Attribution + cite the IROS paper.
PyBullet	**GPL-3.0**	[!] **This is the sharp edge.** `gym-pybullet-drones` is MIT but depends on GPL-3.0 PyBullet. Importing it into your code is a GPL question. For a **course project / report** this is fine (no distribution). If you ever distribute a binary or offer a service, get advice.
Isaac Lab	**BSD-3-Clause** (some parts Apache-2.0)	Very permissive. But Isaac Sim itself is under NVIDIA's EULA, and you need an NVIDIA GPU.
SUMO	**EPL-2.0** (also GPL-2.0)	[!] **EPL-2.0 is weak copyleft.** Modifying SUMO requires you to publish the modified source. If you only `import sumo` / use TraCI, you are fine. Read it if you modify it.
OMNeT++ 6.4.0 binary	APL	Academic use; be careful with redistribution.

The only two that can actually bite you are PyBullet (GPL-3.0) and SUMO (EPL-2.0). For a student report neither matters. If your project ever becomes a public artifact, check.

0.15.10. 9. Traps — a consolidated list

1. `pyopp` **does not exist in OMNeT++ 6.x.** Verified: zero matches in the `omnetpp-6.4.0` tree. Any tutorial recommending embedded Python NED modules is pre-5.0. `pyeval()`/`pycode()` exist but are parameter-expression-only.
2. `sb3-contrib` **has no MAPPO or IPPO.** The directories were deleted. `from sb3_contrib import MAPPO` → `ImportError`. Use `susanbao/mappo` or CleanRL.

3. **AirSim is dead (Microsoft, Jul 2022) and Colosseum — its official successor — was archived 11 Jul 2026.** There is no maintained open-source UE-based UAV simulator. Do not build on it.
4. **Every INET 4.5-era wireless class name in your brief is gone in 4.6/4.7:** `ScalarRadio`, `DimensionalRadio`, `IsotropicScalarRadioMedium`, `IsotropicScalarRadio`, `IdealPropagation`, `BasicRangePropag` `DielectricBreakdownPropagation`, `TernaryPropagation`, `DmgErrorModel`, `PERTableRadioErrorModel`, `TableBasedErrorModel`, `AdditiveRayleighChannel`, `DimensionalChannel`. **And** `SimpleWirelessHost` **never existed.** Tutorials and LLM output are both full of these.
5. `LinearMobilityModule` / `MassMobilityModule` **are C++ class names.** The NED types are `LinearMobility` / `MassMobility`. Writing the C++ name in `typename =` fails.
6. **Never set** `seed = 0` **in** `[General]`**.** It pins every run to the same stream and silently kills all your variance. Set `seed-set`.
7. **Never parse** `.sca`/`.vec` **by hand.** Use `omnetpp.scave.results.read_files()`.
8. `NUM_RNG` **changed from 2 to 1 in OMNeT++ 6.x.** A seed-set means a different seed than it did in OMNeT++ 5.
9. `-march=native` **and LTO change floating-point results between machines.** Final runs only.
10. **Results are not comparable across INET/OMNeT++ versions.** Simu5G's own release notes document which statistics move and by how much.